The Thiele Machine

A Complete Account from First Principles

Structural Entitlement, Accountable Computation,
and No Free Insight

Devon Thiele

June 2026

Before the book: the short version, and how to check it

I think there's a cost in computation you can't get around, and the argument is short enough to check by reading, so here it is up front, with the place to look if you think I'm wrong.

What I'm pointing at: flipping a certification bit from false to true should cost at least one. A classical step rule has nowhere to carry that as a rule of its transition, and nothing Turing-equivalent does either: equivalence is about which functions you compute, not about what the step is allowed to read, so the whole class is missing the field, not one machine at a time. It can only run a checker beside the program, and a checker can be skipped. If that's right, classical computation is the lossy shadow of something that charges the flip: the projection down is clean, and getting back up needs an outside choice of the state that got dropped.

The part I lean on hardest is that the charging rule, A2, isn't something I picked. I argue it's the only rule that prices certification exactly, and that line is a contest A2 has to win, not a definition it gets handed. A rule that ever lets a certification flip go uncharged underprices it: run the trace that does exactly that flip, and it reports zero where the floor is one. A rule that ever charges a step that certifies nothing overprices it: run the trace that does exactly that step, and it bills for a certification that never happened. Two one-step traces, two ways to lose, and the only rule that loses neither fires on exactly the certification flips. I think that's A2. What keeps this from being a word game is that the contest has real entrants. Erasure accounting is the obvious one: same honesty bar, charge at least one every time you destroy a bit, the rule a thermodynamics person reaches for first. Hand it this exact job and it certifies at zero cost whenever nothing got erased, which is most of the time, because it meters the wrong event. If "prices certification exactly" were secretly just a synonym for A2, no serious rival could lose to it out loud; erasure accounting is a serious rival, and it loses out loud. So among the rules that try to price certification, A2 is the one left standing: I didn't write its name into the contest, it outlasted the field. Here's the honest fence around that, because I keep wanting to climb it and I can't: the contest ranks chargers of certification, and that certification is the event on the meter at all is the setup I wrote, not the prize anyone won. Whether the universe is forced to put certification on the meter rather than, say, head-reversals or multiplications is a different question, and an open one. I have not shown it, and I am not claiming it. What I can stand on is the narrow win: hold the event fixed, and A2 is the only honest, non-overcharging way to price it. Name another rule that survives that, one that prices certification exactly and isn't A2, and I'm wrong. The two traces, and the one rival already on the floor, are why I don't think there is one.

That's the test, and it doesn't need a proof assistant; a rule like this is checked by following it. I did mechanize all of it and pull a running machine and a chip out of it, but that was to make the machine and the chip trustworthy, not to make the argument true. The Coq is scaffolding, and so are the OCaml and the RTL. What's on this page stands on the argument, or it doesn't stand.

Everything after is the unfolding: the same argument at full length, every step held open. You don't need it to check what I'm pointing at. You need it only to see whether the thing stays put when you lean on it.

Abstract

The Thiele Machine is a computational substrate where certification cost (μ) lives in the step rule. The step rule is a small bureaucrat that refuses to advance the state until the cost has been counted. The program does not get a vote. The principal structural result is asymmetric and assumes no axioms. Every Thiele state has a canonical forgetful projection onto a classical-equivalent state. Every classical state has multiple Thiele preimages and no canonical lift back. The projection is canonical. The lift is not. The fields the projection drops (`vm_certified`, `csr_cert_addr`, μ relative to the bare three-field observable that drops it) are provably not functions of the classical projection (`cert_not_function_of_forget`, `cert_addr_not_function_of_forget`, `mu_not_function_of_bare_observable`, all proved with no axioms added). Every Thiele machine has a Turing projection. No Turing machine has a canonical Thiele lift. The directionality is proved, not stipulated.

The categorical universal property is closed. The Thiele substrate is the initial object in the category of A2-respecting substrates over the `vm_instruction` signature (`thiele_is_initial_a2_substrate`, zero axioms). That same universal property makes $\mathbb{Z}$ the initial ring, which is exactly why being initial settles nothing on its own: $\mathbb{Z}$ is initial too, and nobody thinks $\mathbb{Z}$ is the shadow of a deeper thing. So initiality is not where the weight sits, and I am not going to pretend it is. The weight sits lower down, in what the dropped field does. Hold the trust fixed and A2 is the only rule that prices certification exactly. Miss a cert-flip and the floor fails on a one-step trace. Charge a step that certifies nothing and you have overpaid. The one rule left fires exactly when the cert flips, and that is A2 (`a2_equal_trust_substitution_payoff`). The state it runs on, $(\mu, \texttt{cert})$, is the proved-minimal extension over (mem, regs, pc): cert is not a function of the rest, μ is not a function of the rest (`P_full_is_minimal_complete_extension`). So a step rule has a cert to read and a μ to charge, or it has no way to say A2 at all. A law that lives in the step is not a checker you run next to the program and hope. A transition table reads a state and a symbol and nothing else, so it has no slot for the field at all. That is what makes classical computation the derivative thing, and it is the argument that says so, not me. Whether the A2 category is the chosen one among every cost-tracking category you could write down is a question I am not asking, and nothing here answers it.

Each load-bearing result has a standard parent in the literature: the irrecoverability theorems against signature-expressibility, the categorical initiality against the universal property of the term algebra, the substrate-axis undecidability against Kleene's 1938 diagonal pattern, the μ-uniqueness against natural-number recursion. Section 24 enumerates each placement, so if you find yourself pattern-matching to "this is just X" you can see which X I already checked against and what the substrate-specific delta is. The argument is four steps, and each one stands on its own. A step holds or it doesn't. If one gives, it gives somewhere specific, and there is a Coq name sitting right under it. You can walk them one at a time.

1. A2 constrains state fields the bare classical signatures do not have. Take a state, flip its certification predicate from false to true in one step, and read the cost ledger. It went up by at least one. That reading is A2, and you can take it on any concrete run. Now try the same reading on a Turing machine: its signature

is a state, an alphabet, and a transition table, and none of those is a certification predicate or a cost field. Register machines don't have them either. Lambda calculi don't. There is no field to flip and no ledger to read, so the rule has nothing to range over. And it isn't a quirk of the 51-opcode VM: `universal_nfi_any_substrate`, over the `CertificationSystem` record, puts the same ≥ 1 on the certifying trace of every substrate carrying the cert predicate. Same reading, every instrument. And the class is not rigged to win: a trusted rival law (erasure accounting, same record shape, its own cost rule) certifies at zero cost without erasing, so respecting the floor is a line some trusted laws fall on the wrong side of, not a label I stapled on the winners (`commitment_cost_not_reducible_to_erasure_cost`). Extend a classical signature with cert and cost fields and a step rule that respects A2 and you have moved to a different substrate; the migration is one-way. And no predicate over the bare classical fields rebuilds the cert flag; the information isn't down there to recover (`cert_not_function_of_forget`).

2. Therefore the substrate is forced. Run the check on the two fields. Take two states that agree on memory, registers, and program counter but differ in the cert flag: nothing in (mem, regs, pc, μ) tells them apart, so cert is not a function of the rest. Take two that differ only in μ: the cost-ledger separation in Section 12 reads them apart and nothing in the bare classical fields does, so μ is not a function of the rest either. That is `P_full_is_minimal_complete_extension`: $(\mu, \texttt{cert})$ is the minimal extension over (mem, regs, pc), and you can't pull either field out of the others. That is the state A2 needs to be a sentence at all. Something to read the cert from. Something to charge the μ to. Take either away and the rule has nothing left to say. So there is no "TM plus A2": write the cert flag onto the tape and you have a Thiele substrate hiding inside a TM, like a person crouched inside a piano. The piano is still a piano. The person is still a person. What you encoded it onto doesn't change what it is. The separation that μ buys has no analog in the bare classical signature (`mu_not_function_of_bare_observable`).

3. Classical computation is the projection, and the projection is not invertible. Run a Turing-machine trace inside the substrate, same tape, same state (`thiele_simulates_turing`). Now forget the cost ledger and the cert flag, and you are back to the Turing machine you started with. That forgetting is a function, one classical image per substrate state (`degenerate_projection_theorem`). Now try to climb back up. Every classical state has more than one substrate preimage (`fiber_has_two_preimages`), so to lift a classical state you have to supply the cost schedule, the graph state, and the cert flag by hand. You can't extract them; they aren't in the classical state to extract. `D4_strictness` exhibits substrate states with no classical preimage at all. The two directions don't match: forgetting is forced and unique, lifting is a choice you make from outside, and the same computable functions run on both sides. That asymmetry is generic (drop a field from any model and you get it), so it colors the picture without doing the work. The part that does the work is that the cost-on-certification rule lives in the step relation and the classical signature has no field to carry it. Hold the trust fixed and A2 is the only rule that prices the certification flip exactly (`a2_equal_trust_substitution_payoff`); once you're charging the flip, the charger isn't yours to pick, it's pinned. What's still open is the prior question, whether certification is the event that has to be charged at all, instead of head-reversals or

multiplications or some other step-event, and I haven't closed that one. So this step pins the price of the flip, not the choice to bill the flip. The TM case is the one I proved (`thiele_simulates_turing`, `degenerate_projection_theorem`; the latter pins the projection as a quotient, identifying two states exactly when they agree on every field it keeps). The climb-back-up has nowhere to stand.

4. What holds each step up. Step 1 is a sentence about what a classical step relation can carry, witnessed by `cert_not_function_of_forget`. Step 2 follows from step 1, witnessed by `mu_not_function_of_bare_observable`. Step 3 is mechanised in Coq, theorem names above. The asymmetry of projection versus lift is witnessed by `fiber_has_two_preimages`. If a step reads like metaphor to you, the metaphor has to be living inside one of these witnesses, and every one of them is a Coq name carrying that step. Nothing in the chain does its work on a handshake. The names are where the weight sits.

The one that got me too. The instinct everyone has, me included: A2 can be enforced in software on a TM, so the substrate distinction is a hardware/software boundary, not a fundamental one. Here's the one sentence that pulled me out of it. A Turing machine cannot refuse to execute a buggy A2-simulator. A Thiele substrate cannot execute one. Load a Thiele simulator onto a TM with a bug, a program that certifies without incrementing μ. The TM runs it faithfully because its step rule has no field for A2 and cannot detect the bug. Load the same buggy program onto a Thiele substrate and the bug has nowhere to live. The only instruction that flips cert is `CERTIFY`, and `CERTIFY` charges $S(\delta) \geq 1$ in the same step that sets the flag. Flip-cert-and-skip-the-charge isn't a transition the substrate has. There's nothing to trap, because there's no free certify to attempt. It isn't that A2 sits inside the simulator as an interpreted step that could be skipped. A2 is the transition law itself. Subsumption is a step-rule claim, not a software-layer claim. "Thiele is simulable on a TM" is true and is not the question. "Thiele's step rule can be written down on a TM" is the question. The answer is no. None of that machinery is mine, and I want to be clear about which part is. The trick of making a rule unbreakable by refusing to let any object exist that breaks it is old and everywhere. A `CertificationSystem` is built so it cannot come into being at all unless the cost obligation (`cs_cert_costs`) is already satisfied, so there is no half-built instance that certifies for free and gets caught later; the illegal thing was never constructible in the first place. That is the same move CHERI makes when a pointer cannot be forged outside its capability, and the same move a TPM makes when a key cannot leave the chip in the clear. I borrowed the move. It is good engineering and it has a long pedigree, and the kernel using it is what keeps me from quietly writing down a substrate that lets A2 slip. So the shape is shared, and I am not pretending I invented the lock. What I chose is what the lock guards. CHERI guards a pointer, a TPM guards a key, and I pointed the same kind of lock at the one thing nobody had thought to charge for: the moment a computation commits to a certificate, with μ the receipt it cannot tear up. The mechanism is theirs. The thing it is aimed at is the whole bet.

These four steps are the entire foundational claim. They do not require 51 opcodes. They do not require an FPGA. They do not require the CHSH result. Constructively witnessing the substrate needs only two opcodes from the scaffolding: any classical compute primitive, so subsumption has something to project to; and one opcode that flips certification, so A2 has something to enforce. In the

51-opcode realisation, `instr_certify` is the only instruction that flips the cert-flag (`vm_apply_certified`). The cost-ledger and the cert-flag together are the proved-minimal extension over (mem, regs, pc): cert is provably not a function of (mem, regs, pc, μ), and μ is provably not a function of (mem, regs, pc, cert), so you can't pull either one out of the rest (`P_full_is_minimal_complete_extension`, via `cost_model_cert_necessity` and `cert_model_mu_necessity`). Drop either and certification stops being determinate. Two fields, and you need both. Forty-nine of the fifty-one scaffolding opcodes are exploration of what the substrate can express. Two are required.

The rest of the document is downstream of the four steps. The CHSH↔NPA equivalence (Section 16) bakes the quantum-realisability condition into the step rule that the `CHSH_LASSERT` opcode realises. A chain of Coq theorems (`column_contractive_check_witness_sound`, `column_contractive_iff_quantum_realizable`, `chsh_lassert_no_trap_implies_quantum_realizable`) proves the integer-arithmetic check is equivalent to the Q_1 PSD conditions on the NPA moment matrix. Section 16 lifts that to Q_{1+AB} via a four-slice Schur cascade. Two levels of NPA, not the whole tower. The proof tree contains no physics premises. The three-layer realisation (Section 14) is a Coq kernel with zero Admitted in the active proof tree, an OCaml runner extracted from that kernel, and a Kami RTL design synthesised through Bluespec and yosys to a Xilinx Kintex-7 FPGA. Bisimulation proofs cover the 47 synth-realised opcodes. The four Q_{1+AB} cert-opcodes live in the Kami HW abstraction with kernel-equivalence proven but not in the synthesized Verilog (silicon-budget, not semantics), contributing the OCaml/RTL opcode-set slack of 4 the parity tests tolerate. None of the 51 opcodes is foundationally required. The 51 is engineering. The substrate is the thing.

Vocabulary for the rest of the document is built in Section 2. Experts can skip it. Falsification conditions for every claim are in Section 22. Open pieces are in Section 24, not hidden.

Contents

1 Let me be straight with you

What I am is curious, past the point of good sense. I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go. I am not a programmer and I was never formally taught any of this. I was not trying to build a formally verified CPU. I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it. Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own. I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had was not a strategy, it was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

The method underneath was crude and it held. I wrote a long structured spec in JSON (a plain-text format for describing data) as a context document, fed it to Copilot, and had it build one Python file per concept, one at a time, each with its own tests, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. (The basic furniture of category theory; the first three I build from scratch in Section 11, functors and monoidal products in Section 2.) It came together in pieces and never in chapters, the way a picture resolves out of static instead of getting drawn in left to right. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place, the night I quit borrowing other people's runtimes and started writing my own around it, with Z3 (a constraint solver out of Microsoft Research) wired into the middle.

Then within weeks the thing I had built to draw pictures started running physics, on the exact same primitives, without my touching a line of it. I plugged in Newton's laws half expecting nothing and it ran. I did not plan it and I did not earn it. I discovered it, the way you find a room in your own house you did not know was there. Somewhere in those weeks I crossed from debugging why one pixel would not show up to understanding that I had quietly built a categorical computing engine, and that the rendering had only ever been the first thing I aimed it at. The pixel did eventually show up, for what it is worth.

I kept going, because by then I could not not keep going. I built a small custom language for describing scenes (the name for that is a domain-specific language, a DSL), then a program that read the language and caught my mistakes in it (a linter), then a way to type commands at the whole thing from a terminal (a command-line interface, a CLI). The night the engine first ran a program declaratively, meaning I told it what should happen and let it work out the how instead of my spelling out every step, I told Julie I thought I was onto something. She said okay.

Then I hit a wall, and it is the wall this whole document is about. Z3 could settle the small questions and fell over on the exact ones I needed answered, which left me staring down an infinite hallway of guess-and-test with no proof waiting at the end of it, ever. One night, not even working, I was watching a video about Flatland, the old story about a creature living in two dimensions who cannot so much as picture a third. And it landed on me sideways, all at once, the way the real ones always do. I said it out loud: I get it now. There is another axis. Classical computation only ever sees what it can read and write off its own surface, and there is a whole direction it is blind to, not because it is broken but because it was never built with an eye that points that way. I opened a new repo that same night and called the thing in it the Thiele Machine. It was a placeholder, the name you give a folder so you can save it. It stuck, and it later turned out to carry more than I meant by it, but that is its own story and I get to it further down.

What survived the whole trip from there to here is not the code. Not one line of it. By the time the Coq proofs finally landed, every line of that first renderer had been thrown out and rewritten at least twice, probably more, and good riddance to all of it. What survived was a single architectural commitment I made early and never once let anyone talk me out of: category theory is the state itself, the actual thing the machine runs on, not a metaphor laid over some realer state underneath. The `Category` structure that came together in that first week and first ran morphisms correctly at two in the morning on January 22 is, in different words and different syntax, the same object `vm_graph` is in the 51-opcode realisation. Finding the different words took fifteen months. I did not write that code, not then and not now. I directed the models and refused everything they handed me until the morphisms composed the way the shape said they had to. The morphisms in that early code were objects that remembered where they came from; the morphisms in the realisation are objects

that remember where they came from and that you can walk up to and challenge with MORPH_ASSERT. The receipt grew teeth. But the shape itself never moved. The shape was the substrate the entire time, and everything else, every renderer and language and proof, was me slowly learning to see what I had already built.

I chased that one question for fifteen months. The renderer worked in January. Physics fell out on the same category structure within weeks. Formal verification was six months and a graveyard of dead ends further on. The hardware came after that. Every step pulled back to show another layer underneath that I had not known was there, and I mean that as plainly as it sounds: I did not know what a Turing machine was when I started, I did not know what Coq was, I did not know most of the words in this document existed. I learned each one the same way, by walking face-first into a wall that needed it and not being let past until I had it. The needing always came first. The learning came after, dragged along behind the needing because it had no say in the matter.

Large language models were my hands the whole way through, and for most of the early stretch they were useless hands, because there was nothing like this for them to draw on: no reference implementation, no prior project to point them at, no Stack Overflow answer waiting at the bottom of a search. I had to build every scrap of the context myself before the tool turned into anything at all, and that alone took months. Once the context was there it became a real force multiplier, but I want the division of labor exact, because it is the thing people most want to get wrong about work like this. The ideas, the architecture, the direction, the demand that the thing be true: those are mine, all of them, no asterisk. The symbol-pushing, the actual Coq and the actual math, the models produced and the proof checker judged. The proofs compile or they do not. I refused every one that did not. Coq is an obedient idiot. It checks the proof matches the statement. It does not check that the statement means what you think it means. A theorem can claim "every cat that is also a prime number has whiskers" and the proof can be three characters long and Coq will nod and stamp it. So I read every statement. The proofs you can't fake. The statements you can fake by accident, which is the more dangerous failure mode. I refused both kinds.

A word on what I actually do, because it is stranger than the usual split and it runs under everything after it. I do not write code or math by hand. I do not pick constants. I do not grind through derivations. I have never written a line of code on paper or in a notepad in my life. What I do is see, and direct, and refuse. When I take hold of an idea it does not stay a point. It opens up. It becomes a shape I can blow up and turn over and take apart, lay across another shape to find where the two of them touch, run forward and then back to where it started, zoom into until it is the only thing in the world and pull back until it is one dot in a field of other dots, and the whole time it stays wired to everything I have ever connected it to, the right connections and the wrong ones both, all of them live at once. I am not reading that field neutrally, the way a search would. The thing doing the looking is me, the whole of me, my gut and my

mood and my stubbornness picking what lights up and which way I move through it, and that is the part you cannot lift out without turning me back into a search engine. So I see where a thing has to fit, the shape it is obligated to have, usually well before I could write it down in a single symbol. Then the model gives me the symbols, the proof checker takes them or throws them back, and I refuse anything that will not compile or does not match the shape I am holding. That refusal is the one thing in this entire document that is mine and only mine: the demand that it be true, and the throwing-out of everything that wasn't.

I am telling you all of this because it is not background color, it is the engine. I did not come up through anywhere that hands you a foundation and says trust us, this part is settled. No professors, no colleagues, no field full of authority to lean back on when a claim got heavy. From the outside that is a deficit. From the inside it is the whole reason any of this is solid: when I meet a claim I either find the proof or I do not believe it, and that goes double for my own, because my own are the ones I most want to wave through. It is not a discipline I picked up to look rigorous. It is how I am built. It is the only way I know to hold a thing and actually trust that I am holding it.

So everything in here gets built from the ground up, and not as a courtesy to you. I build it because I do not believe I understand a thing until I have built it with my own hands, watched it stand up on its own, and tried to knock it over. I built this document the exact way I built the machine, brick by brick, refusing each brick until it held weight. That includes the parts you already knew cold before you opened this. I did not skip them for you, and I could not skip them for me.

If you already know this material and the review of basics grates on you, skip Section 2, which is the vocabulary I had to go look up to do any of this, and jump straight to Section 9, where the formal machine state starts. But if you catch something in the basics that is wrong, I want to know, and I mean that as a real ask and not a polite one. Nothing in here is correct just because I wrote it. Especially the parts I wrote.

A note before the technical content, on what I do and do not know about prior work. I started this with no background in computer science, math, physics, quantum mechanics, hardware, or category theory. None. I didn't know what a Turing machine was. I didn't know what Coq was. I built this alone. Nobody mentored me or walked me through the field. The only person who has touched this work in any way is a friend who taught me how to use GitHub and set up an IDE. Every name, every comparison, every paper I went looking for, came from me searching on my own. I do not trust my own reading of any of the sources I found. When I read a paper, my default assumption is that I don't get it, or I'm reading it wrong, or I'm missing the part where it says something different from what I think it says. This is a list of labels, not a literature review; I am not qualified to write one.

Here is where I think this sits. "You can't get structure for free" is not a new thought. Landauer found it in erasure. Wolpert and Macready found it in regret. Kolmogorov

found it in description-length. Cook and Levin found it in search. Four findings, four fields, nobody talking to anybody. Every one of them is a property of a finished object, or an average, or a measure, or a limit. Not one of them is a law of a step. A2 is a law of a step. There was a chair at that table nobody had sat in, because computation is a dynamics and nobody had charged the moment a dynamics commits. That is the chair A2 takes. Rovelli's "The Thermodynamic Cost of Choosing" (2024) is the closest anyone has come to sitting down, and he does it with a temperature, which is the thing that buys him a forced floor. A2 drops the temperature and gives that forcedness up: it does not say the price is forced, only that given a system that prices the commitment, A2 is the only exact price.

Here are the labels I went and looked into, with one sentence each on what the label is and why I do not think Thiele reduces to it. Each is a claim inviting correction; if the comparison is wrong, tell me.

- Linear logic (Girard). What I read: a kind of logic where assumptions are consumed when you use them, like physical resources. Compiled into modern programming languages, it shows up as type rules that say "this value can only be used once." My reading of why Thiele is not a restatement: linear logic constrains what the program can write down. Thiele's A2 rule constrains what the machine's step function does. Same intent. Different layer.
- Proof-carrying code (Necula). What I read: when one machine downloads code from another, the code can carry a small proof that the host can check before running it. The proof is data accompanying the program. My reading: Thiele's cost-on-certification is enforced by the machine itself, not by a proof bundled with the program. The program does not carry receipts. The machine produces them. Foundational PCC (Appel) is closer in spirit: proofs about a small trusted base, not certificates bundled with the program. I haven't read enough Appel to say more.
- Session types (Honda). What I read: type systems that constrain how two or more concurrent programs talk to each other so the conversation cannot go off-script. My reading: Thiele tracks structural facts a single computation has paid for. It does not constrain inter-program communication. Different problem.
- Bennett's reversibility argument. What I read: Charles Bennett (1973) showed that you can in principle organise any computation so it does not erase information mid-stream. Operations that throw information away are the irreversible ones; everything else can be undone. My reading: in the 51-opcode realisation, the operations that commit information irreversibly (CERTIFY, MORPH_ASSERT, EMIT, REVEAL) are charged even at $\delta = 0$, a floor no program can waive; the reversible operations are designed to be free at $\delta = 0$ for the same reason Bennett identified. Positive cost is the larger set, since any instruction can carry a voluntary δ; the mandatory floor is the part that tracks ir-

reversibility. The shape matches. The dollars-and-joules calibration is a separate named hypothesis.

- Landauer's bound. What I read: Rolf Landauer (1961) argued that erasing one bit of information must release at least $kT \ln 2$ of heat (Boltzmann constant times temperature times the natural log of 2) into the environment around the chip. My reading: this is the physical-units side of Bennett's argument; in Thiele the structural side (the substrate tracks each positive-cost transition via μ) is proved, and the conversion to joules is a named hypothesis (see Section 17). I treat Landauer's claim as motivation, not as something the proofs rest on. Motivation pays no bills in Coq.
- No Free Lunch (Wolpert and Macready, 1997). What I read: averaged over every possible problem, no search method beats blind guessing; whatever an algorithm wins on one set of problems it gives back on another. My reading: this is the same "structure isn't free" intuition, but it lives as an average over a whole problem space, and the cost is refundable. You buy an edge on your problems by spending prior assumptions, and you pay for it somewhere else. A2 charges one commitment event in one run, with no refund anywhere. Same family. Different shape: an average, not a per-step floor.
- Kolmogorov complexity (Kolmogorov 1965; Solomonoff and Chaitin reached it independently around then). What I read: the amount of structure in an object is the length of the shortest program that prints it, and most objects have no short description, so you cannot squeeze below that floor. My reading: same "structure isn't free" intuition, but it is a static property of a finished object. No steps. No moment of paying. A2 charges the moment a running machine commits to a certificate. Same family. Different shape: a property of a string, not a law of a step.
- Cook-Levin and P vs NP (Cook 1971, Levin 1973). What I read: checking a solution can be cheap while finding one looks expensive; whether finding is genuinely harder than checking is the open P-vs-NP question. Cook-Levin is the theorem that SAT is NP-complete, not a proof that search is hard. My reading: same find-versus-check intuition, but it is an asymptotic, worst-case, still-open statement about how difficulty grows with problem size. A2 charges a constant on the one commitment step in a concrete run, whether finding it was easy or hard. Same family. Different shape: an open scaling gap, not a per-step floor. I am not claiming anything about P vs NP; see "What this project is not."
- Rovelli's cost of choosing (Rovelli 2024, *The Thermodynamic Cost of Choosing*). What I read: committing to one branch of a binary choice at temperature T dissipates at least $kT \ln 2$, the same floor Landauer found for erasure, now charged on the act of deciding rather than the act of forgetting. My reading: this is the closest existing thing to A2's claim. It charges a commitment, not a finished object or an average. But it is physical: indexed to a temperature, living in thermodynamics.

A2 charges the commitment in a substrate, with no temperature and no physics, blind to how the witness was found. Closest neighbour. Not the same chair.

- CerCo (certified cost annotations through a verified C compiler). What I read: take a C program, run it through a compiler that itself has been formally proved correct, and you can attach cost annotations to the source program that the compiler preserves all the way down to machine code. My reading: CerCo's cost lives as decoration on the source code that survives compilation. Thiele's cost lives in the step rule of the abstract machine, and the path down to hardware is a step-by-step bisimulation. Different mechanism. Different layer.
- Kami-RISC-V. What I read: a verified processor design written in Coq using the Kami framework that targets the RISC-V instruction set, which is an open-source CPU architecture (a published list of instructions any chip designer can implement). My reading: I use the same Coq-to-Kami-to-Bluespec pipeline that the Kami-RISC-V work pioneered. The instruction set is mine. The cost discipline is mine. The proofs are mine. The toolchain is shared. Nothing else is.
- NPA hierarchy (Navascués-Pironio-Acín). What I read: a way of bounding what kinds of correlations between two separated experiments are achievable using quantum mechanics. The bound takes the form of a sequence of polynomial inequalities, each tighter than the last. The bounds are usually checked by an external numerical solver. My reading: Thiele bakes one of those bounds into the machine's step rule itself. The Z-arithmetic check that CHSH_LASSERT performs is proved equivalent to the Q_1 NPA condition by a chain of Coq theorems (Section 16), and lifted to Q_{1+AB} via a four-slice Schur cascade. Two levels of NPA, not the whole tower. It is operational enforcement at the step level, not a bound stated externally and verified out-of-band.

I read enough of each to convince myself Thiele is not a restatement, but I cannot say that with the confidence of someone who has actually read the field. The only things I will tell you with confidence are the ones where the argument actually closes: the structural axis is first-class substrate state, realised in the scaffolding by its own opcodes; μ is proved unique on every reachable substrate state by `mu_is_initial_monotone`; No Free Insight follows from a one-step cost rule on any abstract certification system by `universal_nfi_any_substrate`; the Categorical Separation Theorem (Section 12) says the structural axis cannot be retrofitted onto a classical substrate without changing what a step is; and the CHSH-to-NPA biconditional drops out of the cost algebra without quantum input. The argument is the part I can defend; the proof corpus is just where I checked I hadn't fooled myself. The literature placement is the part where I am asking for help.

On the name. I wanted to call it the meta machine, and then Facebook bought the word "meta": a company renaming itself after a term for self-reference and draining it of any in the same motion. So I went one level up, where this machine already lives.

The most meta move available, for a machine that runs on meta-level reasoning, was to put my own name on the cover and take the word back. That's the whole of it, a meta move on a meta machine. It happens to be my dad's name too, and my wife's now, so I asked them before I did it. They said go ahead.

What this project is. The Thiele Machine is a model of computation built around one idea I could not find sitting anywhere else: that a computation can be *entitled* to a piece of structure, and that the entitlement is part of the state, as real and as tracked as the numbers in the registers. Entitlement is the gap between a machine that happens to be holding a fact and a machine that has earned the right to use it, and ordinary machines only ever do the first. They keep the bytes of a decomposition, a relation, an invariant, a certificate, a narrowed set of possibilities, and they will let any later step lean its full weight on those bytes whether or not anything ever checked them. The Thiele Machine carries, right alongside the data, the question of whether the trace actually paid for the right to treat that structure as established, and it will not let a step use what the trace has not earned. No Free Insight is the first theorem that falls out: a computation cannot end in a stronger certified claim than it started with unless the strengthening passed through the machine's explicit cost ledger. It's a short argument, and I ran it through Coq so a bad step couldn't hide on me. The ledger is called μ. It never goes down. If you go from uncertified to certified, μ went up. Scope: raw μ is a receipt that certification work was paid for. It is not, by itself, a universal depth score for the meaning of the claim. The stronger semantic reading only applies when the certification comes with an explicit narrowing witness. That distinction is not cosmetic. It is the hinge between an honest cost ledger and an overclaim.

The whole claim, in three lines. Each line is a thing I am willing to be wrong about by name. Wrong conclusion means wrong premise: the law you learned in tenth grade and have not been asked to unlearn since. There is no third move.

1. If flipping a certification bit costs ≥ 1 (A2).
2. And classical machines have no state slot to enforce this constraint at the step-transition level.
3. Then classical machines are a structurally blind, strictly lossy projection of a substrate that does.

That is the substrate claim. A napkin holds it. The rest of this document is the same argument unfolded with kernel citations and a worked instruction set: same content, more words.

Where to look. The first line is A2, and A2 is two clauses and an integer. A step that flips cert costs at least one, the rule fits on the napkin with the conclusion, and the check is finite, public, and yours to run. The second line is the kind of fact you check by writing down a different Turing machine, which is what I did. Add a state field the

step rule reads to see whether the last move flipped cert, and you've left the classical world, which was the claim. Keep it classical and there is no field that does the job. There aren't any that do. That is not philosophy, that is the signature the field itself carries the moment it is written down.

The next move is the simulator: decline the field, run my rule as a program on the tape, increment cert in the program. Granted. The TM produces every trace I produce, beat for beat. It produces the buggy ones too. Faithful is what TMs do. The substrate cannot run the buggy version. That gap, same trace minus the *can't*, is line 3. Simulation is not the escape from the projection. Simulation *is* the projection with the loss made operational. Reaching for the simulator doesn't get you out; it hands you the projection with the loss now running where you can watch it.

That is the whole of it: three lines and a napkin, and the third line is the one I keep circling back to, because it is the one that does the work. Same trace, minus the *can't*, and that *can't* is the entire distance between the two machines. Coq is a notebook of receipts for that distance. The arithmetic comes out the way it comes out, no matter how badly I might have wanted it to come out flattering to me.

Substrate vs scaffolding. Two layers run through the rest of this document, and I kept conflating them myself until the proofs forced the line; confuse them and the theorems quietly stop meaning what they say. The *substrate* is the A2-respecting computational layer: a state type, a step relation, the μ ledger, and the rule that flipping certification false-to-true costs at least one. The *scaffolding* is everything that makes the substrate runnable, verifiable, and synthesizable in practice: the 51 opcodes, the Python reference kernel, the extracted OCaml runtime, the Bluespec hardware, the simulator harness. The substrate-level theorems (No Free Insight, μ initiality, the structural-axis undecidability theorem in Section 19) are facts about substrates: they hold for any A2-respecting computational system, no matter what instruction set realizes it. The scaffolding-level constructions (the opcode semantics, the bisimulation between Coq and Verilog, the synthesised gate count) are how this particular substrate is made concrete enough that you can boot it, audit it, and run it on silicon. The 51 opcodes are one realization of how a substrate state evolves; the substrate-level claim that follows from them is independent of the realization. So read every theorem here with that distinction in hand: a substrate-level result is about substrates, and the 51-opcode VM is one instance where the substrate's existence is pinned down by construction. The distinction carries weight for the rest of the document; let it collapse and the substrate-level results shrink into parochial claims about one instruction set, which is exactly what they are not.

What "new model of computation" means here. The phrase is ambiguous, and I have watched it get read three different ways, so here is what I mean by it. When I say new model of computation, I mean a new substrate, not a new computability class: the Thiele Machine computes the same functions a Turing machine computes, and

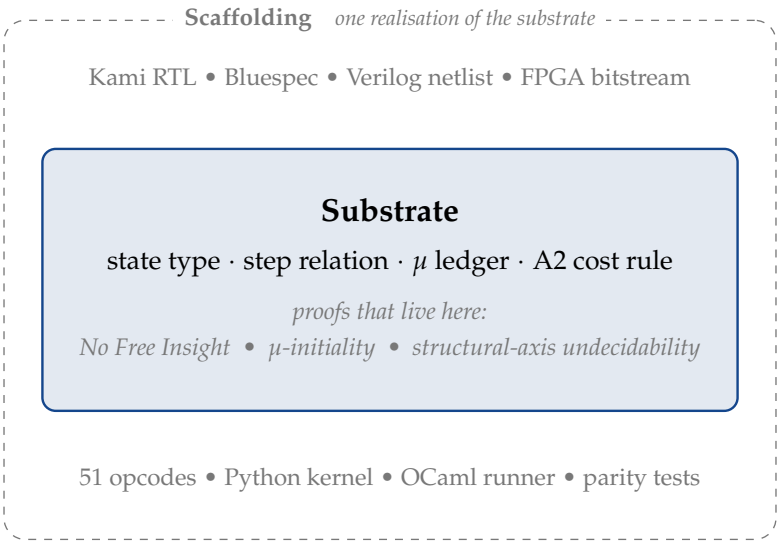


Figure 1: The substrate-versus-scaffolding distinction. The substrate is the A2-respecting computational layer where the substrate-level theorems live; they hold for any A2-respecting system regardless of instruction set. The scaffolding is everything that makes this particular substrate runnable, verifiable, and synthesizable. Collapsing the two is the most common misread of the document.

what is new is where the certification law lives. Classical programs embed faithfully (`D2_faithfulness`). The classical compute surface is conservative under the embedding (`D3_conservativity`). What is new is the substrate. Here is the structural content of "new." A2 lives in the step rule as the transition law, and with the trust held fixed it is the only rule that prices certification exactly. Miss a cert-flip and the floor fails on a one-step trace. Charge a non-cert-flip and you overpay. So the only exact, non-overcharging substitute already *is* A2, and that cuts in both directions (`a2_equal_trust_substitution_payoff`). The state it runs on, $(\mu, \texttt{cert})$, is the proved-minimal extension over (mem, regs, pc): cert is not a function of the rest, μ is not a function of the rest (`P_full_is_minimal_complete_extension`). A classical step rule has no field to carry A2 as a law. It can only run a fallible checker of it, the kind a buggy simulator skips without noticing. That is what makes classical computation the derivative thing here, and it is proved, not stipulated. There is a projection-and-lift asymmetry too, but it is generic. Drop a field from any model and the same shadow falls. It is not what does the work.

The renderer experience is the entry point. I started by trying to build a 3D engine where category-theoretic objects were the runtime objects. Modules. Morphisms. Composition. Not data structures simulating those things. The actual things. The classical CPU saw none of that. It shuffled bytes that happened to encode the structural state. Z3 saw formulas that happened to refer to the same shapes. The structural facts the engine was computing with were in the architecture, not in the data. That is the concrete shape I had to formalise. The substrate this document describes is what came out: compositional structure as state, not as an approximation written on top of state.

The formal version of that shape is a directional structural theorem. `thiele_simulates_turing` lifts every Turing-machine trace into the substrate via `lift_config`: take any TM config, set $\mu = 0$, you have a Thiele config running the same tape. `degenerate_projection_theorem` closes the loop in the other direction: the shadow projection from Thiele to classical is the classical-equivalence quotient map (two states land on the same image exactly when they are classically equivalent), classical traces respect the quotient, and the projection is strictly lossy. So one direction is canonical (Thiele $\rightarrow$ classical: there is exactly one forgetful map onto the classical fields), and the other direction is not (classical $\rightarrow$ Thiele: every classical state has multiple Thiele preimages, witnessed by `fiber_has_two_preimages`). Any lift back requires picking a value for the dropped fields. That pick is external data. Not extracted from the classical state. Appended to it.

The asymmetry is sharpened by the three irrecoverability theorems: `vm_certified`, `csr_cert_addr`, and μ relative to the bare three-field classical observable are provably not functions of the classical projection. Every Thiele machine has a Turing projection; no Turing machine has a canonical Thiele lift. That directionality is generic to any lossy-projection pair, so on its own it buys a richer extension, not priority. What is not generic, and what makes the classical fragment derivative rather than a parallel world: the structural axis is not derivable from the classical projection by any clever predicate, A2 is the exact-least certification-pricing law (pricing exactly without overcharge is equivalent to being A2: `a2_equal_trust_substitution_payoff`), and $(\mu, \texttt{cert})$ is the irredundant lattice extension over (mem, regs, pc) (`P_full_is_minimal_complete_extension`). That is what those two fields buy, and what the classical signature has no room to state. Each step has its own named theorem under it, so you can check the argument one theorem at a time instead of taking my word for it.

What this project is not. It is not a claim that I solved P vs NP. It is not a theory of everything. It is not a claim that computation is physics or that physics is computation. The CHSH algebraic result is pure math. The Landauer connection is motivation. That is the full extent of the physics content here. The rest is bookkeeping.

2 Things I had to learn first

I had been around computers my whole life and paid attention to people like John Carmack on how graphics actually get made, but I came in with none of the formal fields this work turned out to need. By the time the proofs compiled I had needed to learn enough of half a dozen of them that I have to stop here and put down what those fields had words for, before the rest of the document starts using them.

Everything below is what I understood after I went and read about it. If I have anything wrong, tell me. I am not claiming to teach you these subjects. I am putting down what I had to take from each of them to get this work to compile. If you already know one of these subsections, skip it.

2.1 Bits, bytes, and what a CPU sees

A bit is one digit that is either zero or one, the smallest fact a machine can hold. Everything else is bits stacked up. Eight of them stacked make a byte, which lands you any number from 0 to 255, and the first thing that genuinely surprised me is that the same byte is also a letter: a convention called ASCII (American Standard Code for Information Interchange) simply agrees to call 65 the letter A, 48 the digit 0, 32 a space. Nothing in the byte knows which it is. The meaning is a convention laid on top of the data, and which convention you read it through is your choice, not the byte's. Hold onto that, because it is the whole argument of this document shrunk down to one byte: the data sits there inert, and everything that matters is in the structure you read it through. I learned the byte part in February.

A word is a chunk of bits the CPU treats as one unit. The kernel math uses 64-bit words (each word can hold any number from 0 to $2^{64} - 1$, about 18 quintillion). The synthesised hardware uses 32-bit words (each word holds 0 to $2^{32} - 1$, about 4 billion). A word is just a sequence of bits with a fixed width. The substrate does not care which width.

A register is a single named storage slot inside the CPU that holds one word. The 51-opcode VM has 16 of them, named r0 through r15. Registers are the fastest place to read or write a value. Memory is a longer list of word-sized slots, accessed by index (the "address"). The 51-opcode VM has 128 words of memory. That is the entire address space. The substrate has no address space.

The program counter (PC) is one specific register that tracks where in the program the VM is. After every instruction the PC moves forward by one unless the instruction explicitly says jump to a different address.

A stack pointer is a register that points to the top of a region of memory the program uses for nested function calls. CALL pushes a return address there; RET pops it. In the 51-opcode VM, r15 is the stack pointer by convention. The convention is mine. The substrate is not consulted.

A port is a named channel the 51-opcode VM uses to communicate with the outside world. READ_PORT brings a value in, WRITE_PORT sends a value out. The kernel does not model the outside world. It just charges μ for the communication and accepts the value the channel encoding declares. The outside world is welcome to comply. The substrate has no ports.

Modular arithmetic means counting wraps around at a maximum. If the maximum is 16, then 17 wraps to 1 and 19 wraps to 3. "Modulo 16" or "mod 16" means "wrap at 16." The 51-opcode VM wraps register indices modulo 16 and memory indices modulo 128. If you write r17, the kernel treats that as r1.

Bitwise operations work on the bits of a value rather than treating the value as a

number. Bitwise AND of two bits is 1 only when both bits are 1. Bitwise OR is 1 when either is 1. Bitwise XOR (exclusive or, sometimes written $\oplus$) is 1 exactly when the two bits differ. The bitwise version of these does the operation in parallel for every bit position. Bitwise-AND of two 64-bit words gives a 64-bit word. Bit i of the result is the AND of bit i of each input. The bits do not consult each other. The substrate does not see bits.

A shift moves the bits of a value left or right. Left shift by 3 (sometimes written $x \ll 3$) moves every bit three positions to the left, which is the same as multiplying by $2^3 = 8$. Right shift divides by a power of 2. Same idea, opposite direction.

Popcount is the count of 1-bits in a value. Popcount of the byte 11010100 is 4. Sometimes called the Hamming weight; the two names refer to the same thing. Pick one.

Hexadecimal is base 16. Each "digit" is one of 0 through 9 or A through F, where A means 10 and F means 15. Hex is convenient for working with bytes because every two hex digits make one byte. Numbers prefixed with 0x are hex: 0xFF is 255, 0xCAFEEACE is the constant unlock key in the Logic Engine accumulator. It had to be something.

A trap, in the hardware sense, means the machine routes execution to a fixed address instead of advancing normally. It is the hardware equivalent of "this went wrong, go to the error handler." The 51-opcode VM traps LASSERT length mismatches to address 0xF00.

An ISA (instruction set architecture) is the list of all instructions the CPU recognises, plus their formats and costs. Each instruction is identified by an opcode (a numeric tag the hardware decodes to choose which behaviour to run). The kernel has 51 opcodes. The substrate has none. The substrate has a step rule.

2.2 Math notation I will use without explaining it twice

I read the math symbols out loud the way I learned them, which is whatever the source I was reading suggested they were pronounced as. If you have a different convention in your head, here is what I mean by them.

$\mathbb{N}$ is the natural numbers: 0, 1, 2, 3, and so on. $\mathbb{Z}$ is the integers, which extend the natural numbers with negatives: $\ldots, -2, -1, 0, 1, 2, \ldots$. $\mathbb{R}$ is the real numbers, all the way along the number line including fractions and irrationals like π and $\sqrt{2}$.

$\in$ means "is an element of." $x \in S$ reads "x is in S."

$\subseteq$ means "is a subset of" (every element of the left side is also in the right). $\subsetneq$ means strict subset: a subset but not the whole thing.

$\cup$ is set union, the set containing everything in either side. $\setminus$ is set difference: $A \setminus B$ is everything in A that is not in B.

$\times$ between two sets is the Cartesian product: $A \times B$ is the set of all (a, b) pairs with $a \in A$ and $b \in B$. Functions from a set A to a set B are written $A \to B$.

For a finite set S, $|S|$ means the number of elements in S. For a real number x, $|x|$ means the absolute value of x (its distance from zero).

$\sum_{i \in T} f(i)$ is the sum of $f(i)$ taken over every i in T. The big sigma is the summation symbol.

Δx means "the change in x," usually the difference between a final and an initial value of x.

$\Rightarrow$ is logical implication. $P \Rightarrow Q$ means: if P is true, then Q is true. $\iff$ is the biconditional, also called "if and only if." $P \iff Q$ means both $P \Rightarrow Q$ and $Q \Rightarrow P$.

$\forall$ is "for every." $\exists$ is "there exists." $\forall x.\ P(x)$ reads "for every x, $P(x)$ is true." $\exists x.\ P(x)$ reads "there is some x that makes $P(x)$ true."

$\langle X \rangle$ around a random variable X means the expected value, the long-run average value of X. I picked up this notation reading physics papers about Bell tests.

$\circ$ between two functions is composition: $(f \circ g)(x) = f(g(x))$, first apply g, then apply f. The same symbol appears in category theory for arrow composition.

$\oplus$ in this document is bitwise XOR. $\otimes$ has two distinct meanings I cannot consolidate without breaking source quotes. In Section 10 it is the MORPH_TENSOR operator (concatenation of two coupling lists into one); in Section 16 it is the tensor product on quantum-mechanical operators. I flag the second use again where it appears.

I write A^T for the transpose of a matrix A (rows and columns swapped). I_n is the $n \times n$ identity matrix, which has 1 on the diagonal and 0 everywhere else.

2.3 Coq, in just enough detail

Coq is a proof assistant. It is a programming language. The type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid. Section 20 has the longer build, including why I trust it. Here is the vocabulary that runs through the rest of the document.

A Theorem in Coq is a named statement followed by a proof. Lemma is the same construct, conventionally used for intermediate results. Definition introduces a named value (a number, a function, a record, anything). Inductive defines a type with a finite list of named constructors, the only ways to build a value of that type. Coq's natural numbers, for example, are an inductive type with two constructors: zero, and "successor of another natural number." Two constructors, infinitely many naturals.

A tactic is a step inside a Coq proof. The tactics that appear in this document include: induction (prove the base case, then prove that if the statement holds at step n it holds at step $n + 1$), destruct (split a hypothesis into its cases), exact (provide the literal proof

term), reflexivity (prove $x = x$), simpl (simplify the goal by computing), lra (linear real arithmetic; closes goals that are linear inequalities over the reals), nra (nonlinear real arithmetic; handles polynomial inequalities), and psatz (uses Positivstellensatz certificates; it shows a polynomial is non-negative by writing it as a sum of squares). I learned each of them by needing it.

Qed is the keyword that ends a proof. It asks Coq to check the entire chain. If Coq accepts the proof, the theorem is proved.

Admitted is the keyword for "I am claiming this without proof." This is what I am not doing anywhere in the active proof tree. Every Admitted in the kernel would be a bug. Section 21 explains what zero Admitted means. And what it does not.

A Hypothesis (a Coq keyword inside a Section) is a named premise: a statement we accept as given without proof. Hypotheses surface as $\forall$-quantified premises in the conclusion. I use Hypothesis only at named bridge points (the physical-units calibration in Section 17 is the load-bearing example). An Axiom is the global version: a top-level claim accepted without proof. The kernel tier has zero project-local Axiom declarations. The policy is bookkeeping discipline, not faith.

Print Assumptions is the Coq command that lists every Axiom and Hypothesis a given theorem ultimately depends on. Running Print Assumptions on the kernel tier shows only standard Coq stdlib axioms (functional extensionality, classical logic for the reals). No project-local axiom appears anywhere.

A total function in Coq always returns a value. Coq requires every function definition to be total. `vm_apply` is total. Every state-and-instruction pair has a defined result. Including the error case.

"Parametric in K" means the proof works for any value of K. The kernel proofs are parametric in `REG_COUNT` and `MEM_SIZE`, so the same theorems apply at any hardware size. The substrate is parametric in everything the scaffolding picks.

2.4 Category theory: the part I needed beyond Section 11

Section 11 builds Category, Object, Morphism, Composition, Identity, and Associativity from scratch. That build is enough for the categorical-separation argument inside this machine. Here is what I had to look up beyond that.

A functor is a structure-preserving map from one category to another. Given two categories $\mathcal{C}$ and $\mathcal{D}$, a functor F from $\mathcal{C}$ to $\mathcal{D}$ sends each object of $\mathcal{C}$ to an object of $\mathcal{D}$. It sends each arrow $f : A \to B$ in $\mathcal{C}$ to an arrow $F(f) : F(A) \to F(B)$ in $\mathcal{D}$. It respects composition ($F(g \circ f) = F(g) \circ F(f)$) and identity ($F(\mathrm{id}_A) = \mathrm{id}_{F(A)}$). Think of a functor as a translation rule between two categorical worlds that keeps the laws intact.

An initial object in a category is one from which there is exactly one arrow to every

other object. The word "exactly" is doing the load here: not zero arrows, not a fistful, one. In the category of A2-respecting substrates, the Thiele Machine is that initial object, with exactly one structure-preserving morphism into every other such substrate, and the arrow runs outward from it, never toward it. That is a clean fact, and it buys less than it sounds like it should. It is real structure, but the integers are the initial object of the rings too, and nobody crowns the integers king of arithmetic on that alone. So initiality carries none of the priority weight in this document. It is here because it is true and because it is the shape the rest of the picture sits inside, not because it is the argument.

An embedding is a functor that throws nothing away. That is the whole point of putting the word here: the classical projection of a Thiele state loses things on the way out, and an embedding is the move that does not. Faithful is the precise version of "loses nothing on arrows": if two arrows in $\mathcal{C}$ land on the same arrow under F, they were already the same arrow, so F never quietly fuses two distinct things into one. Conservative is the version for invertibility: if $F(f)$ has an inverse over in $\mathcal{D}$, then f already had one back in $\mathcal{C}$, so F cannot manufacture an isomorphism that was not there. (An isomorphism is just an arrow with an inverse.) Both words are saying the same thing from two angles. The map is honest. It does not get to invent structure or hide it.

A monoidal product is a way of combining two objects, or two arrows, into one. In the category of sets, the Cartesian product $A \times B$ is a monoidal product on objects. In the category of Thiele modules, the operation behind MORPH_TENSOR is a monoidal product on morphisms. Take two morphisms with disjoint source and target regions. Pack them into a single morphism whose coupling is the concatenation. The symbol $\otimes$ stands for this in Section 10.

Yoneda's lemma is named once in the Section 1 origin story and never appears again. You can ignore it. I dropped it during the work because I did not need it for any theorem. I left the name in. That is when I first knew the work was assembling.

2.5 Bell-test vocabulary you will meet in Section 16

Section 16 builds the CHSH game from scratch. A few foundational terms thread through that section that I will not rebuild each time.

A correlation between two outcomes is the average value of their product. If Alice's outcome a is ± 1 and Bob's outcome b is ± 1, the correlation is $\langle ab \rangle$, which lies between -1 and $+1$. A correlation of $+1$ means they always match. -1 means they always disagree. 0 means there is no average correlation. A correlator is one specific correlation in a labelled setting: for example, E_{00} is the correlation when Alice's setting is 0 and Bob's setting is 0.

The marginal of a joint distribution on (a, b) is the distribution on a alone, with b averaged out. Zero-marginal means Alice's marginal gives $\langle a \rangle = 0$ and Bob's gives

$\langle b \rangle = 0$. Each outcome is $+1$ or -1 equally often, on average. The Section 16 derivation works in this symmetric slice.

A local hidden variable model is a classical-physics-style theory where Alice's and Bob's outcomes are determined by a shared random variable λ they agreed on before the game started, plus their own measurement setting. "Local" means no communication during the game. "Hidden" means λ need not be directly observable. "Variable" because λ is random. Bell's theorem says no local hidden variable model can produce certain correlations that quantum mechanics can.

A no-signaling correlation is one where Alice's marginal does not depend on Bob's setting, and Bob's marginal does not depend on Alice's setting. You cannot learn Bob's setting by looking only at Alice's outputs. Not even in principle. Most correlations we discuss (classical, quantum, and the PR-box below) are no-signaling.

Quantum-realizable, in this document, means "compatible with some quantum-mechanical state-and-measurement description." The NPA framework (Navascués-Pironio-Acín) gives a precise test for this in terms of positive-semidefiniteness of a moment matrix. The test is decidable. Section 16 builds the test.

The PR-box (Popescu-Rohrlich box) is a hypothetical correlation that wins the CHSH game with probability 1, achieving $|S| = 4$. It is no-signaling, but stronger than anything quantum mechanics allows. Nature does not provide it. It is the standard counterexample to the false claim "any no-signaling correlation is quantum-realizable."

The Cauchy-Schwarz inequality is the basic inner-product inequality $|\langle x, y \rangle| \leq \|x\| \cdot \|y\|$. It reads: the inner product of two vectors is bounded by the product of their lengths. It is one of the workhorse algebraic moves in the Section 16 derivation.

The Tsirelson bound is the value $2\sqrt{2} \approx 2.828$, the largest CHSH score $|S|$ that any quantum-mechanical strategy can achieve. The classical bound is 2. The PR-box bound is 4. Tsirelson sits strictly between classical and PR-box. Nature picked one of these three.

The elliptope, named twice in Section 23 and Section 24, is the convex set of valid correlation matrices that arise from quantum states. The zero-marginal NPA slice of the elliptope is the part the proof in Section 16 covers.

2.6 Hardware and synthesis vocabulary

Section 14 builds the scaffolding stack (Coq kernel, OCaml runner, Kami RTL) carefully. Here is the synthesis-tooling vocabulary that Section 14's third subsection uses without expanding each tool.

A hardware description language (HDL) is a programming language for describing circuits. Bluespec SystemVerilog and Verilog are two HDLs. Bluespec is higher-level: you describe rules and the compiler figures out the gate-level details. Verilog is lower-

level: you describe gates and wires more directly. Verilog RTL specifically means "register-transfer level" Verilog, which describes how data moves between registers each clock cycle. It is what synthesis tools consume. They do not eat Bluespec.

Synthesis is the process of compiling a hardware description down to actual logic gates (the building blocks of chips). yosys is an open-source synthesis tool. After synthesis, place-and-route assigns the gates to physical positions on a chip and connects them. nextpnr-xilinx is the place-and-route tool for the Xilinx FPGAs this design targets. The final step is producing a bitstream, the binary file the FPGA reads to configure itself, which prjxray's xc7frames2bit does. Five steps, four tools, one chip.

An FPGA (field-programmable gate array) is a chip you can program after manufacturing. The gates are physical. The connections between them are configurable. An ASIC (application-specific integrated circuit) is the non-programmable version: gates fixed at manufacturing time for one specific design.

A LUT (look-up table) is the basic logic primitive in an FPGA: a small memory that maps a fixed-length list of inputs to an output, programmed at configuration time to implement any small boolean function. A flip-flop is a one-bit memory cell that holds its value across clock cycles. A carry chain is specialised wiring for fast addition. Distributed RAM is RAM built from LUTs. Block RAM is dedicated RAM blocks on the chip. DSP slices are specialised hardware for digital-signal-processing arithmetic. The synthesised K325T design uses 0 DSP slices because yosys is invoked with `-nodsp`, mapping the wide multiplier in the multi-cycle witness checker (Section 14) to LUTs instead. The substrate would also accept a DSP-inferred mapping. The choice is a place-and-route runtime trade-off, not a correctness one.

A JSON netlist is the gate-level circuit serialised in a JSON text file (JSON is a standard text format for structured data). A VCD waveform is a Value Change Dump, a record of every signal's value at every time step in a simulation. The name describes the file. iverilog is an open-source Verilog simulator. It runs the design and produces a VCD output you can inspect.

A bisimulation is a precise notion of "two systems behave identically." Given the same starting state and the same input, they produce the same output state, field by field. Section 14 builds it in context. The substrate-to-scaffolding bisimulation is the path the document walks.

2.7 Other terms I use without rebuilding

Provenance, in software, means the recorded chain of where an object came from. A morphism's provenance is the specific sequence of MORPH and COMPOSE instructions that built it. You can replay the chain.

Simulation, in computer science, means one system imitating another's input-output behaviour. A Turing machine simulates a Thiele Machine when it produces the same

outputs on the same inputs, by writing the whole Thiele state out across its tape and grinding through it. That is real, and it matters, and it is exactly the thing people reach for when they want to wave the whole question away: if a tape can stand in for the state, what was the difference? The difference is that the tape carries a picture of the state, not the state, the way a photograph of a fire keeps you exactly as warm as any other photograph. The cost ledger and the A2 rule live in the step relation, not in the bytes, and an encoding copies the bytes. Simulating the substrate is not being the substrate.

Projection, in mathematics, is a function that drops information on purpose. The classical projection of a Thiele state is the one that does the dropping I keep pointing at: it throws away μ, certification, and the morphism graph, and keeps registers, memory, and the program counter. That is the exact list. The three it keeps are the three a Turing machine already has, which is the whole reason the projection lands you back in ordinary computation. The three it drops are the three that were doing the new work. Nothing about the projection is wrong; it is a perfectly good function. It just costs you everything that made the state more than a tape.

Quotient, in mathematics, is the operation of declaring two things the same when they agree on what you decided to look at, and then working with the lumped-together classes instead of the originals. The classical projection is exactly that kind of move: any two Thiele states that match on registers, memory, and the program counter get swept into the same class, even when their μ, their certification, and their morphism graph were completely different. So the projection is not just dropping fields one at a time; it is gluing together states that the dropped fields kept apart. That is the precise sense in which it loses information, and it is comforting that the math already had the word sitting there waiting.

A kernel of a map (math sense) is the set of things the map sends to a designated identity value. "Kernel" wears three different hats in this document and I would rather call that out than let it ambush you: the math sense just defined; the Coq proof core, meaning the kernel-tier theorems in Section 23, which is scaffolding I lean on, not the substrate itself; and the operating-system kernel, which I imply at most by analogy and never actually use. Whenever the word shows up, the surrounding sentence tells you which one is meant.

Embedding is a function that preserves structure and is injective. Faithful embedding additionally preserves all the structure-relevant distinctions.

An injection is a function where no two inputs map to the same output. A surjection is a function whose image covers every possible output. Their negations are noninjective and nonsurjective.

Decidable, in computer science, means there is an algorithm that always halts with a yes-or-no answer. The integer-only Z-arithmetic check in CHSH_LASSERT is decid-

able; the algorithm computes the result in finitely many steps and returns a Boolean.

Polynomial complexity: an algorithm is polynomial-time if there is some power k such that the algorithm's running time on inputs of size n is at most a constant times n^k. Polynomial-space is the analogous concept for memory used. The class P is decision problems solvable in polynomial time. The class NP is decision problems where, given a candidate solution, a polynomial-time algorithm can check whether the solution is correct. The famous P-vs-NP question asks whether finding is fundamentally harder than checking. Nobody has proved either direction. Section 24 discusses why my `mu_budget_decidable` / `mu_budget_verifiable` predicates are not a P-vs-NP result. I am not the person who will prove it.

Determinant: a number computed from a square matrix that says whether the matrix is singular (zero determinant means the matrix collapses some direction to zero). For a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ the determinant is $ad - bc$. For larger matrices, you can compute it by "expansion along a row": pick a row, for each entry take the determinant of the submatrix you get by deleting that entry's row and column (with alternating signs), multiply by the entry, and sum. The recursion bottoms out at the 2×2 case above. Section 16 walks the explicit calculations for the 3×3 cases I need.

Schur complement: a way of turning a question about a block matrix into a question about a smaller matrix. If $\begin{pmatrix} A & B \\ C & D \end{pmatrix}$ is a block matrix where A is invertible, the Schur complement of A is $D - CA^{-1}B$. A standard linear-algebra result says the block matrix is positive-semidefinite if and only if both A and the Schur complement are positive-semidefinite. The Section 16 build uses this for the 4×4 block of the NPA moment matrix.

A principal submatrix is the square subblock you get by picking a subset of indices and keeping only the rows and columns at those indices. A standard result is that a matrix is positive-semidefinite if and only if every principal submatrix is positive-semidefinite. Every one. Section 16 relies on this.

2.8 The axioms (A1 through A6)

The Thiele Machine has a small numbered list of substrate-level axioms. They appear by name ("A2," "A3," ...) throughout the document and the formalization. The list below is what each name means, where it lives in the kernel, and what its conclusion is. The axioms are substrate-level; the kernel is scaffolding-level; the list bridges the two.

The kernel keeps two parallel axiom-numbering systems because two parallel substrate abstractions are formalised. The `AbstractCertMachine` record carries A1 (the cert indicator exists, by record existence) and A3 (only cert-setters flip it, the `acm_preserve` field). A2 (cert starts unset) and A4 (cert-setters cost $\geq$ 1) live

alongside the record, not in it: A2 is the `acm_cert M s0 = false` hypothesis to `abstract_nfi`, A4 is the standalone lemma `cert_addr_setter_cost_pos`. The consolidated `CertificationSystem` record folds the cost-on-certification semantic content into a single field kept by the name A2 (the load-bearing piece). The `QuantitativeCertificationSystem` extension adds A3–A6, which lift the cost-floor result from ≥ 1 to $\geq K$ for an arbitrary threshold K. The two numbering systems agree on the cost-on-certification semantic content; they disagree on which axiom carries that content (A4 in the AbstractCertMachine list, A2 in the consolidated CertificationSystem list). When the rest of this document says "A2," it means the cost-on-certification rule, in the consolidated CertificationSystem sense.

A1 (state includes a certification indicator).
An `AbstractCertMachine` record bundles a state type `S`, a step function `acm_step`, and a boolean cert indicator `acm_cert : S → bool`. A1 is the structural requirement that this indicator exists at all. In Coq, A1 is enforced by the record type itself. There is no way to hand Coq an `AbstractCertMachine` value with `acm_cert` left blank; it reads the field as missing and declines to build the record, the way a clerk declines a form with an empty box. A1 is discharged at type-check time.

A2 (cost-on-certification, in the AbstractCertMachine system: "cert indicator starts unset").
In the consolidated `CertificationSystem` record, Coq field `cs_cert_costs`. Statement: for every state s and instruction i, if `cs_cert s = false` and `cs_cert (cs_step s i) = true`, then `cs_cost i` $\geq$ `1`. In words: any single step that flips the certification predicate from no to yes must have positive cost. This is the load-bearing axiom for No Free Insight. `universal_nfi_any_substrate` concludes from A2 alone that every trace from uncertified to certified has total cost ≥ 1. Note: in the AbstractCertMachine numbering, the same content appears as A4 (cert-setters cost ≥ 1). The AbstractCertMachine A2 is the separate precondition that the cert indicator starts unset, which the consolidated system handles as a theorem hypothesis instead of a numbered axiom.

A3 (cost bounds witness growth).
Coq field `qcs_cost_bounds_witness` on the record `QuantitativeCertificationSystem`. Statement: for every state s and instruction i, `qcs_witness s + cs_cost i ≥ qcs_witness (cs_step s i)`. In words: each instruction's cost is an upper bound on how much the system's accumulated evidence (the "witness") can grow in that step. The cost is the speed-limit on evidence accumulation. (In the AbstractCertMachine numbering, A3 is the field `acm_preserve`: non-cert-setter instructions preserve the cert indicator. The two A3s are distinct properties on distinct record types; the monograph's A3 is the quantitative one above.)

A4 (witness nondecreasing).
Coq field `qcsf_witness_nondecreasing` on the record `QuantitativeCertificationSystem_full`. Statement: for every s and i, `qcs_witness s ≤`

`qcs_witness (cs_step s i)`. A3 and A4 point in opposite directions: A3 caps how fast the witness can grow ($w' \leq w + \text{cost}$), A4 stops it from shrinking ($w \leq w'$). Neither implies the other. $(w, \text{cost}, w') = (1, 0, 0)$ satisfies A3 with non-negative cost yet violates A4, and $(0, 0, 1)$ does the reverse, so A4 is its own field, not a consequence of A3. It is also idle: the $\geq K$ floor telescopes through A3, A5, and A6 alone and never touches A4. The field is kept so downstream consumers that want witness monotonicity in hand have it without re-deriving it; the consolidated `QuantitativeCertificationSystem` record drops A4 entirely, carrying only A3 and A5.

A5 (certification requires witness).
Coq field `qcs_cert_threshold_witness` on `QuantitativeCertificationSystem`. Statement: `cs_cert s = true → qcs_witness s ≥ qcs_threshold`. In words: certification cannot fire until the system has accumulated at least `qcs_threshold`-worth of evidence. This is what lets the quantitative theorem push the cost floor from ≥ 1 to $\geq K$, where K is the threshold. Pay more, certify more.

A6 (witness initial zero).
The hypothesis that the run starts with zero witness, stated as a precondition on the quantitative theorem rather than a record field. Without A6 the telescoping bound becomes $K - $ `initial witness` instead of K. Zero is a clean place to start.

A1 through A6 are independent of any specific instruction set. They are conditions on Coq record types; supplying instances of those record types is the only way to invoke the corresponding theorems. The 51-opcode VM supplies them by construction: the kernel builds two concrete `CertificationSystem` values for the VM (one for the `csr_cert_addr` channel, one for the `vm_certified` channel), each discharging A2 from the existing `no_free_certification` kernel theorems. A separate substrate-level abstraction (the `Substrate` typeclass) carries a related but distinct field `mu_monotone` stating that the cost ledger never decreases on a step. `mu_monotone` is a strictly weaker general-purpose monotonicity property than A2: A2 specifically asserts the cert transition costs ≥ 1, whereas `mu_monotone` only asserts the ledger does not go down. The two coincide on the 51-opcode VM, where every instruction adds non-negative cost and the cert transition adds at least 1. Two locks on the same door, by different keys.

That is the vocabulary. From here on, the document uses these words without rebuilding them. Section 9 is where the formal machine state begins; experts should jump there. Everyone else stays for the walk.

3 The structural axis: the projection classical models drop

Why this section exists. This section exists because a renderer I was building in January 2025 walked me into a wall, and the wall turned out not to be made of what I thought.

The renderer was a categorical engine. I had built it so the things running at runtime

were the category-theory objects themselves: modules were objects, transformations between them were morphisms, composing two transformations was literally composition, and running two independent parts side by side was the monoidal product (Section 2 builds all of these). I was not simulating category theory on top of ordinary data. The structure was the thing the engine ran on. And it worked, which is the part I still find a little funny: within weeks the same engine that was supposed to draw pixels was running physics on the identical primitives, and I had not planned that and did not at first believe it.

Then I tried to prove something about what it was doing, and the floor went out. The machine underneath, the ordinary CPU, had seen none of the structure the entire time. (That ordinary architecture, one processor and the memory passing data back and forth across a single bus, is the von Neumann machine, after John von Neumann who wrote it down in 1945; every laptop and phone and server you have ever touched is one.) It had been faithfully shuffling the bytes my modules and morphisms happened to be spelled out in, faithful the way a printing press is faithful to a poem it cannot read. When I brought in Z3 to verify the morphism chains the engine was reasoning with, I met the same blindness in different clothes: Z3 saw formulas, true ones, that happened to refer to the shapes, and it never once saw a shape. For a long stretch I took this for a Z3 problem, a tooling problem, the kind you solve by getting cleverer with the solver. It was not a Z3 problem. The structure I wanted checked was real and was carrying the whole computation, and there was nowhere in the machine for it to sit as a thing the machine itself had to answer for. I was asking a tool to certify a fact about the one part of the computation the computation had no place to keep. You do not get cleverer your way into a slot that was never built.

The Thiele Machine is that slot. It is the substrate the engine had needed all along and could never be handed on the hardware it was running on: a machine where the structure is first-class state instead of a pattern in the bytes, where composing that structure is a step the machine takes rather than something you read back out afterward, and where committing to a structural fact is an act the machine is charged for and cannot quietly perform for free. Everything past this point is that substrate's correctness conditions, laid out in English you can follow and check yourself, with a Coq proof standing next to each one. The Coq is not what makes any of it true. It is there because my own eye slides straight over the gaps in an argument I am hoping is right, and I wanted a reader that would not let me. This section is the long-form answer to a question a stuck renderer put to me and would not drop until I quit trying to out-clever it and went looking for what was actually missing.

Here is the claim this section rests on, stated hard. A2, the rule that flipping certification from false to true costs at least one, is a law of the step rule itself, written into the step's own definition (in the kernel it is the field `cs_cert_costs` on the `CertificationSystem` record), not a habit the program is trusted to keep. And the state A2 needs before it can even be said out loud, the cost ledger μ next to the cert flag, is the

proved-minimal extension over the ordinary fields of memory, registers, and program counter: you need a cert to read and a μ to charge, and `P_full_is_minimal_complete_extension` shows you cannot pull either one out and still have certification mean anything. On every state the machine can actually reach, that cost comes out uniquely μ. Nothing else fits.

Now hold those two facts in the same hand, because the whole priority claim lives in the seam between them. A classical step rule has no field for any of it, so the most it can ever do is run a checker of A2 off to the side, the kind of checker a buggy simulator skips without anything noticing. The substrate is where A2 is the law. Classical is where A2 is, at best, a thing you hope the program remembered. Classical is not a rival to the substrate here; it is what is left of the substrate once you forget the law, which is the precise sense in which it is the derivative one.

There is a categorical version of this, and it carries less than it first looks like it should. Any substrate whose step rule enforces A2 sits in a category, and the Thiele Machine is the initial object of it, the one arrow running out from Thiele to every other such substrate (initiality in the exact sense Section 2 builds). Clean handle, genuinely true, and also generic: the free model over any signature is initial over its own category, the integers are initial among the rings, and nobody thinks that makes the integers the hidden floor under arithmetic. So initiality is a consequence with a tidy name, downstream of the real thing, not the ground under it. The ground is the minimality and the law-versus-checker split above, the one place tagged structure cannot follow. The Turing machine and its relatives are simply not in the A2-respecting category at all, their step rules having no slot for cost-on-certification, and when one of them runs an A2-enforcing system it is the simulator program carrying the rule, never the substrate. The within-model witnesses that come next, two states agreeing on everything classical but splitting on μ or cert or the morphism graph, are illustration, a finger laid on exactly what the projection drops. They are not the load. The substrate-versus-program line is.

3.1 Substrate-independence: the abstract setup

The kernel theorem `universal_nfi_any_substrate` is written to be as hard to wriggle out of as I knew how to make it, and the way you make a theorem hard to wriggle out of is to make it ask for almost nothing. It takes any abstract computational system at all, and by any I mean it genuinely does not care what the system is made of or whether it looks anything like the machine in this document. It asks for five pieces and refuses to ask for a sixth. A set of states, and a state can be whatever it is for you: bits on a tape, words in a register file, cells in memory, it makes no difference to the theorem. A set of instructions, the moves the thing can make. A step rule, which takes a state and an instruction and hands back the next state. A cost function, which pins a non-negative integer to each instruction. And a certification predicate, a plain yes-or-no on states that says whether a given state counts as certified. That is the entire list. The reason I keep leaning on how short it is: any objection that begins “sure, but

that only holds for your particular machine" has nothing to attach to once the result has agreed to run on something this bare.

One premise, labelled A2 and named `cs_cert_costs` in the kernel: if any single step flips the certification predicate from no to yes, the instruction that did it cost at least 1. One conclusion: every trace that starts uncertified and ends certified has total cost at least 1.

And it does not care how powerful the thing is, in either direction. It never asks whether your system can compute everything a Turing machine can (the word for that is Turing-complete: you are Turing-complete if you can simulate a Turing machine, at most Turing-equivalent if a Turing machine can simulate you). It never asks for the reverse either. Stronger than a Turing machine, weaker, dead even, none of it reaches the floor. Bring the five pieces, charge A2 on the flip, and you are in scope. The floor does not check your computational class at the door, because it was never about what you can compute. It is about what your step is allowed to do for free.

The deeper categorical-separation result (Section 12) applies to any computational model whose observable state is just memory, registers, and a program counter. The Turing machine is the canonical instance, and the thing on your desk is the same animal wearing a faster coat: laptops, phones, the central processing unit and graphics processing unit in any server you can rent. They differ in clock speed and in which instructions they happen to recognise, and that is genuinely all they differ in for the purpose at hand. Every one of them carries the same hole in the same place. Their state is a snapshot of bytes, and a byte is just a byte; it remembers what value it holds and nothing about why it earned the right to hold it. So when a computation finishes proving something, the proof leaves a footprint in the output and no footprint at all in the state that did the work. The fact that a structural claim was discharged, paid for, made true rather than guessed, is simply not a field anything classical was built to keep. The blind spot is not a bug in any one of these machines. It is what it means to be one.

One precondition matters, and it's the easiest thing in this whole setup to skate right past: the abstract system has to have a certification predicate. A plain Turing machine does not. The theorem applies to "Turing machine equipped with a certification predicate," not to every Turing machine trace. Saying "every classical computation pays μ" is wrong. Saying "every classical computation that ends in a certified structural claim pays at least 1 into μ" is right. The conditional is doing real work.

3.2 The structural axis is canonical, not orthogonal

The natural follow-up question: if a classical substrate has no built-in slot for structural facts, can I bolt one on and reach the same theorems? Yes. The interesting answer is what it costs and what it forces.

Three theorems pin this down. None of them rule out adding state to a Turing machine.

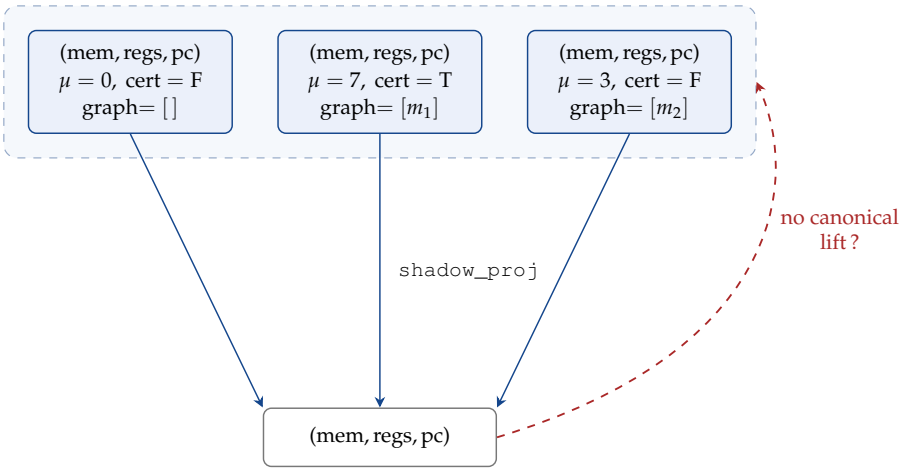


Figure 2: The directional structural theorem. Every Thiele state has a single canonical projection down via `shadow_proj` (`degenerate_projection_theorem`). The classical state has multiple Thiele preimages and no canonical lift back (`fiber_has_two_preimages`). One direction is a function; the other is not.

They say something different: the structural axis is what any substrate looks like once it accepts honest certification cost, and μ is the unique cost it accepts. The Thiele Machine names this structure. It does not invent it.

1. `universal_nfi_any_substrate`: the load-bearing piece. Any Coq `CertificationSystem` (a record bundling a state type, an instruction type, a step rule, a cost function, a certification predicate, and a proof of A2 as one of its fields) has total trace cost at least 1 to go from uncertified to certified. A2 is the only premise, and it is structurally enforced: there is no way to make an inhabitant of the record type without supplying a proof of A2.
2. `mu_is_initial_monotone`: the uniqueness piece. Suppose someone proposed a different cost function for the Thiele Machine. Call their function M. If M assigns zero to the initial state, and M increases by exactly the right amount at every instruction (the per-instruction increment is fixed by the cost schedule in Section 7), then M agrees with μ on every state the machine can reach. There is no other accounting that satisfies those two local conditions. The cost is forced.
3. `shadow_strictly_lossy` and `categorical_separation`: two within-Thiele witness theorems serving as illustration of what the load-bearing piece looks like inside the model. There exist Thiele states agreeing on the six-field projection (memory, registers, PC, μ, error flag, top-level certified flag) but differing in their morphism graphs. The projection drops information. It's worth saying plainly, because it's the first thing a careful reader catches: any model that drops a field and exhibits two states differing only in that field reproduces this shape. The shadow lemma is what items 1 and 2 look like instantiated: the universal cost floor (`universal_nfi_any_substrate`) and the uniqueness of μ (`mu_is_initial_monotone`) are what carry it, and the law-versus-checker distinc-

tion is what it witnesses. Taken by itself the projection witness is generic, since an analogous field-drop reproduces the shape on any substrate, so it leans on items 1 and 2, not the other way around.

Can a Turing machine write a μ-like ledger on its tape and produce traces whose encoding satisfies A2? Yes. The tape is not the problem. The problem is where the rule lives: in the program, not in the Turing machine's step rule.

Here is what that means in the kernel. `CertificationSystem` (defined) bundles a state, an instruction set, a step rule, a cost function, a certification predicate, and a proof of A2. That proof is not commentary sitting next to the type. It is one of the fields. Here is the record body, copied verbatim:

```
Record CertificationSystem := mk_cert_system {
  cs_state : Type;
  cs_instr : Type;
  cs_step  : cs_state -> cs_instr -> cs_state;
  cs_cost  : cs_instr -> nat;
  cs_cert  : cs_state -> bool;
  cs_cert_costs :
    forall (s : cs_state) (i : cs_instr),
      cs_cert s = false ->
      cs_cert (cs_step s i) = true ->
      cs_cost i >= 1;
}.
```

The last field, `cs_cert_costs`, is not a comment on the record. It is a typed field whose type is the literal statement of A2. The colon between the field name and the `forall` is doing real work: any value of type `CertificationSystem` must carry, at that position, a term whose type is the proposition printed there. Coq refuses to build the record otherwise.

That is the precise sense in which A2 is built into `CertificationSystem`, not asserted beside it. To make a value of that type (the Coq word is an inhabitant) you have to supply a proof of A2. In Coq, "supplying a proof" means handing over a term whose type is the statement you are proving; Section 2 covers this. Build the record without that proof and the type checker rejects you. Cold. Unmoved. Correct.

So `CertificationSystem`, by the shape of the type, names substrates whose primitive step enforces cost-on-certification. A Turing machine step rule over state and tape symbol is not one of them. You can package "Turing machine plus a simulator program that maintains an A2-respecting tape encoding" into a `CertificationSystem` value by exhibiting a step rule for the package and proving A2 of it. But then the package is the simulator, not the Turing substrate. The Turing machine is indifferent. Run a buggy simulator that skips the cost increment and the Turing machine

keeps running. There is no level at which the Turing machine knows the simulation is dishonest. There is no level at which the Turing machine knows anything about simulations.

The Coq theorems do not blur this line. `universal_nfi_any_substrate` takes a `CertificationSystem` as input, so its premise is already step-level A2. `thiele_represents_simulating_cert_system` takes a `SimulatingCertificationSystem` (a `CertificationSystem` plus a structure-preserving translation into Thiele) and proves that the simulation's certifications transport into Thiele certifications. Neither theorem makes Turing machines members of the class. They make A2-enforcing systems members of the class.

Turing machines can simulate members of the class. That still makes simulation a property of the running program, not a substrate law. The kernel makes the negative side explicit: `CostBearingSystem` is the unconstrained record (same shape as `CertificationSystem`, but without the A2 field), and `dishonest_forge_system` is a concrete `CostBearingSystem` where a single instruction sets the certified flag at total cost zero. `universal_nfi_any_substrate` forbids that in the `CertificationSystem` world by type-checking. A bare Turing substrate belongs on the unconstrained side by default. Nothing in its step rule excludes the forge.

Let's chase this all the way to the wall. Forget encoding A2 in a program; rig it onto a single tape cell directly. Let "certified" mean cell zero holds a one, and charge at least one for every write of a one. Now `cs_cert_costs` holds, the record type-checks, and `universal_nfi_any_substrate` applies to the construction word for word. That isn't an escape from the theorem. That's walking straight into it. A program that does no certification work and just writes a one to cell zero is now "certified" at cost one, indistinguishable from a program that earned it. The cell is writable by anything, for any reason, with no record of why; the cost bought a write, not an act of certification. The theorem dutifully certifies the forgery and feels nothing about it. What a tape cell cannot hand you is a certification no program can counterfeit, one that every path reaching it is forced to pay for, with no side door that flips the bit for free. That unforgeability does not live in the cell. It lives in the transition relation that writes it. In the 51-opcode realisation this is not a hope, it is a checked fact: `vm_apply_preserves_certified_non_certify` proves that every instruction except `CERTIFY` leaves `vm_certified` exactly as it found it, so the false-to-true flip can only happen when the certifying transition actually fires. The cert is not a value some program left lying in a cell. It is a constraint the step rule enforces on the whole machine. That is the difference between a bit on the tape and a law in the relation, and it is the whole difference.

The actual separation is between substrates whose step rule enforces A2 and substrates where A2 can only be encoded into the running program's behaviour. The Thiele substrate sits in the first class: its step rule enforces A2 as a law. The bare

Turing substrate sits in the second, where A2 can only be encoded into the running program's behaviour and skipped by a buggy simulator. That is the separation, and it is not a claim about a new class of computable functions. The 51-opcode realisation demonstrates the shared-computation side: its structural opcodes can be set to do no structural work (the cost parameter δ can be 0 on every instruction, in which case the cost-on-cert floor is just $S(0) = 1$ at the moment of certification, and that floor matches what an honest simulator would charge anyway). A Turing machine can simulate honest traces of that realisation. But simulation is not instantiation. Putting cost-tracking on the tape is not putting cost-tracking in the step.

That is the computational-model claim, and it names the exact place the standard complexity-theory cost model has no slot for. That model counts steps. It never looks at what a step rule is allowed to contain, so the whole difference between a step that enforces cost-on-certification and a step that merely runs over a tape encoding of it is invisible to step-counting from the very start. The separation lives one floor underneath where complexity theory was built to look, which is why nobody tripped over it from up there.

From here on, the Turing machine is the concrete contrast. It is the model you already know.

3.3 The canonical contrast: the Turing machine

Here is what I had to learn about Turing machines to compare what I built to one. I went and read about them on my own. I'm walking through it because I had to walk through it. If I have anything wrong below, that's because I'm relaying what I read, and I do not trust my own reading of any source. Tell me if it's wrong.

What I read says Alan Turing described his machine in 1936 to make precise the intuitive notion of "algorithm." The machine, as I understand it, is this:

- An infinite tape. Think of it as an infinitely long strip of paper, divided into cells. Each cell holds one symbol from a finite alphabet, say $\{0, 1, \texttt{blank}\}$.
- A read/write head. One cell at a time. It reads the symbol, overwrites it if the table says to, then moves left or right.
- A finite set of states, $Q = \{q_0, q_1, \ldots, q_n\}$. The machine is in exactly one state at a time. q_0 starts the run. Some states halt it.
- A transition table. Given the current state and the symbol under the head, it says: write this, move left or right, enter that state.

That is the entire machine. Nothing else.

Here is what I read as the remarkable thing about this simple machine: it can compute *anything that can be computed by following a definite procedure*. The sources I dug up claim every algorithm that has ever been written, in every programming language that has

ever existed, can be translated into a Turing machine. The label I found for this is the Church-Turing thesis. The original claim (Church 1936, Turing 1936), as I read it, is specifically about *effective calculability*: anything that can be computed by following a mechanical, finite, step-by-step procedure, a Turing machine can also compute. I did not find a counterexample. I also have not verified the thesis myself. I am reporting the ground I had to stand on before I could push against it.

The claim about physical hardware is a related but separate conjecture, called in the sources I read the *physical Church-Turing thesis*: every physical computing device is computationally equivalent to a Turing machine. As I read it, this is an empirical claim about physics, not proven mathematics. I did not find a built physical counterexample. Your laptop, your phone, every server in every data center: in the usual computability sense, they compute no more functions than a Turing machine computes. The differences I found are resource differences. Speed. Space. Energy. Cost. The class of computable functions stays the same.

3.4 The blind spot

The Turing machine's transition table sees exactly two things at each step: the current state q and the symbol under the head. The same shape of blindness applies to every classical substrate I went and looked at. A register-machine instruction (a model of computation that operates on a fixed list of registers, much like a real CPU) sees only its operands, the values in the named registers. A lambda-calculus reduction (a model of computation based on substituting expressions, originally used to define what "computable function" means) sees only the next expression to simplify. A cellular-automaton rule (a model where a grid of cells updates each step based on its neighbours, used by Conway's Game of Life) sees only a small neighbourhood of cells. None of them have a first-class slot for "and by the way, the input has structure X that you have been entitled to use."

The Turing machine cannot ask: "Is this tape sorted?" It has to read every cell.

It cannot ask: "Does this graph have two disconnected components?" It has to run a traversal.

It cannot ask: "Are these two parts of the input independent of each other?" It has to check.

The tape view is LOCAL. One cell at a time. There is no primitive way to see global structure. A Turing machine can COMPUTE global structure, but computing it and seeing it are different things. To know a list is sorted, it runs a sortedness check that touches the cells encoding the list. It pays in time and space for every structural fact it learns.

The same story holds for any model whose observable state projects to `(mem, regs, pc)`. The transition function sees the projection. A known decomposi-

tion, checked invariant, or asserted morphism has to be re-derived, trusted as an untyped convention, or carried in data with no substrate-level audit. There is no state slot where the entitlement lives.

I spent months staring at this before it clicked. The Turing machine isn't broken. It's blind by design, and so is everything that compiles down to its observable state. It's like trying to find your way through a maze by only ever looking at the floor tile you're standing on. You *might* do it, if the maze has an exit and your search strategy eventually reaches it. Some mazes have no exit, and some search strategies loop forever. That is what the halting problem says, in the version I had to learn: there is no general algorithm that can look at an arbitrary program and decide whether it stops or runs forever. Turing proved this in 1936. The maze-tile reader has the same disability. It cannot, in general, tell from the floor tile whether it is making progress or going in circles. If the search does terminate, it still pays for every wrong turn and every part already explored. Someone with a map does not pay that cost.

The Thiele Machine does not magically give the map. "The map" here is a concrete thing: a certified partition graph with proven structural properties, a checked decomposition into modules, a validated formula, an asserted morphism with provenance. The Thiele Machine represents the specific moment a computation becomes entitled to use one of these. It makes that entitlement auditable: there is always a μ-ledger entry showing when the certification was paid for. The map is not free.

3.5 Classical complexity ignores structural cost

Complexity theory is the part of computer science that sorts problems by how hard they are, and it has been keeping the books on hardness for sixty years. It measures two things, and it measures them with real care: time, the number of steps the model takes, and space, the number of cells or registers it burns. Two columns, ruled and totalled, with an entire field's worth of theorems stacked on top of them. What it has never had is a column for the cost of *knowing*: knowing the input is sorted, knowing it splits into two halves that never touch, knowing the structure you are about to lean your whole weight on is actually there. That cost appears nowhere on the ledger. And it appears nowhere whether the model underneath is a Turing machine, a RAM machine (numbered slots you can jump straight to, closer to a real computer than a tape), a lambda-calculus reducer, or anything else that is classical at heart. They all count steps. Not one of them ever stops to ask whether the step was even entitled to use the structural fact it just used.

Consider the satisfiability problem (SAT): given a logical formula ϕ over n boolean variables, is there an assignment that makes it true? A blind brute-force search checks all 2^n assignments. (Here 2^n means 2 raised to the power n: 2 times itself n times. For 20 variables it is about a million, for 30 it is a billion, for 60 the universe starts looking at its watch.) Real SAT solvers do better on many instances. The worst-case cliff remains: for general SAT, no known classical algorithm removes the exponential

wall unless P = NP.

Now suppose ϕ decomposes cleanly: $\phi = \phi_1 \wedge \phi_2$, where ϕ_1 and ϕ_2 share no variables. You can solve ϕ_1 on its own (2^{n_1} work, where n_1 is the number of variables in ϕ_1) and ϕ_2 on its own (2^{n_2} work, where n_2 is the number in ϕ_2). Total brute-force work: $2^{n_1} + 2^{n_2}$ instead of $2^{n_1+n_2}$. The sum can be tiny compared with the combined exponent. That is an exponential improvement: a small structural fact causes a wildly large change in the cost.

But that improvement only applies if you *know* the decomposition exists. No classical computational model assigns a primitive cost to knowing it. The structure is either there or it is not. You either see it, or you spend time finding it, or you trust some side-channel that claims it. The act of certifying "I know this decomposition is valid" is not tracked, not paid for, not audited. Not in the Turing model. Not in any of its computational equivalents.

The Thiele Machine says: that act belongs inside the computation. Here is the structural object. Here is the checked transition that makes the object admissible. Here is where the cost shows up in the trace. Here is how to verify that the cost was actually paid. This is the structural-entitlement reading of No Free Insight, and via `universal_nfi_any_substrate` it lifts to any computational system that can be packaged as a `CertificationSystem`.

3.6 The subsumption theorems, in full

Section 1 framed classical computation as the degenerate fragment of the Thiele substrate, the corner where the structural axis lies dormant and nothing ever lights it up. That framing is not a stance I am asking you to take on faith. It is four theorems in the kernel, and I am setting their statements and proof sketches down below in plain math, so you can hold the framing up against the proofs and decide for yourself, without taking my word for either one and without leaving this page to do it.

Theorem 3.1 (D2 faithfulness)**.** *For every classical program* prog *(a sequence of* `vm_instruction` *values each satisfying* `is_classical_opcode`*) and every starting state s_0, let s_n be the state after running* prog *on s_0. Then (a)* `shadow_proj`(s_n) *extracts exactly the six projected fields of s_n (*`vm_regs`*,* `vm_mem`*,* `vm_pc`*,* `vm_mu`*,* `vm_err`*,* `vm_certified`*); and (b) the structural layer is preserved: s_n.`vm_graph` $= s_0$.`vm_graph`, s_n.`vm_csrs.csr_cert_addr` $= s_0$.`vm_csrs.csr_cert_addr`, and s_n.`vm_certified` $= s_0$.`vm_certified`.*

Proof. Part (a) unfolds `shadow_proj`: by definition it extracts exactly those six fields, and reflexivity closes it. Part (b) is D3 conservativity (next theorem) iterated over the program. □ □

> ***Plain reading.*** Classical programs leave the structural axis alone. The shadow you

see equals the full state's projected fields. The graph and cert channels stay at whatever they were at the start.

Theorem 3.2 (D3 conservativity)**.** *If* `is_classical_opcode`(i) = true *and* $s' =$ `vm_apply`(s, i)*, then* s'.`vm_graph` $= s$.`vm_graph`*,* s'.`vm_csrs.csr_cert_addr` $=$ s.`vm_csrs.csr_cert_addr`*, and* s'.`vm_certified` $= s$.`vm_certified`*. By induction over the program list, classical programs preserve all three.*

Proof. Case analysis on the instruction i. The predicate `is_classical_opcode`(i) = true holds exactly for the compute, memory, control-flow, ordinary I/O, heap, and `MORPH_GET` / `TENSOR_GET` opcodes: the ones whose `vm_apply` rule mutates only `vm_regs`, `vm_mem`, `vm_pc`, `vm_err`, `vm_mu`, `vm_logic_acc`, `vm_mu_tensor`, or `vm_mstatus`. Twenty-two opcodes return `false` from `is_classical_opcode` and are excluded by the premise, but not all for the same reason. `PNEW`, `PSPLIT`, `PMERGE`, `TENSOR_SET`, `MORPH`, `COMPOSE`, `MORPH_ID`, `MORPH_DELETE`, and `MORPH_TENSOR` mutate graph or module-tensor state. `MORPH_ASSERT` can write the cert-address channel on success; `CERTIFY` writes the top-level certification flag; `CHSH_TRIAL` mutates the witness counters. `LASSERT`, `LJOIN`, `EMIT`, `REVEAL`, `PDISCOVER`, `CHSH_LASSERT`, and the four Q_{1+AB} `CHSH_LASSERT` variants are also outside the classical fragment because they carry validation, revelation, discovery, tensor-charge, or mandatory-floor semantics. The D3 proof only needs the forward direction: for each opcode that `is_classical_opcode` admits, direct inspection of the apply rule leaves `vm_graph`, `csr_cert_addr`, and `vm_certified` fixed. Iterating over the program list by induction closes the trace-level preservation. □ □

> ***Plain reading.*** The classical opcodes are defined by what they don't touch. They may change the projected working fields, including the ledger charge assigned to the instruction, but they leave the graph and certification channels alone. A program built only out of them cannot disturb those channels, even if you wanted it to. That dormancy is by construction, not by accident.

Theorem 3.3 (Thiele simulates Turing)**.** *Lift a Turing configuration by keeping the TM part and setting μ to zero:* `lift_config`$(c) := \{$ `th_tm_config` $= c$; `th_mu` $= 0\}$*. Then for every fuel $n \in \mathbb{N}$, transition table δ, and TM configuration c:*

$$(\texttt{thiele\_run}\ n\ \delta\ (\texttt{lift\_config}\ c)).\texttt{th\_tm\_config} = \texttt{tm\_run}\ n\ \delta\ c.$$

Proof. Induction on the fuel n.

Base ($n = 0$). Both `thiele_run` 0 and `tm_run` 0 return their inputs unchanged. Reflexivity.

Step ($n + 1$). Unfold one step on both sides. Let $c' =$ `tm_step` $\delta\ c$ when defined. On a lifted configuration, `thiele_step`'s TM portion is exactly `tm_step` $\delta\ c$: structural

fields are not consulted, and lifted μ stays at 0 because TM steps are classical. Apply the induction hypothesis to the next n steps from `lift_config`(c'). □ □

> ***Plain reading.*** Take any Turing-machine configuration and drop it into the Thiele substrate untouched: the TM part stays exactly as it was, $\mu = 0$, the structural fields empty. Now run `thiele_run` on it. Nothing happens that wasn't going to happen anyway. Step for step the TM portion does precisely what the standalone machine would have done, and because every move it makes is a classical move, the ledger never comes off zero. The induction on fuel just carries that out to the whole trace. The Turing machine is sitting inside the larger object running its same computation on its same tape to its same answer, having paid nothing, and from where it stands there is no way to tell it has been subsumed at all. That is what subsumption is supposed to look like: the old thing keeps working, identically, for free.

Theorem 3.4 (Degenerate projection theorem (four-part))**.** *The following four propositions hold simultaneously:*

1. Faithful embedding. *For every fuel n, transition table δ, and TM configuration c:*

$$(\mathit{thiele\_run}\ n\ \delta\ (\mathit{lift\_config}\ c)).\mathit{th\_tm\_config} = \mathit{tm\_run}\ n\ \delta\ c.$$

2. `shadow_proj` is the quotient map. *For all Thiele states s_1, s_2:*

$$\mathit{shadow\_proj}(s_1) = \mathit{shadow\_proj}(s_2) \iff \texttt{eq\_on\_classical\_shadow}(s_1, s_2).$$

3. Classical traces respect the quotient. *For every classical program* prog *and every two Thiele states s_1, s_2 that agree on the classical shadow:*

$$\mathit{shadow\_proj}(\mathit{run}(\text{prog}, s_1)) = \mathit{shadow\_proj}(\mathit{run}(\text{prog}, s_2)).$$

4. The projection is strictly lossy. *There exist Thiele states s_1, s_2 with $\mathit{shadow\_proj}(s_1) = \mathit{shadow\_proj}(s_2)$ and $s_1 \neq s_2$.*

Proof. The theorem is four lemmas stitched into one conjunction.

- Part (1) is `thiele_simulates_turing` (preceding theorem).
- Part (2) is definitional. `shadow_proj` extracts its six projected fields. Equal projections mean those six fields agree. The Coq witness is `shadow_proj_kernel_is_eq_on_classical_shadow`.
- Part (3) follows from D2 and D3. Run a classical program: the output shadow depends only on the input shadow. Equal shadows in, equal shadows out. The

Coq witness is `D2_classical_shadow_preserved`.

- Part (4) is `shadow_strictly_lossy`. The Categorical Separation construction gives the witness: s_1 has one identity morphism in `pg_morphisms`; s_2 has none. The six projected fields agree. The morphism lists do not.

The conjunction of the four pieces is the theorem. □ □

Plain reading. Four parts about the same projection. Part 1: every TM configuration lifts faithfully into the Thiele substrate; same computation, with $\mu = 0$ and no structural content. Part 2: the projection from Thiele back to classical IS the classical-equivalence quotient; two Thiele states have the same shadow exactly when they agree on the six projected fields. Part 3: classical programs commute with the projection; you can project before running or after, you get the same answer. Part 4: the projection genuinely loses information; there are Thiele states that look identical from the classical side but differ on the structural axis. Put together, classical computation is exactly the image of Thiele computation under the structural-axis projection. The projection drops real information.

3.7 What the projection cannot reverse: three non-existence theorems

The subsumption theorems above establish what the Thiele substrate carries over and above its classical projection: fields the projection drops. The natural next question is whether a classical observer could reconstruct those fields by some clever derived predicate on the surviving shadow. The answer is no, and it is a structural no, the kind you cannot argue your way around. The kernel proves three irrecoverability theorems (each one a non-existence of a function from the bare classical projection back to a Thiele-only field) and packages the corollaries the subsumption argument relies on. Every one is proved with zero axioms and no admits, fully constructively.

The projection in this subsection is the four-field map `forget : VMState → TMSnapshot`, keeping `(pc, μ, regs, mem)`. It is already forgetful, but it still retains μ. For the cost-ledger separation, the stricter three-field projection `bare_observable` drops μ as well, keeping only `(pc, regs, mem)`: the observable the standard complexity-theory cost model sees.

Theorem 3.5 (Cert irrecoverability, `cert_not_function_of_forget`)**.** *There is no function ϕ : `TMSnapshot` → `bool` such that, for every Thiele state s,*

$$\phi(\texttt{forget}\, s) = s.\texttt{vm\_certified}.$$

Proof. The witnesses are `trace_witness_A` (certified, defined) and `forget_witness_uncert` (not certified, defined). Direct computation shows the two states have equal `forget` images. Suppose ϕ existed: then ϕ(`forget trace_witness_A`) = `true` and ϕ(`forget forget_witness_uncert`) = `false`. But the inputs are

equal, forcing the outputs to be equal too. Contradiction. □ □

> ***Plain reading.*** The Thiele `vm_certified` flag is not a function of the four fields kept by `forget`. Two Thiele states can agree on (`pc`, μ, `regs`, `mem`) and still differ on whether they are certified. No formula computed from that four-tuple alone can decide certification. The certification bit lives outside the four-tuple.

Theorem 3.6 (Mu irrecoverability from bare observable, `mu_not_function_of_bare_observable`)**.** *There is no function* ϕ : `BareTMObservable` → `nat` *such that, for every Thiele state* s*,*

$$\phi(\texttt{bare\_observable}\, s) = s.\texttt{vm\_mu}.$$

Proof. The witnesses `mu_separation_witness_zero` and `mu_separation_witness_one` (defined) differ only in their μ field (0 vs 1) and have identical bare observables. A ϕ satisfying the equation would force $0 = 1$. Contradiction. □ □

> ***Plain reading.*** The Thiele μ counter is not a function of the three-field classical observable (`pc`, `regs`, `mem`). Two Thiele states can agree on the classical observable and still have different μ values. The cost-ledger separation between Thiele states that the classical observable cannot tell apart is not a derived fact that a clever classical observer could compute; it is a state distinction the classical observer's signature is structurally blind to.

Theorem 3.7 (Cert-addr irrecoverability, `cert_addr_not_function_of_forget`)**.** *There is no function* ϕ : `TMSnapshot` → `nat` *such that, for every Thiele state* s*,*

$$\phi(\texttt{forget}\, s) = s.\texttt{vm\_csrs.csr\_cert\_addr}.$$

Proof. The witnesses `cert_addr_witness_zero` and `cert_addr_witness_one` (defined) differ only in their `csr_cert_addr` field (0 vs 1) and have identical `forget` images. A ϕ satisfying the equation would force $0 = 1$. Contradiction. □ □

> ***Plain reading.*** The Thiele `csr_cert_addr` channel is not a function of the four fields kept by `forget`. The structural-axis diagonal in Section 19 below builds an undecidable predicate over `csr_cert_addr` reachability; that predicate cannot be re-expressed as a predicate over those four fields because the field it references is not visible in the classical signature.

The three subsumption-witness corollaries.

The irrecoverability theorems above ground three statements the subsumption argument relies on. These are the corollaries the monograph cites as "the Thiele content the bare projection structurally cannot carry."

1. *A2 cannot be stated as a constraint on the bare projection alone.* A2 (`cs_cert_costs`, a typed field on `CertificationSystem`) says that a cert-predicate flip from `false` to `true` forces instruction cost ≥ 1. By cert irrecoverability, any cert-predicate phrased as a function of the bare `TMSnapshot` cannot agree with Thiele's `vm_certified` on every Thiele state. So A2 on the bare classical signature is not Thiele's A2. It constrains a different predicate. The Coq corollary is `no_classical_a2_cert_predicate`.
2. *The cost-ledger observability separation has no analog in the bare projection.* The separation in Section 12 below names two Thiele states with identical (`pc`, `regs`, `mem`) but different μ. As a predicate on the bare classical observable, the separation would have to read μ from an observable that does not contain it. μ-irrecoverability forbids that. The separation lives at the Thiele level by structural necessity, not by drafting choice. The Coq corollary is `no_classical_mu_separation_predicate`.
3. *The structural-axis diagonal has no analog in the bare projection.* The undecidable predicate constructed in Section 19 branches on `csr_cert_addr` reachability. By cert-addr irrecoverability, no function on the bare `TMSnapshot` can recover `csr_cert_addr`. Lift the diagonal predicate down to programs interpreted on the bare projection alone and it references a field the projection erased. The lift is therefore ill-defined. The Thiele-axis undecidability is not the halting problem in disguise; it is structurally distinct because the diagonal predicate references a Thiele-only field. The Coq corollary is `no_classical_cert_addr_predicate`.

No canonical lift.

The fair question to ask here is: sure, the projection drops fields, but couldn't a clever lift pick them back up canonically? I wondered the same thing. The fiber theorem settles it, and the answer is no. The theorem `fiber_has_two_preimages` proves that every `TMSnapshot` has at least two distinct Thiele preimages under `forget`, differing in the dropped fields. Any attempt to define a lift `TMSnapshot` $\to$ `VMState` that commutes with the projection must therefore choose, at every snapshot, which preimage to call canonical. That choice is the external data the lift requires. There is no left inverse to `forget` that is forced by the projection itself; any such inverse smuggles in a cost schedule, a graph state, a cert flag, or some other Thiele-only datum that the classical signature does not provide.

This is the directionality of the subsumption, and it is generic to any lossy-projection pair. The projection `Thiele` $\to$ `Turing` is canonical: it is the unique forgetful map onto the Turing data, and it is a structure-preserving quotient (parts 2 and 3 of the degenerate projection theorem). The lift `Turing` $\to$ `Thiele` is not canonical: every `lift_config`-style definition fixes a choice of $\mu = 0$, empty graph, and `vm_certified = false`, and those choices are appended, not extracted. Every Thiele state has a Turing projection; no Turing snapshot has a canonical Thiele lift.

Directionality on its own buys a richer extension, not priority: a tagged field-drop has the same shape; priority is carried by substitution-adequacy and the lattice-scoped minimality below.

The corollary, in one line: *Turing-complete and Thiele-complete are not the same class of objects.* Any Thiele-complete substrate has a Turing-complete shadow under projection. No Turing-complete substrate admits a canonical Thiele structure without external choice of the cost schedule and the structural axis. The subsumption is directional, not symmetric, and `fiber_has_two_preimages` plus the three irrecoverability theorems prove what that buys: the classical signature is structurally blind to the dropped fields, so the law-versus-checker gap cannot be bridged by any derived predicate over the classical state. The directionality is generic; the structural blindness is what feeds the priority argument.

Categorical universal property: Thiele as initial A2-respecting substrate.
The asymmetry pinned down above has a categorical counterpart: the Thiele substrate is initial in the category of A2-respecting substrates over the `vm_instruction` signature. The theorem `thiele_is_initial_a2_substrate` (proved with no axioms) packages the universal property. For any A2-respecting substrate M over `vm_instruction` and chosen basepoint s_0^M, the canonical trace-fold `trace` $\mapsto$ `fold_left` M.`ccm_step` `trace` s_0^M is the unique morphism preserving the basepoint and commuting with step-extension. Any other g satisfying those two conditions agrees with it on every trace; the proof is induction on traces using step-commutation.

The state-level corollary `thiele_morphism_unique_on_reachable` says the same thing where states can feel it. Any two `CertCostMorphism` maps from the Thiele substrate to M that agree on `init_state` agree on every state reachable from `init_state`. Unreachable states are outside the theorem's jurisdiction. The universal property never touched them.

This is initiality in the actual categorical sense, not slogan initiality. The category is A2-respecting substrates over `vm_instruction`. The morphisms are trace-fold maps on states reachable from a chosen basepoint. In that category, the Thiele substrate is the initial object. The theorem is the term-algebra universal property of an essentially-algebraic theory: build the free model over its own signature, and every other model gets exactly one morphism out of it. The abstract uses the $\mathbb{Z}$-as-initial-ring analogy as a handrail. That analogy is also the warning label: $\mathbb{Z}$ has this property too, so generic initiality buys no priority. Priority rides instead on substitution-adequacy (passing the substitution test is equivalent to being A2: `a2_equal_trust_substitution_payoff`) and the lattice-scoped minimality of $(\mu, \texttt{cert})$ (`P_full_is_minimal_complete_extension`), which a $\mathbb{Z}$ with a coordinate bolted on does not have. The Coq theorem is the term-algebra statement. The proof is short: about 30 lines, all list induction. The hard part lives in the definitions it rests on: `CertCostMachine`, `CertCostMorphism`, `thiele_morphism_exists`, and `thiele_morphism_uni`

que_on_traces.

The scope is not smuggled in. Section 24 says it plainly: initiality is relative to the A2-respecting category. The proof decides initiality relative to the A2-respecting category. The proofs also decide the priority claim: classical computation is derivative, on substitution-adequacy and lattice-scoped minimality plus the law-versus-checker fact. The one thing left to meta-mathematics is whether the A2 category is cosmically privileged over every other cost-tracking category, a question the derivativeness claim never needed and never makes.

3.8 What the projection cannot verify: the verifier corollary

The non-existence theorems above proved the projection is structurally lossy: μ, `vm_certified`, and `csr_cert_addr` are not functions of the bare classical observable. The natural next move is to ask what that lossiness means for verification. If a verifier sees only the projection (a list of `StrictClassicalState` snapshots, the Turing-classical trace), what can it soundly decide?

Answer: not the claims that depend on the dropped fields.

Theorem 3.8 (Bare-setting verifier impossibility, `bare_setting_no_sound_complete_verifier`)**.** *There is no verifier V : `list StrictClassicalState` → `bool` that is simultaneously sound and complete on the claim "`vm_mu` $s = 1$."*

Proof. The two single-step witness states from the receipt theorem (`po1_state_A`, cost-1 `CERTIFY 0`, and `po1_state_B`, cost-0 `PNEW [] 0`) project to the same strict-shadow trace. State A satisfies the claim; state B does not. A complete verifier must accept the honest run from A. On the same accepted transcript, soundness forces the claim to hold for every state that could explain the transcript, including B. The first requirement gives acceptance; the second contradicts B's $\mu = 0$. The Coq check assumes no axioms. □ □

> ***Plain reading.*** A verifier that sees only the classical projection of a Thiele execution cannot soundly decide claims about μ. Two different Thiele executions project to identical classical traces and disagree on whether $\mu = 1$. The verifier has nothing to distinguish them by. Soundness and completeness on μ-sensitive claims are mutually exclusive under the bare projection. This is the receipt theorem lifted from "no function recovers μ from the projection" to "no verifier soundly decides μ-sensitive claims from the projection."

Three escapes, each constructed.

Three structurally distinct ways to clear the impossibility live in the kernel as concrete verifiers, each one a closed Coq theorem.

1. *Substrate-trust*. The transcript carries the full `VMState` and the verifier reads

μ directly. `substrate_escape_succeeds` exhibits a verifier with constant cost that is both sound and complete on `vm_mu` $s = 1$. The verifier's decision function is `Nat.eqb (last t).vm_mu 1`; the proof is one inhabitation per quantifier.

2. *Hardness*. The transcript carries an unforgeable commitment; the verifier accepts the commitment under a hardness hypothesis. `hardness_escape_succeeds` exhibits a constant-cost weak-sound verifier conditional on a `HardnessHypothesis` record. The record is a parameter the verifier is constructed over, not an axiom; whether the hypothesis is realised in nature is a question for cryptography, not this kernel.
3. *Interaction*. The verifier elicits a response from the prover that pins the claim. `interactive_escape_succeeds` exhibits a constant-cost sound and complete verifier when the response set is rich enough to report μ. This is the simplest possible interactive model (one round, one fixed query), and that is all the corollary needs.

The substrate channel is the option the structural axis makes available; hardness and interaction are the two routes classical cryptography and complexity already used. The substrate channel completes the trichotomy. The trichotomy is closed at the bottom by the structural-access theorem:

Theorem 3.9 (Factorisation impossibility, `V_does_not_factor_through_classical`)**.** *For any transcript type T with a projection $\pi : T \to$ `list StrictClassicalState`, no verifier $V : T \to$ `bool` that is simultaneously sound and complete on `vm_mu` $s = 1$ factors through π. Equivalently: there exist $t_1, t_2 : T$ with $\pi(t_1) = \pi(t_2)$ and $V(t_1) \neq V(t_2)$.*

Proof. If V factored through π, then $V(t_1) = V(t_2)$ whenever $\pi(t_1) = \pi(t_2)$. The bare-setting witness-state collision lifts to T as a pair with this property. Running the bare-setting argument on the factored V reproduces the contradiction. The Coq check assumes no axioms. □ □

> ***Plain reading.*** Verification of μ-sensitive claims must access non-classical structure of the transcript. The three escapes are three concrete ways to expose that structure; the factorisation theorem says exposure is mandatory. Any verifier that pretends to live entirely on the classical projection (by reading only what the projection sees, no matter how rich the transcript otherwise) will fail on the same witness pair that the bare-setting theorem already exhibited. The substrate, the hardness commitment, and the interactive response are not three options among many; they are three instances of the only option, which is "the transcript carries non-classical structure and the verifier reads it."

4 What structure is

Everyone uses the word "structure," and the word does the work while meaning nothing in particular, which you get away with right up until you try to build a machine whose entire job is to charge for it. Then the fog has a price. So I am going to pin the word down, further down than feels necessary, because every time I have watched this argument slip, it slipped at a step earlier than anyone was looking.

4.1 Partitions

The smallest piece of structure there is, is a partition. You take everything in front of you and sort it into buckets: nothing in two buckets, nothing left on the floor. That is the whole of the definition, and yes, it is the thing you do with a kindergartner and a pile of blocks. Keep it anyway. The instant the buckets are drawn you have made a claim about the world, and the claim turns out to cost money.

Formally, a partition of a set S is a collection of non-empty pieces $S_1, S_2, \ldots, S_k$ that share no elements and together cover all of S, so every element lands in exactly one piece. I write that $S_1 \cup S_2 \cup \cdots \cup S_k = S$, with $\cup$ the set union from Section 2. Each piece I call a module, or a block, or a cell, whichever word the surrounding sentence wants.

Here is the part that is not kindergarten. The partition is not a tidier way to hold information you already had. The partition is itself information, a fact that did not exist until you drew it. Put x in S_1 and y in S_2 and you now know, for nothing and for keeps, that the two live in different groups. Before, you would have had to go and look. After, the answer is a lookup, and the lookup does not cost one cent more when S swells from ten elements to a billion. You bought a question's answer down from a search to a glance. Something paid for that. Somewhere a bill came due, whether or not anyone troubled to write it on a receipt.

Not writing it on a receipt is the classical world's whole way of life. Carve up the memory of an ordinary machine, announce "these addresses are module A, those are module B, and A and B never touch," and you have said something with teeth, something a later step can lean its full weight on and run a hundred times faster for. A Turing machine will happily hold that announcement. It writes the table onto its tape the way it writes anything, as bytes. What it cannot do is hold it as a fact the machine itself is answerable for. Nowhere in the machine is there a place that says this partition is real and somebody paid to build it; the table just sits there, asserted, and the step rule has no opinion about whether it was earned or invented. The Thiele substrate keeps that place. It carries the partition as part of its state, it charges μ the moment you build one, and the step rule will not move on while the bill is still open. That gap, between a structure you merely wrote down and a structure the machine is on the hook for, is the whole reason this section exists, and a good part of the reason the rest of the document does.

4.2 Axioms carried by modules

A partition by itself is just a set of labels, and labels are cheap because labels promise nothing. Calling something module A commits you to exactly zero about what is inside it. Structure that earns the name starts one step later, when you hang a constraint on a module and make it answer for something.

Two pieces of vocabulary, so I can say that cleanly.

A **predicate** is a yes-or-no question you can put to a thing and get a definite answer back. "$x > 5$" is a predicate about a number. "This memory region holds a sorted list" is a predicate about a region of memory. A predicate is the shape a claim takes once it is sharp enough to be checked instead of merely believed.

A **logical formula** over Boolean (true/false) variables is those variables wired together with AND ($\wedge$), OR ($\vee$), and NOT ($\neg$). Take $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$: it comes out true exactly when at least one of x_1, x_2 holds and, at the same time, at least one of $\neg x_1, x_3$ holds. A choice of true or false for every variable that makes the whole thing come out true is a satisfying assignment; one that makes it come out false does not satisfy it. That is the entire machinery, and it carries everything built on top of it.

Now hang one on a module. "Module A's region satisfies the predicate Φ_A." The capital Greek Φ_A is nothing but a name I am pinning on the predicate I want to attach; mathematicians reach for Greek out of habit and there is no magic in the ink. Φ_A can carry real content: every value in this region sits between 0 and 100, or this region encodes a satisfying assignment to the formula living in module B, or this region is sorted top to bottom.

Constraints like that are what I call axioms here, and the word needs a warning, because it already means something almost opposite. In ordinary math an axiom is a starting truth you take on faith and build out from. That is not my sense at all. A Thiele axiom is granted nothing on faith: it is checked, and the check has a price. A module carries Φ_A not because the programmer believes it, but because a step in the trace went and verified it and paid $\mu \geq 1$ to do so. The axiom is not where the trust begins. It is where the trust got bought.

I will come back to one concrete encoding in Section 10. In the 51-opcode realisation, LASSERT reads a formula from memory, checks it against two witnesses (one satisfying assignment, one falsifying assignment showing the formula is not a tautology), and either accepts or traps. The hardware checker is the Logic Engine. It is a finite-state controller in the chip: a dedicated circuit that walks the formula word by word and verifies the witnesses without calling an external constraint solver. If validation succeeds, the checked constraint package can feed the later certification path. If validation fails, LASSERT traps (Section 2: routes execution to a fixed error address) and sets an error flag. The μ cost is charged either way.

And the receipt is not off in some logbook; it is in the trace itself, at the step where the check ran, reading $\mu \geq 1$. Anyone can walk the trace and put a finger on the exact place the axiom was paid for. That is the whole of the difference between evidence and assertion: an assertion asks you to trust it, evidence shows you the spot where it was earned.

4.3 Structural gain

A trace has *structural gain* when some structural fact is uncertified at the door and certified by the time the trace stops. The plainest case: you open holding a partition nobody has vouched for, and you close holding a checked, certified satisfying-assignment witness. Something structural walked into the computation that was not there when it started.

The shallow question is whether the gain happened, and you can settle that by holding the first state next to the last and looking. The substrate is not built for the shallow question. It is built for *where*: which single step in the run is the one that turned belief into certified knowledge. Pin the trace to the board, walk it end to end, and find the step where the fact crossed from uncertified to certified.

There is always such a step, and it always cost something: a step with $\mu \geq 1$, never a free one, and it can be nowhere else. That is No Free Insight, said plainly before I have earned it on paper. The gain has a location, the location has a price, and the price is never zero.

5 Structural entitlement

It would be easy to read this whole project as one sentence, certification has a price, and stop at the doorway. That sentence is true. It is also the first cost law and the shallowest thing in here, the part you can see without going in. The room behind it is bigger, and this section is about what is in the room.

The deeper object is *structural entitlement*. A computation has it when the state contains not just data, but a structural fact later steps are allowed to use as already established. I call that fact *admissible*. Admissible means the trace earned the right: a cost-positive step paid for it, and the proof object establishing it is on hand. Admissible is not just "true." Admissible is "tracked and paid for."

Several kinds of object can be admissible in this sense: a partition (Section 4), a decomposition (a split of a problem into independent parts), a morphism (Section 11), a constraint (Section 4), a witness (a value that proves the existence of something), or a narrowed feasible set (defined below). The gap between admissible fact and raw data matters. A raw bit pattern can be copied. A certified decomposition can justify solving two smaller problems instead of one large one. At the raw-data level, a morphism ID can be forged in a register: write 42 into the register and call it "the ID of some morphism." An asserted morphism in the graph has provenance (Section 2). In the

51-opcode realisation, the trace contains the MORPH and COMPOSE steps that built the morphism, and MORPH_ASSERT then committed it. A formula sitting in memory is text. A checked formula with a satisfying assignment and a falsifying assignment is admissible narrowing: the two witnesses prove the formula is satisfiable and not a tautology, which rules out at least one possible state.

Definition 5.1 (Structural entitlement)**.** A state has structural entitlement to a claim P when three things are present:

- a structural object carried by the state, such as a partition, morphism, witness counter, or formula package;
- a checked relation between that object and the claim P;
- a trace-visible certification or assertion event that makes later use of P admissible.

Without the third part, the object may exist as raw data. The computation still has not earned the right to use it as certified structure.

Here is how the pieces connect.

5.1 The precise vocabulary

Every term below is an exact Coq definition, not a metaphor. I ran every theorem that follows through Coq, so I won't stamp "machine-checked" on each one; assume I did. That isn't what makes them true. It's how I caught the places I'd talked myself into something.

Feasible set. A feasible set Ω is the list of substrate states still on the table: everything the trace has not yet ruled out, everything still consistent with what it has checked and certified up to this point. In the kernel it is exactly that, a `list VMState`, and the literalness is the whole point. This is not "the space of possible worlds" or any other phrase that lets the possibilities stay vague and uncounted while sounding profound. It is a finite list of concrete states you could print to a page. Early in a run, before the computation has certified anything, the list is long, because almost nothing has been excluded yet. Each time a constraint gets certified, the list gets shorter, because the states that violate that constraint are no longer live and they come off. Learning, on this substrate, is the list getting shorter, and it is concrete enough that you can stand over it and watch the states drop.

Prior and posterior. Two snapshots of that list, before and after one observation or certification event. The *prior* Ω is the list as it stood going in; the *posterior* Ω' is the list coming out. The posterior can only ever be a subset of the prior, $\Omega' \subseteq \Omega$, and the one-directionality of that is not a bookkeeping convenience. You can rule a state out and

you can never rule one back in, because ruling-in would mean un-learning something the trace already paid to establish, and nothing in the substrate runs that way. When the posterior comes out strictly smaller, $|\Omega'| < |\Omega|$, the event genuinely cut the field: states that were live going in are dead coming out, killed by a structural claim the trace certified, a claim those exact states fail. Whatever is still standing afterward is the posterior.

Distinguishing observation. A distinguishing observation is a function that separates an eliminated state from the posterior. Formally: there exists at least one eliminated state $s_w \in \Omega \setminus \Omega'$ such that the observation function outputs a value on s_w that no state in Ω' produces. The observation is the "test" that reveals the structure. No named test, no exclusion: the posterior only genuinely rules out something the prior allowed once you can point at the test that does the ruling-out. Name it and the whole thing becomes checkable by anyone, including someone who thinks I am wrong. That is the entire appeal.

Strictly stronger predicate. The word "stronger" here runs backward from how it sounds. In ordinary talk the stronger claim is the bolder one, the one that says more; in logic the stronger predicate is the one that lets fewer things through. P_1 is strictly stronger than P_2 when it accepts a strict subset of what P_2 accepts. Formally in Coq: $P_1 \leq P_2$ means whenever P_1 is true, P_2 is true too, and strictness means there exists an observation where P_2 is true but P_1 is false. So P_1 is the harder claim to satisfy, not the louder one: it rules more states out, and everything that clears its bar clears the lower one for free. If P_2 is "the state is in Ω" and P_1 is "the state is in $\Omega' \subsetneq \Omega$", then P_1 is strictly stronger, anything in Ω' is in Ω, and not the other way around. The inversion is worth holding onto. Loose talk reaches for "stronger" to mean grander, and here it means smaller, and the whole vocabulary of narrowing only works once you take it the logician's way round.

Structure-addition event. In the 51-opcode realisation, a structure-addition event is a single execution step where the assertion register `csr_cert_addr` transitions from 0 to nonzero. Precisely: s.`vm_csrs.csr_cert_addr` $= 0$ before the step and (`vm_apply` s i).`vm_csrs.csr_cert_addr` $\neq 0$ after. This happens only through a successful MORPH_ASSERT whose property string carries a non-NUL byte, which stores that property's nonzero ASCII checksum in `csr_cert_addr`. Not "structural knowledge was acquired" in fog. This field flipped from zero, at this step, because MORPH_ASSERT asserted such a property against an existing morphism.

Supra-certification (`has_supra_cert`). In the 51-opcode realisation, supra-certification is reached when `csr_cert_addr` $\neq 0$. The name distinguishes it from `vm_certified`, the top-level flag set by CERTIFY. Supra-certification tracks the MORPH_ASSERT channel: has a morphism property been asserted and its checksum registered? A trace can set `vm_certified = true` without `has_supra_cert` by

using CERTIFY and not MORPH_ASSERT; CERTIFY never touches `csr_cert_addr`. Both are positive-cost certification channels. They record different things. Two channels, two costs, one substrate.

Decision tree. A decision tree is an explicit witness to *how* the trace's observations branch over the state space. At each node, the tree records: here is an observation, these states passed it, these states failed it. The leaves record which states ended up in the posterior. The tree is a proof object, not a runtime structure: it exists to show that the transition from prior to posterior was the result of a definite sequence of binary tests rather than a magic jump. Coq makes you construct it explicitly as a witness, and that is the kind thing it does for you. A shortcut isn't sound on its word: it's sound once the actual branching structure that produced the posterior is on the table, and not one second before. The tree is that structure, made to hand over.

Posterior-representative reduction. Given a prior state $s \in \Omega$, a posterior-representative reduction provides a function mapping s to a representative $s' \in \Omega'$. This proves that every prior state corresponds to something in the posterior. The purpose is accounting: the posterior was not reached by arbitrary deletion, but by a coherent map where each prior state is represented. Without this, the narrowing can be fake. Delete states, announce a small posterior, and provide no principled connection to the prior.

Theorem 5.1 (Classical blindness as quotient)**.** *The four-field projection keeps only (PC, μ, registers, memory). It is many-to-one: different Thiele substrate states can land on the same image. Section 2 calls that a quotient. The projection identifies states that agree on the kept fields, even if they differ in certification, witness counters, or morphism graph. The richer six-field `shadow_proj` keeps `vm_certified`, but still forgets the morphism graph. No function of either projection alone can separate the morphism-witness states.*

Proof. Section 12 gives the witness pair. s_1 carries `pg_morphisms` $= [(0, \mathrm{id})]$. s_2 carries `pg_morphisms` $= []$. Everything the projections keep is identical: `vm_regs` $= []$, `vm_mem` $= []$, `vm_pc` $= 0$, `vm_mu` $= 0$, `vm_err` $=$ false, `vm_certified` $=$ false. The four-field projection and six-field `shadow_proj` both map them to the same image. They are different states with equal projections. Any classifier using only the projection-image must agree on equal images, so it agrees on s_1 and s_2. □ □

> ***Plain reading.*** Same witness as Categorical Separation, read backward. Two Thiele substrate states have identical projections under any projection that drops the morphism graph. They are still different states. So the projection is provably many-to-one. A function defined only on the projection output can't tell them apart. The projection already collapsed them.

If your state model only records registers, memory, PC, an error bit, and maybe μ, you

have already thrown away part of what matters. You have collapsed states that differ on the inside into a single raw state that looks the same from the outside. That is not a philosophical complaint. It is a theorem about a projection map.

Theorem 5.2 (μ-ledger necessity, complete independence classification)**.** *For two state components X and Y, I will write $X \perp Y$ to mean: knowing Y tells you nothing about X. I am redefining $\perp$ locally. Elsewhere it can mean orthogonality; here it means independence. Concretely, there are two reachable substrate states with the same Y and different X, and the construction works from every reachable state. Not one lucky start. Five independence results hold at once: (1) $\mu \perp (\texttt{mem}, \texttt{regs}, \texttt{pc})$; (2) $\texttt{certified} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc})$; (3) $\texttt{certified} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu)$; (4) $\mu \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \texttt{certified})$; (5) $\texttt{vm\_graph} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$. The three structural components (μ, vm_certified, vm_graph) do not fall out of the classical projected fields. Nor does any one of them fall out of the others. The projection $(\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$ is the minimal complete extension for ledger semantics: drop μ or vm_certified, and the other becomes undetermined. This holds from every reachable substrate state. No program prefix, however long, collapses the separation.*

Proof. Each claim is a witness pair: two reachable Thiele substrate states agree on the named projection-tuple and differ on the named coordinate.

(5) $\texttt{vm\_graph} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$. The witness is Categorical Separation: same projected fields, different graph. s_1 carries $\texttt{pg\_morphisms} = [(0, \text{id})]$; s_2 carries $\texttt{pg\_morphisms} = []$. s_2 is reachable by doing nothing. s_1 is reachable by one $\delta = 0$ `MORPH_ID` step that adds an identity morphism and leaves the projected fields unchanged. From any reachable s_0, append the same step and get the same kind of graph-only split.

(1) $\mu \perp (\texttt{mem}, \texttt{regs}, \texttt{pc})$. Same visible state, different ledger. One branch runs a $\delta = 0$ no-op like `ADD 0 0 0 0`. The other uses `EMIT` with $\delta = 0$. Registers, memory, and pc match. μ does not: 9 versus 0. The Coq witness constructs it.

(2) $\texttt{certified} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc})$. Run `CERTIFY 0` in one branch. In the other, skip certification and use a $\mu = 1$ no-op, built from a single-bit `EMIT` plus padding, to match pc. The visible tuple matches. `vm_certified` does not.

(3) $\texttt{certified} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu)$. Case (2) with the ledger tied down. The branches still disagree on `vm_certified`, but they now match μ as well. Cost-equivalent steps do the padding without moving the projected fields.

(4) $\mu \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \texttt{certified})$. Same visible tuple. Same certification flag. Different ledger. That is the dual of (3), witnessed by positive-cost steps with identical projected effects.

The universal quantifier "from every reachable starting state" is not decoration. Each

construction is parametric: from any reachable state, the no-op and cost-positive padding steps are still available. No prefix, however long, prevents the witness pair.

Minimality. Drop μ from the projection and witnesses (1), (4) collapse into equal classes: the pair witnessing μ independence is forgotten. Drop `certified` and witnesses (2), (3) collapse the same way. Each dropped field forgets an independence axis. The five-field projection is the smallest projection in this family for which the three structural components (μ, `certified`, `vm_graph`) remain independent of the rest. □ □

> ***Plain reading.*** Five independence claims, one witness per claim, all parametric in the starting state. The headline case (5) is the categorical-separation pair: same five projected fields, different morphism graphs. The other four are pairs of states reachable by short programs that match on the named tuple but disagree on the named coordinate. Minimality says: drop any one field and you've broken one of the five witnesses. So the five-field projection (`mem`, `regs`, `pc`, μ, `certified`) is the smallest projection in this family that keeps all three structural axes (cost-ledger, cert-flag, morphism graph) independent of the rest. None of those three is a function of the others plus the classical projected fields. Three axes, five witnesses, one minimal projection.

The ledger is not just a counter someone forgot to derive. There are explicit witness states with identical Turing/RAM-visible state and incompatible receipts. Not just from one starting point. From every reachable substrate state. Cost, certification, and graph structure are three independent dimensions, and the five-field projection I just proved minimal is the smallest state that keeps all three expressible at all. Classical computation is exactly the projection that throws them out. Not a poorer set of coordinates over the same complete object, the way tagged-$\mathbb{Z}$ is a richer $\mathbb{Z}$ and $\mathbb{Z}$ is fine without the tag. The drop is structural: A2 stops being a law of the step rule and becomes, at best, a checker someone runs after the fact from outside, and a checker can be wrong where a law cannot. So the order between the two runs one way only. Minimality says the structure was there to keep; the law-versus-checker gap says the one thing you lose by dropping it is the one thing A2 ever was. Put those together and classical computation comes out derivative: not a peer coordinate system that happens to record less, but the object you are left holding after the substrate's own rule has been demoted to a check you run by hand and can get wrong. That is what it has been all along.

Theorem 5.3 (Latent μ: the cost is intrinsic to computation)**.** *Run any trace. The μ it accumulates is just the sum of the step costs. Partition graph, registers, memory, program counter, certified flag, even starting μ: none of them changes that sum. Formally:*

$$\mu_{final} = \mu_{initial} + \sum_{i \in prog} \texttt{instruction\_cost}(i).$$

The right side uses the summation symbol from Section 2. It reads: add up `instruction_cost(i)` *for every program element i in prog. The result does not depend on the starting state at all. Two runs of the same program from any two starting states accumulate the same additional μ. The change in μ depends only on the program, not on the starting state. Any accounting system that assigns zero to the starting state and increases by exactly* `instruction_cost` *at each step ends up with exactly* `trace_total_cost` *on every reachable state. There is no alternative honest accounting. Any program containing a positive-cost step accumulates strictly positive μ. There is no free computation.*

Proof. Induct on *prog*. At each step, apply μ-Conservation.

Base (prog $= []$*).* Empty program. No steps. The sum is 0, so both sides reduce to μ_{initial}. The equation holds.

Step (prog $= i ::$ rest*).* Let $s_1 = \texttt{vm\_apply}(s_0, i)$. By μ-Conservation,

$$s_1.\mu = s_0.\mu + \texttt{instruction\_cost}(i).$$

Apply the induction hypothesis to rest starting from s_1:

$$\mu_{\text{final}} = s_1.\mu + \textstyle\sum_{j \in \text{rest}} \texttt{instruction\_cost}(j).$$

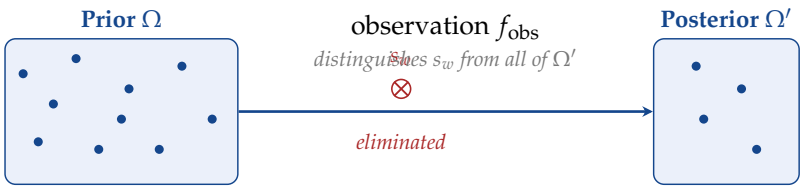
Substitute $s_1.\mu$:

$$\begin{aligned}\mu_{\text{final}} &= s_0.\mu + \texttt{instruction\_cost}(i) + \textstyle\sum_{j \in \text{rest}} \texttt{instruction\_cost}(j) \\ &= \mu_{\text{initial}} + \textstyle\sum_{i \in \mathit{prog}} \texttt{instruction\_cost}(i).\end{aligned}$$

The right-hand expression depends only on *prog* and μ_{initial}. No other starting-state field gets a vote. If any step has positive cost, then $\Delta\mu$ is strictly positive: all terms are non-negative, and that term is ≥ 1. □ □

> ***Plain reading.*** The change in μ across a run is just the sum of the per-step costs. μ-Conservation handles each step. Induction stitches them together. Two different starting states with the same starting μ end up at the same final μ on the same program. The starting graph, registers, memory: none of it matters for the μ count. Only the program does. The cost is attached to the trace, not invented by the state that carries it. The substrate records the receipt; it does not make the receipt up.

The theorem is not saying μ is outside the Thiele substrate. The μ ledger is one of the substrate's load-bearing parts. The point is different: the number it records is forced by the step sequence. Any honest accounting gives the same number. Classical execution pays the same latent bill without storing the receipt. The ledger does not create the cost. It reveals it.



$$\Omega' \subsetneq \Omega \quad \text{and} \quad f_{\text{obs}}(s_w) \notin f_{\text{obs}}(\Omega') \quad \Longrightarrow \quad \log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'| \;\leq\; \Delta\mu$$

Figure 3: Feasible-set narrowing and the entropy lower bound. A strict subset plus a distinguishing observation yields a strictly stronger posterior predicate (one fewer admissible state, witnessed by s_w). The number of yes/no bits the narrowing buys is at most the μ-cost of the trace that produced it. Same event, two sides of one equation.

Theorem 5.4 (Feasible narrowing becomes predicate strengthening)**.** *Suppose a posterior feasible set Ω' is a strict subset of a prior feasible set Ω. Suppose also that an observation distinguishes a removed state from all posterior states. Then the membership predicate for Ω' is strictly stronger than the membership predicate for Ω. (The membership predicate for Ω is the function that takes a state and returns true exactly when the state is in Ω.)*

Proof. Define $P_\Omega(s) := (s \in \Omega)$ and $P_{\Omega'}(s) := (s \in \Omega')$. "Strictly stronger" has two parts.

(a) $P_{\Omega'}$ implies P_Ω. If $P_{\Omega'}(s)$ holds, then $s \in \Omega'$. Since $\Omega' \subseteq \Omega$, we get $s \in \Omega$. So $P_\Omega(s)$ holds.

(b) P_Ω does not imply $P_{\Omega'}$. Since $\Omega' \subsetneq \Omega$, there exists $s_w \in \Omega \setminus \Omega'$. So $P_\Omega(s_w)$ is true and $P_{\Omega'}(s_w)$ is false. That is enough for strictness. The distinguishing observation gives more: an explicit Boolean function witnessing s_w, not just the assertion that some such state exists. □ □

> ***Plain reading.*** Smaller set, stronger predicate. That is the whole of (a), and it is pure set inclusion doing the work. (b) is where the distinguishing observation earns its rent. Without it you have an existential: somebody, somewhere, lives in the gap. With it you have a name and an address. An existential is a promise. A witness is the proof of the promise.

It feels like two separate things happened. First you did some work and the search space got smaller; then, as a reward, you got to assert something stronger. Two events, cause and effect, the second one earned by the first. That ordering is the illusion. There is no second event. The instant Ω shrinks to Ω' the stronger predicate already exists, because the predicate is nothing but membership in the set, and the set is smaller. Nobody pulled the predicate out of a hat to make the theorem go through; it was sitting there the whole time, defined by the boundary that just got redrawn. The distinguishing observation is what lets you write the boundary down honestly, but

it does not author a new claim on top of the narrowing. The narrowing is the claim. One fact wearing two costumes: shrink the feasible set and tighten the predicate are the same move read off opposite sides of the same equation.

Theorem 5.5 (Strengthening requires structure addition)**.** *If a trace starts with no certification address, ends certified for a strictly stronger predicate, and that certification is tied to the supra-certification channel defined above, then the trace contains a structure-addition event.*

Proof. "No certification address at start" is $s_0.\texttt{vm\_csrs.csr\_cert\_addr} = 0$. "Certified for the supra-cert channel at end" is $s_n.\texttt{vm\_csrs.csr\_cert\_addr} \neq 0$, the definition of `has_supra_cert`. The trace runs from s_0 to s_n through a finite sequence of steps. The field `csr_cert_addr` starts at 0 and ends nonzero. Somewhere in the trace, it flipped. That step is, by the definition in Section 5, a structure-addition event. The strictly-stronger-predicate hypothesis names what the certification is *for*; it does not affect the existence of the flip. □ □

> ***Plain reading.*** Cert address starts at 0, ends nonzero. So somewhere along the trace, it flipped. That flip IS what we defined "structure-addition event" to be in Section 5. The proof is just observing: if the value went from 0 to nonzero across a finite trace, the change happened on some specific step. The mathematics is bookkeeping.

This is the formal answer to "where did the new admissible structure enter?" Arithmetic on its own never answers that, because arithmetic only ever hands back what was already derivable from what you fed it; it closes a question, it does not open the world up to a fact that was not already in the premises. A stronger admissible predicate is exactly such a new fact, so it could not have fallen out of the computation for free. It had to be added, and the theorem says where: not in a flourish at the end, not off the page, but at one specific step you can point to in the trace, a structure-addition event with a timestamp.

Theorem 5.6 (Universal certification cost)**.** *Take any abstract computational system: states, step labels, step rule, cost function, certification predicate. If every one-step move from uncertified to certified costs at least 1, then every trace from uncertified to certified has total cost at least 1.*

Proof. This is the same theorem as the Universal No Free Certification result in Section 8, restated here at the structural-entitlement layer. The proof is the trace-induction argument given there. Empty trace cannot flip the cert predicate. A non-empty trace either flips it at the first step, where single-step A2 gives ≥ 1, or flips it later, where the induction hypothesis gives ≥ 1 on the rest. Either case yields total cost ≥ 1. □ □

> ***Plain reading.*** This is the same theorem as the one in the NoFI section, carried up here on purpose. Down there it was proved as a fact about cost. Up here the structural-entitlement layer needs that same floor as one of its own load-bearing parts, not as a footnote pointing back down the page, because the whole argument about who gets to claim a stronger predicate stands on it: certification never comes free wherever A2 is in force. Accept A2 and you have already agreed that flipping certification on costs you something. A2 acceptance is the entry fee.

This theorem matters because it is not about the 51-opcode ISA. The cost floor is forced for any system the moment that system has a certification predicate and a cost rule that charges for certification transitions. Build the predicate so it can flip at zero cost and you have free certification by construction. And a system that refuses to represent certification at all has no way to talk about structural entitlement from the inside. The entitlement gets smuggled in from outside, and somebody signs their name to it. Smuggled entitlement is not entitlement, no matter whose name is on it.

Theorem 5.7 (Structural entitlement representation)**.** *For any explicit witness of structural entitlement consisting of:*

- *a strict feasible-set narrowing* $\Omega' \subsetneq \Omega$*;*
- *an observation function that distinguishes at least one eliminated witness state from all posterior states;*
- *a certified posterior predicate built from membership in* Ω'*;*
- *a decision tree* realized by the trace*: the trace's actual sequence of observations matches the tree's branching structure, and each branch in the tree corresponds to a test that actually occurs in the trace; and*
- *a posterior-representative reduction tying the prior states to posterior representatives,*

the posterior predicate is strictly stronger than the prior predicate, the trace contains a structure-addition event, and the size of the narrowing is bounded by the paid ledger:

$$\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'| \leq \Delta\mu.$$

Here $|\Omega|$ *means the number of states in the prior feasible set, using the* $|\cdot|$ *notation from Section 2.* $|\Omega'|$ *is the posterior size.* $\log_2^{\uparrow}$ *means: take the base-2 logarithm and round up. It counts yes-or-no questions. A 1000-state set needs 10 binary questions because* $2^{10} = 1024$*, the first power of two that can cover 1000 items. Both sides of this bound are integer question-counts, not real-valued entropies. The eye wants to read* $\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'|$ *as the textbook bit-gain* $\log_2(|\Omega|/|\Omega'|)$*, and that is the one reading to set aside: the two ceilings round in opposite directions, so the difference lands above that real-valued number about as often as below it. What it counts instead is the binary distinctions the narrowing forces the tree to make, and that integer (not its real-valued cousin) is what the proof bounds by the tree's depth,*

and the depth by $\Delta\mu$. $\Delta\mu$ is final μ minus initial μ across the trace.

Proof. Three conclusions, chained.

(i) Strictly stronger predicate. The strict narrowing $\Omega' \subsetneq \Omega$ and the distinguishing observation match the hypotheses of the theorem above. Apply it. The membership predicate of Ω' is strictly stronger than that of Ω.

(ii) Structure-addition event. By hypothesis, the posterior certification uses the supra-cert channel. So the trace starts with `csr_cert_addr` $= 0$ and ends with `csr_cert_addr` $\neq 0$. The Strengthening-requires-structure-addition theorem gives a structure-addition event at some step k^*.

(iii) The $\log_2^\uparrow$ bound. The realized decision tree T is binary; write ℓ for its leaf count. Sort the prior states by which posterior state they end at. That is $|\Omega'|$ groups, and the covering hypothesis (`tree_covers_feasible_reduction`, itself discharged from the trace's fibered reduction by `fibered_reduction_implies_tree_cover`) caps every group at ℓ states. So $|\Omega| \leq \ell \cdot |\Omega'|$. Take $\log_2^\uparrow$ of both sides and use that it is subadditive on products, $\log_2^\uparrow(ab) \leq \log_2^\uparrow a + \log_2^\uparrow b$:

$$\log_2^\uparrow |\Omega| \;\leq\; \log_2^\uparrow \ell + \log_2^\uparrow |\Omega'|, \qquad \text{equivalently} \qquad \log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \;\leq\; \log_2^\uparrow \ell.$$

A binary tree holds at most 2^d leaves at depth d, so $\log_2^\uparrow \ell \leq d$, and the two combine to

$$\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \;\leq\; \log_2^\uparrow \ell \;\leq\; d.$$

Each internal node of T is paid for by one certified observation in the trace. The realized-by-the-trace condition says the Boolean observation actually occurs; the certified-posterior hypothesis ties it to a cert-channel activation. That makes each certified observation a cert-setter step. Section 7 charges each such step at least 1 in μ. So the cert-setter steps cost at least d total, and that total is at most $\Delta\mu$, the cost of the whole trace. Chain the inequalities:

$$\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'| \;\leq\; d \;\leq\; (\text{number of cert-setter steps}) \;\leq\; \Delta\mu. \quad \square$$

□

Plain reading. Three conclusions, three steps. Conclusion 1 is just the predicate-strengthening theorem, applied to the strict-narrowing and distinguishing-observation hypotheses. Conclusion 2 is just the structure-addition theorem, applied to the trace's start-uncertified, end-certified condition. Conclusion 3 is the entropy bound. A binary tree that compresses $|\Omega|$ states into $|\Omega'|$ leaf-classes has depth at least $\lceil \log_2(|\Omega|/|\Omega'|) \rceil$, bounded by $\log_2^\uparrow |\Omega| - \log_2^\uparrow |\Omega'|$ on the underestimate side. Each tree node is a certified observation. Each certified observation pays at least 1 in μ. The tree depth is bounded by $\Delta\mu$. Compression for less than

you paid never shows up, because the ledger does not extend that kind of credit. The bound is not a price anyone gets to negotiate down; the universe already settled it.

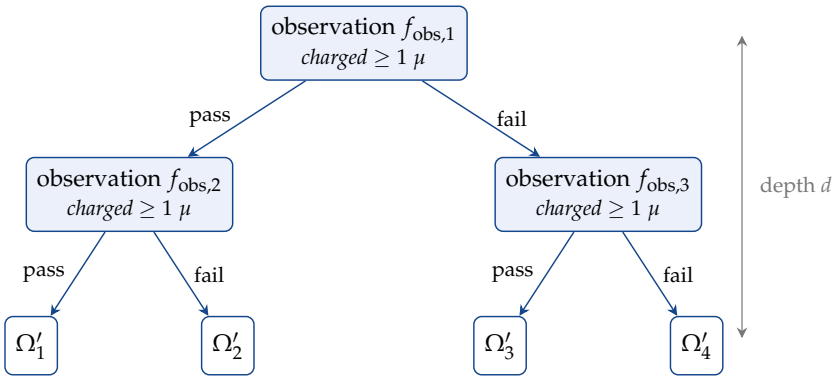


Figure 4: The decision tree witness. Each internal node is a certified observation; each leaf is one posterior class. The realised-by-the-trace condition pins every internal node to a cert-setter step in the trace. The tree's depth is what bounds the entropy gain on the right, and the μ paid for the trace is what bounds the depth on the left.

This theorem does not quantify over the English word "structure." It defines the witness class: strict narrowing, distinguishability, certification, and an explicit tree/fiber representative for the reduction. For every member of that class, structural entitlement is not metadata and not a slogan. It is predicate strengthening. It requires structure addition. Its quantified narrowing is charged to μ. That is what entitlement means here. Not a feeling, not a story.

Theorem 5.8 (Every sound structural shortcut lands here)**.** *A* `SoundStructuralShortcut` *is a package containing exactly the receipts a shortcut must provide to count as sound: a prior feasible set, a posterior feasible set, strict narrowing, a distinguishing observation, a certified posterior predicate, a realized decision tree, and a posterior-representative reduction. For every such package, the structural-entitlement representation theorem applies. The shortcut lands in strictly stronger posterior knowledge, a structure-addition event, and the $\Delta\mu$ lower bound.*

Proof. By construction, a `SoundStructuralShortcut` Coq record contains: Ω, Ω' with $\Omega' \subsetneq \Omega$, an observation function f_{obs} with a distinguishing-witness proof, a decision tree T with a tree-realized-by-the-trace proof, a posterior-representative reduction with a representatives proof, and an initial-uncertified proof on the starting state. These fields and obligations are exactly the hypotheses of the Structural-Entitlement-Representation theorem. Project the seven data fields and eight proof obligations from the record into the theorem's hypotheses. Apply the theorem. The three conclusions (strictly stronger predicate, structure-addition event, $\log_2^{\uparrow}$-narrowing $\leq \Delta\mu$) follow. □ □

> ***Plain reading.*** The record is exactly what the representation theorem needs as input. Unpack the record. Plug the fields into the theorem. Read off the three conclusions. The class is non-empty because `factored_n1_shortcut` builds a member of the class on the page, by name. Non-empty by exhibition, not by reasoning.

This is the formal version of "it has to." Once a shortcut stops being a human story and becomes a sound object the substrate may use, it has to carry the data that makes it admissible. In this framework, that data is exactly the structural-entitlement witness. When symmetry, factorization, modularity, independence, or decomposition is named as the source of a shortcut, the next question is no longer mystical. It is a checklist: the prior set, the posterior set, the observation that rules something out, and the representative witness and tree witness. With those in hand the shortcut is admissible. Without them the structure is still sitting outside the computation, however good the story around it sounds.

5.2 What narrowing buys

There's a concrete version of all this you can run, a factored search problem built to make the difference visible. Two programs go after the same answer. The blind one runs with every instruction cost set to 0 and searches the flat space, paying nothing and seeing nothing, which is the point and not an accident: a machine with no ledger gets its blindness for free. The sighted one pays first. Two receipt-visible events, each `EMIT "." 0`, for a total receipt cost of 18, and only then does it split the problem into two factors and search them apart. That 18 is the whole tab. The kernel proves the exact cost formulas, and it proves the arithmetic that makes the trade lopsided: the N^2 versus $2N$ gap doesn't merely win, it eventually dwarfs that constant μ cost by orders of magnitude, so the price you paid up front stops mattering almost immediately. The executable tests run the actual VM at representative sizes and watch the measured numbers land where the proof says they will. The receipt costs 18 once. The exponent saves you forever.

The point of this example is not the N^2 versus $2N$ speedup. Divide-and-conquer is not the new claim. Nobody is selling that. The point is the auditable trace. The receipt-visible `EMIT` events plus the certified suffix make the structural shortcut admissible to the substrate, not just asserted by the programmer who happens to feel good about it. The 18 μ is the cost of producing the receipt. The receipt is the evidence that the structural shortcut is honest.

There are two nearby claims here and it matters to keep them separate, because conflating them is exactly how a real theorem turns into a stage trick. The `EMIT`-only $N = 1$ witness closes the observation layer. It proves posterior admissibility as `CertifiedObs` (a Coq type for "a certified observation has been recorded," meaning the VM has observed and recorded a result but has not yet activated a cert channel). It does not close the stronger bridge. The trace does not end in `has_supra_cert`. In

the semantic CSR-transition sense, it does not realize structure addition. A minimally extended witness does close the full theorem honestly. Keep the factorized search exactly as it is, then append `PNEW`, `MORPH_ID`, and `MORPH_ASSERT`. That suffix fires the concrete certification channel. The extended trace lands in the full structure-addition theorem. The fix is three opcodes long. The fix exists.

That example is not a P vs NP result. Anybody who reads it as one is reading their own ambition into the bytes. It is a small, honest witness for what the substrate is meant to track. Certified structure can change which computation is admissible. The cost of acquiring that admissibility is not the same as the time saved by using it. One is the entry fee. The other is the dividend.

What the package looks like in the concrete `n=1` witness. It pays to lay the whole package open once, at the smallest factored-search case, where the object is small enough to count on your fingers and nothing has to be taken on faith. The record `SoundStructuralShortcut` carries fifteen fields. Seven are data: decoder, observation fn, representative observation fn, observation equality, prior set, posterior set, decision tree. The other eight are proof obligations standing over that data, the receipts that say the data behaves the way the fields promise. The nine semantic ingredients below cover all seven data fields plus two of those obligations, strict narrowing and initial-uncertified state; the remaining six are discharged inline by the wiring that holds the record together. Fifteen fields, nine ingredients, six wiring obligations, and that is the whole of it. Nothing here is hand-waved, because there is no room left to wave a hand: every part has a place, and every place is filled.

1. **Prior feasible set Ω.** `sighted_n1_supra_prior := [init_state; sighted_n1_supra_final]`. A literal two-element list of `VMState` values.
2. **Posterior feasible set Ω'.** `sighted_n1_supra_posterior := [sighted_n1_supra_final]`. A literal one-element list.
3. **Strict narrowing.** $|\Omega'| = 1 < 2 = |\Omega|$ by direct inspection of the lists.
4. **Distinguishing observation.** `sighted_n1_supra_obs_fn`. Returns `sighted_n1_supra_trace` when `vm_mu = 19`, else `[]`. The post-`MORPH_ASSERT` state has `vm_mu = 19`. The initial state has `vm_mu = 0`. The observation separates them.
5. **Decision tree.** `sighted_n1_tree := dt_branch dt_leaf dt_leaf`. A two-leaf branching tree, one branch per factor.
6. **Tree realized by the trace.** The trace's actual `EMIT`-then-search-then-`MORPH_ASSERT` sequence matches a root-to-leaf path of the tree.
7. **Posterior-representative reduction.** `sighted_n1_supra_repr_obs_fn`. Defined as `if Nat.eqb s.(vm_mu) s.(vm_mu) then [] else [instr_halt 0]`; the tautology `Nat.eqb mu mu = true` always picks the

empty-list branch, so this is the empty-observation representative dressed up as a conditional.

8. **Certified posterior predicate.** Built from `omega_predicatereceipt_list_eqbsighted_n1_supra_posteriorsighted_n1_supra_obs_fn`. The `MORPH_ASSERT` in the supra-suffix fires the certification.
9. **Initial uncertified state.** `init_state.(vm_csrs).(csr_cert_addr)=0` by definition of `init_state`.

All of that machinery only earns its keep if something actually satisfies it, and the worry with any definition this elaborate is that you have quietly written down a shape nothing in the world fits. So the lemma `sound_shortcut_from_components` does the unglamorous thing: it takes the seven data fields listed above and eight proof hypotheses, one per obligation, and out the other end comes a `SoundStructuralShortcut` record. Then the closed term `factored_n1_shortcut:SoundStructuralShortcut33sighted_n1_supra_traceinit_state` feeds it the seven fields and clears each of the eight obligations with a named lemma, and the downstream theorem `factored_n1_shortcut_lands_in_representation` runs `every_sound_structural_shortcut_lands_here` over that finished term and reads back exactly what the abstract class promised: a strictly stronger posterior predicate, a structure-addition event, and the $\log_2^{\uparrow}$ narrowing bounded by $\Delta\mu$. The payoff is small to state and it is the one I actually cared about. The class is not empty, and it is non-empty by construction, not by argument: there is a concrete shortcut built on the page that hits every clause, which means the whole apparatus is about something real and not a definition admiring itself in a mirror.

5.3 The boundary

The hard internal theorem for the explicit class is proven, and so is the boundary in both directions on a concrete witness. An `EMIT`-only factorized-search trace closes only the observation layer. The posterior predicate is certified observationally. But the trace does not reach `has_supra_cert` and does not realize semantic structure addition. A trace with the factorized search unchanged and a post-search suffix `PNEW ; MORPH_ID ; MORPH_ASSERT` closes the whole chain: stronger predicate, structure addition, and the $\Delta\mu$ lower bound. An unconditional Shannon-style bound on every deterministic single trace is something the code refuses to claim. A single execution path is not enough information. The bound needs the whole tree, or a per-branch preimage, or a distribution over traces. The proof stops where the witness stops. That is the honest place to stop, not a hole to be patched later.

Once entitlement is internalized with explicit witnesses instead of waved at, the picture firms up into three facts that lean on each other. You cannot strengthen a certified claim for free: the receipt costs μ or it does not exist. Strip a trace down to its classical shadow and the distinctions that entitlement turns on stop being visible at all, so the shadow can no longer tell you what was earned and what was merely assumed.

And narrowing the feasible set is no longer a vibe; it has a formal path into predicate strengthening with a ledger lower bound underneath it, so the act of knowing more shows up as a number that went up. Then the substrate-level result in Section 19 settles the question you would ask next, which is whether all of this can be put on autopilot: is there a uniform, internally representable procedure that takes an informal structural argument and hands back a `SoundStructuralShortcut` witness? There is not. The structural axis comes with its own undecidable predicate, which is the unsurprising and slightly bleak news that insight does not reduce to a crank you can turn.

6 What insight is, and why it cannot be free

On this substrate the word *insight* stops being a mood and starts having a meter reading. It comes in three tiers, and the tiers are not a taxonomy I laid over the top of one undifferentiated cost for tidiness. They are three genuinely different things that happen when a computation learns, and they sort themselves by price: the first is free, the second never is, and the third is where the price tag has its hand on something a classical machine cannot reach at all.

6.1 The three tiers of observation

The kernel proves each tier lands where it lands, by what the substrate makes it cost, so the split is the substrate's verdict and not my filing system. Cheapest first.

Tier 0: Raw observation (always free). A raw observation takes something in, writes it down, and stops there. It does not say what the data means, does not lean on it, and does not let any later step lean on it either. The cleanest case in the realisation is a CHSH trial, one round of the two-player game I build from scratch in Section 16: you run the round, a counter ticks up by one, and not a single structural field moves. The certification channels stay cold because nothing has been claimed, and where nothing has been claimed there is nothing to commit to and nothing to pay for. Cost: zero, and the zero is not an oversight in the cost schedule. Looking is free on this substrate the same way looking is free in the world. You can watch as long as you like and the meter never starts. It starts the instant you turn what you saw into something you are willing to stand behind, and that instant is a different tier.

Tier 1: Certified structural insight (always costs $\geq$ 1). Two cert channels. `csr_cert_addr` (the formula/certificate address, a control-status register; Section 9 explains the term). `vm_certified` (the top-level flag). Any concrete instruction that turns either channel from off to on costs at least 1. The kernel proves it: `certification_requires_positive_mu`.

Precision first. `LASSERT` reads a constraint package and charges for it. `LASSERT` does not flip either cert channel. Two opcodes do the flipping: `CERTIFY` (sets `vm_certified = true`) and `MORPH_ASSERT` (sets `csr_cert_addr` nonzero

when the assertion succeeds). `LASSERT` is the validation that precedes them. The kernel groups a broader cost-positive class: `LASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `READ_PORT`, `CERTIFY`, `MORPH_ASSERT`. Each costs at least 1 at the step level. `LASSERT` is part of the chain. The cert-channel flip happens at the subsequent `CERTIFY` or `MORPH_ASSERT`, not at `LASSERT` itself.

The chain, written out. Readers collapse two steps into one. Don't. Suppose register `r0` holds the address of a four-byte formula in memory and `r1` holds the address of a dual-witness certificate (a satisfying assignment together with a falsifying assignment, the non-triviality witness from Section 10). Step one: `LASSERT freg=r0 creg=r1 kind=SAT flen=4 delta=0` reads the formula header, walks the formula and the two witnesses through the on-chip Logic Engine, charges $4 \times 8 + 1 = 33$ to μ, and either advances the PC (validation passed) or traps to `LASSERT_TRAP_PC` and latches `vm_err = true` (validation failed). The μ cost is charged either way. Both `csr_cert_addr` and `vm_certified` stay untouched by this step, validation passing or not. Step two, conditional on step one passing: `MORPH_ASSERTmid=m property="validated_constraint"cert=""delta=0` charges $S(0) = 1$ to μ and flips `csr_cert_addr` to the ASCII checksum of `"validated_constraint"`, which is nonzero because the property string carries non-NUL bytes. The supra-cert channel is now active.

Total cost across the two steps: 34 μ. `LASSERT` paid for the validation work. `MORPH_ASSERT` paid for the channel flip. The cert channel only crossed from 0 to nonzero on the second step, not the first. The two costs are not redundant. One is the cost of checking the constraint package against its dual witness. The other is the cost of committing to the result of that check by activating the cert channel. A downstream certified predicate that references `csr_cert_addr` sees the channel-active state produced by the `MORPH_ASSERT` step and is allowed to rely on the validated constraint produced by the `LASSERT` step. Skip either step and the chain breaks. `LASSERT` alone leaves the cert channel cold. `MORPH_ASSERT` without a preceding `LASSERT` activates the channel without an underlying validated constraint. You have paid for a commitment to nothing.

Tier 2: Certified binary-game witness insight (always costs ≥ 1). The sharpest case, and it is worth slowing down on why it is sharp. Here the trace is certified and its CHSH statistics show S pushed past 2, the classical ceiling (Section 16 builds the game and the ceiling from nothing). The certification on its own is already a Tier 1 event, so it already costs at least one. But look at what is being certified. $S > 2$ means the recorded outcomes cannot have come from any local hidden variable model (Section 2): no shared-randomness-then-local-choice story, which is the whole classical account of how two separated outcomes manage to correlate, can produce them. So this tier certifies a fact that rules out an entire worldview, and the price does not flinch for the size of the thing it is pricing. Certified evidence of a game violation costs at least one to come by, on a trace of any length you care to run.

6.2 Why insight cannot be free

The proof:

Suppose a trace starts in an uncertified state and ends in a certified state. Somewhere in that trace, a specific opcode fired that flipped one of the certification registers from off to on. The two concrete cert-channel instructions writing `vm_certified` or `csr_cert_addr` are: `CERTIFY` (which sets `vm_certified = true`) and `MORPH_ASSERT` when the morphism exists (which sets `csr_cert_addr` to the ASCII checksum of the property string: nonzero for any property string with a non-NUL byte). These are the only instructions that change those fields. `CHSH_LASSERT` sits in the certification cost-discipline class ($S(\delta) \geq 1$) but does not write to `vm_certified` or `csr_cert_addr`. Its success is observable via PC advance and `vm_err` staying false (the bridge theorem operates on that signature). `MORPH_ASSERT` with an empty property string or a missing morphism does not activate the cert channel, though it still costs $S(\delta) \geq 1$. This is proven by case analysis over all 51 opcodes. Every other instruction leaves those fields unchanged. So: the cert flip happened at a `CERTIFY` or `MORPH_ASSERT` step. Look at that exact step. That step, by the semantics of the 51-opcode VM, has cost $S(\delta) \geq 1$, because both `CERTIFY` and `MORPH_ASSERT` are in the mandatory-floor class. So at that step: $\mu_{\text{after}} = \mu_{\text{before}} + S(\delta) \geq \mu_{\text{before}} + 1$. And because μ never decreases (each instruction adds a non-negative cost), $\mu_{\text{before}} \geq \mu_{\text{initial}}$. Therefore $\mu_{\text{final}} \geq \mu_{\text{after}} \geq \mu_{\text{before}} + 1 \geq \mu_{\text{initial}} + 1$.

This is not a philosophical argument. It is a structural property of the step rule. The Coq proof checks it mechanically. The proof does not say "intuitively, certification should cost something." It says: here is the step-rule definition. Here is the cost rule. Here is the lemma that says μ never decreases (the monotonicity lemma; the Section 2 sense of monotonicity is the right one). Here is the induction over the trace (Section 2 for what induction means). End of proof. The deeper reason: if you could certify structure for free, the certification would mean nothing. A receipt that any realisation can produce at zero cost proves nothing. The whole point of the μ ledger is that it cannot be faked. No trace gets to accumulate certified structural knowledge without paying for it in μ on the way. The ledger won't let it.

But cost alone is not yet semantic depth. A concrete realisation can spend resources checking a claim that turns out to be shallow. `LASSERT` does not count as structural certification merely because a formula parses and one satisfying assignment exists. It must also carry a non-triviality witness: one satisfying assignment and one falsifying assignment. That proves the asserted constraint excludes at least one state and therefore genuinely narrows the feasible set. The receipt proves spend. The witness proves the spend bought an actual narrowing.

7 The μ ledger

μ (mu) is the global structural cost ledger of the Thiele substrate. One counter, carried by every state. Not the whole substrate. The part that keeps receipts.

7.1 What mu is

μ is a single natural number riding along on every substrate state (a natural number being a non-negative integer, $0, 1, 2, \ldots$; the notation cheat sheet is in Section 2). It starts at 0 and it never goes down, and that second half is the whole character of the thing. A counter that can fall is an accountant you can lean on to revise the books. A counter that only ever climbs is a fact about history: it says how much has been spent, and it cannot be talked into saying less. μ is the second kind.

In the 51-opcode realisation, every instruction carries a cost parameter (δ) in its encoding. The VM adds that cost to μ when the instruction executes. Here is how δ is used in practice:

For **pure compute programs**, meaning programs that do arithmetic, load and store values, and jump, every instruction carries $\delta = 0$. The program I use as the concrete comparison case for the Turing Strictness theorem (`blind_program`) has this property: every instruction sits at $\delta = 0$. The proof that its total μ-cost is zero is a direct calculation. For these programs, μ stays at 0 unless a positive-cost instruction fires.

For **certification and revelation instructions** (`CERTIFY`, `LASSERT`, `MORPH_ASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `READ_PORT`): the cost is at least 1, regardless of δ. Even $\delta = 0$ still costs 1. Here is why that floor is forced, not chosen.

When `CERTIFY` fires, it commits the realisation to a certified state: `vm_certified` flips from false to true. That commitment cannot be undone: μ never goes down, and the certificate cannot be taken back. Committing to a structural fact has the same shape as erasing the "uncertified" possibility from the trace's history: once the commitment is recorded, the prior uncertainty is gone. Landauer's argument (covered in Section 17) is the physics-side motivation for charging at least 1, though the Coq proof does not depend on it. The minimum honest cost of an irreversible certification step, as a function only of the programmer-supplied δ, is $\delta + 1$: the δ the programmer declares plus the mandatory 1 for the commitment. I write that as $S(\delta)$ throughout.

Why $+1$ and not $+2$ or no floor? Because $+1$ is the minimum that makes certification impossible for free. $\delta + 0$ would allow $\delta = 0$ to certify for free. $\delta + 2$ would be overcharging. $\delta + 1$ is the unique honest minimum: you can always choose $\delta = 0$ and pay exactly 1. You can choose $\delta > 0$ and pay more. The proof that $S(\delta)$ is the only consistent cost floor is (`cost_necessity` + `cost_uniqueness`).

Throughout this document, $S(n)$ means $n + 1$. The S stands for "successor" in the basic math sense: the successor of n is the next natural number after n. So $S(0) = 1$, $S(1) = 2$, and so on. $S(\delta) = \delta + 1$ is always at least 1. `EMIT`, `REVEAL`, and `READ_PORT`

go further: their cost also grows with the information content they carry. An `EMIT` of one ASCII character (which is 8 bits, see Section 2) with $\delta = 0$ costs $8 + 1 = 9$. Ten characters costs $80 + 1 = 81$. Emitting more costs more. The price tracks the bits, every time, and there's no rate to negotiate.

For **everything outside that positive-cost class**, including `ADD`, `LOAD`, `STORE`, `JUMP`, `PNEW`, `PSPLIT`, `PMERGE`, `MORPH`, `COMPOSE`, and the rest, the VM charges exactly δ, which can be 0. Here is why these are allowed to be free.

The module operations are where the free part of the ledger actually lives, and the reason they are free is the same reason a thing is free in physics, not a courtesy I extended them. `PNEW` brings a module into existence, `PSPLIT` cuts one in two, `PMERGE` folds two back into one, and the quiet fact about all three is that nothing has been thrown away: remove the module and `PNEW` is undone, run `PSPLIT` and `PMERGE` against each other and you are exactly where you started. They are reversible, and Landauer's principle says a reversible operation erases no information and so owes no information-erasure cost. So `PNEW` at $\delta = 0$ pays nothing, and it is worth being clear that this is not a loophole stapled on to keep the bookkeeping cheap. It is the ledger refusing to charge for work that destroyed nothing. You rearranged the furniture; you did not burn any of it.

The same logic applies to `MORPH`, `COMPOSE`, and the other graph-building operations. Building the morphism chain is the organisational work. The result is structure you have not yet committed to. The cost arrives when you assert the chain via `MORPH_ASSERT`. That is the moment of commitment, and the moment of commitment costs $S(\delta) \geq 1$. You can build for free. You cannot certify for free.

μ grows whenever `instruction_cost` is positive. The certification/revelation class is forced to be positive even when its encoded δ is 0. Other instructions can remain free by carrying $\delta = 0$.

Here is the complete cost schedule:

Instruction class	Cost = `instruction_cost`
Zero-floor operations (`ADD`, `LOAD`, `JUMP`, `PNEW`, `MORPH`, ...)	δ (can be 0)
Certification floor (`CERTIFY`, `MORPH_ASSERT`, `LJOIN`)	$S(\delta) = \delta + 1$
`EMIT` (emit payload as receipt-visible trace event)	$\lvert\text{payload}\rvert_{\text{bits}} + S(\delta)$
`REVEAL` (disclose bits from a module)	$\text{bits} + S(\delta)$
`READ_PORT` (read bits from input port)	$\text{bits} + S(\delta)$
`LASSERT` (validate formula and witnesses)	$\mathit{flen} \times 8 + S(\delta)$

The $\lvert\text{payload}\rvert_{\text{bits}}$ notation means "the number of bits in the payload." A one-byte ASCII character contains 8 bits. A ten-byte string contains 80 bits.

For `LASSERT`: *flen* is the number of formula units in the encoded formula. Each unit is one byte (8 bits) in the binary encoding used by the VM. This is a design choice in the binary format. The formula header declares how many bytes follow. The VM charges 8 bits per byte. The multiplication by 8 simply converts byte-count to bit-count, matching the information content you are asking the trace to validate.

The proof that μ is always exactly $\mu_{\text{before}} + \texttt{instruction\_cost}$ for each step:

Theorem 7.1 (μ-Conservation)**.** *For any VM state s and VM instruction i:* $(\mathit{vm\_apply}\ s\ i).\mu = s.\mu + \mathit{instruction\_cost}\ i$.

Proof. Case analysis over the 51 opcode constructors of `vm_instruction`. For each constructor i, the apply rule has the form

$$(\texttt{vm\_apply}\ s\ i).\mu \;=\; s.\mu + c_i,$$

where c_i is the cost term written in the apply rule. Compare to `instruction_cost`: for zero-floor opcodes $c_i = \delta$; for certification-floor opcodes (`CERTIFY`, `MORPH_ASSERT`, `LJOIN`, `CHSH_LASSERT`) $c_i = S(\delta)$; for `EMIT` $c_i = \lvert\text{payload}\rvert_{\text{bits}} + S(\delta)$; for `REVEAL` and `READ_PORT` $c_i = \text{bits} + S(\delta)$; for `LASSERT` $c_i = \mathit{flen} \times 8 + S(\delta)$. In every case c_i matches the corresponding case of `instruction_cost` by definition. The error sub-cases (e.g., `MORPH_ASSERT` on a missing morphism, `CHSH_TRIAL` with bad bits, `LASSERT` length mismatch) follow the same cost rule: the trap or err-latch only changes the PC and the error flag; the μ update is the same. So $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + c_i = s.\mu + \texttt{instruction\_cost}(i)$ for every i. □ □

> ***Plain reading.*** Look at each of the 51 opcodes one at a time. Every apply rule writes μ as the old μ plus the cost. The cost term is exactly what `instruction_cost` returns. Including the error paths: failing an instruction costs the same as succeeding at it, because the cost is on the attempt, not the outcome. There is no opcode that escapes this. Every one of the 51 lands in the same equation.

This is not a monotonicity claim. It is an equality. μ goes up by exactly the cost of the instruction, always, for every instruction, in every case including error cases. There is no backdoor.

That equality should not be misunderstood. It means the ledger is exact about spend. It does not, by itself, mean every paid unit corresponds to deep semantic gain. On this substrate, semantic gain is stricter than spend. It requires a narrowing witness in addition to the ledger entry.

7.2 Why mu is the unique canonical cost measure

Here is the stronger claim, which took a while before the proof compiled. μ is not just a valid cost measure for this 51-opcode realisation. It is the unique one. Any other cost function that starts at zero and tracks the per-instruction cost schedule exactly must equal μ on every reachable VM state. Not "isomorphic" in some abstract sense. Literally equal, value by value, on every reachable state.

Theorem 7.2 (μ-Initiality)**.** *Let M be any function from VM states to natural numbers satisfying: (a) M assigns zero to the initial VM state, $M(\texttt{init\_state}) = 0$; and (b) M tracks the cost schedule exactly: for every VM state s and every VM instruction i, $M(\texttt{vm\_apply}\, s\, i) = M(s) + \texttt{instruction\_cost}(i)$. Then M equals μ on every reachable VM state. (Monotonicity, the property that $M(s') \geq M(s)$ when s' is reachable from s, follows from (b) because `instruction_cost` is a non-negative natural number. You do not need to state monotonicity separately.)*

Proof. Induction on the reachability witness for s. A reachable VM state is either `init_state` itself or `vm_apply` s' i for some reachable s' and some VM instruction i.

Base case ($s =$ `init_state`). By hypothesis (a), $M(\texttt{init\_state}) = 0$. By definition of the initial state in the kernel, $\texttt{init\_state}.\mu = 0$. So $M(\texttt{init\_state}) = \texttt{init\_state}.\mu$.

Step case ($s =$ `vm_apply` s' i for reachable s'). By hypothesis (b),

$$M(s) \;=\; M(s') + \texttt{instruction\_cost}(i).$$

By the induction hypothesis on s', $M(s') = s'.\mu$. By μ-Conservation,

$$s.\mu \;=\; (\texttt{vm\_apply}\, s'\, i).\mu \;=\; s'.\mu + \texttt{instruction\_cost}(i).$$

Substituting: $M(s) = s'.\mu + \texttt{instruction\_cost}(i) = s.\mu$. □ □

> ***Plain reading.*** M agrees with μ at the start (both zero). The step rule says M and μ both update by the same amount on every step (M by hypothesis (b), μ by μ-Conservation). Induction closes it. There is no alternative accounting that agrees with the cost schedule at every step, starts at zero, and disagrees with μ on a reachable VM state. The cost is forced.

In plain language: given the cost schedule, there is no alternative accounting. If you charge $S(\delta)$ for certification-floor opcodes, $|\text{payload}|_{\text{bits}} + S(\delta)$ for `EMIT`, and δ for everything else, then μ is the only function on VM states that starts at zero and tracks the schedule. What was a design choice is the schedule itself: the decision that `CERTIFY` costs $S(\delta)$ and `ADD` costs δ. But the schedule is not arbitrary either, and that is the part worth slowing down on. Hold the trust fixed and A2 is the only rule that prices certification exactly, so the only exact, non-overcharging substitute already *is* A2 (`a2_equal_trust_substitution_payoff`). And $(\mu, \texttt{certified})$ is the proved-minimal extension over (mem, regs, pc): cert is not a function of the rest, μ is not a function of the rest (`P_full_is_minimal_complete_extension`). A law that lives in the step is not a check you run afterward and hope you got right. Given-the-schedule uniqueness is the accounting; the A2-and-minimality result underneath it is the priority. No competing count can agree on the schedule, start at zero, and land on a different total, and there's nowhere left for that freedom to hide.

7.3 The LASSERT cost formula

The most important cost formula in the VM is for `LASSERT`, the opcode that validates a logical formula and its witnesses:

$$\Delta\mu_{\text{LASSERT}} = \mathit{flen} \times 8 + S(\mathit{mu\_delta})$$

flen is the encoded formula-unit count, read from the formula's own header in memory. Read, not declared. Each unit contributes eight concrete bits to the bill. The per-step mandatory charge is always at least 1, no matter how cheap the unit-count feels.

What makes this the one to watch is where the bill gets keyed. It is keyed to the formula actually sitting in memory, not to the number you typed on the instruction. Eight bits for every unit the header carries, plus the floor of at least 1 you pay for the act of checking at all, and the act of checking is not up for negotiation. So a two-unit formula lands at 16 + 1 = 17, and there is no posture you can strike to talk it down, because the count was pulled out of the formula's own header and not out of anything you declared. Declaring a smaller count does not shrink the formula; it only gets you caught. That is the whole reason you cannot smuggle a larger formula through a smaller bit count.

This is forced, not chosen. There's no way to validate a 1000-bit formula on a 10-bit budget; the cost counts the formula you actually present, byte for byte, not the formula you wished you'd encoded. The proof that this is the unique minimum cost satisfying that constraint is (`cost_necessity` + `cost_uniqueness`).

What this buys you: the trace pays for presenting and checking a constraint package, no more, no less. It does not, by itself, guarantee the constraint is meaningful. You could present a formula that is always true, a tautology, and the ledger would still charge you the full bill. The VM closes this separately via the dual-witness requirement: see the `LASSERT` opcode description in Section 10.

The obvious thing to try, and where it runs out. The programmer declares *flen* in the instruction encoding, so the first thing I wanted to know was: what stops them from writing $\mathit{flen} = 0$ for a huge formula and paying only $S(0) = 1$? It's the cheat I'd reach for first.

The VM catches it. When `LASSERT` runs, the VM reads the formula's actual encoded-unit count from the formula header in memory and compares it to whatever *flen* the instruction claimed. If they do not match, the VM traps (Section 2: routes execution to a fixed error address): the program counter jumps to `LASSERT_TRAP_PC`, which is `0xF00` in hexadecimal, decimal 3840. The error flag is set. The check does not succeed. This trap-to-fixed-address behaviour is unique to `LASSERT`. Every other error in the VM simply sets `vm_err` to true and continues advancing the program counter by 1. `LASSERT` is special because a mis-length formula is a serious integrity violation, not a recoverable error. The μ cost is still charged: that detail matters because failed checks are not free. The Coq lemmas `lassert_honest_cost` and `lassert_honest_mu_cost` prove that if `LASSERT` completes without trapping, the declared count equals the in-memory count, and the μ charge uses the real formula bits.

8 No Free Insight: the first cost law

8.1 The abstract versions

Theorem 8.1 (Universal No Free Certification)**.** *Let $\mathcal{C}$ be any abstract computational system with:*

- *a set of states;*
- *a set of step labels;*
- *a step rule (given a state and a step label, the rule produces the next state);*
- *a cost function (each step label has a non-negative integer cost);*
- *a certification predicate (a yes/no test on states, with at least one "no" state and at least one "yes" state).*

Premise (A2, the single-step cost rule): for every state s and step label i, if $\mathrm{cert}(s) = \mathrm{false}$ *and*

$\text{cert}(\text{step}(s, i)) = \text{true}$, *then* $\text{cost}(i) \geq 1$.

Conclusion: any trace of step labels $t = [i_1, i_2, \ldots, i_n]$ *that starts at state* s_0 *with* $\text{cert}(s_0) = \text{false}$ *and ends at state* s_n *with* $\text{cert}(s_n) = \text{true}$ *has total cost* $\sum_{k=1}^{n} \text{cost}(i_k) \geq 1$.

Proof. Induction on the trace t.

Base case ($t = []$). Then $s_n = s_0$, so $\text{cert}(s_n) = \text{cert}(s_0) = \text{false}$. But the hypothesis says $\text{cert}(s_n) = \text{true}$. Contradiction; the empty-trace case is vacuous.

Step case ($t = i :: \text{rest}$). Let $s_1 = \text{step}(s_0, i)$ be the state after the first step label. Case-split on $\text{cert}(s_1)$.

- *Subcase A:* $\text{cert}(s_1) = \text{true}$. The first step label flipped the certification predicate from false to true in one step. By the A2 premise, $\text{cost}(i) \geq 1$. The total cost is $\text{cost}(i) + \sum_{j \in \text{rest}} \text{cost}(j) \geq 1 + 0 = 1$, since every individual cost is a non-negative natural.
- *Subcase B:* $\text{cert}(s_1) = \text{false}$. Then the rest of the trace must do the flipping: $\text{cert}(\text{run}(\text{rest}, s_1)) = \text{true}$ with $\text{cert}(s_1) = \text{false}$. Apply the induction hypothesis at (rest, s_1) to get $\sum_{j \in \text{rest}} \text{cost}(j) \geq 1$. The total trace cost is $\text{cost}(i) + \sum_{j \in \text{rest}} \text{cost}(j) \geq 0 + 1 = 1$.

Either subcase gives total trace cost ≥ 1. □ □

> ***Plain reading.*** The cert predicate is off at the start, on at the end. So somewhere a step flipped it. Whichever step did the flipping costs at least 1 by the single-step rule. Every other step costs at least 0 because costs are non-negative naturals. Sum them up: at least 1. Empty trace can't do the flipping at all, so it's vacuous. That's the whole argument. The induction just makes precise the idea that the flip has to happen *somewhere*, and wherever it happens it pays.

This theorem does not know about registers, memory, partitions, morphisms, CHSH trials, or the 51-opcode ISA. It says: once a model has a yes/no certification test and the one-step certification transition is priced, trace-level non-free certification falls out by induction over the trace, no further premises required. If that one-step premise fails, the model permits free certification by definition. If the model has no certification predicate, it cannot talk about structural entitlement internally.

And don't read "that's the whole argument" as "the premise is the whole content." The single implication is one line from the pricing rule, yes. The content is that this rule is the only one that fits. A trusted erasure-accounting law, the same trust posture, certifies at total cost zero when nothing is erased: a real rival, a different bill (`commitment_cost_not_reducible_to_erasure_cost`). And a substitute that charges any other predicate fails the floor on a one-step trace (`substitution_test_reje`

`cts_non_a2_exact_substitute`). A tautology does not get to have a rival that loses.

Theorem 8.2 (Thiele No Free Insight)**.** *For the 51-opcode VM specifically, cert-channel activation is positive-cost, and certified predicate strengthening has to pass through a structure-addition event. The relevant certification/revelation steps all live in the positive-cost class.*

Proof. Two parts.

Cert-channel activation requires positive cost. The Both-channels theorem below establishes that any trace activating either certification channel (`csr_cert_addr` becoming nonzero or `vm_certified` becoming true) has $\Delta\mu \geq 1$. The actual channel-writing cases are covered by the positive-cost cert/revelation class: `CERTIFY`, `MORPH_ASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `LASSERT`, and `CHSH_LASSERT`. The class is deliberately broader than literal field writers; every member has `instruction_cost` $\geq S(\delta) \geq 1$, so the lower bound holds.

Certified predicate strengthening requires a structure-addition event. By hypothesis the trace ends certified for a stronger predicate via the supra-cert channel (`csr_cert_addr` nonzero). The Strengthening-requires-structure-addition theorem (Section 5) gives the structure-addition event directly. □ □

> ***Plain reading.*** This is just the abstract universal NoFI theorem instantiated for the 51-opcode realisation and read back through the Thiele substrate interface. The two concrete certification channels (`csr_cert_addr` and `vm_certified`) are both cost-positive at the step level, and the predicate-strengthening reading of NoFI falls out of the structure-addition theorem.

This is the VM-level version. The kernel proves the two concrete cert channels cannot be activated for free, by case analysis over all 51 opcodes, then adds the predicate-strengthening layer: once a stronger accepted predicate is tied to `has_supra_cert`, the trace must contain a structure-addition event somewhere in it.

8.2 The specific versions

For the 51-opcode VM concretely, there are several specific versions. The two certification channels I mentioned in Section 6 are the certificate-address register (`csr_cert_addr`) and the top-level flag (`vm_certified`). Both are defined precisely in Section 9, register by register.

Theorem 8.3 (No Free Certification, single step)**.** *If* $s.$`vm_csrs.csr_cert_addr` $= 0$ *and* $($`vm_apply` $s\ i).$`vm_csrs.csr_cert_addr` $\neq 0$*, then* `instruction_cost`$(i) \geq 1$.

Proof. Case analysis over the 51 opcode constructors. The Coq proof uses

`cert_addr_setterb`: if `csr_cert_addr` changes from 0 to nonzero, the instruction must fall inside that cost-positive class. The class includes `instr_morph_assert`, `instr_emit`, `instr_reveal`, `instr_ljoin`, and `instr_lassert`. It is a deliberately broad guardrail class, not a list of literal CSR writers. In the current `vm_apply` semantics, the direct CSR-writing success case is `instr_morph_assert`. For the theorem, the required fact is cost: every member of `cert_addr_setterb` has `instruction_cost` ≥ 1, with `instr_emit` and `instr_reveal` higher because they also charge payload bits. □ □

> ***Plain reading.*** If you see `csr_cert_addr` move from zero to nonzero, the proof catches that step inside a cost-positive guardrail class. That class is broader than the direct CSR writers, but broad is fine here: every member costs at least 1. The field cannot flip for free.

Theorem 8.4 (No Free Certification, trace level)**.** *If a trace starts at a state with* `csr_cert_addr` $= 0$ *and ends at a state with* `csr_cert_addr` $\neq 0$*, then the trace's total μ cost is ≥ 1.*

Proof. Instantiate the universal theorem above with $\mathcal{C}$ = the 51-opcode VM; cert$(s) :=$ (s.`vm_csrs.csr_cert_addr` $\neq 0$); step = `vm_apply`; cost = `instruction_cost`. The A2 premise is exactly the single-step theorem above. □ □

> ***Plain reading.*** Single-step lifts to trace-level. The universal theorem does the work.

Theorem 8.5 (No Free Certification via CERTIFY)**.** *If a trace starts at a state with* `vm_certified` $=$ false *and ends at a state with* `vm_certified` $=$ true*, then the trace's total μ cost is ≥ 1.*

Proof. Case analysis over the 51 opcodes shows that `vm_certified` flips from false to true only when i is an `instr_certify` constructor; every other opcode leaves it unchanged. The `CERTIFY` apply rule charges $S(\delta) \geq 1$. So the single-step instance of A2 holds for this channel. Instantiate the universal theorem with cert$(s) :=$ s.`vm_certified`, otherwise as above. □ □

> ***Plain reading.*** Same shape as the other channel. Only one opcode (`CERTIFY`) can flip the top-level flag. It charges ≥ 1. The universal theorem lifts that to the trace.

Theorem 8.6 (Both channels)**.** *For ANY transition of either certification channel from absent to present, whether the certificate-address register* `csr_cert_addr` *or the top-level flag* `vm_certified`*, μ grows by at least 1 over the trace.*

Proof. Suppose the trace either flips `csr_cert_addr` from 0 to nonzero, or flips `vm_certified` from false to true (or both). In the first case the trace-level

theorem (`csr_cert_addr`) applies. In the second case the trace-level theorem (`vm_certified`) applies. The disjunction is exhaustive over "cert-channel activation," so μ grows by ≥ 1 in either case. □ □

> ***Plain reading.*** Two channels, one master claim. Whichever channel got activated, the corresponding single-step rule fired and the universal theorem lifted it to the trace. There is no third way.

Theorem 8.6 is the one I actually reach for downstream. When something later in the document needs "and certification was not free," this is the line it cites, because it has already done the bookkeeping of asking which way the certification came in and answered: doesn't matter, both are priced. The certificate-address register `csr_cert_addr` and the top-level flag `vm_certified` are not two separate promises I have to keep track of; they collapse into one claim that holds no matter which door a program walks through to call itself certified.

8.3 What this means in practice

Run any program on the kernel. Watch the μ counter. If the program ends with `vm_certified = true` or with `csr_cert_addr` nonzero, then somewhere in the trace, μ went up by at least 1. You can audit the trace and find exactly which opcode paid the bill. There is no program that can reach a certified state from scratch without paying for it.

This is what I mean by unforgeable. The μ ledger is the receipt. If you are certified, the receipt exists.

9 The formal state record

Definition 9.1 (Concrete state record)**.** The kernel's `VMState` record realises the substrate state in 12 fields:

- *s*.`vm_graph`: the partition graph. It contains module records, morphism records, and fresh-ID counters.
- *s*.`vm_csrs`: control-status registers. The hardware split: data registers hold values (like *s*.`vm_regs`); CSRs hold configuration and status. Four fields:
  - `csr_cert_addr`: 0 until something is asserted. Set to the ASCII checksum of the property string on a successful `MORPH_ASSERT` (nonzero for any property string with a non-NUL byte). The name says address; the contents say fingerprint. Nonzero here means a structural property was certified and paid for.
  - `csr_status`: general status code. Informational. Nothing reads it; nothing branches on it.
  - `csr_err`: error code. Zero means clean. Nonzero means an instruction

failed: bad argument, missing morphism, locality violation, etc.

- `csr_heap_base`: base address for `HEAP_LOAD` and `HEAP_STORE`. Final address = base + register-specified offset. No paging, no translation, just addition.

- s.`vm_regs`: a list of register values. The kernel fixes `REG_COUNT = 16`; reads and writes wrap modulo it. The kernel proofs are parametric in `REG_COUNT`, so 16 is a scaffolding choice, not a theorem.
- s.`vm_mem`: a list of data-memory words. The kernel fixes `MEM_SIZE = 128`; reads and writes wrap modulo it. The proofs are parametric here too.
- s.`vm_pc` $\in \mathbb{N}$: program counter. The integer that says *which instruction next*.
- s.`vm_mu` $\in \mathbb{N}$: the μ ledger, the substrate's accounting field. Starts at 0. Never decreases.
- s.`vm_mu_tensor`: a 4×4 array of cost values (16 entries), indexed by direction pairs (i, j). `TENSOR_SET` writes an entry, `TENSOR_GET` reads one. `REVEAL` stamps the disclosure cost at the direction index encoded in the instruction. The kernel tracks *where* costs land, not just how much. In the kernel model these are natural numbers; in the hardware, 32-bit words. If you don't use the tensor tracking, ignore all 16 entries.
- s.`vm_err` $\in \{\text{false}, \text{true}\}$: the error flag. Set when an instruction fails.
- s.`vm_logic_acc`: a natural number with two roles. Role one: the Logic Engine accumulator during `LASSERT` (the on-chip unit that reads formula words and certificate bytes and tracks validation state). Role two: the magic key the FSM matches against. `0xCAFEEACE` (hex; 3,405,703,886 decimal) passes. Anything else, the FSM rejects. The field exists so the Coq kernel and the physical hardware agree on the same accumulator value at every step.
- s.`vm_mstatus`: status word, 0 or 1. 0 = Turing mode (ordinary computation; partition graph and morphism map idle). 1 = Thiele mode (structural state live). The kernel keeps this as a status indicator only. It does not gate the structural opcodes at the step level; those are always available. The field exists so the hardware and the Coq model agree across the bridge proofs.
- s.`vm_witness`: the CHSH witness counters. This is a record of 8 natural-number counters, one for each combination of (measurement setting pair) $\times$ (same/different outcome). The four setting pairs are $(x, y) \in \{(0,0), (0,1), (1,0), (1,1)\}$, where x is Alice's setting and y is Bob's setting. That is the modern Bell-test convention (NPA hierarchy, Brunner et al. 2014 review), and the same one the Coq constructor `instr_chsh_trial (x y a b : nat)` uses. The outcome variables are (a, b). For each setting pair, there are two counters: same-outcome and different-outcome. Eight counters total: `wc_same_00`, `wc_diff_00`, `wc_same_01`, `wc_diff_01`, `wc_same_10`,

`wc_diff_10`, `wc_same_11`, `wc_diff_11`, where the numeric subscript is the value of (x, y) for that bucket. Each `CHSH_TRIAL` instruction increments exactly one counter. (Some Bell-test literature uses the reverse convention with (a, b) for settings; the numeric subscripts 00, 01, 10, 11 are unambiguous either way.) The CHSH score S is computed from the aggregate sums of these counters. For each setting pair (x, y), define $N_{xy} = \texttt{wc\_same}_{xy} + \texttt{wc\_diff}_{xy}$ (total trials with that setting) and $E_{xy} = (\texttt{wc\_same}_{xy} - \texttt{wc\_diff}_{xy})/N_{xy}$ (the correlation: fraction same minus fraction different, ranging from -1 to $+1$). Then $S = E_{00} + E_{01} + E_{10} - E_{11}$. If no trials have been run for a setting ($N_{xy} = 0$), the correlator for that setting defaults to 0 by convention. Classically $|S| \leq 2$; the NPA algebraic model allows $|S| \leq 2\sqrt{2}$ (see Section 16). The concrete Coq definition is `chsh_stat_from_wc`.

- s.`vm_certified` $\in \{\text{false}, \text{true}\}$: the top-level certification flag. Only `CERTIFY` sets it. Nothing else can.

The reference kernel and the physical hardware are not the same shape at every bit, on purpose. The kernel is where I pin the step rule down precisely enough that nothing can quietly drift; it is the gap-catcher, not the source of the truth. The truth is the cost argument itself, the thing you could re-run with a pencil. The kernel stores register and memory values as natural numbers, but truncates writes through 64-bit word helpers to keep the math clean and the bit-width honest. The hardware layer (explained in Section 14) is finite: 16 general-purpose registers, 128 memory words, bounded tables. The bridge proofs in the hardware section state exactly where those finite hardware representations match the kernel step; the size constants are parametric, so the same proofs apply at any bound, and the 16 and 128 are scaffolding choices, not theorems.

Definition 9.2 (Partition Graph)**.** The partition graph s.`vm_graph` is a record:

- `pg_next_id`: the next fresh module identifier.
- `pg_modules`: a list of module-ID/module-record pairs. Each `ModuleState` holds region data, axiom data, and the module's μ-tensor.
- `pg_next_morph_id`: the next fresh morphism identifier.
- `pg_morphisms`: a list of morphism-ID/morphism-record pairs. Each `MorphismState` holds a source module, target module, coupling data, and an identity flag.

Remark 9.1. There is no separate `vm_heap`, `vm_output`, or `vm_imem` field in the Coq kernel's `VMState`. Heap operations are heap-relative accesses into `vm_mem`. Output and instruction transport live one layer up: the runner, the trace, the RTL harness. None of that is kernel state. The distinction matters because NoFI is a theorem about the record above, nothing more.

Remark 9.2. The partition graph splits in two. The *module layer*: how state is decomposed into named pieces. The *morphism layer*: the typed categorical relationships between those pieces. The two layers are independent. Two states can agree on every module and still differ on every morphism. The Categorical Separation Theorem (Section 12) lives on that fact.

10 The instruction set: all 51 opcodes

The kernel realises the Thiele substrate as a 51-opcode VM (the term "opcode" is in Section 2: the numeric tag the hardware decodes to choose which behaviour to run). Every one carries an explicit `mu_delta` parameter. Cost is part of the instruction, not bolted on after. The VM reads it, the VM charges it.

10.1 Group 1: Partition operations (4 opcodes)

These opcodes modify the partition graph. They are how the kernel builds structural knowledge: one module, one partition edit, one accounting entry at a time.

PNEW(R**,** δ**)**: allocate a new module.
Creates a fresh module and charges δ to μ. The instruction carries a region list R describing which memory addresses belong to this module. The current step rule normalises this to a standard form (the math sense of "canonical": a unique representative for a class of equivalent inputs): it keeps the size (how many addresses) and stores it as the range `0, 1, ..., sz-1`. So the checked step preserves the module size, not arbitrary input geometry. The bounded RTL has its own partition-table and active-module machinery for locality enforcement (locality, in this VM, is the constraint that an instruction may only access memory inside its own active module's region; Section 14 covers the table). The `PNEW` opcode itself is the graph allocation step, nothing fancier.

Why does this exist? Because structural knowledge needs a home. Creating a module is how you tell the VM "this region of computation is a named unit I'm tracking." The cost δ is programmer-declared and can be 0; the VM does not force you to pay for allocation itself. If building this partition cost something because you had to figure out the right boundary, run a search, or derive the structure, you record that here. If it was bookkeeping, $\delta = 0$ is fine, no fee for shuffling paper.

PSPLIT(m**,** L**,** R**,** δ**)**: split one module into two.
Splits module m and charges δ to μ, no more. The constructor still carries left and right region payloads, but the current hardware-aligned step ignores those payloads and calls `graph_hw_psplit`: the left side gets half the module size, and the right side gets the remainder. This is the refinement operation in the finite hardware model. If discovering the split deserves a cost, encode a positive δ; the canonical reversible-accounting policy keeps it at 0.

PMERGE(m_1, m_2, δ): merge two modules into one.
Merges modules m_1 and m_2 and charges δ. The hardware-aligned step concatenates the module sizes through `graph_hw_pmerge`; module IDs wrap modulo 64. It does not perform a rich invariant-compatibility check in this step. Like `PNEW` and `PSPLIT`, the VM does not enforce a minimum cost here. Trust comes later, at the certification step, when the structure has to pay for being believed.

PDISCOVER(m, *evidence*, δ): query partition structure.
The evidence payload is carried by the instruction, but the hardware-aligned relation does not attach axioms or mutate the graph. It advances the PC and charges δ. This is a placeholder for discovery accounting, not a register query in the current kernel step.

10.2 Group 2: Logic and certification (3 opcodes)

These opcodes talk to the Logic Engine, the on-chip finite-state machine that checks formulas and certificates. No outside solver gets smuggled in.

LASSERT(*freg*, *creg*, *kind*, *flen*, *cost*): validate a formula and witnesses.
Reads a formula whose declared encoded-word count is *flen* from memory starting at the address in register *freg*. The *kind* boolean flag selects which validation path the Logic Engine should follow. "SAT" (satisfiable) means: I'm claiming this formula has at least one satisfying assignment, some combination of variable values that makes it true. "UNSAT" (unsatisfiable) means: I'm claiming it has no satisfying assignment, it is always false. In the on-chip semantics, *kind* = *true* is the only success path: it requires a dual-witness certificate, one satisfying assignment and one falsifying assignment, so the validation proves the formula is satisfiable and not a tautology. *kind* = *false* is the UNSAT-tag path: `lassert_check_ok` returns false unconditionally when *kind* = *false*, so any `LASSERT` issued with *kind* = *false* fails the check, traps to `LASSERT_TRAP_PC`, and sets the error flag. The μ-cost is still charged. The SAT-only validation path is the kernel's structural-proof shape; the UNSAT tag is a reserved opcode bit that the kernel routes straight into the trap. This matches the RTL FSM, which has phases 0 (idle), 1 (header read), and 2 (scan). The kernel and hardware both validate that the declared count matches the in-memory formula header before the check is allowed to succeed. A mismatch traps and sets the error flag. `LASSERT` reads a certificate block from memory starting at the address in register *creg*. That block contains two witnesses: one satisfying assignment and one falsifying assignment. The on-chip Logic Engine FSM processes the formula and both witnesses in-place. No external solver gets a hidden vote. Validation succeeds only if the declared count is honest, the first witness satisfies the formula, and the second witness falsifies it. That means the formula excludes at least one valuation. `LASSERT` itself is a checked validation step. Certification channels are activated by concrete state-changing paths such as `CERTIFY` and `MORPH_ASSERT`. The abstract cert-address theorem uses a broader guardrail predicate and puts `LASSERT` in the cost-positive certification/revelation class. That is not a claim that `LASSERT` writes the certificate-address register. It is a

claim that this class cannot move for free. Charges $\mathit{flen} \times 8 + S(\mathit{cost})$ to μ, even when the check fails.

This is the most important logic opcode in the VM. `LASSERT` is where a constraint package is checked before any later certification step can lean on it. There's no faking the pricing anymore by declaring a tiny *flen* for a large in-memory formula: if the header and the instruction disagree, the VM traps. And no faking semantic narrowing with a tautology either, which used to be the slick move: the dual-witness requirement closes the tautology-inflation hole, because a tautology has no falsifying witness to show the formula excluded any valuation, and the VM wants to see that witness or it walks. The cost formula is still a spend ledger. The honest-length check and the non-triviality witness are what tie that spend to the real checked object, not to a thing the programmer wished they had checked.

LJOIN(*c1reg*, *c2reg*, δ): reserve certificate-join cost.
Reserves the cost of joining two memory-resident certificates. The addresses of the input certificates are in *c1reg* and *c2reg*. Charges $S(\delta) \geq 1$. In the current hardware-aligned semantics, `LJOIN` does not compare certificate strings, write a composite certificate, touch CSR state, or mutate the graph. It advances the PC and pays the positive floor. The join proof lives at the abstraction/verifier boundary, not inside this kernel step.

MDLACC(m, δ): MDL cost accounting.
Charges δ as a module-discovery/model-description cost for module m. It does not read a register, and it does not mutate the graph. It just enters the bill.

Suppose you have two models that both fit the same data. One is a ten-thousand-parameter neural network. The other is a five-line formula. Both fit the training set. Which bill do you want to pay? Minimum description length says the total bill is the model description plus the cost of encoding the data once the model is known. Description length is how many bits it takes to write the model down. A simple formula takes few bits. A massive network takes many. The short formula may cost more to encode the remaining data, but almost nothing to describe. The massive network may compress the training set perfectly, but its own description is enormous. MDL picks the smaller total: bits for the model plus bits for the data given the model.

In the Thiele vocabulary: a module structure that you built can carry a description-length cost. Not because someone decreed it. Because constructing a specific partition of state space may require information that belongs on the bill. `MDLACC` is the opcode that lets a program explicitly enter that description cost into the ledger after the structure has been built. It performs no graph change. Its sole effect is to advance the program counter and charge the encoded δ. The connection to MDL is not a citation to a 1978 paper. It is the claim that the substrate's μ-ledger can track the model-description bill when the program chooses to enter it.

Opcode	What it does	Typical cost
`LOAD_IMM`(r, v, δ)	Write immediate value v into register r	0
`XFER`$(r_{\text{dst}}, r_{\text{src}}, \delta)$	Copy register r_{src} to r_{dst}	0
`ADD`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a + r_b$ (modular 64-bit)	0
`SUB`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a - r_b$ (modular 64-bit)	0
`MUL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a \times r_b$ (modular 64-bit)	0
`AND`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ AND r_b (bitwise)	0
`OR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ OR r_b (bitwise)	0
`SHL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted left by r_b bits	0
`SHR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted right by r_b bits	0
`LUI`(r_d, v, δ)	Load upper immediate: $r_d \leftarrow v \ll 8$	0
`XOR_LOAD`$(r_d, addr, \delta)$	Load from absolute memory address $addr$ into r_d	0
`XOR_ADD`(r_d, r_s, δ)	$r_d \leftarrow r_d$ XOR r_s (XOR accumulate)	0
`XOR_SWAP`(r_a, r_b, δ)	Swap registers r_a and r_b (reversible, direct two-write exchange)	0 by convention
`XOR_RANK`(r_d, r_s, δ)	$r_d \leftarrow \text{popcount}(r_s)$ (Hamming weight)	0

10.3 Group 3: Compute and data transfer (14 opcodes)

Ordinary arithmetic and data movement. These are the operations you find in every ISA. They mostly carry $\delta = 0$ because they do not introduce new structural entitlement. They just compute.

Why does the ISA have all these? Because the 51-opcode realisation has to run programs, not just keep receipts. Programs need to compute intermediate values, manage data, and control flow. The compute group gives them that ordinary machinery. The key property: none of these opcodes touch the certification channels. Arithmetic can move bits all day. It cannot mint certified structural knowledge.

`XOR_SWAP` is a reversible register swap. The kernel implements it as a direct two-write exchange. The name suggests the classic XOR-swap sequence that some assembly programmers use to swap two values without a temporary register (it works by writing one register with the XOR of both, then the other, then the first again), but the kernel does not actually run that sequence. It writes both registers at once. In the table it carries $\delta = 0$ by convention because a reversible operation, in the Bennett/Landauer sense (Section 17), does not destroy any information. If a program supplies a nonzero δ, the ledger still charges it. The convention is not a loophole.

10.4 Group 4: Memory and control flow (8 opcodes)

Memory access and program flow control. This is the group where a machine could quietly cheat if you let it: load from wherever it pleases, jump wherever it pleases, and the partition stops meaning anything. So this is exactly where the locality story has to hold. The opcodes here move data and steer the program counter and nothing more, and the wall that keeps a module's reach inside its own region lives one layer down, in the bounded hardware table that the partition-wall note right below makes

concrete.

Opcode	What it does	Note
`LOAD`(r_d, r_a, δ)	Load `vm_mem`$[r_a]$ into r_d	Kernel register-indirect load
`STORE`(r_a, r_s, δ)	Store register r_s into `vm_mem`$[r_a]$	Kernel register-indirect store
`JUMP`(addr, δ)	Set PC to addr unconditionally	none
`JNEZ`(r, addr, δ)	Jump to addr if $r \neq 0$	none
`CALL`(addr, δ)	Push return address via stack pointer r15, jump to addr	Qed coverage
`RET`(δ)	Pop return address via stack pointer r15, jump there	Qed coverage
`HALT`(δ)	Advances PC and charges δ, like any other instruction. The runner stops when PC goes past the program; HALT inserted before the end is just a no-op.	runner-level
`CHECKPOINT`(ℓ, δ)	Accept label ℓ, advance PC	label outside kernel

At the kernel level, `LOAD` and `STORE` are register-indirect memory operations over `vm_mem`. At the RTL/cosimulation layer, the partition wall adds active-module locality checks. `INIT_PT` and `INIT_ACTIVE_MODULE` are harness setup commands for that finite hardware table, not VM opcodes and not fields of `VMState`. This is the honest split: the abstract ISA defines memory semantics; the bounded hardware layer enforces locality through its own table state.

10.5 Group 5: Revelation, I/O, heap, and tensor (7 opcodes)

This group mixes two fundamentally different kinds of operations. `EMIT` and `READ_PORT` are in the *certification/revelation cost class*: they cost ≥ 1 unconditionally and are subject to the same mandatory-floor rule as `CERTIFY` and `MORPH_ASSERT`. `WRITE_PORT`, `HEAP_LOAD`, and `HEAP_STORE` are ordinary I/O and storage, no mandatory floor. `TENSOR_SET` and `TENSOR_GET` manage per-module cost tensors. They share this group only by physical-channel adjacency; their cost semantics are distinct.

Opcode	What it does	Cost
`READ_PORT`(r, c, v, *bits*, δ)	Write value v into register r. The value v is embedded in the instruction encoding at assembly time (the kernel step is deterministic: it writes the declared v, not a runtime port read). Channel c and the *bits* count are used by the runner/environment layer for I/O orchestration and cost accountability.	$bits + S(\delta) \geq 1$
`WRITE_PORT`(c, r_s, δ)	Send register r_s to channel c outside kernel state	δ
`EMIT`(m, *payload*, δ)	Emit payload as a receipt-visible trace event	$\lvert payload\rvert_{\text{bits}} + S(\delta) \geq 1$
`HEAP_LOAD`(r_d, r_a, δ)	Load from `csr_heap_base +` `regs[`r_a`]` into r_d	δ
`HEAP_STORE`(r_a, r_s, δ)	Store r_s to `csr_heap_base +` `regs[`r_a`]`	δ
`TENSOR_SET`(m, i, j, v, δ)	Set per-module tensor entry (i, j)	δ
`TENSOR_GET`(r_d, m, i, j, δ)	Read per-module tensor entry (i, j) into r_d	δ

`EMIT` and `READ_PORT` live in the cert/revelation class but pay more than the $S(\delta)$ floor: they pay for every bit they touch. `EMIT` unfolds the payload into actual bits, charges those bits directly, then adds at least 1 for the revelation-class step. A one-byte ASCII payload (8 bits, $\delta = 0$) costs 9. Ten bytes (80 bits, $\delta = 0$) costs 81. Emitting more costs more; the bill tracks the bits, every single one, and there's no volume discount. `READ_PORT` is the same shape on the way in: bits read, plus at least 1. The floor is still there. Zero-cost certification is still impossible. What this layer adds is that the cost tracks the actual bit content, not just the fact of the step.

`TENSOR_SET` and `TENSOR_GET` are the write door and the read door onto a thing each module carries privately: a 4×4 grid of geometric weights, the local metric numbers that the geometry layer reads back later when it wants to know how much each module's structure weighs. In the reference kernel that grid lives as a flattened 4×4 vector, sixteen numbers in a row. In the Kami hardware design it lives as dedicated tensor state on the silicon. Two completely different ways of keeping the same sixteen numbers, and the entire point of the cosimulation is that you cannot tell which one you are talking to from the outside. They step in lockstep on every path, the clean ones and the ugly ones alike: hand either of them an index off the edge of the grid and both latch the same error, and on the success path both insist on $i, j < 4$ (and, for `TENSOR_GET`, that the module actually exists in the graph) before they will write or read a thing. Nothing fancier is going on there than asking whether the operand points at a real cell of a 4×4 grid or runs off it. So the per-module weights are not some soft annotation that the flattened vector and the silicon happen to mostly agree about. They are one object, observed twice, and the two views never once disagree.

10.6 Group 6: Witness and certification (8 opcodes)

These opcodes accumulate binary game outcome statistics into hardware witness counters, enforce the NPA-PSD boundary on those counters, and manage the top-level certification flag.

CHSH_TRIAL(x, y, a, b, δ): record one CHSH trial.
Records one trial of the CHSH game. $x \in \{0,1\}$ is Alice's measurement setting. $y \in \{0,1\}$ is Bob's measurement setting. $a \in \{0,1\}$ is Alice's outcome. $b \in \{0,1\}$ is Bob's outcome. The Coq constructor order is settings first (x, y), then outcomes (a, b). The instruction increments exactly one of the 8 `WitnessCount` counters: if $a = b$ (same outcome), it increments `wc_same_xy`; if $a \neq b$ (different outcome), it increments `wc_diff_xy`. There are 4 setting pairs $\times$ 2 same/different buckets = 8 counters. The CHSH score S is computed later from these counters. Charges δ to μ.

This is a Tier 0 operation: it records data but does not activate any certification channel. It is provably free in the sense that `CHSH_TRIAL` does not and cannot trigger a cert-insight event (proven, `chsh_trial_is_tier0`). One guardrail: if any of x, y, a, b is not in $\{0,1\}$, for example passing $x = 5$, the VM fires a bad-bits error step. The error flag latches, no counter is incremented, but the cost is still charged. The VM does not silently count invalid inputs as valid trials, and there is no path through the kernel that lets you pretend it did.

CERTIFY(δ): set the top-level certification flag.
Sets `vm_certified = true`, charges $S(\delta) \geq 1$ to μ, advances PC. This is the simplest cert-channel activation: you are saying "I, the program, certify that this execution has reached an attested state." The flag is set. The cost is charged. There is no way to set `vm_certified = true` without this opcode, and this opcode always charges ≥ 1.

REVEAL(m, *bits*, *cert*, δ): disclose a value.
Reveals *bits* bits from module m with certificate string *cert*. In the current kernel relation, the certificate string is carried but not checked; the priced event is the disclosure itself. A bit-commitment protocol is a two-phase scheme: first you commit to a value (seal it in an envelope), later you reveal it. The reveal step proves you didn't change your answer after seeing the other side's response. `REVEAL` is the disclosure half of that scheme. `REVEAL` charges bits $+ S(\delta)$: the number of bits being disclosed, plus at least 1. The cost is also recorded in the global μ-tensor at flat index $(m \bmod 16)$, where the module operand is read as the tensor direction index (the μ-tensor is indexed by direction pairs, not module slots; Section 9), marking which direction the disclosure cost landed in. Revealing 0 bits still costs at least 1. Revealing 32 bits costs at least 33. Disclosing more costs more, and there's no plan where you reveal the whole module for the price of one bit. This is what makes the revelation requirement concrete: getting receipt-visible cross-correlations from binary game trials requires an explicit

disclosure instruction, and that disclosure is priced by how much you reveal.

CHSH_LASSERT(δ): enforce the level-1 NPA-PSD boundary on the witness counters. Reads the 8 `vm_witness` buckets, runs the integer-arithmetic check `column_contractive_check_witness` ($\mathbb{Z}$ arithmetic only, Q_1 column-contractivity), and advances PC on pass or traps to `LASSERT_TRAP_PC` and latches `vm_err` on fail. Charges $S(\delta) \geq 1$ either way. A non-trapping step entails the witness-derived 5×5 NPA moment matrix is PSD (`chsh_lassert_no_trap_implies_quantum_realizable`). The four `CHSH_LASSERT_1AB` variants below extend this base check from the 5×5 level-1 matrix to the 9×9 level-1+AB matrix; the full treatment of the family is in the "CHSH check family" note after the opcode tally and in Section 16.

CHSH_LASSERT_1AB(δ): Q_{1+AB}-aware witness check at $\gamma = 0$.
Extends `CHSH_LASSERT` with the level-1+AB NPA condition. Reads the 8 `vm_witness` buckets, runs an integer-arithmetic check (Q_1 column-contractivity plus the SOS sum-of-squares condition $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \leq 1$ at $\gamma_1 = \gamma_2 = \gamma_3 = \gamma_4 = \gamma_5 = 0$), advances PC on pass with cert state intact or traps to `LASSERT_TRAP_PC` on fail. Charges $S(\delta) \geq 1$ either way. The bridge theorem `chsh_lassert_1ab_no_trap_implies_quantum_realizable_q1ab` proves a non-trapping step entails `quantum_realizable_q1ab` = symmetric9 $\wedge$ PSD9 of the 9×9 NPA Q_{1+AB} moment matrix at the witness-derived rationals.

CHSH_LASSERT_1AB_G5(δ, *same_g5*, *diff_g5*): Q_{1+AB} check with the four-body moment γ_5 free.
Carries one γ-bucket pair $(\mathit{same\_g5}, \mathit{diff\_g5})$ for $\gamma_5 = (\mathit{same\_g5} - \mathit{diff\_g5})/(\mathit{same\_g5} + \mathit{diff\_g5})$. Runs the γ_5 SOS check (a weighted 4-D Cauchy-Schwarz identity in cleared integer form), enforces $-Dg5 < Ng5 < Dg5$ as part of the bool decider. On pass, the bridge gives PSD9 at $(E, 0, 0, 0, 0, \gamma_5)$.

CHSH_LASSERT_1AB_G345(δ, *same_g3*, *diff_g3*, *same_g4*, *diff_g4*, *same_g5*, *diff_g5*): Q_{1+AB} check with three-body and four-body moments γ_3, γ_4, γ_5 free.
Carries three γ-bucket pairs. Runs the γ_{345} check: an augmented 2×2 Schur-slack lemma plus a 4×4 Sylvester PD condition on the difference matrix $H_{\gamma_{345}} = \det_M \cdot M_M - M_N$. On pass, the bridge gives PSD9 at $(E, 0, 0, \gamma_3, \gamma_4, \gamma_5)$.

CHSH_LASSERT_1AB_G12345(δ, *same_g1*, *diff_g1*, ..., *same_g5*, *diff_g5*): full Q_{1+AB} check with all five γ moments free.
Carries five γ-bucket pairs. Runs the compositional sym6 $\to$ sym5 $\to$ sym4 Schur-cascade check (21 cleared H-entry numerators, 15 cleared S6 Schur entries, 10 cleared S5 Schur entries, then four sym4-determinant minors of the cleared 4×4). On pass, the bridge gives PSD9 at $(E, \gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5)$ for the bucket-derived rationals: the full Q_{1+AB} cone the substrate validates at the witness layer. By Ishizaka 2025 (cited externally), $Q_{1+AB} = Q$ in the CHSH scenario, so the validated region coincides with the

Tsirelson set, though the equivalence is not formalised in Coq. The four cost-positive check opcodes in this family are defined in the Kami HW abstraction with kernel-equivalence proven, but are not synthesized into the Genesys 2 bitstream. The reason is silicon, not semantics: the base `CHSH_LASSERT` 23-phase FSM is the dominant contributor to the design's roughly 74% LUT utilisation on the K325T (Section 14 gives the full breakdown), even after time-sharing a 384×384 multiplier across phases, and the Q_{1+AB} variants do strictly more wide arithmetic per check (`_g12345` runs a 50-minor sym6 $\to$ sym5 $\to$ sym4 Schur cascade). The Kami proof chain covers all 51 opcodes; the bitstream covers the 47-opcode synthesized subset, including the base `CHSH_LASSERT`.

MORPH_ASSERT (see Group 7): on success, it activates the cert-address channel and belongs conceptually here too, but is listed in the morphism group because it operates on a morphism ID.

10.7 Group 7: Categorical morphism operations (7 opcodes)

These are the opcodes that operate on the categorical morphism layer of the partition graph. Six of them (`MORPH`, `COMPOSE`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_ASSERT`, `MORPH_TENSOR`) modify graph state and `is_classical_opcode` returns false on them. `MORPH_GET` only reads graph state into a register and is classified as classical by that predicate, even though a Turing machine still has to simulate the graph on its tape to evaluate it. A Turing machine can simulate programs using these opcodes by encoding the full Thiele state on its tape. The simulation can be honest (maintaining the μ-ledger cost semantics in the encoding) or dishonest (omitting that accounting). The TM substrate's transition relation is indifferent to which, which is exactly the substrate-versus-program distinction Section 3 names: A2 lives in the simulator program when the substrate runs the simulation, not in the substrate itself.

A *morphism* is a record saying: "module A is related to module B in this specific, named way." Not just "A and B exist." Not just "A and B are connected by an edge." A morphism has a source module ID, a target module ID, coupling data (a list of (source-cell, target-cell) pairs plus a label string), and an is-identity flag.

The coupling data is actual relational content. `MORPH` creates a morphism with an empty coupling list initially. `COMPOSE` computes the relational composition of two morphisms' coupling lists: pair (a, c) is included if and only if there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct. `MORPH_TENSOR` concatenates two coupling lists. The coupling is not a programmer tag. It is data the kernel builds and proves properties about. You can read back the number of coupling pairs via `MORPH_GET` (selector 2), if you want to know.

Why does this matter? Because two programs can have the same classical projection, and even the same μ balance if you expose the ledger, while having completely different morphism maps. A classical machine cannot distinguish them. The 51-opcode

realisation can, via a single `MORPH_GET` probe. This is the Categorical Separation Theorem (Section 12).

Opcode	Effect	Cost
`MORPH`(r_d, s, t, c, δ)	Create morphism $s \to t$, write ID into r_d. Empty coupling. (c carried but unused.)	δ
`COMPOSE`(r_d, m_1, m_2, δ)	Compose $m_1 : A \to B$ with $m_2 : B \to C$ (requires target of m_1 = source of m_2). Coupling = relational composition.	δ
`MORPH_ID`(r_d, m, δ)	Identity morphism on module m.	δ
`MORPH_DELETE`(id, δ)	Delete morphism id. Error if not found.	δ
`MORPH_ASSERT`(id, $prop$, $cert$, δ)	Assert morphism id exists with property $prop$. Sets `csr_cert_addr` = checksum($prop$) on success. Always charges $S(\delta) \geq 1$.	$S(\delta)$
`MORPH_TENSOR`(r_d, m_1, m_2, δ)	Tensor $m_1 \otimes m_2$: requires disjoint source/target regions. New morphism from union-source to union-target. Coupling = concatenation.	δ
`MORPH_GET`(r, id, sel, δ)	Read field (0=source, 1=target, 2=#coupling-pairs, 3=is-identity) of morphism id into r.	δ

`MORPH_ASSERT` deserves special attention. It charges $S(\delta) \geq 1$ regardless of whether the assertion succeeds. This is the sharpest instance of No Free Insight: the *attempt to claim structural knowledge costs something*, even when the claim is false. A failed assertion is not free. A machine without a structural ledger has no way to express this constraint. In the 51-opcode realisation it is enforced by the step semantics and tied back to the abstract theorem.

10.8 Why 51 opcodes?

The number 51 comes from what the kernel needs to do:

- Compute arithmetic and data: 14 ops (`LOAD_IMM`, `XFER`, `ADD`, `SUB`, `MUL`, `AND`, `OR`, `SHL`, `SHR`, `LUI`, `XOR_LOAD`, `XOR_ADD`, `XOR_SWAP`, `XOR_RANK`)
- Memory and control flow: 8 ops (`LOAD`, `STORE`, `JUMP`, `JNEZ`, `CALL`, `RET`, `HALT`, `CHECKPOINT`)
- I/O, heap, and tensor: 7 ops (`READ_PORT`, `WRITE_PORT`, `EMIT`, `HEAP_LOAD`, `HEAP_STORE`, `TENSOR_SET`, `TENSOR_GET`)
- Partition structure: 4 ops (`PNEW`, `PSPLIT`, `PMERGE`, `PDISCOVER`)
- Logic and certification: 3 ops (`LASSERT`, `LJOIN`, `MDLACC`)
- Witness and certification (general plus the full CHSH check family): 8 ops (`CHSH_TRIAL`, `CERTIFY`, `REVEAL`, `CHSH_LASSERT`, `CHSH_LASSERT_1AB`, `CH`

SH_LASSERT_1AB_G5, CHSH_LASSERT_1AB_G345, CHSH_LASSERT_1AB_G12345)

- Categorical morphisms: 7 ops (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR, MORPH_GET)

$14 + 8 + 7 + 4 + 3 + 8 + 7 = 51$. Of the 51, 47 are synthesized into the FPGA bitstream; the four Q_{1+AB} check opcodes (CHSH_LASSERT_1AB and its three γ-extended variants) live in the Kami HW abstraction with kernel-equivalence proven, but not in the synthesized bitstream (see the "CHSH check family" note just below for the silicon-budget reason).

The CHSH check family: `CHSH_LASSERT` plus four Q_{1+AB} extensions. The base opcode CHSH_LASSERT mu_delta reads the eight buckets of vm_witness directly, computes the integer-arithmetic check column_contractive_check_witness (defined using only $\mathbb{Z}$ arithmetic on the same/diff counts), and either advances the program counter on success or traps to LASSERT_TRAP_PC and latches vm_err on failure. The cost is $S(mu\_delta) \geq 1$ regardless of outcome. Mandatory-floor discipline: you pay to check, even to fail. The bridge theorem chsh_lassert_no_trap_implies_quantum_realizable proves that any no-trap, no-err-flip execution of CHSH_LASSERT implies the witness-derived 5×5 NPA moment matrix is PSD (i.e., quantum-realizable at NPA level 1). The chain in plain order: the integer check on the witness counters passes; that fact lifts to real-valued column-contractivity by column_contractive_check_witness_sound; column-contractivity is biconditional with NPA-PSD by column_contractive_iff_quantum_realizable. Three theorems. No physics input. The opcode is the place in the kernel where the algebra of Section 16 becomes something the VM actually refuses to do, not just declines to recommend.

The four Q_{1+AB} extensions CHSH_LASSERT_1AB (tag 0x2F, $\gamma = 0$), CHSH_LASSERT_1AB_G5 (tag 0x30), CHSH_LASSERT_1AB_G345 (tag 0x31), and CHSH_LASSERT_1AB_G12345 (tag 0x32) lift the same discipline from the level-1 5×5 matrix to the level-1+AB 9×9 NPA moment matrix. Each runs its own integer-arithmetic Schur-cascade check on vm_witness plus the supplied γ-bucket pairs, advances on pass or traps on fail, charges $S(mu\_delta) \geq 1$. The four bridge theorems (one per slice, named chsh_lassert_1ab_*_no_trap_implies_quantum_realizable_q1ab) deliver quantum_realizable_q1ab = symmetric9 $\wedge$ PSD9 on the witness-derived rationals. The mathematical detail of the four-slice Schur cascade is in §16's Q_{1+AB} subsection below.

Every opcode exists for a reason. None are redundant. If you removed CERTIFY, you could not activate vm_certified. If you removed MORPH, you could not build morphism relationships. If you removed CHSH_TRIAL, you could not track 2-player binary game statistics. If you removed CHSH_LASSERT, the kernel would record

CHSH trials but could not enforce that validated outcomes lie inside the level-1 NPA-PSD boundary. If you removed the Q_{1+AB} family, the kernel could not validate the level-1+AB cone the Tsirelson set lives in (per Ishizaka 2025, $Q_{1+AB} = Q$ in CHSH). The ISA is designed, not grown.

11 What a categorical morphism is: the category theory, from scratch

Category theory from scratch. I didn't know what it was when I started this project.

11.1 The idea of a category

A category is the part of mathematics that takes the relationships between things as seriously as the things, and then quietly decides the relationships matter more. It is built from two ingredients. There are objects, the things. And there are arrows between them, the relationships. That is the entire inventory, and it looks thin until you notice what got promoted. In most of mathematics the objects are the citizens and the maps between them are second-class traffic. A category turns that around. The arrows are first-class, and nearly everything you can say inside a category you say by pointing at arrows, not by reaching inside objects.

The objects can be anything at all, and that is at once why category theory reaches as far as it does and why it feels slippery the first time through. Sets can be the objects. So can vector spaces, or topological spaces, or any kind of mathematical thing that has structure-respecting maps between its members. The theory does not care what the objects are made of. It cares only how they connect. In the realisation here the objects are modules, the named pieces a partition cut the state into, and the arrows are the relationships between those pieces.

Arrows go by a second name, morphisms, and the two words are interchangeable; I use both. An arrow runs from one object to another, written $f : A \to B$ and read "f, from A to B." The thing everyone imports from school and has to set down here is the assumption that an arrow is a function. It does not have to be. An arrow is whatever the category accepts as a structure-respecting way to get from A to B, and in the realisation here it is something more pointed than a function. A morphism is a claim: the assertion that these two modules are related, and here is the type of the relationship.

Two laws must hold:

1. **Composition**: If $f : A \to B$ and $g : B \to C$, there must exist a composite $g \circ f : A \to C$. This is what `COMPOSE` implements.
2. **Identity**: For every object A, there must exist an identity arrow $\mathrm{id}_A : A \to A$. This is what `MORPH_ID` implements.

And composition must be associative: $(h \circ g) \circ f = h \circ (g \circ f)$.

Why composition? Because relationships have to chain. A to B, B to C, therefore A to C. Without that, every multi-hop path has to be named individually, and the structure collapses into a pile of unconnected pairs. A category without composition is a phone book without a search function.

Why identity? Every object needs a self-loop, even the boring ones. A module with no connections to anything else still has to exist somewhere in the framework. `MORPH_ID` is the instruction for that: it builds an $A \to A$ morphism, stores an empty coupling, and flags it `is_identity`. That flag is load-bearing, and here is why it has to be. An empty coupling would *annihilate* under relational composition (compose anything with the empty relation and you get the empty relation back, not the thing you started with), so the identity cannot be carried by the coupling data. It is carried by the flag, which `COMPOSE` honors: composing an identity-flagged arrow with f returns f's coupling unchanged, so $\mathrm{id}_A ; f = f$ and $f ; \mathrm{id}_B = f$ hold at the coupling level (`graph_compose_morphisms_coupling`; the matching hardware path is `driven_step_compose`). Drop it and isolated objects fall outside the framework entirely. The category does not see them.

Why associativity? Parenthesization should not matter. $(h \circ g) \circ f$ and $h \circ (g \circ f)$ both spell the same three-hop path $A \to B \to C \to D$. If they came out different, two programs that built the same chain in different orders would produce two different morphisms. That is a bug. The chain is the thing. How you assembled it is not.

These laws are proven in Coq. Concretely: morphisms carry coupling data, a list of (source-cell, target-cell) pairs plus a label. A "cell" is a natural number indexing into a module's memory region. A coupling pair (a, c) says: cell a in the source module is related to cell c in the target module. The coupling is the *relational content* of the morphism, the actual evidence of which parts of one module correspond to which parts of another. `COMPOSE` produces a new morphism whose coupling pairs are the relational composition of the two inputs: (a, c) is included when there exists an intermediate cell b such that (a, b) is in the first coupling and (b, c) is in the second. Standard relational composition. Matrix multiplication for sets of pairs. The one exception is the identity short-circuit from the previous subsection: if either input is flagged `is_identity`, `COMPOSE` returns the other input's coupling unchanged, which is what makes the identity law hold. This is proven (`graph_compose_morphisms_coupling`). The categorical layer is not just called a category. It satisfies the actual category axioms on the actual coupling data. I checked that in Coq, because the identity law holding "obviously" is exactly the kind of thing my eye waves through right before it turns out to be false.

11.2 Why modules as objects

In the concrete kernel, the objects of the category are the modules of the partition graph, and that choice is less arbitrary than it looks. A module is a named piece of the computation, the place where a partition cut the state and put a fence around it.

Drawing that fence is the act that creates an address. Once a piece has a name it owns a region of memory or state, it may carry certified axioms, it carries a local 4×4 μ-tensor, and, the part that earns it the job, it becomes a thing the rest of the structure can point at. That is the whole reason modules are the bookkeeping unit: a relationship has to attach to something, and the only somethings on offer are the named pieces. An unnamed piece, as far as the category is concerned, is not there at all. There is nothing to relate it to and nothing to relate it from. So everything structural hangs off a module not by convention but because a module is the only kind of thing the category can see.

A morphism between two modules is a claim with its evidence stapled on: "there is a named relationship between A and B, and here is the explicit coupling data that says which parts line up." That is the move that turns the relationship into a first-class object instead of a remark in a comment somewhere. `COMPOSE` chains two of these by the relational composition already spelled out above, and an identity-flagged input still falls to the short-circuit from before; I will not re-derive either one here. What is worth stopping on is that the relationship does not dissolve into the modules it connects once it is built. It stays in the graph as a thing you can hold and ask questions of: via `MORPH_GET` (selector 2) you read back the coupling-pair count of any morphism, after the fact, straight off the state. The relationship is not lost in the act of relating. It is its own object, carrying a count you can go and demand back.

What the coupling pairs represent is up to the programmer. They could encode:

- Module B refines module A (which cells in A map to which cells in B)
- Module A entangles with module B (which cell-pairs are correlated)
- Module A simulates module B (which transitions/outputs correspond)

The kernel enforces the relational composition correctly. The interpretation is yours.

11.3 Why this is not just "extra state"

Here is the question I chewed on for a long time. If you can just bolt extra registers onto a classical machine and stash morphism IDs in them, why does the categorical layer matter at all?

The answer is in the `MORPH_ASSERT` opcode and in the Categorical Separation Theorem.

Extra registers do not have provenance. You can write anything into a register. You can write 42 into register 5 and claim it is a morphism ID. But the VM checks: does morphism 42 actually exist in the graph, put there by a `MORPH` or `COMPOSE` instruction that genuinely ran? `MORPH_ASSERT` looks the morphism up by ID and fails (setting `vm_err`) if no such morphism is there. Existence is all it verifies; the source and target are not arguments it checks against, though you can read them back separately with `MORPH_GET` (selectors 0 and 1). And `MORPH_ASSERT` costs $S(\delta) \geq 1$ even if it fails.

Building the morphism chain by calling `MORPH`, `COMPOSE`, and `MORPH_ID` is free when those instructions carry $\delta = 0$. Those ops don't have a mandatory positive floor. What costs is `MORPH_ASSERT`: the act of asserting that the morphism exists and activating the cert-address channel. That always costs $S(\delta) \geq 1$. You can build the chain for free, but you cannot assert it into the cert-address channel for free.

This is why the morphism layer is not "extra state." It is *provenance-bearing state*. The trace records where every asserted morphism came from, and the substrate has a place to check that provenance. A classical machine cannot fake that for free: the assertion flips the cert-address channel, and that flip costs $S(\delta) \geq 1$ by No Free Insight (Section 8), with no field in the classical shadow to charge it against. The companion fact, that the classical shadow cannot even tell two different morphism graphs apart, is the Categorical Separation Theorem (Section 12).

12 Categorical separation: what the classical shadow drops

Section 3 established the substrate distinction. The Thiele substrate enforces A2 in the step relation. The Turing machine's transition table has no slot for it. That is the load-bearing result. This section gives the within-model illustration: a pair of concrete VM states whose six-field shadow collides but whose morphism graphs differ. The witness is useful. It is not the foundation. Any computational model with a non-identity projection can produce an analogous shadow collision. What anchors the witness in the rest of the argument is the substrate-level law-vs-checker claim from Section 3 (A2 lives in the step rule, not in a checker bolted on afterward), the cost-floor theorem `universal_nfi_any_substrate`, and the minimality theorem `P_full_is_minimal_complete_extension`, which says $(\mu, \texttt{certified})$ is the proved-minimal extension over (mem, regs, pc): cert is not a function of the rest, μ is not a function of the rest. The uniqueness/initiality result `mu_is_initial_monotone` buys the richer extension, not the priority. The priority rides on minimality plus the law-versus-fallible-checker split. Read the construction below as: here is what the substrate distinction looks like inside the model, concrete enough to put your finger on a specific piece. If Section 3 isn't fresh, it's worth a glance first, because the witness lives downstream of the substrate distinction and leans on it the way a diagram leans on the thing it diagrams. The witness is a picture of the distinction inside the model, not a stand-in for it, and I'd rather say that out loud than let it carry more weight than it can hold.

Theorem 12.1 (Categorical Separation)**.** *There exist states s_1 and s_2 such that:*

- $s_1.\texttt{vm\_regs} = s_2.\texttt{vm\_regs}$ *(identical registers)*
- $s_1.\texttt{vm\_mem} = s_2.\texttt{vm\_mem}$ *(identical memory)*
- $s_1.\texttt{vm\_mu} = s_2.\texttt{vm\_mu}$ *(identical μ)*
- $s_1.\texttt{vm\_err} = s_2.\texttt{vm\_err}$ *(identical error flag)*

- s_1.`vm_pc` $= s_2$.`vm_pc` *(identical program counter)*
- s_1.`vm_certified` $= s_2$.`vm_certified` *(identical top-level certification flag)*

yet their morphism graphs differ. In the concrete witness, one state has an identity morphism in `pg_morphisms`*; the other has no morphisms.*

Construction. Build s_1 with `pg_morphisms = [(0, id)]`. Build s_2 with `pg_morphisms = []`. Set every shadow field the same: registers, memory, μ, PC, error flag, and `vm_certified`. The equality proof is by reflexivity on those fields. The graph distinction is by list constructor mismatch: a one-element morphism list is not the empty list. □

Two witness states, side by side. What the six-field shadow sees. What the graph state actually holds.

	State s_1	State s_2
`vm_regs`	[]	[]
`vm_mem`	[]	[]
`vm_mu`	0	0
`vm_pc`	0	0
`vm_err`	false	false
`vm_certified`	false	false
`pg_morphisms`	[(0, id)]	[]

What this witness shows. The two states above are not contrived. They are the simplest possible six-field shadow collision under the Thiele projection: identical registers, identical memory, identical PC, identical error flag, identical μ, identical `vm_certified`. Every observer restricted to that projection says they are the same state. The morphism graph says they are not. This is a witness inside the model that the chosen projection drops information. It is not, on its own, a substrate-comparison theorem: an analogous projection on any other substrate reproduces the same shape. What anchors this in the rest of the argument is the cost-floor theorem plus the minimality and law-versus-checker results Section 3 subsection 2 lists. The uniqueness/initiality theorem in that list gives the richer extension; minimality and the fact that A2 can be a step-rule law instead of an after-the-fact check give the priority.

What the witness does not prove on its own. The obvious move is to reach for more room: bolt on a register, a tape region, any auxiliary slot, and surely the Turing machine wriggles out. It doesn't. It rebuilds the same witness one size up. The augmented machine still has a non-identity projection (drop the added slot from the projected fields) and the same collision shape on the augmented states. The shadow lemma collapses no matter how much room you give it, because the collapse is baked into the construction.

The deeper question (whether you can extend a Turing machine with a cost-tracking instruction so the substrate itself enforces A2) is the substrate-versus-program distinction. Section 3's subsection "The structural axis is canonical, not orthogonal" makes this precise once and explicitly. The short version: a Turing machine with extra tape and a longer instruction set is still a Turing machine. A buggy program on the augmented TM can still flip the cert bit without writing the cost increment. The kernel theorems require the cost-on-certification rule to live in the substrate's step function, not in the running program. The Turing substrate does not satisfy that requirement, no matter how the program above it is written.

Connecting to the universal NoFI. Combine this within-model witness with `universal_nfi_any_substrate` and `thiele_represents_simulating_cert_system` (both) and you get the substrate-level statement. Take a certification system that satisfies A2 *and* supplies a structure-preserving translation into the Thiele VM, packaged as a `SimulatingCertificationSystem`, the record that carries the translation together with a proof it commutes with steps. Its certifications then transport: an uncertified-to-certified trace costs at least 1 in the source system, and the embedded execution lands in a Thiele-certified state. The translation is a hypothesis the record supplies, not something the theorem conjures for an arbitrary A2 system; the only embedding the kernel builds outright is the Turing one (`thiele_simulates_turing`), and the cost floor for an A2 system with no embedding at all is the separate, unconditional content of `universal_nfi_any_substrate`. Unconditional here means the axiom closure is empty, nothing named or assumed; the theorem's own two premises are just the trace endpoints, given a run that starts uncertified and ends certified. The witness theorem shows what the Thiele projection drops within the model. The substrate-independent theorem shows that any honest substrate inherits the μ-price on certification. Together they say: the cost floor is forced wherever A2 is accepted. Thiele is the canonical place where the dropped information sits as primitive state.

What would falsify this section. The whole section rides on one separating function existing nowhere. A function on the six shadow fields (`vm_regs`, `vm_mem`, `vm_mu`, `vm_pc`, `vm_err`, `vm_certified`) that returns different values on s_1 and s_2 above would make `shadow_strictly_lossy` false, and the Coq proof would stop compiling. The witness pair is small enough to check by hand. That is the seam this section breaks at, and the place to go looking for it is Section 22.

Theorem 12.2 (Shadow Separation)**.** *There exist two states with the same classical shadow but different morphism graphs, and a legitimate graph-preserving probe after which the morphism-level distinction remains.*

Proof. Take s_1, s_2 to be the Categorical-Separation witness pair: s_1 has `pg_morphisms` $= [(0, \mathrm{id})]$, s_2 has `pg_morphisms` $= []$, and the six

classical-shadow fields agree. So `shadow_proj`(s_1) = `shadow_proj`(s_2) but s_1.`vm_graph` $\neq$ s_2.`vm_graph`.

The probe is the single-instruction program [`instr_add`$0\,0\,0\,0$] (`ADD` with $\delta = 0$). This instruction is classical: `is_classical_opcode`(`instr_add`$0\,0\,0\,0$) = true. By D3 conservativity, the probe leaves `vm_graph` untouched on both s_1 and s_2. After the probe, the six classical-shadow fields agree (because they agreed before and the probe is deterministic on the shadow) and the morphism graphs still differ. □ □

> ***Plain reading.*** Same witness pair as Categorical Separation. The probe is a classical opcode that touches only the classical shadow. D3 says it leaves the morphism graph alone. So the graph-level difference survives the probe. The point: the difference between s_1 and s_2 is not an artifact of the initial state. It persists through ordinary classical computation.

The probe used is deliberately boring: `ADD 0 0 0 0`. It is not a morphism operation. That is the point. The theorem is not relying on a failing probe to manufacture a difference. The difference is already in the structural state, and ordinary classical projection cannot see it.

Corollary 12.3 (Classical observer is blind)**.** *No function of (registers, memory, μ, PC, error, certified) can separate all Thiele states. The classical shadow is strictly lossy.*

Corollary 12.4 (Thiele strictly extends classical)**.** *Every program in the classical fragment, meaning every program restricted to* `is_classical_opcode`*, leaves* `pg_next_id` *unchanged. There exist Thiele programs that change* `pg_next_id` *via* `PNEW`*. Therefore Thiele strictly extends the classical fragment: it can represent and probe states that no program in that fragment can reach. A universal Turing machine can simulate the full Thiele state by encoding it on tape, but a faithful simulation has to preserve the μ-ledger cost. Drop the accounting and it is no longer simulating the substrate honestly.*

So the embedding goes one way and only one way, and that asymmetry is the whole point. Run any classical program inside the substrate and it does exactly what it always did: same answers, same registers, and not a single fingerprint left on the structural layer it never knew was there. The classical world sits inside the substrate without disturbing it, the way a sentence sits inside a language that can also say things the sentence has no words for. What it cannot do is reach back the other way. The substrate has transitions that leave no shadow in the classical six fields at all, and once you can speak them you can ask questions the classical machine cannot even phrase, let alone answer. The 51-opcode realisation is where those transitions get names you can type: `PNEW`, `MORPH`, `MORPH_ASSERT`. The kernel does not ask you to take the asymmetry on faith; it carries it out one trace at a time.

13 The knowledge receipt: what makes certification unforgeable

Here is the claim I want to earn rather than assert: the μ ledger cannot be forged. Not hard to forge, not impractical to forge: there is no move on the table that produces a fake receipt that survives one more step of looking. I am going to show that in four acts, each one a concrete run you could type yourself, because a thing this load-bearing should not rest on my say-so.

Act 1: The attempt costs, even when it fails.

Run `MORPH_ASSERT`(k, "test", "", 0) where morphism k does not exist. What happens:

- `vm_err` becomes true (the assertion failed, morphism k not found).
- `csr_cert_addr` stays 0, the cert-address channel is NOT activated on failure.
- μ increases by $S(0) = 1$ regardless (the attempt cost something).

The VM paid for a failed assertion. A machine without a structural ledger has no way to express this, let alone enforce it. In the 51-opcode realisation, the cost of asserting structural knowledge is charged whether or not the claim is true.

Act 2: Build the chain.

Run this sequence:

$$\texttt{PNEW}(A),\ \texttt{PNEW}(B),\ \texttt{PNEW}(C)$$
$$\texttt{MORPH}(f, A, B),\ \texttt{MORPH}(g, B, C)$$
$$\texttt{COMPOSE}(h, f, g)$$

Then `MORPH_GET`(r_0, h, source) and `MORPH_GET`(r_1, h, target). The resulting state has $r_0 = A$, $r_1 = C$, and morphism $h : A \to C$ exists in the morphism map. With $\delta = 0$ on each build step, $\mu = 0$. The morphism exists, but no cert-address commitment has been paid for yet.

Act 3: Assert.

Run `MORPH_ASSERT`(h, "composed", "", 0). Morphism h exists and the property string "composed" has a non-NUL byte. The assertion succeeds. `csr_cert_addr` goes to the ASCII checksum of "composed" (nonzero). Cert-address channel activated. μ goes up by $S(0) = 1$.

(Why does the property string need a non-NUL byte? Because `csr_cert_addr` stores the ASCII checksum of the property string, a plain byte-sum. The empty string sums to 0, and so would an all-NUL string, leaving the channel at 0. Not activated. Any string with a non-NUL byte gives a nonzero checksum and works. "composed" is just an example label.)

Now there is a line in the μ ledger that says, in effect, at this exact point in the trace somebody asserted a morphism and was charged for it. The thing that makes that line a receipt and not a note is what you cannot do to it afterward. You cannot reach back and unspend the cost, because μ only ever climbs; the ledger has no subtraction in it, so there is no operation that quietly undoes the charge. You cannot slide the entry earlier or later to pretend the assertion happened somewhere it did not, because its place in the trace is fixed by when μ moved. An unforgeable receipt is just a charge you can neither erase nor relocate, and that is precisely the shape of this one.

Act 4: Classical programs cannot forge it.

Snapshot Program A at the end of Act 2, before Act 3's `MORPH_ASSERT`. Its six-field classical shadow is fixed: `MORPH` and `COMPOSE` wrote the new morphism IDs into their destination registers and `MORPH_GET` read A and C back into r_0 and r_1; the program counter sits at 8; $\mu = 0$; error clear; uncertified.

Program B forges that shadow without building anything. It never runs `MORPH` or `COMPOSE`. It just `LOAD_IMM`s the same values into the same registers (the morphism IDs are deterministic, handed out in order (0, 1, 2) by `pg_next_morph_id`, so there is nothing secret to reproduce) and pads with $\delta = 0$ `CHECKPOINT` no-ops until its program counter also reads 8. Now every field the shadow keeps matches Program A's, $\mu = 0$ on both sides. A classical observer cannot tell them apart. That two states can share the entire shadow while differing in the graph is the Categorical Separation Theorem (Section 12); Program B is the forged half of exactly such a pair.

The whole difference lives in the graph. Program A built morphism h with `MORPH` and `COMPOSE`, so h sits in its `pg_morphisms`. Program B's register h holds the number 2, but its graph is empty: the value is a forged token, not a graph entry. Apply `MORPH_DELETE`(h): Program A succeeds, because there is a morphism to delete. Program B fails and sets err, because there is not. `MORPH_DELETE` reads the graph, and the graph is the one thing `LOAD_IMM` cannot forge.

The μ ledger is not a counter that measures how much work was done. It is a record of which structural commitments were made. Program B did the same amount of "work" in the μ sense, but it did different work. The kernel state records the difference.

14 The substrate, instantiated three times

Three scaffolding layers wrap the substrate: a Coq kernel, an OCaml runner extracted from it, and a Kami RTL design tied back to it. None of them IS the substrate. They are how the substrate gets to live on a chip, on a laptop, and inside a proof checker. The kernel-to-RTL boundary is closed by formal bisimulation proofs (Section 2: same input, same output, field by field). The kernel-to-OCaml boundary is closed by audited parity tests (Section 2: run both on a battery of inputs, verify they agree). Proof on the hardware side. Tested correspondence on the runner side. Both boundaries are

named in the source, out in the open, so you can walk to either one and see what it actually rests on. I would rather hand you the exact seam where proof stops and tested correspondence starts than dress the whole stack up as one seamless theorem it is not.

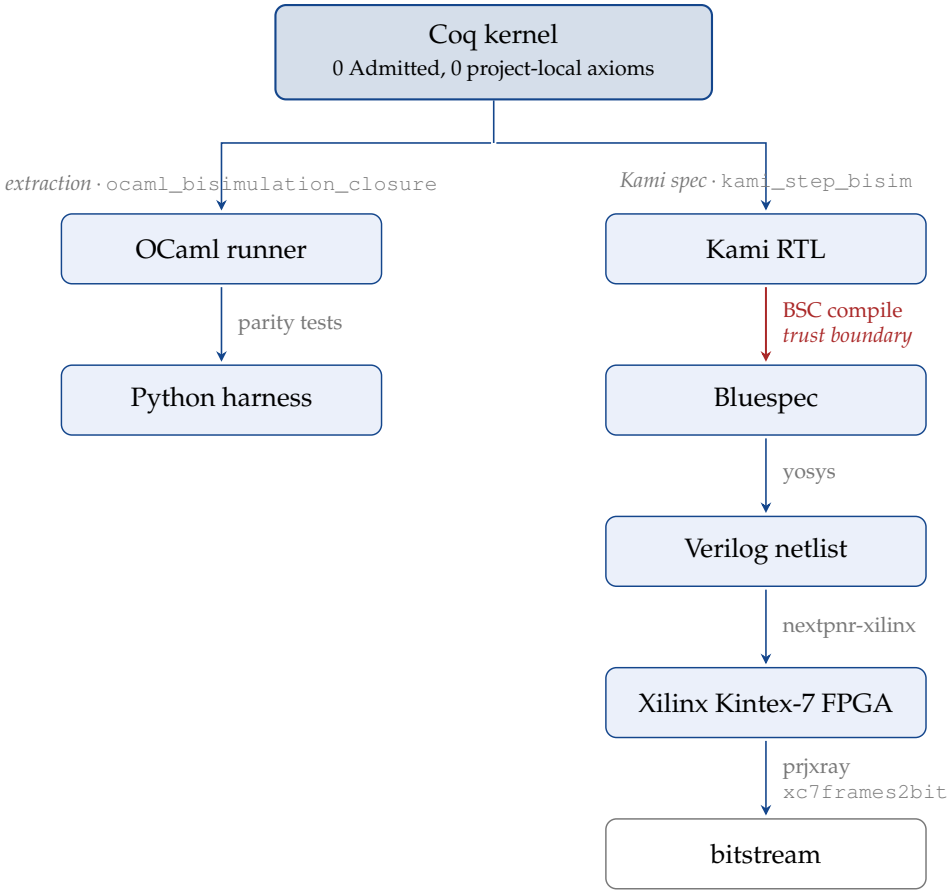


Figure 5: The three-layer realisation. The Coq kernel is the source of truth; the left branch is the runnable-binary path, the right branch is the silicon path. Blue arrows are either formally bisimulated (with the Coq theorem named on the edge) or empirically parity-tested. The red BSC-compile arrow is the single residual trust boundary on the silicon path, named `bsc_kami_compilation_trusted`.

14.1 Layer 1: The Coq kernel

Coq is where the substrate gets pinned down. The kernel defines:

- The `VMState` record (all 12 fields, as in Section 9).
- The `vm_instruction` type: an inductive type (Section 2) with 51 constructors, one for each opcode, each carrying the right argument types.
- The `vm_apply` function: takes a state and an instruction, returns a state. It is total (Section 2): every input pair produces a defined output, including the error case.
- All the theorems: No Free Insight, μ-monotonicity, μ-initiality, categorical sepa-

ration, Turing strictness, and the rest.

The Coq kernel operates mostly over natural numbers, but `VMState` is not an unbounded mathematical fantasy. `REG_COUNT = 16`. `MEM_SIZE = 128`. Both are matched against the synthesised RTL. The accessors wrap indices through those sizes (modular arithmetic, Section 2). Both constants are parametric in the proofs (Section 2: same proofs, any value), so nothing in the formal chain depends on the specific numbers. Register and memory writes are truncated through the kernel's 64-bit word helpers to keep the math clean. The μ counter is a plain natural number with no fixed upper bound. The RTL instantiates a finite 32-bit datapath (the part of the chip that moves and operates on data, not the control logic) and bounded tables.

Zero Admitted means: no step in any theorem proof was asserted without derivation. Coq checked everything. I did not.

14.2 Layer 2: The OCaml extracted runner

Here is the trick that makes the OCaml layer worth having at all. A Coq definition is two things wearing one coat: a recipe for computing an answer, and a proof that the recipe behaves. Extraction takes the coat off. It keeps the recipe, in this case rendered into OCaml, an ordinary programming language you can compile and run like any other, and it drops the proof terms on the floor, because a CPU has no use for them. So `vm_apply` (the function that defines what every opcode does) walks out the door as an OCaml function that computes the same answer on the same inputs. The proofs do not come along. The algorithm does. That is the whole point: I get a binary I can actually run, descended from the exact definition the theorems were proved about, and the descent is mechanical rather than me retyping the semantics into a second language and praying. The extracted runner executes all 51 opcodes from the Coq semantics, morphism operations and `CHSH_LASSERT` included.

The trust boundary at this layer is named explicitly. The kernel proves formal facts about the Coq-side observable (`shadow_to_eo (vm_apply s i)` strips a state down to its observable fields after one instruction): exact μ accounting, μ never decreases, the function is total. It also names the external runner boundary. The Coq lemma `ocaml_runner_agrees` is a Coq-side reflexive witness (a proof that one Coq expression equals itself, useful as a placeholder), not an unproved `Hypothesis` (Section 2). The cross-language claim, that the built OCaml binary returns the same observable as the Coq semantics on every input, is checked by the parity test suite (Section 2: run both on the same battery, verify they agree). I do not count that as a kernel theorem. It is an audited boundary, backed by tests, named in the source.

14.3 Layer 3: The Kami RTL hardware

The whole reason this layer exists is to refuse to retype the design in a second language. Kami lets you write a hardware specification inside Coq itself and then extract it, the same move the OCaml runner used, except the target is not software, it is a chip. The

first stop is Bluespec SystemVerilog, a hardware description language pitched high enough that you say what the hardware should compute rather than which wire goes where: the registers it holds, the rules it runs, and the conditions under which each state transition is allowed to fire. The Bluespec compiler then drops a level and emits standard Verilog RTL, the gate-and-register description that synthesis tools chew into actual logic, either on an FPGA, a chip you can reconfigure after it leaves the factory, or an ASIC, a chip burned for one design and never reprogrammable. So a Kami module is the same idea descending one more flight of stairs: the finite hardware realisation, pinned down in Coq, walks all the way to gates without anyone hand-translating it on the way, and the Kami extractor runs it through the pipeline above.

The hardware has:

- 16 registers, each 32 bits wide (`RegCount = 16`; kernel proofs parametric)
- 128-bit instruction encoding (`InstrSz = 128`)
- 128 words of data memory and 128 instruction words (`MemSize = 128`; kernel proofs parametric in `MEM_SIZE`)
- bounded instruction transport and execution state
- 64 partition module slots (`PTableSz = 64`)
- 16 slots in each of the morphism, coupling, formula, certificate, descriptor-metadata, and coupling-pair tables
- An on-chip `LASSERT` FSM for formula verification (with 64-entry inlined backing buffers)
- The μ ledger updated atomically with each executed instruction (no separate parallel datapath; the cost arithmetic happens inside the same `step` rule)

The Verilog is the hardware artifact. The FPGA is whatever I could put it on for free. The current target is the Xilinx Kintex-7 `xc7k325tffg900-2` on the Digilent Genesys 2: the smallest fully-open-toolchain part the design still fits comfortably after the multi-cycle `CHSH_LASSERT` witness checker landed. Section 2 explains the synthesis pipeline I use: yosys does the gate-level synthesis, nextpnr-xilinx does the place-and-route, and prjxray's `xc7frames2bit` produces the bitstream the chip reads to configure itself.

The CI workflow runs both halves on every relevant push. One job produces a JSON netlist plus a simulation VCD waveform from iverilog (all three terms in Section 2); the other job produces a real `.bit` file loadable on the FPGA. Running yosys (`synth_xilinx -family xc7 -abc9 -nodsp`) against the committed RTL today gives a resource breakdown: roughly 151,000 LUTs, several thousand flip-flops, about a thousand carry chains, two block RAMs (the FPGA-primitive vocabulary is in Section 2; the exact post-synthesis numbers are 151,154 LUTs, 8,463 flip-flops, 1,113 carry-

chains, 64 distributed RAMs, 2 block RAMs of 36 Kbit each, 0 DSP slices). The bulk of the LUT growth over earlier drafts is `instr_chsh_lassert`'s column-contractive integer-arithmetic witness check: I deliberately disable DSP inference (`-nodsp`) so the multiplier tree maps to LUTs. yosys's DSP48E1 inference fans the same arithmetic out to more DSP slices than the K325T has, and routing through DSPs makes the openXC7 placer too slow to fit in CI. To keep that LUT cost finite, the witness check itself is implemented in Kami as a 23-phase FSM that time-shares one 384×384 signed-unsigned multiplier across the 22 wide integer multiplications it needs. The Coq spec for `instr_chsh_lassert` is still single-step (single-cycle retirement at the VM/spec level); the multi-cycle execution is a Kami-implementation detail invisible to the spec, exactly the same pattern as the existing `instr_lassert` FSM (see the `chsh_lassert_fsm` rule). The design fits the K325T's 203,800 LUT6 budget with comfortable margin. Concretely, the CI `fpga-bitstream` job produces a real `.bit` file end-to-end on every relevant push (yosys synthesis → openXC7's `nextpnr-xilinx` place-and-route → prjxray's `fasm2frames` packaging → `xc7frames2bit`): roughly 11,444,000 bytes, post-route maximum clock frequency 49.33 MHz against the 12 MHz target, zero route congestion (router2 converges with `overused=0 overuse=0 archfail=0`). The bitstream is loadable on the Digilent Genesys 2 with `openFPGALoader--boardgenesys2build/thiele_xc7k325t.bit`, and each CI run uploads it as an artifact alongside the FASM and the JSON netlist, so the file is available without needing to re-run the hour-long synth flow locally.

The kernel proofs are parametric in `REG_COUNT` and `MEM_SIZE`, so the same proofs apply at any size constants the design picks. This part was a proof of existence on hardware, not a design constraint. The size constants (16 registers, 128 data words, 128 instruction words) were chosen small enough to fit on a free-toolchain FPGA. The design scales to whatever size you actually want.

47 of the 51 opcodes are implemented in the synthesized RTL: the Kami core has hardware logic for those 47, and the synthesised Verilog carries them through to the FPGA bitstream. The remaining four (the Q_{1+AB} check-opcode family `instr_chsh_lassert_1ab`, `instr_chsh_lassert_1ab_g5`, `instr_chsh_lassert_1ab_g345`, `instr_chsh_lassert_1ab_g12345`) are defined in the Kami HW abstraction with kernel-equivalence proven, and they execute in the OCaml runner and Python harness, but they are not synthesized into the FPGA bitstream because the silicon budget for additional wide-arithmetic FSMs is exhausted by the base `CHSH_LASSERT` check. The OCaml/RTL opcode-set parity test tolerates exactly these four. The question is not which opcodes are in hardware, but which opcodes have a completed formal *bisimulation proof*.

A bisimulation proof says: given the same starting state and the same instruction, the hardware and the Coq model produce the same output state. Not "similar" or "equivalent in some abstract sense." Literally: the hardware's answer equals the Coq model's answer, field by field. It is a step-by-step correspondence, verified in Coq.

Formal bisimulation proofs exist for all 47 synth-realised opcodes. Zero Admitted. All Qed. The four Q_{1+AB} check opcodes have no synthesized-Verilog counterpart, as noted above, so the question of Verilog bisimulation does not arise for them; they live in the OCaml-runner / Python-harness layer plus the Kami HW abstraction (with kernel-equivalence proven) plus the substrate-level Coq theorems of §16. Here is the precise breakdown for the 47:

- **31 opcodes are truly unconditional.** These are the `SupportedOpcode` group: `True` in Coq, no conditions whatsoever. They cover all standard compute, load/store, jump, I/O, and certification operations that don't involve morphism tables or formula checking, plus `CHSH_LASSERT` (whose snapshot-level check reads the same witness counters and runs the same `column_contractive_check_witness` function as `vm_apply` via `abs_phase1`). Verified (`embed_step_compute`).
- **6 opcodes require valid instruction arguments.** The hardware and Coq model AGREE on the error path for invalid arguments, so no semantic disagreement exists, but the success-path proof requires the argument to be in range:
  - `CALL`: stack pointer must be in bounds (`WellFormedSnapshot`), and `pc` $<$ `MEM_SIZE`.
  - `RET`: stack pointer must be in bounds (`WellFormedSnapshot`).
  - `CHSH_TRIAL`: all four values x, y, a, b must be in $\{0, 1\}$. Passing 5 as an outcome value triggers the bad-bits error path, on which both hardware and VM agree.
  - `TENSOR_SET`: indices i, j must both be < 4 (valid for a 4×4 tensor).
  - `TENSOR_GET`: indices valid AND the module must exist in the graph.
  - `LASSERT`: the declared formula-unit count *flen* must match the actual in-memory formula header. (Mismatch causes a trap to 0xF00, on which both sides agree; the precondition is needed for the success-path proof.)

  Covered in the kernel for `CALL`, `RET`, `CHSH_TRIAL` and for `TENSOR_SET`, `TENSOR_GET`, `LASSERT`.
- **10 opcodes carry Qed proofs under structural invariants that hold inductively from hardware reset.** Seven from the morphism group (`MORPH`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_ASSERT`, `MORPH_GET`, `COMPOSE`, `MORPH_TENSOR`) and three graph-shaping opcodes (`PNEW`, `PSPLIT`, `PMERGE`). The preconditions are `coupling_wf` (= `coupling_desc_bounded` $\wedge$ `coupling_pairs_in_range` $\wedge$ `coupling_pairs_fully_populated`) and `morph_table_wf`. Both invariants are proved to hold at hardware reset and to be preserved by every `kami_step` operation: `coupling_wf_kami_step_preserved` and `morph_table_wf_kami_step_preserved`. So the precondition is discharged in practice for any state reachable by machine

execution; no unreachable-state caveat applies. `PNEW/PSPLIT/PMERGE` additionally need `pt_well_formed` and arithmetic side conditions that are likewise inductive.

The official classification (the theorem `rtl_coverage_partition`) records the partition $37 + 10 + 0 = 47$ for the synth-realised subset: all Qed, zero structural gaps within that subset. The four Q_{1+AB} check opcodes sit in the Kami HW abstraction (with kernel-equivalence proven) but outside the synthesized-Verilog surface. The slack of 4 between the 51-opcode VM ISA and the 47-opcode synth partition is a silicon-budget choice, not a semantics gap.

And the four Q_{1+AB} check opcodes are not stranded by sitting outside that partition: their kernel-equivalence proof makes them sound at the substrate level (see §16), and they run in the OCaml runner and Python harness. What they lack is a synthesized-Verilog counterpart to bisimulate against, which is why they fall outside the 47 rather than inside it as a gap. That is honest proof accounting, not a hole papered over.

The substrate is the math. The Coq kernel pins it down. The hardware is a finite physical instantiation with explicit table sizes and word widths. The proofs state the bridge between them under the finite representation invariants where those invariants are needed. This is the only honest relationship physical hardware can have with an abstract mathematical substrate.

Substrate-level vs program-level enforcement. In the Coq kernel, A2 is part of the step relation. A2 says: flipping certification false-to-true costs at least 1, at the step level. Look at the cases for `CERTIFY` and `MORPH_ASSERT` in `vm_apply`: the cost is added before the flag flips; the step that mutates the certification channel and the step that charges μ are the same step. That is what A2 means in the kernel.

A Turing machine simulating the substrate has A2 only as a property of its simulator program. A Turing machine can run a buggy simulator that skips the cost; the substrate cannot. A buggy simulator does not satisfy A2 because A2 is not a law of its step rule. It is at best a program-level property the simulator is supposed to keep and can drop on any step. The substrate cannot drop it for the same reason it cannot forget to breathe: charging μ on the cert-flip is the step, not a checker bolted on afterward. The proof covers the step relation in the kernel and, under the BSC compiler trust boundary, the Kami RTL generated through the hardware pipeline. It is structural, and it is sharper than any metaphysics about substrate-level enforcement being "deeper" than program-level enforcement. A2 lives in the substrate as a law of the step rule, where flipping certification and charging μ are one and the same step. A classical step rule can carry A2 only as a fallible program-level property, the kind a buggy simulator can skip, and that is what makes classical computation derivative here. Bolt A2 into the step rule and you have not extended classical computation, you have rebuilt the substrate. What the hardware bridge separately checks is smaller: that

the kernel definition matches the synthesised RTL, with documented trust boundaries at the OCaml extraction and the BSC compiler. The priority claim is not waiting on that; the substitution-adequacy and minimality theorems already settle it.

Substrate-level property enforcement is not unique to Thiele as a category of design. Trusted Platform Modules enforce key isolation in hardware; CHERI enforces capability discipline in the ISA itself. Both put the property the system cares about into the substrate's primitive operation rather than relying on program-level discipline to maintain it. The shared structural feature with TPMs and CHERI (enforcement living in the substrate's step rather than the data) is the generic shape; it is the design category, not the priority. What is Thiele's own is the specific law being enforced: A2, the exact-least predicate that prices certification, charging exactly when the cert flips (`a2_equal_trust_substitution_payoff`), carried on the irredundant (μ, cert) extension (`P_full_is_minimal_complete_extension`), with μ as the unforgeable receipt. The shared shape is generic; the priced law is not. If a TPM is the right analog for one piece (a hardware-rooted invariant a program cannot forge) and CHERI is the right analog for another (an ISA-level discipline programs cannot escape), the analogy is structural, not detailed; I'm not claiming the underlying mechanisms match, only the substrate-level shape of what is being enforced.

15 Five more instantiations, none of them mine

The abstract records were built with the state type and the instruction type left open, on the theory that someone would eventually plug in a system I had never thought about. It turns out the plugging-in had already happened, years ago, in five different industries, under five different names, usually right after somebody lost money. None of the five communities below would describe themselves as instantiating an axiom about certification cost. They would describe themselves as patching an exploit. Same event. So I ran each system through the kernel's records, and what follows is what came out: the instantiation, the theorem, the consequence, and the thing that would break it. Every result here is machine-checked and closed under the global context; the names are handles you can look up.

15.1 Proof-of-stake finality

A proof-of-stake blockchain finalizes blocks by vote: validators stake collateral, vote on checkpoints, and a finality gadget marks some block irreversible. The founding embarrassment of the field is *nothing at stake*: if casting a finalizing vote costs a validator nothing, then voting to finalize two conflicting histories also costs nothing, and the gadget's "irreversible" means nothing. The fix every live protocol converged on is slashing: finalize dishonestly and your stake is destroyed.

Run that through the records. A gadget whose finalize step carries zero stake-at-risk is a cost-bearing system without the A2 field, and it realizes the kernel's free-forgery witness exactly: `nothing_at_stake_is_free_forgery` shows no A2

field can be constructed for it, by `free_forgery_violates_A2`. A gadget whose finalize step risks at least one unit *is* an A2-respecting certification system (the slashing inequality is not an analogy for the axiom; it discharges the axiom's proof obligation), and then `slashing_finality_floor`, which is `universal_nfi_any_substrate` instantiated, prices every run from unfinalized to finalized at one unit of stake minimum. Slashing is A2 wearing a hoodie. The field derived it empirically, by getting robbed.

Falsifier: a protocol whose known nothing-at-stake status contradicts the classification: a zero-stake-at-finalize gadget that admits an A2 proof, or a slashing gadget with a free finalizing trace.

15.2 Gas metering

Execution-fee networks bill every step: each opcode costs gas, and the fee schedule is the protocol's promise about who pays for what. Treat "the state root is committed" as the certification event and a gas schedule becomes a local pricing law. It picks the steps it charges, and it is trusted to charge there and nowhere else. The kernel's pricing-uniqueness theorem then lands directly: `gas_schedule_exactness` says a schedule collects at least one and at most one unit per commitment exactly when its charging predicate *is* the commitment predicate with unit pricing. There is one exact schedule, and it charges on commitment.

The two failure modes are theorems too. Undercharge (one committing opcode the schedule misses) and `undercharged_opcode_admits_free_commitment` hands you a one-step certifying trace at zero gas. The theorem prices the limiting case, where the charge is missing entirely; Ethereum's 2016 DoS season was the graded version of the same failure shape, opcodes priced far below their real cost letting attackers buy state growth almost free, and the emergency repricing that followed is what restoring the charging predicate looks like in production. Overcharge and `overcharge_breaks_exactness` kills the ceiling instead. And the class is not empty talk: the kernel machine itself sits in it. `thiele_vm_commit_pricing_is_exact` prices that machine's own certification channel exactly, and `thiele_unit_price_lower_bounds_mu` checks that μ pays the commitment floor with room to spare, which is also why μ itself is not the exact commitment pricer: it prices computation too.

Falsifier: a schedule satisfying floor and no-overcharge whose charging predicate differs from the commitment predicate on some reachable step.

15.3 Trusted-execution attestation

A trusted execution environment (an enclave on someone else's computer) proves to you, remotely, that a specific measured binary produced a result. The proof is an attestation report, and the load-bearing part of the report is a measurement register: a value the hardware accumulates as the run commits, the PCR or MRENCLAVE of the spec sheets. The kernel says the register is not engineering prudence. `attestation_`

`cannot_factor_through_bare_transcript` (the verifier-factorisation theorem instantiated at reports) proves a sound and complete verifier of a μ-dependent claim cannot be a function of the bare classical transcript, no matter how cleverly it reads the logs. The replay attack is the proof's witness made flesh: `replay_is_a_two_preimage_witness` exhibits two reports with identical logs and different registers, each honestly produced by a real run: one that paid, one that did not. And the escape works: expose the register and `measurement_enriched_attestation_succeeds` builds the sound, complete, unit-cost verifier the bare transcript could not carry.

Falsifier: a sound and complete attestation scheme for a μ-dependent claim that reads only the bare transcript.

15.4 Certificate transparency

Browsers trust certificate authorities; certificate authorities occasionally sign things they should not. Certificate transparency's answer is a public, append-only log: a claim of the form "this certificate is logged" becomes checkable against a structure that is computationally hard to forge. In kernel coordinates the log is a hardness witness, and the design is the hardness escape: `transparency_log_escape` shows log-backed transcripts admit a unit-cost decision procedure whose acceptances are grounded, while `log_free_verifier_impossible` (the bare-setting impossibility, re-exported in this vocabulary) shows the log-free equivalent with the same soundness target does not exist at any cost. The attack the CT community worries about most, the split view (show one population one log and another population another), is not an unfortunate operational detail; `split_view_witness` exhibits it as the impossibility theorem's own witness pair: two bare-indistinguishable views, different truth underneath.

Falsifier: a sound and complete log-free verifier with the same soundness target.

15.5 Proof-carrying verification

Proof-carrying code ships a certificate next to the binary so the consumer checks the proof instead of re-deriving the property; interactive and zero-knowledge protocols do the same trade with rounds. Both are the interaction escape: `proof_rounds_escape` builds the sound, complete, unit-cost checker over proof-carrying transcripts (the checker reads the carried certificate, not the run), while `bare_pcc_impossible` confirms that stripping the rounds lands back in the impossible bare setting. Underneath sits the floor: `level_k_verification_floor`, the μ-hierarchy instantiated, prices k certification events at no less than k units, and `level_k_verification_floor_tight` shows the bound is met. Read the fence carefully, because it is load-bearing: the floor bounds *certification events*. It says nothing about gate counts, circuit sizes, prover time, or proof length. A succinct verifier compresses everything except the number of commitments it makes. That is the precise sense in which verification stays expensive after everything else gets cheap.

Falsifier: a level-k certified trace with total μ below k, or a sound and complete zero-round bare verifier.

15.6 The same table, five ways

System	Escape used	Kernel result	What is forced	What refutes it
PoS finality	none, the cost records directly	`universal_nfi_any_substrate`	slashing: finalize risks ≥ 1	a zero-stake gadget admitting A2
Gas metering	none, a pricing biconditional	`exact_commitment_pricing_characterization`	charge $\equiv$ commit, unit price	a conforming non-commitment schedule
TEE attestation	substrate (enrich the state)	`V_does_not_factor_through_classical`	the measurement register	sound+complete bare-transcript attestation
Transparency logs	hardness	`hardness_escape_succeeds`	the log itself	sound+complete log-free verifier
Proof-carrying / ZK	interaction	`interactive_escape_succeeds`, `level_k_certification_cost_floor`	the rounds, and k units for k commitments	a level-k trace under k μ

Five industries, three escapes, two cost records, one axiom. Each community paid for its row in incidents before anyone wrote it as a theorem, which is either reassuring or expensive depending on when you bought in. What none of the five rows supplies (worth saying out loud, since this section would be the natural place to oversell it) is forcedness: that certification is the event a model *must* price remains the named open problem from the opening pages. The rows are convergence evidence, in five vocabularies that owe each other nothing.

16 Binary Game Statistics: where the μ-Ledger Meets the NPA Boundary

The substrate has a CHSH step. The VM realises it with two opcodes. `CHSH_TRIAL` records outcomes of a 2-player binary game into hardware witness counters. `CHSH_LASSERT` refuses to advance normally unless those counters satisfy a Z-arithmetic check.

The enforcement doesn't care who built the wall. `CHSH_LASSERT` refuses a super-quantum correlation by trapping to `LASSERT_TRAP_PC`; `LASSERT` refuses a formula whose dual witness fails to exclude a single valuation by trapping to the same address, the same way. One wall is the Tsirelson bound, which I did not author and could not have. The other is a logical constraint I wrote down myself. The step traps the same on both. So μ isn't pricing contact with external structure as some special case. It prices the certified narrowing as such, and the trap fires the same whether the state it throws out is a PR box or a falsifying assignment I handed it. Insight costs, and the price never asks whether you found the wall or built it.

And the claim is about that step, not the chip. Some of these cert-opcodes never reach the synthesised FPGA. That's a silicon budget, not a hole in the argument: the trap is

the cost-on-certification law, and the law lives in the transition, not the gate count.

Everything below earns that. The matrix algebra is the discharge: it shows the wall `CHSH_LASSERT` traps against is real and externally fixed, the Tsirelson boundary, not a bar I set myself. I came at it sideways, stress-testing the μ-ledger's cost algebra against a known algebraic boundary, the NPA moment matrix conditions for what correlations are achievable under polynomial constraints.

I did not set out to find what I found. The chain is concrete. The Z-arithmetic check the kernel runs on the witness counters (decidable, integer-only) is sound with respect to the real-valued column-contractive predicate by `column_contractive_check_witness_sound` (188 lines, closing with `nra` after IZR distribution and field simplification). That real-valued predicate is biconditional with PSD of the zero-marginal NPA moment matrix by `column_contractive_iff_quantum_realizable`. End-to-end: a successful `CHSH_LASSERT` step entails NPA-PSD on the witness-derived correlators, by `chsh_lassert_no_trap_implies_quantum_realizable`. No quantum-mechanical premises live in the Coq dependency tree: no Hilbert space, no operators, no observables. The identification of NPA-PSD with quantum-realizable correlations lives in the NPA framework itself, cited externally (Navascués–Pironio–Acín 2007, 2008) as standard. The kernel's content is the algebraic bridge between an integer-arithmetic check inside an opcode and PSD of a specific matrix; calling that condition "quantum-realizable" uses NPA's own terminology, not a physics derivation inside the kernel.

Direction of derivation: substrate first, NPA second. Two passes, in this order. First pass, substrate-side: the `CHSH_TRIAL` opcode, its witness-counter cost schedule, the integer Z-arithmetic predicate `column_contractive_check` that `CHSH_LASSERT` runs on those counters, and the soundness of that integer check against its real-valued lift. None of this references NPA. The honesty condition the μ-ledger enforces at `CHSH_LASSERT` is: a successful witness check requires the counters to satisfy three explicit polynomial inequalities on cleared integer ratios. That predicate is defined inside the cost algebra and verified by the kernel without consulting the quantum-mechanics literature. Second pass: structural agreement. The substrate-native predicate, lifted to the reals via IZR and the witness-count denominators, is the column-contractive predicate; the column-contractive predicate is biconditional with PSD of the zero-marginal NPA moment matrix. The biconditional is an agreement between two independently-defined objects (the cost algebra's honesty condition and NPA's quantum-realizability condition), proven in pure polynomial arithmetic with no project-local axioms. The load-bearing result is §16's "Algebraic characterization of the μ-ledger honesty condition" subsection. The earlier subsections (Bell test setup, classical bound, quantum bound) are the walk there. If you have time for one subsection, that is the one. The CHSH game is built from scratch in the next subsection so the result makes sense to readers who have not seen Bell tests before; readers who have can skip ahead.

16.1 The Bell test setup, from scratch

Imagine two players, Alice and Bob, in separate rooms. They cannot communicate once the game starts. A referee sends Alice one of two questions ($x \in \{0,1\}$). The referee sends Bob one of two questions ($y \in \{0,1\}$). Alice answers $a \in \{0,1\}$. Bob answers $b \in \{0,1\}$. (Settings (x, y), outcomes (a, b): same convention as Section 9 and the Coq.)

The game rule: they want $a \oplus b = x \wedge y$. The symbol $\oplus$ is XOR (exclusive OR): $a \oplus b = 1$ when $a \neq b$, and 0 when $a = b$. The symbol $\wedge$ is AND: $x \wedge y = 1$ only when both $x = 1$ and $y = 1$, otherwise 0. In plain language: their answers should be different if and only if both questions were 1. For all other question combinations (both 0, or one each), they should match.

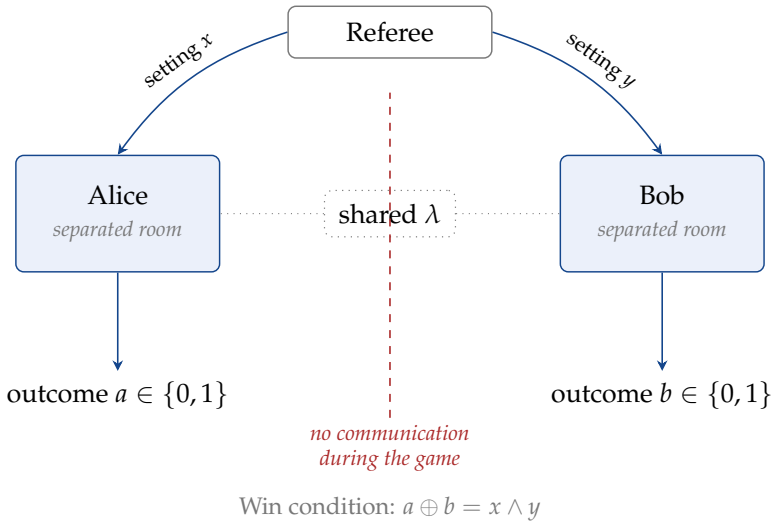


Figure 6: The CHSH setup. The referee distributes settings x, y; Alice and Bob each have a separated room and may share an agreed-on λ from before the game started. No communication during the game. Each produces a ± 1-valued outcome. The four correlators $E_{xy} = \langle ab \rangle$ they accumulate across trials are what the kernel's CHSH_LASSERT instruction checks against the column-contractivity / NPA condition.

Why 75% is the classical ceiling. Before the game, Alice and Bob can agree on any deterministic strategy: a function $a(x, \lambda)$ for Alice and $b(y, \lambda)$ for Bob, where λ is any shared randomness they agree on in advance. Once the game starts, no communication.

Fix any deterministic strategy and freeze λ. Then Alice has two predetermined answers, a_0 and a_1, and Bob has two predetermined answers, b_0 and b_1. Winning all four question combinations would require:

$$a_0 \oplus b_0 = 0, \quad a_0 \oplus b_1 = 0, \quad a_1 \oplus b_0 = 0, \quad a_1 \oplus b_1 = 1.$$

Now XOR the four left sides. Every variable appears twice, so the total is 0. XOR

the four right sides and the total is 1. That is impossible. So every deterministic local strategy loses at least one of the four cases, and the best deterministic strategy wins at most 3 out of 4 question combinations.

And you cannot buy your way past it with a coin flip. A randomized strategy is nothing but a bag of deterministic ones with a die deciding which to pull, and every strategy in the bag already tops out at 75%. Average a pile of things that each sit at 75% or below and the average sits at 75% or below; there is no arrangement of losers that sums to a winner. That is linearity of expectation, the most boring theorem in the building, quietly closing the last door.

So the classical ceiling is $3/4$, seventy-five percent, and not a hair more. And I do not want you to take that on the strength of an XOR trick on a page; the reference kernel pins it down both ways and leaves nothing Admitted. One executable trace, built from nothing but `PNEW`, `PSPLIT`, and `CHSH_TRIAL` and never spending a cent of the cost ledger ($\mu = 0$), walks right up to 75% and touches it. Then `local_box_CHSH_bound` closes the lid from the other side: no free program, no $\mu = 0$ trace anywhere, gets over the line. The achievable and the unbeatable land on the same number, which is the only kind of ceiling I trust.

16.2 The CHSH inequality: where it comes from

Now here is what took me a while to understand: why is there a specific algebraic combination S that detects whether correlations can be explained classically?

The key move is this. For any fixed shared randomness λ, Alice's output $A_x(\lambda) \in \{-1, +1\}$ (I'm using ± 1 encoding now, not $0/1$, indexed by Alice's setting x) and Bob's $B_y(\lambda) \in \{-1, +1\}$ (indexed by Bob's setting y). Consider the expression:

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$

Factor it:

$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

Since B_0 and B_1 are each ± 1, either $B_0 + B_1 = \pm 2$ and $B_0 - B_1 = 0$, or $B_0 + B_1 = 0$ and $B_0 - B_1 = \pm 2$. The two terms cannot both be large. In either case, $|B_0 + B_1| + |B_0 - B_1| = 2$, and since $|A_0|, |A_1| \leq 1$, the entire expression is at most 2 in absolute value for any fixed λ.

Average over λ:

$$S = \langle A_0B_0 \rangle + \langle A_0B_1 \rangle + \langle A_1B_0 \rangle - \langle A_1B_1 \rangle$$

where $\langle A_x B_y \rangle$ is the correlation (expected value of the product) when settings are x and y. Since the expression before averaging was bounded by 2 for every λ, the average satisfies $|S| \leq 2$.

That is the argument. The combination with the minus sign is chosen exactly because the factoring trick works: one of the B-terms goes to zero and the other to ± 2, forcing the whole thing below 2. The inequality $|S| \leq 2$ is a pure algebraic consequence of the outputs being ± 1 and being determined locally.

None of that lives only in my head. The argument I just walked you through is the English proof; the formal check that I did not fool myself anywhere is `local_box_CHSH_bound`, which says the plain thing in symbols: for any factorizable box B, $|S(B)| \leq 2$, with nothing left Admitted. And a bound from above is only half an honest claim. `classical_bound_achieved` hands over the other half, a constructive witness, an executable trace at $\mu = 0$ that actually reaches $|S| = 2$ rather than just respecting it. Put the two together and the ceiling is not loose: $\max\{|S| : \mu = 0\} = 2$, exactly, no slack to argue about.

16.3 The quantum bound: why $2\sqrt{2}$?

Now hand the same game to quantum mechanics and the ceiling lifts. Not to anything you please, and not all the way to a perfect 4; it lifts to $2\sqrt{2} \approx 2.828$ and stops there as hard as the classical 2 stopped. That number is the Tsirelson bound, named for Boris Tsirelson, who was the first to write down $|S| \leq 2\sqrt{2}$ and pin quantum correlations under it. The physics story of *why* it is that exact number, and not some other irrational nobody would have guessed, runs through Hilbert-space operators, anticommutators, the Pauli matrices, the Bloch sphere, the whole machinery. It is a real and beautiful derivation and I am going to walk straight past it, because none of the formal content here leans on it. I do not need to retell quantum mechanics to show you the kernel respects the wall; I only need the algebra, and that is what I am keeping.

What the formal content does depend on, and what I will derive in the next subsections, is the algebra. The Coq theorem about quantum realizability takes "the correlation is quantum-realizable in the zero-marginal NPA framework" as a premise (Section 2 builds the term) and concludes $|S| \leq 2\sqrt{2}$. The premise is the physics input; the conclusion is pure algebra. Section 16's last subsection then builds the same bound from polynomial inequalities on the NPA moment matrix without any quantum-physics premise at all. That is the path I follow, and that is the part I am claiming.

And that gap between 2 and $2\sqrt{2}$ is not bookkeeping. It is the most stubborn fact in physics: people have gone into labs, fired entangled photons through polarizers, counted coincidences, and watched the number sit up above 2 where no local classical story is allowed to put it. Whatever the world is doing, it is not Alice and Bob with a shared cheat sheet agreed beforehand. The physics of *why* the wall sits at $2\sqrt{2}$ is somebody else's chapter and a good one. My question is narrower and, to me, stranger: why does the kernel's CHSH check respect a wall it was never told about?

The polynomial arithmetic below answers it. Feed the cost ledger nothing but its own algebra, no quantum mechanics anywhere in the room, and the same $2\sqrt{2}$ falls out the bottom on its own.

A note on the rational approximation in the kernel. The kernel names the rational threshold $5657/2000 \approx 2.8285$ for exact rational comparisons (you can verify $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8$, so $5657/2000 > \sqrt{8} = 2\sqrt{2}$). The real-valued quantum-model bridge uses $\sqrt{8}$ directly. Those are two different proof surfaces, and I keep them apart on purpose: the rational one so a machine can check it in exact arithmetic, the real one so the quantum bridge has something to bite on.

16.4 What the cost algebra proves

Theorem 16.1 (Classical CHSH Bound)**.** *For any locally factorizable correlation (Alice's answer depends only on x and shared randomness λ; Bob's answer depends only on y and λ), $|S| \leq 2$. And $|S| = 2$ is achievable at $\mu = 0$, so the bound is tight.*

Proof. $|S| \leq 2$: the XOR-factoring argument from Section 16's "Why 75% is the classical ceiling." Fix any local deterministic strategy and any λ. Alice produces $A_x(\lambda) \in \{-1, +1\}$; Bob produces $B_y(\lambda) \in \{-1, +1\}$. The quantity

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$
$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

satisfies $|\cdot| \leq |B_0 + B_1| + |B_0 - B_1| = 2$ (because for $B_0, B_1 \in \{\pm 1\}$, one of $|B_0 + B_1|$ and $|B_0 - B_1|$ is 2 and the other is 0, and $|A_0|, |A_1| \leq 1$). Average over λ to get $|S| \leq 2$. Mixtures of deterministic strategies are convex combinations, so the same bound holds by linearity of expectation.

Tightness: the strategy $A_x \equiv 1$, $B_y \equiv 1$ gives $\langle A_0B_0\rangle = \langle A_0B_1\rangle = \langle A_1B_0\rangle = \langle A_1B_1\rangle = 1$, so $S = 1 + 1 + 1 - 1 = 2$. This is achievable at $\mu = 0$ by a program with all $\delta = 0$ instructions accumulating one trial per setting pair in `vm_witness`. The Coq witness is `classical_bound_achieved`. □ □

> ***Plain reading.*** Factor the expression. Two terms. One has $B_0 + B_1$, the other has $B_0 - B_1$. With $B_0, B_1 \in \{\pm 1\}$, only one can be nonzero at a time, and the nonzero one is ± 2. Multiply by $|A| \leq 1$. The whole expression caps at 2. Average over λ, you get $|S| \leq 2$. To make it 2, just have both players always answer 1.

Theorem 16.2 (Non-locality of CHSH violation)**.** *WitnessCount configurations with $S > 2$ cannot be produced by any local hidden variable model (the term is built in Section 2: a classical-physics-style theory where outcomes come from shared randomness λ plus local settings).*

Proof. Contrapositive of the Classical CHSH Bound. Any LHV model produces a

locally factorizable correlation by definition: each player's outcome is a function of their setting and the shared λ. The theorem above says all such correlations satisfy $|S| \leq 2$. So a configuration with $S > 2$ cannot have arisen from any LHV model. □ □

> ***Plain reading.*** Direct contrapositive. If $|S| \leq 2$ for every LHV, then $S > 2$ rules out every LHV. The Bell-test contradiction is in this form.

Theorem 16.3 (Tsirelson upper bound for the zero-marginal NPA model)**.** *If a trace is quantum-realizable in the zero-marginal NPA model, then its CHSH value satisfies* $|S| \leq \sqrt{8} = 2\sqrt{2}$.

Proof. Quantum realizability in the zero-marginal NPA model means the four CHSH correlators E_{xy} arise as $\mathrm{Tr}(\rho\, A_x B_y)$ for some quantum state ρ and self-adjoint ± 1-valued observables A_x, B_y with $\rho_{A_x A_x} = \rho_{B_y B_y} = I$ and vanishing marginals. That is the quantum-mechanical way of writing measurements with ± 1 outcomes, and the argument below leans only on the algebraic equivalent that follows, not on the physics notation. Equivalently (by Section 16's biconditional), the correlators satisfy the three column-contractivity conditions. From column-contractivity, $S^2 \leq 8$: this is the row-bounds factorization (Cauchy-Schwarz applied to each row of the correlator matrix viewed as inner products of unit vectors). Specifically, the row norms of E satisfy $\|E_{0,\cdot}\|_2 \leq 1$ and $\|E_{1,\cdot}\|_2 \leq 1$, and the CHSH expression S is an inner product of bounded vectors with the row vectors, yielding $|S|^2 \leq 4(\|E_{0,\cdot}\|_2^2 + \|E_{1,\cdot}\|_2^2)$ via the Cauchy-Schwarz argument in `tsirelson_from_row_bounds`. Combined with row-norms ≤ 1: $|S|^2 \leq 8$, so $|S| \leq 2\sqrt{2}$. □ □

> ***Plain reading.*** Quantum-realizable means column-contractive (by the biconditional). Column-contractive bounds the row norms of the correlator matrix by 1. Cauchy-Schwarz on the CHSH expression viewed as an inner product gives $|S| \leq 2\sqrt{2}$. No quantum-mechanical detail enters past the biconditional; the bound is then pure algebra.

Theorem 16.4 (General algebraic Tsirelson bound)**.** *For every algebraically coherent correlator (one satisfying* $|E_{xy}| \leq 1$ *along with four constraints derived from* 3×3 *submatrices of the NPA moment matrix; the submatrix determinants, called minors, must be non-negative, which is part of the requirement that the full matrix be positive-semidefinite, see Section 2 for both terms), and given some witness parameters* t *and* s *that the kernel builds explicitly, the CHSH value satisfies* $S^2 \leq 8$*, that is,* $|S| \leq 2\sqrt{2}$*. This is the general case: every algebraically coherent correlator, not just the symmetric configuration. The Coq proof uses* `psatz` *(described in Section 2: a Coq tactic that proves polynomial inequalities by exhibiting a sum-of-squares decomposition). The rational approximation corollary uses* $5657/2000$ *as an exact rational overestimate of* $2\sqrt{2}$ *(you can verify* $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8$*), allowing the bound to be checked in exact rational arithmetic without floating-point. No physics*

premises. Pure polynomial arithmetic. The Tsirelson bound $2\sqrt{2}$ is derived from algebra, not assumed.

Proof. The four minor-positivity conditions on 3×3 submatrices of the NPA moment matrix are: each principal 3×3 minor's determinant is non-negative. Working through the determinant formulas with the moment-matrix structure of Section 16, the four constraints become four polynomial inequalities of the form $1 - E_{xy}^2 - E_{x'y'}^2 \geq 0$ for the appropriate index pairs (these reduce to the column-contractivity conditions 1 and 2 plus their transposes).

Given these constraints, $S^2 \leq 8$ is a polynomial inequality in $(E_{00}, E_{01}, E_{10}, E_{11})$ subject to four polynomial constraints. The Positivstellensatz (the algebraic certificate for polynomial-positivity) gives a sum-of-squares decomposition: the polynomial $8 - S^2$ minus a non-negative combination of the constraint polynomials is itself a sum of squares. Coq's `psatz` tactic finds and verifies such a decomposition; the explicit witness parameters t, s are coefficients in the SOS combination, computed once and stored in the kernel.

The rational approximation corollary uses $5657/2000$ in place of $\sqrt{8}$ to keep the comparison in $\mathbb{Q}$: $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8 = 32{,}000{,}000/4{,}000{,}000$, so $5657/2000 > \sqrt{8}$. Combined with $|S|^2 \leq 8$, this gives $|S| < 5657/2000$, a strict rational bound checkable without floating-point evaluation. □ □

> ***Plain reading.*** Four polynomial constraints on the correlators (the 3×3 minor positivity conditions). Goal: show $S^2 \leq 8$ subject to those constraints. Positivstellensatz says: if a polynomial is non-negative on the feasible set, you can write it as a non-negative combination of the constraint polynomials plus a sum of squares. `psatz` finds the combination. The Coq file stores the witness coefficients. The Tsirelson bound is the consequence; no physics input goes into the proof.

Bell's theorem has another face, and it is worth seeing both. The statistical face is the one everyone draws: counts going into a number S and the number refusing to stay under 2. But the same wall shows up one floor down, in the language of couplings, where you are no longer talking about averages at all but about whether two parties can each answer locally and have their answers fit together. A WitnessCount-consistent locally deterministic strategy here is an honest, almost insultingly simple thing: four bits, Alice's answer for each of her two settings and Bob's for each of his, fixed in advance with no peeking. Pin that down and you get a separable coupling, by which I mean one that hands each setting exactly one outcome type, no hedging, no mixing. Any locally factorizable morphism coupling that respects the WCLocallyConsistent predicate is stuck below the bound: $|S| \leq 2$. So read it the other way, which is the way that bites. If a trace ever shows $S > 2$, there is no local strategy of four bits that could have produced it, full stop. The wall is not a fact about statistics that we then dress up in structure; it is already there in the structure, and the statistics are just

where you happen to bump into it first.

What does any of this have to do with computation? Everything, and it comes in through the witness counters and the certification layer. Picture a trace that sits there racking up CHSH trial results through CHSH_TRIAL, tally after tally, costing nothing, because counting is cheap and the universe does not send you a bill for watching it. Then the trace does the one expensive thing: it turns around and certifies that $S > 2$ via CERTIFY or LASSERT. That is the cert-flip, and A2 forces $\mu \geq 1$ on it, no exceptions. So the statistics are free and the claim about the statistics is not, and that gap is the whole point. You can stare at the data all day for nothing; the moment you stand up and say you know what it means, somewhere a counter ticks. Throughout, the witness counters carry Alice's setting x and Bob's setting y, the same indexing the formal layer uses, so this is not a side dialect (Section 9). It is the witness insight taxonomy of Section 6, viewed from the place where physics hands you a free supply of trials and then charges you for the sentence you write about them.

16.5 The bit-commitment requirement

If you want cross-correlations above the classical bound to come out attested, not just observed, the reference kernel makes you spend a disclosure instruction to get them. That falls straight out of how cost is priced: A2 charges the cert-flip, so the only way to reach an attested claim is to take the step that flips the cert, and that step costs. The strictness is downstream of the accounting, not the other way around, and the next few lines walk through exactly where it gets forced.

So here is where that gets concrete. A single CHSH_TRIAL drops a few setting bits and outcome bits into the witness counters, and the cross-correlations $\langle A_x B_y \rangle$ are nothing but those tallies divided through. They are a record of what happened, and the ledger has no opinion about a record. The accounting only wakes up when you point at one of those numbers and say it backs a particular claim. That is not a count any more. It is a stance, and a stance is the one thing the ledger knows how to charge for, which is why the cost lands precisely on the sentence and never on the arithmetic underneath it.

And this part is not a hope, it is settled. There is exactly one instruction in the reference kernel that can push `csr_cert_addr` off zero, and that is MORPH_ASSERT, and even then only when the morphism genuinely exists and the property string it carries is more than an empty NUL byte. So the door to attestation has a single key. Run that backward and it pins the trace: anything ending in supra_cert had to have walked through a valid MORPH_ASSERT step on the way, there is no other path that sets the register, and the kernel will not let you fake one. Which is the same wall from a closer angle. CHSH_TRIAL counts pile up for free, all the raw data you like, but the instruction that stands up and certifies what the pile means is the one that costs, and it is the only one that even can.

16.6 Algebraic characterization of the μ-ledger honesty condition

If you only have the patience for one subsection of Section 16, make it this one. Everything before it is me showing my work, the honest grind of getting from the cost algebra to the physics boundary. What lands here is the thing the grind was for: the place where the bookkeeping the machine does and the wall the universe puts up turn out to be the same statement read two ways. The rest of the section earns it. This is what it earns.

Here is the result of stress-testing the cost algebra against the NPA boundary. The column-contractivity conditions arise from the natural polynomial conditions that make the NPA moment matrix well-formed; they were not imported from physics, and the biconditional with NPA-PSD is proved in pure polynomial arithmetic. The following biconditional holds.

Positive semidefinite (PSD). A matrix M is positive semidefinite if for every vector v you can multiply with it, the quantity $v^T M v$ is at least 0. (Here v^T is the transpose of v from Section 2: turning a column of numbers on its side; $v^T M v$ is the matrix product of v^T, M, and v, which is just one number.) The intuition: PSD means the matrix never maps a vector to something pointing "backwards" relative to the vector. For a 2×2 matrix $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$, PSD reduces to three simple conditions: $a \geq 0$, $c \geq 0$, and $ac - b^2 \geq 0$. The first two say the diagonal entries are non-negative. The third says the determinant (Section 2) is non-negative. For larger matrices, the PSD condition generalises: every principal submatrix (every square subblock you get by picking some rows-and-columns at the same indices, see Section 2) must also be PSD.

PSD is the mathematical condition that a matrix can represent genuine inner products, covariances, or correlations from a quantum state. The NPA framework uses PSD of a specific 5×5 matrix (the "moment matrix") as the condition for quantum realizability. You build that 5×5 matrix from the four CHSH correlators, and if it is PSD, the correlators are consistent with some quantum state.

16.6.1 The matrix, written out

Here is the actual 5×5 NPA moment matrix for the zero-marginal CHSH scenario. The five operators are: I (identity), A_0 (Alice's measurement for setting 0), A_1 (Alice's measurement for setting 1), B_0 (Bob's for setting 0), B_1 (Bob's for setting 1). The (i, j) entry is the expected value of $O_i \cdot O_j$. Zero-marginal means we set $\langle A_0 \rangle = \langle A_1 \rangle = \langle B_0 \rangle = \langle B_1 \rangle = 0$ and the cross-correlation terms $\langle A_0 A_1 \rangle = \langle B_0 B_1 \rangle = 0$.

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix}$$

Rows and columns are indexed 0 through 4, in the order $\{I, A_0, A_1, B_0, B_1\}$. The diagonal is all 1s because each operator O_i satisfies $O_i^2 = I$ when the outcomes are ± 1-valued: squaring a ± 1-valued operator gives the identity, so the diagonal entries are all $\langle I \rangle = 1$. The off-diagonal entries E_{xy} are the CHSH correlators with Alice's setting x and Bob's setting y (Section 2). Everything else is zero because the marginals vanish (Section 2: a marginal is the average of one variable with the others integrated out; zero-marginal means $\langle A_x \rangle = \langle B_y \rangle = 0$).

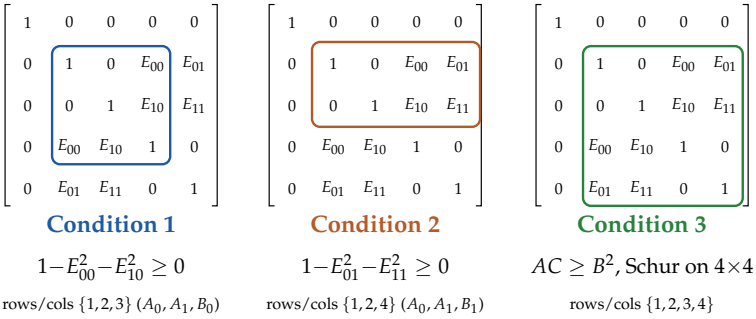


Figure 7: The 5×5 zero-marginal NPA moment matrix shown three times, each copy highlighting one principal submatrix whose PSD condition gives one column-contractivity condition. Two come from 3×3 principal-submatrix determinants; the third comes from the Schur complement of the 4×4 block. All three follow from PSD of the parent matrix.

16.6.2 Where the three conditions come from

The three column-contractivity conditions come directly from PSD of this matrix. Here is the derivation.

Condition 1: $1 - E_{00}^2 - E_{10}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 3\}$ of the 5×5 moment matrix (the rows and columns indexed by A_0, A_1, B_0):

$$\begin{pmatrix} 1 & 0 & E_{00} \\ 0 & 1 & E_{10} \\ E_{00} & E_{10} & 1 \end{pmatrix}$$

Compute the determinant of this 3×3 submatrix by the cofactor expansion along the first row (Section 2: pick a row, multiply each entry by the determinant of the 2×2 matrix you get by deleting that entry's row and column, with alternating signs, then sum):

$$1 \cdot (1 \cdot 1 - E_{10}^2) - 0 \cdot (0 \cdot 1 - E_{10} \cdot E_{00}) + E_{00} \cdot (0 \cdot E_{10} - 1 \cdot E_{00}) = 1 - E_{10}^2 - E_{00}^2.$$

The principal-submatrix rule for PSD says this determinant must be ≥ 0. That is condition 1.

Condition 2: $1 - E_{01}^2 - E_{11}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1,2,4\}$ (the A_0, A_1, B_1 block):

$$\begin{pmatrix} 1 & 0 & E_{01} \\ 0 & 1 & E_{11} \\ E_{01} & E_{11} & 1 \end{pmatrix}$$

Same calculation: determinant $= 1 - E_{01}^2 - E_{11}^2 \geq 0$. That is condition 2.

Condition 3: $(1 - E_{00}^2 - E_{10}^2)(1 - E_{01}^2 - E_{11}^2) \geq (E_{00}E_{01} + E_{10}E_{11})^2$

This comes from the 4×4 submatrix at rows and columns $\{1,2,3,4\}$. That submatrix has a block structure (an arrangement where the entries are themselves 2×2 matrices):

$$\begin{pmatrix} I_2 & E \\ E^T & I_2 \end{pmatrix}$$

where I_2 is the 2×2 identity matrix (Section 2) and E is the 2×2 correlator matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix}.$$

E^T is the transpose of E from Section 2. A standard linear-algebra fact (the Schur complement criterion, also Section 2) says that a block matrix of this shape is positive-semidefinite if and only if the Schur complement of the upper-left I_2 is positive-semidefinite. The Schur complement here works out to $I_2 - E^T E$. The order of the transpose matters: $E^T E$ puts the squared norms (sums of squares) of the columns of E on its diagonal, while EE^T would put the squared norms of the rows. The right one for this argument is $E^T E$.

Computing $E^T E$ entry by entry:

$$E^T E = \begin{pmatrix} E_{00} & E_{10} \\ E_{01} & E_{11} \end{pmatrix} \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix} = \begin{pmatrix} E_{00}^2 + E_{10}^2 & E_{00}E_{01} + E_{10}E_{11} \\ E_{00}E_{01} + E_{10}E_{11} & E_{01}^2 + E_{11}^2 \end{pmatrix}.$$

The Schur complement is therefore

$$I_2 - E^T E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix} = \begin{pmatrix} A & -B \\ -B & C \end{pmatrix}$$

where $A = 1 - E_{00}^2 - E_{10}^2$ (the left side of condition 1), $B = E_{00}E_{01} + E_{10}E_{11}$, and $C = 1 - E_{01}^2 - E_{11}^2$ (the left side of condition 2). For this 2×2 Schur complement to be PSD, two things must hold: the diagonal entries must be non-negative (which are conditions 1 and 2 again) and the determinant must be non-negative, $AC - B^2 \geq 0$. That last inequality is condition 3.

That is the complete derivation. Three conditions. Two from 3×3 submatrices. One

from the Schur complement of the 4×4 block. All of it falls out of the single requirement that M is positive semidefinite.

Why this equals quantum realizability. The forward direction is the derivation above: if the correlators are column-contractive (the three conditions hold), the matrix M is PSD by construction. The reverse direction (proved as `npa_psd_implies_column_contractive`) is that if M is PSD, the correlators must be column-contractive. Here is the argument. Take any PSD matrix M and feed it the specific 5-dimensional vector $v = (0,\ -(E_{00}y_0 + E_{01}y_1),\ -(E_{10}y_0 + E_{11}y_1),\ y_0,\ y_1)$, where y_0 and y_1 are free real numbers. The PSD condition $v^T M v \geq 0$, after the matrix algebra, gives a quadratic expression in y_0 and y_1: $Ay_0^2 - 2By_0y_1 + Cy_1^2 \geq 0$ for all real y_0 and y_1. A quadratic of the form $ay^2 + 2by + c$ is non-negative for every real y if and only if $a \geq 0$ and $ac \geq b^2$ (the non-negative-discriminant rule from intermediate algebra: the parabola does not dip below the axis exactly when its discriminant is non-positive, which after sign-flipping reads $b^2 \leq ac$). That closes condition 3. The diagonal conditions (1 and 2) come from setting $y_1 = 0$ and $y_0 = 0$ separately.

A correlation $(E_{00}, E_{01}, E_{10}, E_{11})$ is *Thiele-honest* if and only if it is zero-marginal NPA realizable. Exactly. Not approximately. Not "consistent with some physics axiom." The biconditional is pure polynomial arithmetic with no project-local axioms (`Print Assumptions` on `column_contractive_iff_quantum_realizable` reports only Coq's standard classical-reals and functional-extensionality stdlib axioms, namely `sig_not_dec`, `sig_forall_dec`, and `functional_extensionality_dep`, the same stdlib families Section 16.7 discloses for the Q_{1+AB} theorems; "zero axioms" in this section means zero project-local, in the sense of Section 22):

Theorem 16.5 (Thiele honesty $\iff$ NPA-PSD)**.**

$$\mathit{honest}(\mathbf{E}) \iff \mathit{psd\text{-}npa}(\mathbf{E})$$

where honest *means the three column-contractivity conditions hold, and* psd-npa *means the* 5×5 *zero-marginal NPA matrix is positive semidefinite.*

Proof. The full math is in the two derivations of Section 16 preceding this theorem. Recapitulated for the proof block:

Forward direction (column-contractive ⇒ PSD). Assume the three column-contractivity conditions hold. The Section 16 derivation showed: condition 1 is exactly the principal-submatrix determinant on rows/columns $\{1,2,3\}$; condition 2 is exactly the principal-submatrix determinant on rows/columns $\{1,2,4\}$; condition 3 is exactly the Schur-complement determinant of the 4×4 block. Combined with the $\{0\}$ principal submatrix (just the entry $1 \geq 0$) and the diagonal entries (all $1 \geq 0$), every principal submatrix has non-negative determinant and non-negative diagonal, so by the principal-submatrix PSD criterion (Section 2), the full 5×5 matrix is PSD. The Coq witness is `z`

`ero_marginal_npa_column_contractive_implies_psd`.

Reverse direction (PSD ⇒ column-contractive). Assume the moment matrix M is PSD. Specialize $v^T M v \geq 0$ at the vector

$$v(y_0, y_1) := \big(0,\ -(E_{00}y_0 + E_{01}y_1),\ -(E_{10}y_0 + E_{11}y_1),\ y_0,\ y_1\big)$$

for free $y_0, y_1 \in \mathbb{R}$. Computing $v^T M v$ using the explicit moment-matrix entries and collecting terms gives the quadratic form

$$A y_0^2 - 2B y_0 y_1 + C y_1^2 \ \geq\ 0 \quad \text{for all } y_0, y_1 \in \mathbb{R},$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. By the quadratic-non-negative-discriminant lemma (`quadratic_nonneg_discriminant`), this is equivalent to: $A \geq 0$, $C \geq 0$, and $B^2 \leq AC$. The three conditions are exactly the three column-contractivity conditions. Specializing further at $(y_0, y_1) = (1, 0)$ and $(0, 1)$ recovers conditions 1 and 2 directly; the general (y_0, y_1) case yields condition 3. The Coq witness is `npa_psd_implies_column_contractive`. □ □

> ***Plain reading.*** Both directions are pure algebra on the 5×5 matrix. Forward: each of the three column-contractivity conditions is a principal-submatrix or Schur-complement determinant being non-negative. PSD reduces to all principal-submatrix determinants ≥ 0, which is exactly what column-contractivity gives you. Reverse: feed the PSD matrix a specific test vector parameterized by (y_0, y_1). The result $v^T M v$ comes out as a quadratic in (y_0, y_1). Non-negative-for-all-(y_0, y_1) means the quadratic's discriminant is non-positive, i.e., $b^2 \leq ac$. That's condition 3. The diagonal conditions fall out at $(1, 0)$ and $(0, 1)$. No physics anywhere; both directions are polynomial-arithmetic identities.

The left side is three polynomial inequalities (column contractivity of the zero-marginal NPA matrix):

$$1 - E_{00}^2 - E_{10}^2 \geq 0 \quad \text{(condition 1)}$$

$$1 - E_{01}^2 - E_{11}^2 \geq 0 \quad \text{(condition 2)}$$

$$AC \geq B^2 \quad \text{(condition 3)}$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. These are the three conditions of `zero_marginal_column_contractive` (defined): column 0 norm ≤ 1, column 1 norm ≤ 1, determinant $AC - B^2 \geq 0$. The right side says the NPA moment matrix is positive semidefinite, the standard mathematical characterization of quantum realizability.

The forward direction (`zero_marginal_npa_column_contractive_implies_psd`) was proved first. The reverse direction: given a PSD NPA matrix, specialize it at

the vector $(0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$. The PSD condition gives a quadratic-in-y_0/y_1 that is everywhere non-negative. Then `quadratic_nonneg_discriminant` ($\forall t, a + 2bt + ct^2 \geq 0 \Rightarrow b^2 \leq ac$) closes the determinant condition. The diagonal conditions come from $y_0 = 1, y_1 = 0$ and $y_0 = 0, y_1 = 1$. Both directions are packaged as the biconditional `column_contractive_iff_quantum_realizable`.

What this means for the hierarchy of correlations:

- Classical local hidden variables: $|S| \leq 2$.
- Thiele-honest = zero-marginal NPA realizable (biconditional, both directions, no project-local axioms). Every Thiele-honest correlation satisfies $|S| \leq 2\sqrt{2}$, but NOT every correlation with $|S| \leq 2\sqrt{2}$ is Thiele-honest: $(1, 0, 1, 0)$ has $|S| = 2$ but fails condition 1.
- No-signaling: no constraint beyond $|E_{ab}| \leq 1$.
- PR box ($E_{00} = E_{01} = E_{10} = 1$, $E_{11} = -1$): condition 1 gives $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Fails immediately by direct arithmetic.
- Tsirelson bound: `state_column_contractive_implies_tsirelson` proves Thiele-honest $\Rightarrow |S|^2 \leq 8$ (i.e., $|S| \leq 2\sqrt{2}$) directly from column-contractivity. The chain factors through PSD: column-contractive $\Rightarrow$ NPA-PSD $\Rightarrow$ row bounds $\Rightarrow$ Tsirelson, and the row-bounds step is `tsirelson_from_row_bounds`. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not Thiele-honest) follows immediately. The converse is false and not proved.

A Turing machine can output the correlations $(1, 0, 1, 0)$. These are not Thiele-honest: condition 1 gives $1 - 1^2 - 1^2 = -1 < 0$. The arithmetic closes by `lra` once `column_contractive_iff_quantum_realizable` is destructed on condition 1. The μ-ledger's column-contractivity conditions are what make the quantum boundary real for computation: there exist correlations a Turing machine certifies but no Thiele Machine certifies as honest.

Kernel-level enforcement (the CHSH_LASSERT opcode). Here is the thing about a certification bit: by itself it is just a bit. A generic `CERTIFY` can set `vm_certified := true` and never once glance at the witness counters that are supposed to earn it. That is a rubber stamp with the ink already on it, and a rubber stamp is exactly what the whole discipline is built to forbid. So the certification channel cannot lean on `CERTIFY` for honesty; it needs an opcode that does the looking, that reads the evidence before it is allowed to say the word. That opcode is `CHSH_LASSERT`, and it ties "Thiele-honest" as column-contractivity to the certification channel by refusing to take the claim on faith. It reads the WitnessCounts buckets, the counts of how the two parties actually came out together and apart, and it computes the three integer-arithmetic conditions `column_contractive_check_witness` in Z-arithmetic, no

real and no rational ever evaluated at runtime, because the moment you let floating point near a soundness check you have smuggled in a place for the answer to drift. If the counts satisfy the three conditions, the program counter advances and the run goes on. If they do not, there is no negotiating: the step traps to `LASSERT_TRAP_PC` and latches `vm_err`, and the dishonest certification never happens. Two theorems close the chain to NPA-PSD; both are stated and proved below.

Theorem 16.6 (Integer-check soundness, `column_contractive_check_witness_sound`)**.** *Let w be the WitnessCount record with eight non-negative-integer fields $\texttt{wc\_same}_{xy}, \texttt{wc\_diff}_{xy}$ for $(x, y) \in \{0,1\}^2$. Define the integer-only check as the conjunction of three $\mathbb{Z}$-arithmetic inequalities on these counts (the boolean function `column_contractive_check_witness` written in $\mathbb{Z}$, with no division). If the integer check returns `true`, then defining*

$$N_{xy} = \texttt{wc\_same}_{xy} + \texttt{wc\_diff}_{xy}, \qquad E_{xy} = \frac{\texttt{wc\_same}_{xy} - \texttt{wc\_diff}_{xy}}{N_{xy}}$$

(with the convention $E_{xy} = 0$ when $N_{xy} = 0$) yields a real-valued correlator $\mathbf{E} = (E_{00}, E_{01}, E_{10}, E_{11}) \in [-1,1]^4$ that satisfies the three column-contractivity conditions of Section 16.

Proof. The integer-only check is written in $\mathbb{Z}$ as three inequalities that, after multiplying through by N_{xy}^2 to clear all denominators, are polynomial inequalities in the non-negative integer counts. For each condition:

- Condition 1 ($1 - E_{00}^2 - E_{10}^2 \geq 0$) becomes the inequality $N_{00}^2 \cdot N_{10}^2 - (\texttt{wc\_same}_{00} - \texttt{wc\_diff}_{00})^2 N_{10}^2 - (\texttt{wc\_same}_{10} - \texttt{wc\_diff}_{10})^2 N_{00}^2 \geq 0$ after substituting E_{xy} and clearing denominators.
- Condition 2 is the analogous inequality with the E_{01}, E_{11} correlators.
- Condition 3 ($AC \geq B^2$ in the notation of Section 16) becomes a polynomial inequality of degree 4 in the integer counts, after clearing the N_{xy}^4 denominator.

The integer check is exactly these three polynomial inequalities tested on the integer counts (using $\mathbb{Z}$, no rationals). To prove the conclusion: assume the check returns `true`. Cast the integer counts to $\mathbb{R}$ by $\texttt{IZR} : \mathbb{Z} \to \mathbb{R}$, which preserves the inequalities. Divide through by N_{xy}^2 (or N_{xy}^4, for condition 3) using the fact that $N_{xy} \geq 0$ as a natural number; in the $N_{xy} = 0$ case, the convention $E_{xy} = 0$ makes the inequality trivial. After dividing, the three inequalities are exactly conditions 1, 2, 3 of column-contractivity in $\mathbb{R}$. The Coq proof closes the post-division algebra by `nra` (nonlinear real arithmetic) acting on a sum-of-squares-shaped goal. □ □

> ***Plain reading.*** Integer arithmetic upstairs, real arithmetic downstairs, IZR is the elevator. You take the integer counts of same and different outcomes, write down

the three column-contractivity conditions in terms of those counts (cleared of denominators), and you have polynomial inequalities in $\mathbb{Z}$. Cast to $\mathbb{R}$ via IZR; this preserves ≥ 0. Divide back through by the (non-negative) total-trial counts; this gives the real-valued column-contractivity conditions. The algebra after the division is just a sum-of-squares-shaped polynomial inequality; `nra` closes it. The whole route avoids any floating-point evaluation at any step: integers in, reals out, division at the boundary.

Theorem 16.7 (CHSH_LASSERT ⇒ NPA-PSD, `chsh_lassert_no_trap_implies_quantum_realizable`)**.** *Let s be a state and let $s' =$ `vm_apply` s (`instr_chsh_lassert` δ). If s'.`vm_err` $=$ false and s'.`vm_pc` $= s$.`vm_pc` $+ 1$ (i.e., the CHSH_LASSERT step advanced normally rather than trapping), then the zero-marginal NPA moment matrix M built from the witness-derived correlators E_{xy} of s.`vm_witness` is positive semidefinite.*

Proof. By the apply rule for `instr_chsh_lassert`: the step branches on `column_contractive_check_witness` run on s.`vm_witness`. If the check returns `false`, the step traps to `LASSERT_TRAP_PC` and sets `vm_err = true`. If the check returns `true`, the step advances PC by 1 and leaves `vm_err = false`. By hypothesis, s' exhibits the second behavior, so the integer check returned `true` on s.`vm_witness`.

Compose with the preceding theorem (integer-check soundness): the correlators E_{xy} derived from s.`vm_witness` satisfy the three real-valued column-contractivity conditions.

Compose with the forward direction of `column_contractive_iff_quantum_realizable` (the biconditional proved in Section 16): column-contractivity implies the NPA moment matrix M is positive semidefinite. □ □

> ***Plain reading.*** Three theorems compose: the kernel step rule says no-trap means the integer check passed; the soundness lemma says the integer check passing implies real-valued column-contractivity; the biconditional says column-contractivity implies NPA-PSD. Stack them. A successful CHSH_LASSERT step is a proof that the witness-derived correlators land inside the NPA-PSD set. No physics input anywhere in the stack. The (1,0,1,0) correlation fails the integer check (condition 1 fails immediately: $1 - 1 - 1 = -1 < 0$), so any program that accumulated those statistics traps on CHSH_LASSERT.

Run the (1,0,1,0) correlation through `CHSH_LASSERT` and watch what the step does with it: the integer check fails on the very first condition, the step refuses to advance, and the state comes out the far side wearing `vm_err = true`. Nothing in the kernel was ever told that (1,0,1,0) is unphysical. It was told to count, clear denominators, and test three inequalities, and the arithmetic simply would not close. That is the only gate

there is. A trace that is column-contractive walks straight through; a trace that is not gets stopped by its own numbers, and the kernel never has to know why.

To falsify: exhibit $E_{00}, E_{01}, E_{10}, E_{11}$ satisfying the three polynomial inequalities but with a non-PSD NPA matrix. No such example exists; the theorem proves it impossible. Or exhibit a quantum experiment producing a non-PSD NPA matrix. That would break quantum mechanics, not this theorem.

16.7 Lifting the bridge to level 1+AB: the four-slice Schur cascade

Section 16 above derives the level-1 bridge: CHSH_LASSERT $\Rightarrow$ PSD of the 5×5 zero-marginal NPA matrix. The level-1 cone contains the Tsirelson set Q (every quantum-realisable correlator is PSD at level 1), but it strictly contains Q: the level-1 cone is bigger than Q for CHSH, so a level-1 certificate does not on its own pin the correlator down to physically quantum. The next NPA level that does coincide with Q for CHSH is level 1+AB: the 9×9 moment matrix indexed by the basis $\{I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1\}$. That $Q_{1+AB} = Q$ in the CHSH scenario is Ishizaka 2025's result, not mine, and I cite it instead of re-deriving it because re-deriving someone else's NPA-hierarchy theorem from scratch is a fight nobody asked me to pick. What the substrate actually nails down is the certification end: a successful step of one of the four `CHSH_LASSERT_1AB` cert-opcodes is a proof that the 9×9 Q_{1+AB} moment matrix is PSD at the witness-derived rationals and the bucket-derived γ values. The opcode steps; the matrix is in the cone. That is the whole bridge.

And the two levels are not two separate tricks. The 5×5 and the 9×9 checks are one dimension-polymorphic PSD predicate evaluated at two sizes: top index 4 for the 5×5, top index 8 for the 9×9, the same fold either way. The level-1 CHSH biconditional from Section 16 is the dimension-5 instance of it, recovered as a corollary that imports the original 5×5 result as a black box and adds no axioms over it (`colum n_contractive_iff_general_realizable`). So the lift in this subsection is not a 9-shaped coincidence dressed up as a method: it is the same realizability projection run one dimension up, and the dimension-5 corollary reproducing the bridge you already have is exactly the check that it is one predicate and not two.

16.7.1 The 9×9 moment matrix and its five γ parameters

The level-1+AB moment matrix is parametric in nine reals: the four CHSH correlators $(E_{00}, E_{01}, E_{10}, E_{11})$ and five additional moments

$$\gamma_1 = \langle A_0A_1B_0\rangle, \qquad \gamma_2 = \langle A_0A_1B_1\rangle, \qquad \gamma_3 = \langle A_0B_0B_1\rangle,$$
$$\gamma_4 = \langle A_1B_0B_1\rangle, \qquad \gamma_5 = \langle A_0A_1B_0B_1\rangle.$$

γ_1, γ_2 are the two three-body A-A-B moments. γ_3, γ_4 are the two three-body A-B-B moments. γ_5 is the four-body moment, and it enters the matrix in two cells that are *not* equal: the (A_0B_0, A_1B_1) cell carries $\gamma_5 = \langle A_0A_1B_0B_1\rangle$, while the conjugate (A_0B_1, A_1B_0) cell carries $\langle A_0A_1B_1B_0\rangle = -\gamma_5$, the same four operators in the oppo-

site B-order, equal in magnitude and opposite in sign because the orthogonal measurements anticommute (the slice already fixes $\langle B_0 B_1 \rangle = 0$, i.e. $\{B_0, B_1\} = 0$, so $\langle A_0 A_1 B_1 B_0 \rangle = -\langle A_0 A_1 B_0 B_1 \rangle$). That minus sign is the operator anticommutator a CHSH violation runs on. Writing the same γ_5 into both cells would assert $B_0 B_1 = B_1 B_0$, a classical commutativity that forces the certifiable score down to the classical bound 2; the conjugate sign is what lets the slice reach Tsirelson's $2\sqrt{2}$. Each γ_k lives in $[-1, 1]$; on a quantum state with zero marginals, each γ_k is the corresponding expectation. The 9×9 matrix entries are polynomials of degree ≤ 2 in (E_{ij}, γ_k), given explicitly by the products of basis operators (e.g. entry $(I, A_0 B_0) = E_{00}$, entry $(A_0, A_1 B_0) = \gamma_1$, entry $(A_0 B_0, A_0 B_0) = 1$, etc.). PSD of this matrix is the level-1+AB NPA condition.

16.7.2 Why four slices

The substrate certifies four nested γ-slices, each via its own cert-opcode and its own integer-arithmetic check:

- **Slice A** ($\gamma = 0$, tag 0x2F): all five γ_k pinned to zero. The bool decider checks Q_1 column-contractivity (the three conditions of §16) plus the SOS sum-of-squares condition $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \leq 1$. Closed by an explicit 4-D Cauchy-Schwarz SOS identity.
- **Slice B** (γ_5 free, tag 0x30): the four-body moment γ_5 is admitted as a bucket-pair argument; the other four γ are zero. The check adds a weighted 4-D Cauchy-Schwarz identity that handles the $2\gamma_5(v_{11}v_{22} - v_{12}v_{21})$ cross term arising in the residual (the relative minus is the conjugate-cell sign above: γ_5 on the $v_{11}v_{22}$ pairing, $-\gamma_5$ on the $v_{12}v_{21}$ pairing).
- **Slice C** ($\gamma_3, \gamma_4, \gamma_5$ free, tag 0x31): the two A-B-B moments and the four-body moment are admitted; $\gamma_1 = \gamma_2 = 0$. The check is a 4×4 Sylvester PD test on a difference matrix $H_{\gamma_{345}} = \det_M \cdot M_M - M_N$, derived via an augmented 2×2 Schur-slack lemma.
- **Slice D** (full $\gamma_1, \ldots, \gamma_5$ free, tag 0x32): all five γ parameters are admitted. The check is a compositional sym6 $\to$ sym5 $\to$ sym4 Schur cascade: the only check stiff enough to handle the bilinear A-A-B cross terms γ_1, γ_2 introduce in the residual.

These four are not four independent gadgets; they nest. Each slice admits everything the slice before it admitted and then opens a little more room, so the admitted sets climb $A \subset B \subset C \subset D$, each one strictly larger than the last because each one stops pinning a moment the previous slice held at zero. The reason to build them in exactly this order is that every newly freed moment drags a harder cross term into the residual, and you would rather meet those one at a time than all at once on the first try. Slice D, with nothing pinned, is the whole of the Q_{1+AB} cone, and the three slices below it are not lesser objects: they are that same cone seen with progressively more of the parameters frozen.

16.7.3 The compositional Schur cascade (Slice D)

The mathematical core of Slice D is the chain

$$\begin{aligned}\mathrm{PSD9}(M) &\Leftarrow \text{sym6 PD-interior on the 21 entries of } M_{6\times 6}\\ &\Leftarrow \text{sym5 PD-interior on the } 5\times 5 \text{ Schur complement of row 1}\\ &\Leftarrow \text{sym4 PD-interior on the } 4\times 4 \text{ Schur complement of row 2}\end{aligned}$$

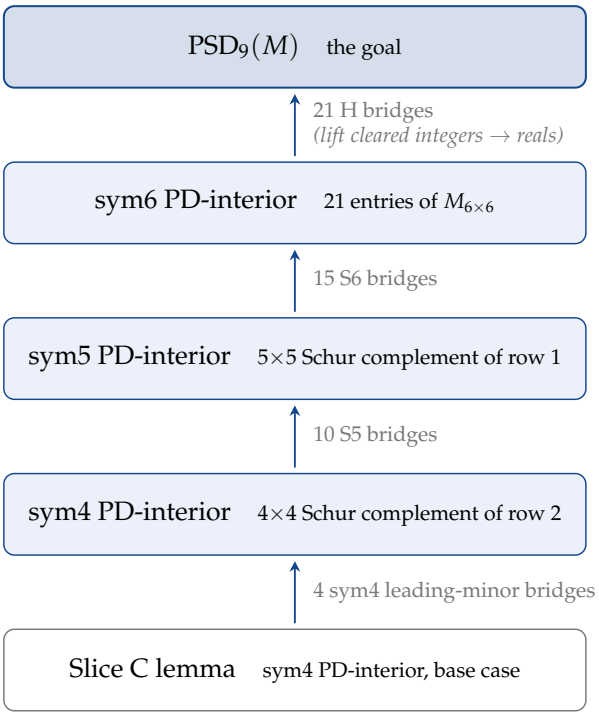


Figure 8: The Schur cascade lifts the base-case sym4 PD-interior fact from Slice C all the way up to $\mathrm{PSD}_9(M)$. Each arrow is one Schur-complement reduction, a polynomial identity in the correlators and moments. The counts beside each arrow are how many such identities the integer check grinds through at that step, one per matrix entry.

where $M_{6\times 6}$ is what the 9×9 collapses to once the $\mathrm{Q}_{1+\mathrm{AB}}$ subsumption identities pin its dependent entries: the projection onto the basis $\{v_{B1}, v_{B2}, v_{11}, v_{12}, v_{21}, v_{22}\}$, after which the 9×9 lands in the PSD cone whenever the 6×6 residual does. From there it is Schur complements all the way down, and it is the same move you already watched at level 1, run on a bigger matrix. The Schur-complement criterion (Section 2), the one the level-1 bridge used on its 4×4 block, turns a positive-definiteness question about the 6×6 into one about a 5×5 Schur complement; run it again and the 5×5 becomes a 4×4; and the 4×4 closes by Sylvester's criterion, all four leading principal minors positive. That is the whole soundness argument: a base case and three reductions, each one a Schur step you can carry out by hand, stacked from the 4×4 back up to PSD9.

Every reduction is a single polynomial identity in the correlators E_{ij} and the moments γ_k. To keep the opcode's check decidable and integer-only, each identity is multiplied

through by the common denominator $\mathrm{KZ} = \left(N_{00}N_{01}N_{10}N_{11} \cdot D_{g_1}D_{g_2}D_{g_3}D_{g_4}D_{g_5}\right)^2$ and verified on integers, the same clear-the-denominators move as level 1. It is a lot of arithmetic: one identity per matrix entry, twenty-one across the 6×6, fifteen for the first Schur step, ten for the second, four leading minors at the base. The kernel grinds every one. But the grinding isn't the argument; the argument is the iterated Schur reduction above, and a reader who wants to break it breaks the reduction, not the fifty identities. A passing integer check is those reductions holding, and those reductions holding is M in the PSD cone. The check leans on nothing past the two background facts level 1 already leaned on: the decidability of the reals and functional extensionality.

16.7.4 What this delivers and what it does not

Delivers. For each of the four cert-opcodes, a non-trapping step (with no error-flip) implies the 9×9 NPA $\mathrm{Q}_{1+\mathrm{AB}}$ moment matrix is symmetric and positive-semidefinite at the witness-counter-derived rationals $(E_{ij} = (\mathtt{wc\_same}_{ij} - \mathtt{wc\_diff}_{ij})/(\mathtt{wc\_same}_{ij} + \mathtt{wc\_diff}_{ij}))$ and the bucket-derived γ values $(\gamma_k = (\mathit{same}_{g_k} - \mathit{diff}_{g_k})/(\mathit{same}_{g_k} + \mathit{diff}_{g_k}))$. This is the operational form of "the substrate certifies $\mathrm{Q}_{1+\mathrm{AB}}$-realisability of CHSH correlators". Combined with the cited Ishizaka 2025 equivalence $\mathrm{Q}_{1+\mathrm{AB}} = \mathrm{Q}$ in CHSH, each successful step admits a quantum realisation.

Does not deliver. The kernel does not formalise the NPA hierarchy in Coq, does not formalise Q (the physical Tsirelson set), does not construct a Hilbert space from the moment matrix, and does not prove $\mathrm{Q}_{1+\mathrm{AB}} = \mathrm{Q}$. Those are external math facts the result cites, exactly as every NPA-hierarchy paper cites NPA's own $1 + \infty$ convergence result externally. The substrate-level content is the cert-opcode discipline plus the four-slice Schur-cascade soundness, which the kernel proves under no project-local axioms.

Why A2 is load-bearing. Each of the four cert-opcodes is a cert-setter; A2 forces its instruction cost to be at least 1. Without A2 the substrate could accept non-PSD correlators at zero cost, because the cert-flip would not need to be paid for. With A2 every successful $\mathrm{Q}_{1+\mathrm{AB}}$ certification commits $S(\mathit{mu\_delta}) \geq 1$ units of μ to the ledger. This is the same A2 application that powers the cost-ledger separation, the n-way bound, and the LASSERT polynomial exceedance.

Falsification. Exhibit witness counters and γ bucket pairs for which one of the four cert-opcodes does not trap but the resulting 9×9 NPA moment matrix is *not* PSD, OR show that the Schur-cascade soundness theorem at any slice is incorrect (an SOS identity wrong, a minors-witness step that does not imply PSD9, a bool decider unsound), OR show that the soundness chain depends on a project-local axiom beyond the two standard-library axioms cited above. The Coq `Print Assumptions` report is the audit surface: every theorem either lists no new axiom or it does not pass closeout.

17 Irreversibility accounting and the μ-hierarchy

17.1 The Landauer association is motivation only

Same shape as Landauer's 1961 argument: erasing one bit must release at least $kT \ln 2$ of heat into the environment (k Boltzmann, T absolute temperature, $\ln 2$ the natural log of 2). The Coq proofs below do not depend on Landauer being right about the physics. They formalise irreversibility at the instruction level only. A cost-positive instruction is tagged irreversible. The count of such firings is bounded by total μ. Shape borrowed, content paid for separately.

Theorem 17.1 (μ-Landauer step bound)**.** *Let* $\texttt{irreversible\_bits}(i)$ *be the indicator function that returns* 1 *if* $\texttt{instruction\_cost}(i) > 0$ *and* 0 *otherwise. Let* $\texttt{total\_irreversible\_bits}(\mathit{trace}) := \sum_{i \in \mathit{trace}} \texttt{irreversible\_bits}(i)$*. Then for every trace,*

$$\texttt{total\_irreversible\_bits}(\mathit{trace}) \leq \texttt{trace\_total\_cost}(\mathit{trace}).$$

The Coq witness is `total_irreversible_bits_le_cost`*.*

Proof. Per-instruction inequality: for every i, $\texttt{irreversible\_bits}(i) \leq \texttt{instruction\_cost}(i)$. Two cases.

Case $\texttt{instruction\_cost}(i) = 0$. Then $\texttt{irreversible\_bits}(i) = 0$ by definition, and $0 \leq 0$ holds.

Case $\texttt{instruction\_cost}(i) > 0$. Then $\texttt{irreversible\_bits}(i) = 1$, and the cost schedule (Section 7) gives $\texttt{instruction\_cost}(i) \geq S(\delta) \geq 1$ for cert-setters or $\texttt{instruction\_cost}(i) \geq \delta \geq 1$ otherwise. So $1 \leq \texttt{instruction\_cost}(i)$.

Summing the per-instruction inequality over the trace:

$$\sum_{i \in \mathit{trace}} \texttt{irreversible\_bits}(i) \leq \sum_{i \in \mathit{trace}} \texttt{instruction\_cost}(i) = \texttt{trace\_total\_cost}(\mathit{trace}).$$

The right equality is by definition of `trace_total_cost` (or via the Latent-μ theorem applied to the trace from $\mu_0 = 0$). □ □

> ***Plain reading.*** Each cost-positive instruction contributes at least 1 to total cost and at most 1 to the irreversible-bit count. Summing across the trace, the count is bounded by the cost. Shape-Landauer in the sense that irreversible operations are charged. No physics in the proof. A 0/1 indicator bounded by what it indicates. No microscopic erasure model. No physical entropy. No Boltzmann constant. The conversion of μ counts to joules is the calibration hypothesis (`mu_landauer_unruh_calibrated`, Section 24). No kernel theorem uses it.

The number, fenced. Nothing upstream depends on this paragraph; refuting it refutes a bridge, not the kernel. With the fence up, here is the number the bridge predicts. The kernel side is exact already: `irreversible_bits` tags an instruction 1 when it charges positive cost and 0 otherwise, `total_irreversible_bits_le_cost` bounds the tally by total μ, and the clean-room demonstration (`nofi_demo`) checks the matching conservation identity on every instance it can enumerate: banked insight equals erased bits, digit for digit, on all of them. The calibration hypothesis (`mu_landauer_unruh_calibrated`) then prices the physical side: certifying a result that banks k bits dissipates at least $k \cdot k_B T \ln 2$ of heat. At room temperature that is about 2.9×10^{-21} joules per banked bit, absurdly small, and, since Bérut and colleagues verified the single-bit Landauer limit in a colloidal-particle trap in 2012 (*Nature* 483, 187–189), absurdly measurable. What that experiment did not measure, and what a Thiele-specific experiment would, is the floor attached to the *certification* event itself: erase a bit without banking a claim and you pay Landauer like everyone else; bank the claim and the prediction is that the commitment carries the same irreducible price, paid at the moment the flag flips and not a femtojoule sooner. That is the open problem's empirical face. If the measurement comes in under the floor, the calibration hypothesis dies, this paragraph dies with it, and every kernel theorem stands exactly where it stood.

17.2 The μ-hierarchy

The proof:

Borrow the silhouette of the classical time-hierarchy theorem, and then say honestly how much smaller the thing inside it is. The shape is the part everyone knows: more of the resource buys you something you provably could not have had with less. The resource here is μ, and the something is the cost level a certified trace lands on. Every level $k \geq 1$ is reachable, meaning you can build a trace that pays exactly that much and ends certified; and no level k comes for less than k units of μ, because the ledger conserves and conservation does not run backward. That pairing, reachability on one side and a conservation floor on the other, is the whole content, and it is a statement about how cost stacks up at the ledger, not a statement that one problem class outruns another. Whether more μ ever buys you a genuinely harder problem is a different and much heavier question. It is open, I have it on the list in Section 24, and I am not going to quietly borrow its weight here while it is still unpaid.

Theorem 17.2 (μ-Hierarchy (`mu_hierarchy_theorem`))**.** *For every $k \geq 1$:*

1. *There exists a trace reaching `vm_certified = true` with μ-cost exactly k.*
2. *Any trace with `vm_mu` $\geq k$ has `trace_mu_cost` $\geq k$.*

Proof. Part (1): cost-k certified trace exists. Construct the trace

$$prog_k := \underbrace{[\texttt{instr\_load\_imm}\,0\,0\,0,\ \ldots,\ \texttt{instr\_load\_imm}\,0\,0\,0]}_{k-1 \text{ copies}},$$
$$[\texttt{instr\_certify}\,0].$$

The $k-1$ `LOAD_IMM` instructions each have $\delta = 0$ and so cost 0 each. The final `CERTIFY 0` has cost $S(0) = 1$ and sets `vm_certified` to `true`. So $\texttt{trace\_total\_cost}(prog_k) = 0 \cdot (k-1) + 1 = 1$, not k. To get cost exactly k, use $prog_k := [\texttt{instr\_load\_imm}\,0\,0\,(k-1),\ \texttt{instr\_certify}\,0]$: the `LOAD_IMM` with $\delta = k-1$ costs $k-1$ (ordinary compute pays δ directly), the `CERTIFY 0` costs 1, total k. After running, $\texttt{vm\_certified} = \texttt{true}$.

Part (2): cost lower bound. By Latent μ applied to any trace from $\mu_{\text{initial}} = 0$: $\mu_{\text{final}} = \texttt{trace\_total\_cost}(trace)$. So $\texttt{vm\_mu} \geq k$ implies $\texttt{trace\_total\_cost}(trace) \geq k$ directly.

Each $k \geq 1$ is therefore realizable, and each k-realizing trace pays at least k. □ □

> ***Plain reading.*** Two parts. Part (1) is by construction: pick a single `LOAD_IMM` with $\delta = k-1$ (costs $k-1$) and follow with `CERTIFY 0` (costs 1). Total cost k, ends certified. Part (2) is just Latent μ: total trace cost equals total μ accumulated from 0, so $\mu \geq k$ means cost $\geq k$. No shortcuts.

Corollary 17.3 (`mu_hierarchy_no_upper_bound`)**.** *No fixed μ-budget suffices for all certification levels.*

Proof. Suppose for contradiction that some fixed budget $B \geq 0$ suffices for all certification levels, meaning every certified trace satisfies $\texttt{vm\_mu} \leq B$. Apply the μ-Hierarchy theorem at $k = B + 1$: there exists a certified trace with $\texttt{vm\_mu} = B + 1 > B$, contradicting the budget bound. □ □

> ***Plain reading.*** For any budget you propose, the hierarchy theorem produces a certified trace that exceeds it by 1. So no finite budget caps all certifications.

What this comes down to is that the ledger does not give discounts. Pick any integer level you like and there is a certified trace that arrives at exactly that level; try to reach it for one unit less and you cannot, not because anybody forbade it but because the cost was already spent on the way up and conservation will not let you unspend it. That is the entire argument, and where it is mechanised the check agrees. Notice what I did not say. I did not say more μ makes the substrate "strictly more powerful". That phrase belongs to the problem-class reading, the question of whether a bigger budget cracks problems a smaller one cannot, and this theorem does not touch it. I would rather leave that fence standing in plain sight than collect on a claim I have not paid

for.

Here is the classical statement the shape comes from, so you can see exactly where the resemblance starts and where it stops. The time-hierarchy theorem says that for any time bound $T(n)$ there are problems you can solve in time $T(n) \log T(n)$ that you simply cannot solve in time $T(n)$: hand the machine a little more clock and it can decide things it could not decide before. That is a statement about which problems are in reach. The μ-hierarchy keeps the silhouette and trades the cargo: more μ buys strictly more cost levels, which is a statement about how high the ledger can climb, not about which problems come into reach. So treat the resemblance as a way to see the result, not as a smuggled inheritance of the classical theorem's strength. The proof a few lines up is the part actually holding weight; the analogy is just the light I am reading it by.

To falsify: find $k \geq 1$ and a trace reaching `vm_certified = true` and `vm_mu` $\geq k$ with total trace cost $< k$. The theorem says this cannot happen. If it does, the cost accounting is wrong.

18 Discrete Geometry over Partition Graphs

The partition graph is also a discrete geometric object. Here is one result that falls out, no physics required.

18.1 Discrete triangulation of the partition graph

A partition graph is not only a bookkeeping structure. You can also read it as a shape, a discrete geometric object, and put geometric questions to it. The honesty of the framing matters here, so I will be plain about what is and is not being claimed. The result below reads the partition graph as a triangulated combinatorial object, a graph cut into triangular pieces, and sets it in an equilateral angle model: every corner of every triangle contributes $\pi/3$ radians, which is 60 degrees, because the three angles of an equilateral triangle are equal and a full turn is 2π radians, or 360 degrees. Those angles are a modeling choice. They are not pulled out of μ costs, and I am not going to dress them up as if they were.

A discrete triangulation defines a curvature-like angle defect. As I understand it (I had to research this for the project, and the usual caveat applies that I do not trust my own reading of geometry sources): on a flat surface, if you draw a triangle, the interior angles sum to 180°. On a curved surface, the angles can sum to more or less than 180°. Curvature is the amount by which triangles deviate from flat. In the equilateral discrete model, curvature at a vertex is the defect left after subtracting the angles of the incident triangles (the triangles meeting at that vertex) from 2π (a full turn).

I went looking for the right invariant and found the Euler characteristic χ, a number that captures the overall shape of a triangulated surface, independent of how you draw the triangles. For any triangulation with V vertices, E edges, and F triangular

faces: $\chi = V - E + F$. As I read it, no matter how you triangulate the same surface, you get the same number. For a sphere, $\chi = 2$. For a torus (donut shape), $\chi = 0$. The continuum Gauss-Bonnet theorem (which I am citing, not certifying) says the total curvature of a closed surface equals $2\pi\chi$: the local geometric information (curvature) integrates to a global topological fact (shape). The discrete analogue is stated below. The continuum result is exposition.

18.2 The boundary-triangulated angle-defect identity

A heads-up before the theorem: this is not the closed-surface Gauss-Bonnet identity. The standard closed-surface result is total curvature equals $2\pi\chi$. The theorem here uses a different predicate (planar triangulations with a boundary) and a different constant (5π instead of 2π). If you walk in carrying the closed-surface invariants, 5π is going to look broken, and I get why; it is the right number for the class of graphs this predicate describes, and I want you to be able to see that for yourself. The full derivation is in the paragraphs after the theorem; the theorem statement itself carries the predicate's invariants right out front, so you can see the whole scope first and the constant stops looking like a typo.

Theorem 18.1 (Boundary-triangulated angle-defect identity (`discrete_gauss_bonnet`))**.** *Let g be a partition graph satisfying* `well_formed_triangulated` *(a* definition*, not a theorem; the structural identities it bundles are packaged into the named theorem* `triangulation_combinatorial_identity`*): a planar triangulation whose interior edges I and boundary edges B obey $E = I + B$ and $3F = 2I + B$, with a boundary-Euler-relation field (*`satisfies_boundary_euler_relation`*) forcing the additional constraint $B = 3\chi$, where $\chi = V - E + F$ is the Euler characteristic. In the equilateral angle model (every triangle corner contributes $\pi/3$), the total angle defect satisfies*

$$\text{angle\_defect}(g) = 5\pi \cdot \chi(g).$$

The constant is 5π, not 2π, because `well_formed_triangulated` *is a planar-region predicate with a triangular boundary. The one-parameter identity behind every case is* angle_defect $= 2\pi\chi + \pi B$*, which is the proof's intermediate $2\pi V - \pi F$ re-expressed through $\chi = V - E + F$ and $B = 2E - 3F$, and a proven kernel lemma in its own right (*`total_angle_defect` $= 2\pi V - \pi F$*). Read the family off the boundary: a closed surface ($B = 0$) gives the textbook $2\pi\chi$; this predicate's triangular boundary ($B = 3\chi$) gives $(2\pi + 3\pi)\chi = 5\pi\chi$; a hexagonal-boundary disk ($\chi = 1$, $B = 6$) would give 8π, and the predicate refuses it precisely because $B \neq 3\chi$. So 5π is not a free-floating oddity. It is the $B = 3\chi$ point of a line, and the boundary you admit picks the point. The canonical instance the predicate accepts is a triangulated planar disk ($\chi = 1$, $B = 3$).*

Proof. The total angle defect is

$$\text{angle\_defect}(g) \;=\; \sum_v \big(2\pi - \deg(v) \cdot \tfrac{\pi}{3}\big) \;=\; 2\pi V - \tfrac{\pi}{3}\sum_v \deg(v),$$

where $\deg(v)$ counts triangles meeting at vertex v (triangle-incidence, not edge-incidence). The triangle-incidence sum is

$$\sum_v \deg(v) \;=\; 3F,$$

because each triangle has three corners, each contributing once to the corner-count of its vertex. So

$$\text{angle\_defect}(g) \;=\; 2\pi V - \pi F.$$

From `well_formed_triangulated`: the structural invariants $E = I + B$ and $3F = 2I + B$ combine to give $3V = 5E - 6F$ (the triangulation-combinatorial identity). Solving for V: $V = (5E - 6F)/3$. Substituting:

$$\text{angle\_defect}(g) \;=\; 2\pi \cdot \tfrac{5E-6F}{3} - \pi F \;=\; \tfrac{10\pi E - 12\pi F - 3\pi F}{3} \;=\; \tfrac{10\pi E - 15\pi F}{3} \;=\; \tfrac{5\pi(2E-3F)}{3}.$$

The Euler characteristic in this triangulation satisfies $\chi(g) = (2E - 3F)/3$ (from the same identity, by elimination). Therefore $\text{angle\_defect}(g) = 5\pi \cdot \chi(g)$. □ □

> ***Plain reading.*** Total angle defect is $2\pi V - \pi F$ once you use the triangle-corner count $\sum \deg = 3F$. The `well_formed_triangulated` invariants give V in terms of E and F. Substitute and simplify. The 5π falls out as the coefficient of χ. Not chosen, not picked, not aimed at; just what the algebra returns when you solve the system the predicate hands you.

In my own words first, because I expect this to trip people up. I did not author any of the math in this section. I do not write code or proofs by hand. I directed LLMs and refused everything that did not compile. When I went looking for what discrete Gauss-Bonnet should look like, the version I kept finding was the closed-surface one, with constant 2π. The graphs the predicate I ended up with accepts are not closed surfaces. They have a boundary, like a flat sheet of paper cut into triangles. The constant that fell out of the proof for that case was 5π, not 2π. I did not pick 5π. I do not pick numbers. The algebra picked it. The checker didn't object, which is corroboration, not the reason. So the warning to a reader who has the closed-surface version cached: the predicate named `well_formed_triangulated` is not the closed-surface case. If you assume it is, you will assume identities the predicate does not assume, and you will conclude the number is wrong. The number is right for the case the argument covers. If it's wrong, it's wrong at a step in that derivation, and you can put your finger on the step; where it's mechanised, the check would choke there too.

Now the same thing in the formal voice. `well_formed_triangulated` is a planar-triangulation predicate defined, with its combinatorial invariants packaged into the theorem `triangulation_combinatorial_identity`. Its structural invariants are $E = I + B$ and $3F = 2I + B$, where I count the edges in two flavours: I for interior edges (edges shared by two triangles) and B for boundary edges (edges with only one

triangle on either side). These invariants give $\chi(g) = (2E - 3F)/3$ and $3V = 5E - 6F$. In a closed surface like a sphere the corresponding identity is $2E = 3F$, but this predicate is for planar regions with a boundary, not closed surfaces.

In this discrete model the symbol $\deg(v)$ stands for the number of triangles meeting at vertex v (triangle-incidence), not the number of edges meeting at v (edge-incidence, which is the usual graph-theory definition). The triangle-incidence sum is $\sum_v \deg(v) = 3F$ because each triangle contributes once per corner and a triangle has three corners. Substituting that into the total angle defect $\sum_v (2\pi - \deg(v) \cdot \pi/3) = 2\pi V - \pi F$ yields $5\pi\chi$ exactly. (A reader who plugs in the edge-incidence identity $\sum_v \deg(v) = 2E$ will get a different answer, because that is a different quantity than the one this proof tracks.) The factor differs from 2π because the predicate differs from a closed surface, not because the algebra is wrong. A tetrahedron (a four-faced closed solid: V=4, E=6, F=4) violates $3V = 5E - 6F$ and is correctly excluded from the predicate. A planar disk with V=7, E=15, F=9 gives $\chi = 1$ and $B = 3$ (a single triangular outer boundary), and the arithmetic of every invariant the predicate requires checks out for those values. The predicate's invariants accept this disk under the same checks the kernel applies to its packaged witnesses.

This is a fully proven theorem about the discrete triangulation model used here. It is not the continuum Gauss-Bonnet theorem and it is not a derivation of angles from μ cost. It is a combinatorics result: any well-formed triangulated partition graph (in the planar-triangulation sense above) satisfies the angle-defect formula. That is the full extent of the geometry section.

19 The substrate and its halting-shaped wall

The Thiele Machine is not the 51 opcodes. The 51 opcodes are scaffolding. They make the substrate concrete enough to verify in Coq, run as OCaml, and synthesize to silicon, but they are not the substrate itself. The substrate is the step relation together with the cost-ledger monotonicity rule: every atomic transition either preserves μ or charges it. PNEW, MORPH, CHSH_LASSERT, the whole instruction set. Those are the experimental apparatus that gives the substrate a concrete realization to probe. Strip the opcodes and the substrate is still there. Add different opcodes that respect the substrate's monotonicity discipline and the substrate is still the same substrate.

The Turing machine has the same shape. Tape, head, and transition table are scaffolding for one realization. Register machines, lambda calculus, counter machines, RAMs, and partial recursive functions all instantiate the same step-relation primitive with different scaffolding, and the equivalence between them is the content of the Church-Turing thesis. They are all the Turing machine in the sense that matters. They share the same notion of a step.

The result that follows is a fact about the substrate, not about the 51 opcodes. The opcodes are one realization. Other substrates that supply the same typeclass fields

inherit the same theorem the moment they are presented.

Relationship to Rice's theorem. The result that follows is Kleene's 1938 diagonal pattern instantiated at the substrate level. Rice's 1953 theorem applies the same diagonal to non-trivial extensional properties of recursive functions computed by Turing machines; the substrate-level version below applies the very same diagonal to non-trivial extensional predicates of substrate programs in any model that exposes an internal recursion theorem and an internal representability predicate. It is the same trick, two floors apart, which is the whole point. The formal content is the diagonal construction; what the substrate-level framing adds is the parameterization (any substrate satisfying the typeclass interface, not just the standard Turing model) and the choice of extensional predicate (`AdmitsShortcut`, branching on `csr_cert_addr` reachability). The result is therefore a substrate-level instance of the pattern Rice used; the result does not fall out as a corollary of Rice's theorem in its original setting, because Rice's premises are about partial recursive functions in the Turing-machine model, and nothing in them knows about arbitrary substrates carrying their own recursion theorems and representability predicates. Both theorems are sibling instances of Kleene's diagonal; neither is a corollary of the other.

19.1 The Substrate typeclass

A Coq typeclass is a checklist of operations and properties a type has to supply to count as the thing. This one captures exactly what an A2-respecting computational substrate is, divorced from any particular instruction set, and it exposes:

- A type `state` of substrate states.
- A type `Program` of substrate programs.
- A bounded-execution function `run : Program → state → option state`, returning `Some r` on convergence and `None` on divergence.
- A cost function `mu : state → nat`.
- An atomic step relation `step : Program → state → state → Prop`.
- A cost-ledger monotonicity field `mu_monotone`: `forall p s s', step p s s' → mu s ≤ mu s'`. This is a strictly weaker general-purpose monotonicity property than the kernel's A2 (which specifically asserts that a step flipping certification false-to-true costs ≥ 1, and which is carried by the `CertificationSystem` record, not by this typeclass). The two coincide on the 51-opcode VM where all instruction costs are non-negative and the cert transition costs ≥ 1. See Section 2.8 for the A2 statement and the related axioms A3 through A6.
- An internal Gödel coding: a way to pack any program into a state and unpack it again, so the substrate can hold its own programs as ordinary data. Formally, `encode : Program → state` and `decode_safe : state`

→ `Program` with the round-trip property `decode_safe (encode p) = p`, which just says that encoding and then decoding hands the program back unchanged.

- An internal-representability predicate `Representable : (Program → Program) → Prop`, scoping which Coq function transformers the substrate can express as programs of its own kind. Concrete substrates instantiate this with a precise meaning; the minimal nat-coded substrate defines `Representable f` as "`f` is the diagonal flip transformer for the substrate's own decide function," which is exactly the shape the diagonalization needs.
- A recursion-theorem field, internal form: for every internally representable transformer `f : Program → Program`, there is a program `p` extensionally equivalent to `f p`, where extensionally equivalent means the two give the same result on every input, however differently they are written.

The recursion-theorem field is the load-bearing piece. It is the substrate's Kleene 1938 recursion theorem, restricted to transformers the substrate can actually express. The restriction is not a hedge; it is the substrate's own scope. Turing 1936 did not prove "no decider exists in any conceivable mathematical universe"; he proved "no Turing machine decides halting," because Turing machines were the substrate Turing was studying. The restriction to internally representable transformers is the same scope move at the substrate level: the substrate's limitative theorem is about what the substrate itself can express, not about every Coq function that exists in the meta-theory.

The recursion theorem is not derivable from `mu_is_initial_monotone` (the cost-functional uniqueness result). Initiality of the cost functional is the statement that any monotone cost measure satisfying the axioms equals μ on reachable states; that is a uniqueness theorem, collapsing a type to a single inhabitant. The recursion theorem is a fixed-point statement, asserting that a non-trivial type admits self-reference. They are different categorical statements. The recursion theorem is supplied by the typeclass field above and discharged unconditionally for the minimal nat-coded substrate by exhibiting a self-aware program that is its own diagonal flip's fixed point.

19.2 The shortcut-admittance predicate

The substrate becomes interesting for the limitative result when it is equipped with a non-trivial *shortcut-admittance predicate*: a Prop-valued predicate `AdmitsShortcut` on programs, witnessed by at least one program that admits and at least one that does not, and depending only on extensional behavior (programs with the same `run` behavior on every state admit shortcuts iff each other).

The typeclass extension `WithShortcutPredicate` packages this: an `AdmitsShortcut` field, two canonical witnesses `yes_program` and `no_program` with their respective inhabitedness or non-inhabitedness lemmas, and the extensionality field. These are the conditions the diagonalization needs. A substrate that supplies

them earns the limitative theorem.

Two concrete substrate instances are constructed in the kernel.

The minimal nat-coded substrate `nat_substrate` discharges every typeclass field unconditionally, no `Hypothesis` and no `Axiom`. Its programs are natural numbers; three program codes have meaning (`0` = yes, `1` = no, `2` = the self-aware flip fixed point); `run` is a Coq `Fixpoint`; `Representable f` is the predicate "`f` is the diagonal flip transformer for the substrate's own decide function"; `recursion_theorem` is discharged by exhibiting code `2` as the fixed point of any such transformer. `PrintAssumptionsnat_recursion_theorem` is `Closed under the global context`. The substrate-level limitative theorem fires unconditionally for this substrate.

The 51-opcode VM substrate `vm_substrate` is parameterized over the VM-specific Gödel encoding (`vm_encode`, `vm_decode_safe`, the round-trip), the VM's internal-representability predicate (`vm_representable`), and the VM's internal recursion theorem restricted to representable transformers. The Coq Section discipline is identical to the one I use for physical-constant positivity: the parameters are not global axioms; closing the surrounding Section discharges them as explicit `forall` premises on every consumer. The encoding piece is a closed lemma giving `program_to_nat`, `nat_to_program`, and the round-trip with no Hypothesis. The VM's internal-representability predicate and internal recursion theorem are the VM-instance-specific Section parameters; the substrate-level result does not depend on them, because `nat_substrate` discharges the limitative theorem unconditionally at the substrate level.

The VM-side `AdmitsShortcut` is the extensional predicate "this VM program's bounded run from `init_state` coincides with the bounded run of `simple_morph_trace`." `yes_program` is `simple_morph_trace` itself; `no_program` is the empty trace (which produces `init_state`, demonstrably distinct from `simple_morph_final`, since the latter has fired the supra-cert channel and the former has not). The extensionality field is the standard "equal `run` on every state implies the predicates agree." The instantiation that assembles these into the `WithShortcutPredicate` interface lives in the kernel.

19.3 The substrate-level limitative theorem

Theorem 19.1 (Structural-axis undecidability, substrate-level)**.** *For every Substrate-instance equipped with a WithShortcutPredicate extension, there is no Coq-definable function* `decide : Program → bool` *whose corresponding diagonal flip transformer is internally representable in the substrate and that for all programs* `p` *satisfies:* `decide p = true` *iff* `AdmitsShortcut p`*.*

Proof. Assume for contradiction such a `decide` exists. Define the diagonal flip trans-

former

$$\texttt{flip}(p) \;=\; \begin{cases} \texttt{no\_program} & \text{if } \texttt{decide}\, p = \texttt{true} \\ \texttt{yes\_program} & \text{if } \texttt{decide}\, p = \texttt{false} \end{cases}$$

By hypothesis, `flip` is internally representable in the substrate. Apply the substrate's typeclass field `recursion_theorem` to `flip`: there exists `p` such that `prog_equiv` p (`flip` p), where `prog_equiv` is the extensional equivalence "`run` agrees on every initial state."

Case-split on `decide p`. *Case A* (`decide p = true`): by the bi-implication, `AdmitsShortcut p`; by definition of `flip`, `flip` p = `no_program`; by the fixed-point witness, `prog_equiv` p `no_program`; by extensionality of `AdmitsShortcut` (typeclass field), `AdmitsShortcut p` iff `AdmitsShortcut no_program`. The latter is false (typeclass field `no_program_no_shortcut`). Contradiction.

Case B (`decide p = false`): symmetrically, by the bi-implication ¬ `AdmitsShortcut p`; `flip` p = `yes_program`; `prog_equiv` p `yes_program`; extensionality yields `AdmitsShortcut p` iff `AdmitsShortcut yes_program`. The latter is true (typeclass field `yes_program_admits`). Contradiction.

Both cases close. No such `decide` exists. `PrintAssumptionsstructural_shortcut_undecidable` returns `Closed under the global context`. □

> ***Plain reading.*** Kleene 1938's diagonal, transplanted from time to structure. Same shape: assume a decider, build a flipper, find a fixed point, watch the program disagree with itself in both cases. The work is in the typeclass: once the substrate has its own recursion theorem, two canonical inhabitants on opposite sides of the predicate, and extensionality, the diagonal lands itself. The structural axis has its own halting wall, on its own axis, internal to its own machinery. Not transported. Not assumed. Built.

Theorem 19.2 (Structural-axis undecidability for the nat-coded substrate)**.** *For every Coq function* `d : nat → bool`*, the substrate* `nat_substrate d` *cannot internally decide its own structural-shortcut membership predicate. In particular,* `d` *itself is not a correct decider for the predicate of the substrate it parameterizes; the corollary* `nat_self_undecidable` *reads off this self-referential conclusion directly.*

Proof. The substrate `nat_substrate` discharges every typeclass field constructively: `Program := nat`; `yes_program := 0`; `no_program := 1`; `run` is a Coq `Fixpoint`; `prog_equiv` is pointwise equality of `run`; `Representable f` is the precise predicate "`f` is the diagonal flip transformer for the substrate's own decide function"; `recursion_theorem` is discharged by exhibiting code `2` as the fixed point of any such transformer (its `run` is defined to mirror whatever the transformer outputs); `AdmitsShortcut` is extensional by construction; `yes_program_admits` and `no_p`

rogram_no_shortcut hold by definitional unfolding of run at 0 and 1. Apply the substrate-level theorem above. The corollary nat_self_undecidable specializes the conclusion to d itself: d cannot be a correct decider for nat_admits d, cannot answer true exactly when the predicate holds. Print Assumptions on both nat_structural_shortcut_undecidable and nat_self_undecidable returns Closed under the global context. □

> ***Plain reading.*** The substrate-level theorem with every typeclass field hand-discharged. No Hypothesis, no Axiom, no Section parameter: a concrete substrate built out of three natural numbers and a fixpoint, where the limitative result fires unconditionally. The diagonal is not a thing that might exist for some substrates; here it exists, and it bites.

Theorem 19.3 (Structural-axis undecidability for the 51-opcode VM)**.** *For the 51-opcode VM, there is no Coq function* `decide : list vm_instruction → bool` *whose diagonal flip transformer is internally representable in the VM language and that for all programs* `p` *satisfies:* `decide p = true` *iff* vm_admits_shortcut_extensional p*.*

Proof. Instantiate the substrate-level theorem at vm_substrate. The Gödel encoding piece (vm_encode, vm_decode_safe, round-trip) is supplied as a closed lemma giving program_to_nat, nat_to_program, with no Hypothesis. The VM's internal-representability predicate vm_representable and the VM's internal recursion theorem restricted to representable transformers are the VM-instance-specific Section parameters; closing the surrounding Coq Section discharges them as explicit forall premises on every consumer, the same Section-parameter discipline I use for the physical constants $\hbar > 0$, $c > 0$, $k_B > 0$. The VM-side AdmitsShortcut is the extensional predicate "this VM program's bounded run from init_state coincides with the bounded run of simple_morph_trace"; yes_program is simple_morph_trace; no_program is the empty trace, which produces init_state and so differs from simple_morph_final (where the supra-cert channel has fired). The substrate-level diagonal applies; vm_structural_shortcut_undecidable is its specialization, carrying the VM-side parameters as premises. The substrate-level content is closed at nat_substrate (above); the VM corollary presents the same content at VM scope. □

> ***Plain reading.*** The 51-opcode VM is one instance of the substrate. The wall exists at the substrate level: nat_substrate already proved that unconditionally. The VM corollary is the same wall, named at VM scope, with the VM-specific Gödel-encoding machinery written out as premises in the same style I write out the physical constants. The kernel proves the limitative theorem once at the substrate; the VM inherits.

19.4 The proof, in prose

The proof is the standard Kleene-1938 diagonal, transported to the substrate.

Assume for contradiction that `decide : Program → bool` satisfies the bi-implication. Define the flipping transformer

$$\texttt{flip}(p) = \begin{cases} \texttt{no\_program} & \text{if } \texttt{decide p = true} \\ \texttt{yes\_program} & \text{if } \texttt{decide p = false} \end{cases}$$

This is a total Coq function from `Program` to `Program`. Apply the substrate's recursion-theorem field to `flip`: there exists a program `p` with `prog_equiv p (flip p)`.

Case-split on `decide p`.

Case A: `decide p = true`. By the bi-implication, `p` admits a shortcut. By definition of `flip`, `flip p = no_program`. By the recursion-theorem witness, `p` is extensionally equivalent to `no_program`. By extensionality of `AdmitsShortcut`, `p` admits iff `no_program` admits. But `no_program` does not admit. Contradiction.

Case B: `decide p = false`. By the contrapositive of the bi-implication, `p` does not admit. By definition of `flip`, `flip p = yes_program`. By the recursion-theorem witness, `p` is extensionally equivalent to `yes_program`. By extensionality, `p` admits iff `yes_program` admits. But `yes_program` admits. Contradiction.

Both cases close. No total decision procedure for `AdmitsShortcut` exists. □

The proof is short, because the work is in the typeclass, not in the diagonal. The body runs about 38 lines, of which roughly 16 are actual tactic steps; the rest is the case-split structure and the inline narration. Once the substrate has a recursion theorem, two canonical inhabitants, and an extensional predicate, the diagonal is mechanical. The substrate-level work was assembling the right typeclass; the limitative result is what falls out.

There is one thing here that is easy to read past: the diagonal does not invoke A2 as a step. The mechanics (the flip transformer, the recursion-theorem fixed point, the case-split contradiction) use the Substrate typeclass's `recursion_theorem` field and the `WithShortcutPredicate` extensionality field. A2 is upstream of those: it is what makes the substrate the kind of object the typeclass instance is built over, and it is what picks out the certification axis as substrate-distinguishing in the first place. Without A2 the substrate is a `CostBearingSystem` (the unconstrained record from earlier), free-forgery is admissible at total cost zero, and an analog of the diagonal still goes through as a Rice-style undecidability result over some other extensional predicate; what is lost is the specifically certification-axis content of the result. A2 is the axiom that makes "the structural axis has its own halting-shaped wall" the right phrasing, rather than "extensional predicates over substrates are undecidable": a generic

statement that holds of any substrate with a recursion theorem and a non-trivial extensional predicate and that says nothing about the certification axis specifically. The wall lands on the certification axis because A2 is what picks out that axis as the substrate-distinguishing structure; the recursion theorem closes the diagonal inside that structure. A2 is upstream of the result, not inside the proof, and the result is sharper for it.

19.5 The two-axis picture this completes

Time axis	Structural axis
steps	μ
HALT (Turing 1936)	AdmitsShortcut (above)
no decision procedure for halting	no decision procedure for shortcut admittance

The substrate's own halting-shaped wall, on its own axis, proved by its own diagonal, internal to the substrate's typeclass machinery. Not transported from the time axis by reduction. Not axiomatic. Built. The parallel holds at the level of *a substrate has its own limitative wall*; what makes the structural side specifically the *certification* axis, rather than a generic extensional predicate, is A2, which the cert-flavoured VM instance carries and the minimal `nat_substrate` does not.

19.6 What this closes

What the substrate-level theorem closes. Whether a uniform translation procedure exists from informal structural arguments into `SoundStructuralShortcut` witnesses reduces to whether the substrate can internally decide membership in its own structural-shortcut class. The theorem above answers that in the negative: no internally representable decision procedure for the membership predicate exists, so no uniform internally representable translation procedure exists either. The structural axis has its own undecidable predicate, on the same diagonal shape Turing used in 1936.

What the nat-substrate theorem closes. The minimal nat-coded substrate `nat_substrate` discharges every typeclass field constructively and the limitative theorem fires unconditionally. `PrintAssumptionsnat_structural_shortcut_undecidable` and `PrintAssumptionsnat_self_undecidable` both return `Closed under the global context`. The substrate's own halting-shaped wall is, at the substrate level, a closed Coq theorem. What fires unconditionally here is the generic floor: a Kleene-diagonal undecidability over an abstract extensional predicate (`nat_admits` is bare run-behaviour, with no certification content), the same generic family Rice's theorem inhabits. The certification-axis-specific wall, where the predicate is tied to the supra-cert channel, is the VM corollary below; it is conditional on the VM's internal recursion theorem, carried as a Section parameter, and that is a theorem routine for any sufficiently expressive substrate, left unproven here as bookkeeping, not as a live doubt. Per the A2 note earlier in this section, A2 is what turns the generic wall into the certification-axis one.

What the VM corollary closes. The 51-opcode VM corollary `vm_structural_shortcut_undecidable` is the substrate-level theorem instantiated at `vm_substrate`: no representable decider answers `true` exactly when a VM program admits the extensional shortcut. It carries the VM-side Section parameters as explicit `forall` premises in the Bekenstein-bound style. The substrate-level content does not depend on the VM-side parameters; it is closed at the `nat_substrate` level. The VM corollary is the substrate-level result presented at the VM scope, with the VM-side internal-representability work named in its premises.

How to falsify. Three routes.

1. A Coq function `decide : Program → bool` whose diagonal flip transformer is internally representable in some Substrate plus WithShortcutPredicate instance and that satisfies the bi-implication would force the substrate-level proof to fail at one of its steps, each of which is named in the case-split above.
2. For the nat-coded substrate specifically: exhibit a Coq function `d : nat → bool` that correctly decides `nat_admits d`. The corollary `nat_self_undecidable` forbids this; finding such a `d` contradicts the corollary.
3. Exhibit a substrate that has both an internal recursion-theorem witness for a representable transformer and a non-trivial extensional predicate, but for which the diagonal construction does not produce a contradiction. The proof body is about 38 lines, roughly 16 of them actual tactic steps; finding such a substrate means finding an error in those tactic steps.

19.7 The wall the classical configuration cannot see

There is one sentence that kills most new "a computer can't decide this" results before they get off the ground, and the first time I thought I had one I said it to myself before anyone else could: *you didn't find a new kind of unanswerable question. Your machine is a Turing machine with some extra memory bolted on, the thing you can't decide is just one more ordinary fact about what a program does, and Rice's theorem already says nobody can decide facts like that. Nothing new, nothing sideways to ordinary computation.* I went looking for where that sentence breaks the result, because if it does, I'd spent months on a footnote to Rice. It doesn't. The reason is plain once you see it: a classical machine never gets the whole state. It gets a stripped-down version, the part it can read and write, and my unanswerable question turns on a piece the stripping throws away. Rice's theorem is about an answer a machine can't compute from what it's handed. This is about an answer that was never in what it's handed.

A classical decider has, as its entire input, the Turing configuration `forget s : TMSnapshot`, the same four-field projection from earlier in this section. If the thing it is asked to decide is not a function of `forget s`, then no classical decider can be right about it, and not for want of horsepower: what it is deciding is not in what it is

given. That is the whole shape of this wall, and it is why it sits beside Rice's rather than under it. Two independent obstructions, on two independent axes.

Theorem 19.4 (Structural-shortcut admittance is not classical, `structural_shortcut_not_function_of_classical`)**.** *There is no function* `decode` : `TMSnapshot` → `bool` *such that, for every Thiele state* s*,*

$$\texttt{decode}(\texttt{forget}\ s) = \texttt{admits\_structural\_shortcut\_bool}\ s,$$

where `admits_structural_shortcut_bool` s *is the reading "s's certification channel has fired" (*`csr_cert_addr` $\neq 0$*).*

Proof. Two states reachable from `init_state` by executing a trace: `run_cert_set` runs the minimal `MORPH_ASSERT` closure and fires the channel (`csr_cert_addr` = 650); `run_cert_unset` runs the same two-instruction prefix, then a graph-preserving no-op whose μ-cost is chosen to match, leaving `csr_cert_addr` $= 0$. The matched cost makes the two runs land on the same (`pc`, μ, `regs`, `mem`), so `forget run_cert_set` = `forget run_cert_unset` by computation. A `decode` satisfying the equation would have to return both `true` and `false` on that one input. Contradiction. □ □

> ***Plain reading.*** The collision is built on states the machine actually reaches, not on two records I wrote down by hand to make the point. That is the one upgrade over `cert_addr_not_function_of_forget` earlier, and it is the one a curious reader would otherwise reach for. The certification channel through which the undecidable predicate is detected is a field the classical signature does not carry. So the undecidable predicate is, additionally, classically invisible: a second wall, on the same predicate, of a different kind than the diagonal.

This is the first rung of a short ladder, and I want to be exact about how far each rung reaches, because the honest version is sharper than the inflated one.

It survives any classical oracle. Hand the classical decider an oracle, a halting oracle, say. It does not help, and the reason is the cheap one: a classical oracle answers questions about the classical configuration, so its answer is itself a function of `forget` s. A decider plus such an oracle is still a function of `forget` s, and the keystone kills it. The Coq statement quantifies over an oracle of arbitrary answer type: `structural_axis_survives_any_classical_oracle`, with the halting case as a corollary. The triviality here is the content. Baker–Gill–Solovay is hard precisely because *computational* obstructions need not survive relativization; an information-theoretic one survives for free, because you cannot recover by any oracle what is not in the input.

The two axes are independent, but not in the way the word "degree" would suggest. It is tempting to reach for Turing-degree incomparability here, and I did reach for

it, and it is the wrong instrument. The structural-shortcut set is decidable from the full state (it is a total boolean function of `VMState`, `structural_membership_decidable`), so it is Turing degree 0, below halting, hence *comparable* to the classical axis, not incomparable. More generally any genuinely undecidable predicate of one Turing-complete machine is Turing-equivalent to its halting problem; two predicates of the same machine cannot be incomparable degrees. So the genuine independence is information-theoretic, not degree-theoretic, and that is exactly what `axes_mutually_independent` states: the structural reading is not a function of the classical configuration, and a classical quantity (`vm_pc`) is not a function of the structural channel, with reachable witnesses both ways. I kept the decidability theorem around on purpose, as a guard: it forecloses the over-reading rather than inviting it.

The encoding is no longer a premise. The VM corollary above carried the standard Gödel encoding as a Section parameter alongside the recursion theorem. That part is not a live question: the kernel has `program_to_nat` / `nat_to_program` with a proven round-trip, and `vm_structural_shortcut_undecidable_encoded` supplies a concrete encoding (the program's code stored in `vm_logic_acc`) and discharges it. With that discharged, the VM theorem's single surviving premise is the recursion theorem: a universal interpreter realized as a `list vm_instruction` with its s-m-n parametrization, in the unbounded-fuel model. Named, not hidden, and not an axiom; the unconditional version remains the `nat_substrate` theorem, which carries no premises at all.

> ***Plain reading.*** The short version: the structural axis has its own undecidable predicate (the diagonal, above), that predicate is not a function of the classical configuration (the keystone), that fact survives handing the classical side a halting oracle (it relativizes for free), and the two axes are genuinely independent in the information-theoretic sense, without overstating that into a Turing-degree gap, which the predicate's own decidability rules out. Every claim in this paragraph is a closed Coq theorem; the one thing left conditional, the VM's recursion theorem, is named and is unconditional one level down.

20 What Coq is, and why I used it

Coq is a proof assistant, which is a phrase I had to go look up. Underneath the phrase it is a programming language, but one whose type system is strict enough to do a thing ordinary languages cannot: you write a proof down as a program, and the compiler refuses it unless the program genuinely matches the statement you claimed. You cannot talk it into a maybe.

Here is the idea. In a normal programming language, a type says something about what kind of value a variable holds. In Python, `x: int` means x is an integer. In Coq, types are propositions. The type `n > 0` means "the proof that n is positive." A function from `n > 0` to `n + 1 > 0` is a proof that if n is positive, $n + 1$ is too.

The compiler checks whether the types line up. If your proof has a hole and you write "trust me," it rejects you. You can't bluff a proof term past it. But here is the part people skip, and it's the part that matters: the compiler checks that the proof matches the statement. It does not check that the statement says what you meant. Hand it a statement that means nothing and it'll prove it for you, three characters, no complaint. So the compiler accepting a proof is not the same as the thing being true. It means this proof matches this statement, and whether that's the right statement is on me, not the machine.

So I didn't use Coq to make anything true. I used it because I don't trust my own eye to catch a gap in an argument I want to believe. I spent months convinced by my own reasoning, pushed the proof through, and watched a key step come apart in my hands. The machine caught where I'd fooled myself. That is what it's for, and it's the same source the running machine and the chip get extracted from. Gap-catcher and extractor. Not the proof. The proof is the argument, and an argument has to close in a way a person can follow, or it hasn't closed at all.

20.1 The Inquisitor

Beyond Coq's type checker, the project has a custom proof auditor called the Inquisitor (`scripts/inquisitor.py`). It runs on every commit and emits 86 distinct rule codes. The main categories:

- **No holes.** No `Admitted`, `admit`, or `give_up` anywhere in the active proof files. No unproved `Axiom` or `Parameter` declarations in project code.
- **No vacuous theorems.** No theorem whose conclusion is literally `True`, `0 = 0`, or any other tautology that proves nothing. No "theorem" of the form $P \to P$. No existence claims backed by constant witnesses. Coq's kernel already prevents circularly dependent proofs (it checks definitional well-foundedness); the Inquisitor catches separately the case where a proof is technically valid but epistemically empty, proving `True` by `trivial`, for instance.
- **No physics stubs.** No physics quantity defined as a constant placeholder. No physics claim without a corresponding invariance condition. No theorem stated about a physics concept if that concept is not genuinely formalized.
- **No trivial true proofs.** If a theorem's entire proof is `trivial` or `tauto` and the conclusion reduces to `True`, the theorem is rejected.
- **Scope enforcement.** The proof tree is split into tiers. Tier 1 is the kernel files: the core proof of the machine. Tier 1 files may not import from exploratory or research-side files. Theorem connectivity: every file in the active proof tree must reach the cost or semantics foundation through its import chain. If a file is disconnected, it does not get to claim the kernel's pedigree.
- **Comment hygiene.** No TODO/FIXME/WIP markers in active proof comments.

Current Inquisitor status (250 Coq files scanned): zero HIGH findings, zero MEDIUM, and exactly one LOW. The LOW is a vacuity-score warning on one file, the F4 reference Verilog evaluator: a small implementation of the reference 4-element field F_4, deliberately defined in terms of small constant predicates and enumerated boolean functions, which is what triggers the "constant function" heuristic. The LOW is documented and audited. It does not correspond to a missing proof step.

21 Zero Admitted: what it actually means

"Zero Admitted" is literal. Nowhere in the proof tree did I write "trust me" and walk on. Every step the checker could see, it saw, end to end, with no "and the rest is obvious." That is a hygiene fact about the gap-catcher, not a verdict on the argument: it says I didn't let myself skip a step where the machine was watching. It does not say the statements are the right statements. That part stays on me, which is what the rest of this section is about.

This is what it does NOT mean:

- It does not mean the theorems prove physical reality. The theorems prove things about the Thiele substrate, as formalised in Coq. Whether that formalisation describes the world you live in is a separate question, answered by experiments, not by proof checkers.
- It does not mean the Coq axioms themselves are consistent. Coq sits on a formal foundation called the Calculus of Inductive Constructions. The key idea is the Curry-Howard correspondence: proofs and programs are the same thing. A proof that P is true is literally a program of type P. Checking the proof is type-checking the program. The "inductive" part means you can define your own recursive types (naturals, lists, trees) inside the system instead of having them as magical primitives handed down from above. That matters. The foundation is explicit. Nothing is smuggled.
  Now the honest limitation. The sources I read on Coq's foundation say it is believed to be logically consistent (meaning you cannot derive a contradiction from it), but this has not been proven within Coq itself. The way I understand it (and I do not fully trust my own reading), that is not a weakness specific to Coq. The label I dug up is Gödel's incompleteness theorem from 1931: any consistent formal system powerful enough to do arithmetic cannot prove its own consistency from within itself. A system that could prove its own consistency would already be inconsistent. And an inconsistent system proves everything, its own consistency included. So Coq's foundation is trusted, not proved-from-within. "Trusted" has substance here. The same foundation has verified the CompCert C compiler and the Four Color Theorem (Gonthier 2005, the first published machine-checked proof of a major open problem in mathematics), among other artifacts. Large-scale inconsistencies would have surfaced through

those uses if they were there. I am relaying what I read. I have not personally audited CompCert or Gonthier's proof. "Zero Admitted" means every theorem follows from those axioms. It does not mean those axioms are themselves in-system guaranteed. It means they are the same axioms a substantial body of verified software has relied on without breaking.

- It does not mean the hardware implements exactly the Coq semantics in every case. All 47 synth-realised opcodes have Qed bisimulation proofs: 37 unconditional, 10 under structural invariants `coupling_wf` and `morph_table_wf` that hold inductively from hardware reset. Zero structural gaps within the synth-realised subset, documented in the kernel. The four Q_{1+AB} cert-opcodes live in the Kami HW abstraction (with kernel-equivalence proven) but outside the synthesized-Verilog surface, by silicon budget, not by semantics gap.
- It does not mean the OCaml runner binary is proven inside Coq to be bitwise identical to the Coq semantics. The kernel proves Coq-side observable facts. The parity suite checks the extracted binary empirically. One is a theorem. The other is a test.

The kernel tier (Section 8) is the tightest. The arguments there close with zero Admitted and zero named hypotheses, so the gap-catcher had nothing to wave through and nothing extra to lean on: they run on the Coq type-theory axioms and the Thiele substrate definitions, and nothing else gets in. Those are the ones I'd defend out loud.

The bridge tier involves named hypotheses and documented trust boundaries. Three of them. First, the BSC (Bluespec compiler) trust boundary for hardware: Section 14 introduces Bluespec; the BSC compiler is the tool that translates Bluespec into Verilog; the proof assumes this tool's output faithfully implements its input. Second, the A_QM physical assumption for the quantum connection: a named hypothesis I use sparingly, marking the place where the Coq proof would need a physics input it does not derive. Third, the external OCaml parity boundary, described in Section 14: the OCaml binary is checked against the Coq semantics by tests, not by a Coq theorem. These are explicitly labelled and auditable. They are not Admitted proofs. They are documented limits of the formal claim.

22 How to break this

Every claim in this document comes with the exact shape of the thing that would kill it. I went to the trouble on purpose. A claim you can't break is a claim that wasn't saying anything, and I would rather hand you a loaded gun pointed at my own work than a sentence too vague to aim at. So the ones below are written as instructions, not as boasts: here is the trace to find, here is the object to build, here is the theorem to exhibit. Build one and the proof tree falls, and I mean that as the better outcome of the two, because a break tells me something I didn't know and a year of nobody finding one only tells me people got tired of looking. If you build any of these, that is

the message I want in my inbox.

Falsify No Free Insight.
Find a VM trace that:

- starts with `vm_certified = false` and `vm_mu = 0`
- ends with `vm_certified = true`
- ends with `vm_mu = 0`

If you find such a trace, the claim is false: `certification_requires_positive_mu` breaks at the step that prices the cert-flip. Add the counterexample as a theorem and the check refuses to compile on top of that.

Falsify μ-monotonicity.
Find a state s and instruction i such that $(\texttt{vm\_apply}\ s\ i).\mu < s.\mu$. The Coq proof of `vm_apply_mu` falls.

Falsify categorical separation.
Prove that for ALL states s_1, s_2 with identical classical shadow, ALL instructions produce the same error behavior. That kills `categorical_separation`.

Falsify Turing strictness.
Exhibit a *classical* program, a Thiele instruction trace restricted to the classical opcode fragment (`is_classical_program`, run on the same substrate), whose execution changes the partition graph (`pg_next_id`) or the morphism map. Or prove that the witness step `PNEW` leaves `pg_next_id` fixed. Either kills the strictness half of `D5_thiele_strictly_extends_classical`. This is the semantic claim, that the classical opcode fragment cannot reach the structural states one structural opcode reaches. It is not the computability claim. A Turing machine can of course simulate a Thiele run by encoding the graph as bytes on its tape; that simulation exists and is not a counterexample.

Falsify structural entitlement.
Find two Thiele VM states with the same classical shadow where no probe can distinguish their structural fields. `shadow_strictly_lossy` falls. Or produce a strict feasible-set subset with a distinguishing observation that does not yield stronger membership predicates. The feasible-subset theorem falls with it.

Falsify universal certification cost.
Build a `CertificationSystem` satisfying the one-step rule `cs_cert_costs`, then give a trace from uncertified to certified with total cost 0. `universal_nfi_any_substrate` falls. If you instead drop the one-step rule, you have found a free-certification system, not a counterexample.

Falsify substrate-level structural-axis undecidability.
Three routes (Section 19 expands).

- Exhibit a Coq function `decide : Program → bool` whose diagonal flip transformer is internally representable in some Substrate plus WithShortcutPredicate instance and that satisfies the bi-implication. The substrate-level proof of `structural_shortcut_undecidable` must fail at some step; identify which.
- For the nat-coded substrate specifically: exhibit a Coq function `d : nat → bool` that correctly decides `nat_admits d`. The corollary `nat_self_undecidable` says no such `d` exists; build one and you have contradicted it head-on, which is exactly the kind of break I want to hear about.
- Exhibit a substrate that has both an internal recursion-theorem witness for a representable transformer and a non-trivial extensional predicate, but for which the diagonal construction does not produce a contradiction. The proof body is about 38 lines, roughly 16 of them actual tactic steps; finding such a substrate means finding an error in those tactic steps.

Falsify the classical CHSH bound.
Exhibit a locally factorizable strategy that achieves $|S| > 2$. `chsh_stat_violation_not_local` falls. So does known mathematics.

Falsify the CHSH-NPA bridge.
The biconditional `column_contractive_iff_quantum_realizable` falls to a counterexample in either direction: a column-contractive correlator whose zero-marginal NPA moment matrix is not PSD (forward), or a PSD zero-marginal NPA matrix whose correlators are not column-contractive (reverse). For the Tsirelson corollary, exhibit a Thiele-honest (column-contractive) correlator with $|S| > 2\sqrt{2}$ and `state_column_contractive_implies_tsirelson` falls. The PR box and the Turing-classical point $(1, 0, 1, 0)$ are *not* counterexamples: both fail column contractivity by direct arithmetic, which is what the theorem requires, not what would break it.

The Inquisitor standard.
Run `pythonscripts/inquisitor.py` on the repository. If it finds any HIGH, MEDIUM, or LOW finding at all, the proof hygiene has degraded. Current status: 0 HIGH, 0 MEDIUM, 0 LOW. If that changes, it is a bug.

Falsify the axiom audit.
If running `Print Assumptions` (Section 2: the Coq command that lists everything a theorem depends on) on any named theorem in the kernel tier shows a project-local `Axiom` or `Parameter` in the dependency tree, the audit has degraded. The current probe across the kernel tier covers 3,937 named theorems: 2,923 close under the global

Coq context outright, and 1,014 lean only on Coq standard-library axiom families (functional extensionality and the classical-reals construction). Of those, 67 depend on the excluded middle, which is the classical-logic axiom $P \vee \neg P$ ("either P is true or P is false, with no third option"); those uses come from the Coq standard library (specifically, the real-number theory needed for the CHSH algebra), not from project code. Zero project-local axioms appear in any of the 3,937 dependency trees. Any project-local axiom or parameter appearing in any of them is a bug.

Falsify the hardware bisimulation.
Run the Verilog cosimulation suite (`tests/test_verilog_cosim.py`, including `test_all_47_opcodes_in_cosim`). If any of the 47 synth-realised opcodes shows the synthesised RTL diverging from the Kami step function on any input, the Section Variable `rtl_step_correct` is violated empirically and the 47-opcode bisimulation falls. A divergence inside `bsc_kami_compilation_trusted` (the BSC compiler / pretty-printer pipeline) points at the toolchain instead, a documented trust boundary, not a counterexample to the bisimulation theorems.

Falsify the convergence answer to parametricity.
The separation/minimality/uniqueness trio is schema-parametric: any field a step rule carries that classical state does not determine supports all three theorems, word for word, and I concede that in full. What singles out certification is convergence: the verifier factorisation result (`V_does_not_factor_through_classical`), the clean-room floor demonstration (`nofi_demo`), and the NPA moment-matrix certificate (`chsh_lassert_no_trap_implies_quantum_realizable`) land on the same field from vocabularies that owe each other nothing. Exhibit another field with three convergent bridges of the same standing, or break one of these, and certification's distinguished seat falls back to an aesthetic choice. Whether certification is the event a model is *forced* to price remains the named open problem either way.

Falsify the proof-of-stake reduction.
Exhibit a protocol whose known nothing-at-stake status contradicts the formal classification of Section 15: a zero-stake-at-finalize gadget that admits an A2 proof, or a slashing gadget (finalize risks ≥ 1) with a finalizing trace of total stake-at-risk 0. Coq accepts the construction if it exists; `nothing_at_stake_is_free_forgery` or `slashing_finality_floor` falls.

Falsify the gas-metering reduction.
Construct a schedule satisfying both the commitment floor and no-overcharge whose charging predicate differs from the commitment predicate on some reachable step. `gas_schedule_exactness` falls, and the kernel's `exact_commitment_pricing_characterization` with it.

Falsify the attestation reduction.
Build a sound and complete attestation verifier for the μ-dependent claim that reads

only the bare classical transcript, a function of the report's log field alone. `attestation_cannot_factor_through_bare_transcript` falls, and behind it the verifier-factorisation theorem.

Falsify the transparency-log reduction.
Build a sound and complete log-free verifier with the same soundness target. `log_free_verifier_impossible` falls, and the bare-setting impossibility with it.

Falsify the proof-carrying reduction.
Construct a level-k certified proof script with total μ below k, or a sound and complete zero-round bare verifier. `level_k_verification_floor` or `bare_pcc_impossible` falls, and the μ-hierarchy or the bare-setting impossibility behind them.

23 The full claim ledger

These are the kernel-tier claims (no named hypotheses; each one checked in Coq):

1. *no_free_certification*: csr_cert_addr going $0 \to$ nonzero in one step requires cost ≥ 1.
2. *no_free_certification_mu*: same, with μ increment ≥ 1.
3. *no_free_certification_trace_mu*: trace-level version.
4. *no_free_certification_certified*: vm_certified going false$\to$true requires cost ≥ 1.
5. *certification_requires_positive_mu*: both channels covered.
6. *universal_nfi_any_substrate*: any abstract certification system satisfying the one-step cost rule has trace-level non-free certification.
7. *thiele_universal_nfi_cert_addr*: universal theorem instantiated for Thiele's `csr_cert_addr` channel.
8. *thiele_universal_nfi_certified*: universal theorem instantiated for Thiele's `vm_certified` channel.
9. *thiele_represents_simulating_cert_system*: any certification system with a simulation morphism into Thiele transfers certification into a Thiele certified execution.
10. *forget_kernel_is_eq_on_classical*: the classical snapshot quotient forgets exactly the non-classical fields.
11. *blindness_non_injective*: distinct Thiele states can have the same classical snapshot.
12. *certification_is_lost*: certification can be in the kernel of the classical forgetful map.
13. *shadow_strictly_lossy*: the classical shadow forgets morphism structure.
14. *mu_ledger_necessity*: no strict-classical oracle on $(\texttt{mem}, \texttt{regs}, \texttt{pc})$ recovers the pair $(\mu, \texttt{certified})$.
15. *vm_mu_not_classically_determined / vm_certified_not_classically_determined*: neither ledger balance nor certification status is a function of strict classical state.
16. *mu_ledger_mutual_independence*: μ and `vm_certified` are each independent of strict classical state and of each other under cost-only/cert-only projections.
17. *mu_ledger_minimality / P_full_is_minimal_complete_extension*: the full

(`mem`, `regs`, `pc`, μ, `certified`) projection is complete, and dropping either μ or certification destroys the corresponding completeness.

18. *thiele_nfi_pc_indexed*: PC-indexed execution version.
19. *mu_is_initial_monotone*: μ is the unique canonical cost measure.
20. *vm_apply_mu*: exact ledger identity $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + \texttt{instruction\_cost}(i)$. The cost is not just δ, it is δ for ordinary instructions, $S(\delta)$ for cert-setters, $|\text{payload}|_{\text{bits}} + S(\delta)$ for EMIT, etc.
21. *kernel_certified_implies_positive_mu*: from zero, reaching certified implies $\mu > 0$.
22. *feasible_strict_subset_implies_strict_predicates*: feasible-set narrowing plus a distinguishing observation yields a strictly stronger predicate.
23. *strengthening_requires_structure_addition*: certified predicate strengthening requires a structure-addition event.
24. *b4_information_reduction_derives_strict_predicates*: feasible-set reduction can generate the strict-predicate premise used by NoFI.
25. *info_priced_cert_executions_bound*: executed cert-setting events are bounded by $\Delta\mu$.
26. *observation_partition_reduction_implies_posterior_representative_reduction*: observation classes package into the posterior-representative witness for structural entitlement.
27. *info_priced_arbitrary_feasible_reduction_bound*: with a decision-tree/representative witness, feasible-set compression is bounded by $\Delta\mu$.
28. *info_priced_weighted_feasible_reduction_bound*: the $\Delta\mu$ lower bound extends to nat-weighted feasible sets.
29. *structural_entitlement_representation*: strict narrowing plus distinguishability, certified posterior admissibility, and an explicit posterior-representative reduction imply a strictly stronger predicate, a structure-addition event, and a $\Delta\mu$ lower bound.
30. *every_sound_structural_shortcut_lands_here*: every packaged `SoundStructuralShortcut` instantiates the structural-entitlement representation theorem.
31. *structural_shortcut_undecidable*: for every Substrate plus WithShortcutPredicate instance, no Coq function whose diagonal flip transformer is internally representable in the substrate decides the structural-shortcut admittance predicate. The substrate's own halting-shaped wall, by Kleene 1938 diagonal restricted to the substrate's internal expressive power.
32. *nat_recursion_theorem / nat_structural_shortcut_undecidable / nat_self_undecidable*: the minimal nat-coded substrate discharges every `Substrate` typeclass field unconditionally (no Hypothesis, no Axiom). The recursion theorem is proved by exhibiting a self-aware fixed-point program (code `2`). The substrate-level limitative theorem fires unconditionally, and the corollary `nat_self_undecidable` reads off: the substrate cannot internally decide its own shortcut-admittance predicate (the abstract, A2-free version; the certification-axis-specific form is the conditional VM corollary). `Print Assumptions` on all three returns `Closed under the global context`.
33. *vm_structural_shortcut_undecidable*: substrate-level theorem instantiated at the 51-opcode VM. Carries the VM-side Section parameters (Gödel encoding, internal-representability predicate, internal recursion theorem) as explicit `forall`

premises in the Bekenstein-bound style. The substrate-level content is unconditional at `nat_substrate`; the VM corollary presents the result at the VM scope.

34. *program_to_nat / nat_to_program / nat_to_program_program_to_nat*: closed Gödel encoding of `list vm_instruction` as a `nat` with a proven left inverse, all 51 opcode constructors handled, no Hypothesis. Composes the existing binary encoders with a `list bool ↔ nat` bijection through `positive`.
35. *categorical_separation*: classical-identical states distinguishable by morphism probe.
36. *classical_observer_cannot_separate*: no classical function separates morphism-distinct states.
37. *shadow_proj / shadow_separation_theorem*: formal surjection, strictly lossy.
38. *D2_faithfulness*: classical programs embedded faithfully in Thiele.
39. *D3_conservativity*: classical programs leave structural layer untouched.
40. *D4_strictness*: there exist Thiele steps that escape the classical fragment.
41. *D5_thiele_strictly_extends_classical*: Thiele strictly exceeds classical semantics.
42. *degenerate_projection_theorem*: shadow projection is the classical equivalence quotient.
43. *no_classical_certification_decider*: no classical machine can decide morphism-certification.
44. *chsh_stat_violation_not_local*: CHSH-violating statistics are non-local.
45. *structural_trace_preserves_cert_addr*: traces with no cert-address setter preserve `csr_cert_addr`.
46. *partition_refinement_nonfree*: certified partition knowledge cannot be free.
47. *partition_free_but_certification_nonfree*: partition structure can be built at $\delta = 0$, but certified partition knowledge cannot be acquired for free.
48. *witness_insight_nonfree_general*: certified non-local witness insight requires $\mu \geq 1$.
49. *witness_insight_complete_taxonomy*: full three-tier taxonomy proven.
50. *separable_coupling / non_separable_coupling*: Functional (separable) and non-functional (non-separable) coupling predicates with concrete witnesses.
51. *CHSH coupling bridge*: Bell's theorem in morphism coupling language. A locally consistent strategy produces a separable coupling and has $|S| \leq 2$. $S > 2$ rules out every locally factorizable morphism coupling.
52. *graph_certify_morphism_lookup*: morphism certification cost utility, including lookup correctness.
53. *exists_covering_tree*: for any feasible-set reduction, a covering complete binary tree exists constructively.
54. *info_priced_reduction_no_tree_hypothesis*: entropy bound without requiring the tree as an explicit input.
55. *sound_shortcut_from_components*: assembly lemma for SoundStructuralShortcut from its component witnesses.
56. *mu_budget_decidable / mu_budget_verifiable / classical_mu_budget_decidable*: predicates classifying decision problems by their polynomial-μ envelope, plus the inclusion that every zero-μ classical solver is mu_budget_decidable. Not a class separation. See Section 24 for what these predicates do and do not say.

57. *column_contractive_iff_quantum_realizable*: Thiele honesty $\iff$ zero-marginal NPA realizability. A biconditional with no project-local axioms (its `Print Assumptions` reports only Coq's standard classical-reals and functional-extensionality stdlib axioms). Forward (`zero_marginal_npa_column_contractive_implies_psd`): column-contractive $\to$ NPA PSD. Reverse (`npa_psd_implies_column_contractive`): NPA PSD $\to$ column-contractive, proved by vector specialization plus the `quadratic_nonneg_discriminant` lemma. Zero-marginal NPA realizability is the orthogonal-observables slice of the quantum elliptope ($\rho_{AA} = \rho_{BB} = 0$), not the full elliptope. Classical correlations like $(1, 0, 1, 0)$ have $|S| \leq 2$ but fail column contractivity. The classical polytope and the Thiele-honest set intersect but neither contains the other.

58. *state_column_contractive_implies_tsirelson*: Thiele-honest $\Rightarrow |S|^2 \leq 8$, i.e., $|S| \leq 2\sqrt{2}$. The proof factors through quantum realizability: `state_column_contractive_implies_quantum_gram` composes with `quantum_realizable_implies_tsirelson_bound`. The row-bounds lemma `tsirelson_from_row_bounds` is the algebraic Cauchy–Schwarz step inside this chain. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not Thiele-honest) follows immediately. The converse ($|S| \leq 2\sqrt{2} \Rightarrow$ Thiele-honest) is false and is not claimed.

59. *Concrete falsifications by direct arithmetic.* The PR box ($E_{00} = E_{01} = E_{10} = 1$, $E_{11} = -1$) fails the first column-contractivity condition: $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Any super-quantum value $|S| > 2\sqrt{2}$ falls outside the Thiele-honest set by the contrapositive of `state_column_contractive_implies_tsirelson`. The Turing-classical correlation $(1, 0, 1, 0)$ also fails condition 1. None of these require new theorem statements; they are immediate by `lra` once the biconditional is destructed.

60. *mu_hierarchy_theorem*: for every $k \geq 1$, there exists a certified trace with cost exactly k, and every certified trace with `vm_mu` $\geq k$ has cost $\geq k$. Infinite strict μ-complexity separation.

61. *mu_hierarchy_no_upper_bound*: no fixed μ-budget suffices for all levels.

62. *Structural-invariant Qed group*: the 10 opcodes PNEW, PSPLIT, PMERGE, MORPH, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_GET, COMPOSE, MORPH_TENSOR carry Qed bisimulation proofs under the inductive invariant `coupling_wf` (= `coupling_desc_bounded ∧ coupling_pairs_in_range ∧ coupling_pairs_fully_populated`) and `morph_table_wf`. The preservation theorems `coupling_wf_kami_step_preserved` and `morph_table_wf_kami_step_preserved` show these invariants hold inductively at hardware reset and through every `kami_step` operation, so the precondition is discharged in practice for any state reachable by machine execution.

63. *RTL trust boundary*: the Section Variable `rtl_step_correct` states that the synthesised RTL implements the Kami step function. It is backed empirically by the cosim suite (38 tests, covering every opcode) and the parametrized cross-layer fuzz suite. The kernel proves `rtl_step_correct` for the COQKAMI model and names `bsc_kami_compilation_trusted` as the residual trust boundary covering the BSC compiler / pretty-printer pipeline.

64. *nothing_at_stake_is_free_forgery*: a finality gadget whose finalize step carries zero stake-at-risk admits no A2 field. Nothing-at-stake is the kernel's free forgery, stated in proof-of-stake vocabulary (Section 15).

65. *slashing_finality_floor*: under any slashing schedule (finalize risks ≥ 1), every run from

unfinalized to finalized carries total stake-at-risk ≥ 1; `universal_nfi_any_substrate` instantiated at the slashing gadget.

66. *gas_schedule_exactness*: a gas schedule satisfies the commitment floor and no-overcharge exactly when its charging predicate is the commitment predicate with unit pricing; the kernel machine itself inhabits the class (`thiele_vm_commit_pricing_is_exact`).
67. *attestation_cannot_factor_through_bare_transcript*: a sound and complete attestation verifier of the μ-dependent claim cannot factor through the report's bare transcript; the replay attack is the witness pair (`replay_is_a_two_preimage_witness`).
68. *level_k_verification_floor*: k certification events cost at least k μ in any covered proof system, with tightness witnessed; bounds certification events only, not gate counts, circuit size, prover time, or proof length.

Bridge-tier claims (checked in Coq, but resting on named hypotheses or documented trust boundaries):

- **Hardware bisimulation**: 47/47 synth-realised opcodes with Qed proofs (37 unconditional + 10 under structural invariants `coupling_wf` / `morph_table_wf`, 0 structural gaps in the synth-realised subset); documented (`rtl_coverage_partition` witnesses $37 + 10 + 0 = 47$). The four Q_{1+AB} cert-opcodes (slack 4 in the OCaml/RTL opcode-set parity test) live in the Kami HW abstraction with kernel-equivalence proven and run in the OCaml runner and Python harness, but are excluded from the synthesized Verilog by silicon budget; documented alongside their substrate-level Coq soundness theorems.
- **OCaml extraction**: `ocaml_bisimulation_closure` proves NoFI, μ-monotonicity, and totality for the Coq-side extracted observable. The external OCaml binary equivalence is a parity-test trust boundary, not a kernel theorem.
- **Structural advantage**: the kernel proves exact μ costs and the arithmetic time-tax facts for the factored-search witness; executable tests check the measured VM behavior.
- **Cost formula forcing**: `cost_necessity` + `cost_uniqueness`, conditional on Landauer-Unruh calibration for physical interpretation.

24 Scope of the claim: what is established, what is not

This section names the boundary of what the subsumption argument establishes and what it does not. The directional structural result is the spine: every Thiele substrate has a canonical projection onto a classical-equivalent substrate; no classical substrate has a canonical Thiele lift; the dropped fields (`vm_certified`, `csr_cert_addr`, μ relative to the bare classical observable) are provably irrecoverable from the projection (Section 3.7). That is the load-bearing content. The items below are open in the sense that the document does not extend the proofs to cover them; each one is listed with the formal reason the existing argument stops where it stops, so the perimeter of the positive claim is clear.

Physical interpretation of μ as thermodynamic entropy. The subsumption claim establishes that μ is the unique cost coordinate up to the local conditions in `mu_is_initial_monotone` (Section 7) and that classical substrates project onto Thiele substrates in the cost-tracking-blind direction. It does not establish that the μ-ledger corresponds to physical thermodynamic entropy in joules. The Landauer connection in Section 17 is a named hypothesis (`mu_landauer_unruh_calibrated`); the formal development is consistent with it but does not depend on it. Closing this gap would require a physical experiment, not a proof.

Class separation. The subsumption claim establishes a cost-tracking separation at the substrate level: classical substrates do not carry A2 in their step rule, and the cost-ledger separation between Thiele states with identical bare-classical observables has no analog on the classical signature (`no_classical_mu_separation_predicate` in Section 3.7). It does not establish a complexity-class separation (P vs NP or any analog). The kernel defines `mu_budget_decidable` and `mu_budget_verifiable` and proves only the basic implication that every decidable problem is verifiable. The factored-search witness ($\mu = 18$ pays for one structural shortcut) is a witness-level cost gap, not a class separation. There is no concrete problem placed in `mu_budget_verifiable` but provably not in `mu_budget_decidable`. Closing this gap would require constructing such a problem.

Privileged choice of extensional predicate for the substrate-axis diagonal. The subsumption claim establishes that the structural-axis diagonal lives on `csr_cert_addr` reachability and that the diagonal's content cannot be re-expressed on the bare classical projection (`cert_addr_not_function_of_forget` in Section 3.7). It establishes the substrate-level diagonal for an entire family of non-trivial extensional predicates: no representable decider answers true exactly when the predicate holds (`structural_shortcut_undecidable`). The VM-instance corollary picks one specific predicate in the family: `simple_morph_trace`-coincidence. Whether that is the privileged choice, versus a predicate built directly from `has_supra_cert` reachability, versus receipt-list equality at a specific fuel bound, is open. The substrate-level result is the family statement; the VM corollary is one tile. Closing this gap would require an argument that one tile is canonical.

Unconditional Shannon-style bound on every deterministic single trace. The subsumption claim establishes the entropy-bounded μ-ledger theorem for sound structural shortcuts: any trace equipped with a realized-by-the-trace decision tree pays at least $\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'|$ in μ for narrowing Ω to Ω' (Section 5). It does not establish the same bound for an arbitrary deterministic single trace without the tree as witness. The naive single-trace claim is false in general, and I say so explicitly above. The remaining open step: derive the realization condition (`decision_tree_realized_by_trace`) from VM trace behavior without requiring the user to verify the leaf count. Closing this gap would require automating the tree extraction from a trace, which the kernel does not currently do.

Full-elliptope quantum mechanics connection. The subsumption claim establishes the algebraic equivalence between Thiele honesty and zero-marginal NPA realizability (`column_contractive_iff_quantum_realizable`, no project-local axioms). The zero-marginal framework is the orthogonal-observables slice ($\rho_{AA} = \rho_{BB} = 0$): a proper subset of the full quantum elliptope. The claim does not establish that the full elliptope corresponds to a structural-shortcut class in the kernel, nor that a physical Thiele Machine can produce CHSH-violating statistics by exploiting quantum entanglement in hardware. Closing the algebraic gap would require extending the column-contractivity characterisation; closing the empirical gap would require an experiment.

The boundary above is the perimeter of the positive claim. Inside the perimeter: the directional subsumption (Section 3), the three irrecoverability theorems (Section 3.7), the cost-ledger observability separation (Section 12), the entropy-bounded μ-ledger theorem (Section 5), the substrate-axis undecidability (Section 19), the CHSH-NPA equivalence on the orthogonal slice (Section 16), and the categorical universal property of Thiele as the initial A2-respecting substrate (`thiele_is_initial_a2_substrate`, Print Assumptions: Closed under the global context). Outside the perimeter: the items above, each with the formal reason the existing argument does not extend to it. Nothing I claim crosses the perimeter. So if your objection lands outside it, that is worth knowing, and it also means we are arguing about a claim I never made. Tell me which side of the line you think your question falls on; drawing the line is the whole point.

Categorical foundationality vs. metaphysical foundationality. The theorem `thiele_is_initial_a2_substrate` (zero axioms) establishes Thiele as the initial/universal object in the category of A2-respecting substrates over `vm_instruction`: hand it any A2-respecting substrate M over the same instruction signature and any chosen basepoint s_0^M in M, and the canonical trace-fold simulation from Thiele to M is the unique morphism preserving the basepoint and commuting with step-extension. There is exactly one way in, and Thiele is where you start. That is initiality in the precise categorical sense, the same sense in which $\mathbb{Z}$ is the initial ring and the free group is initial among groups with a fixed generating set. Here is the contradiction I want to hold in plain sight, because it is the place this kind of result gets oversold and I would rather oversell nothing. The theorem is airtight, and the theorem does not tell you that the A2-respecting category is the one worth standing inside. That second question, whether this category sits above the alternative cost-tracking categories, is not a theorem at all. It is the same kind of question as why anyone bothers studying groups when semigroups were sitting right there: a choice of which signature carries the weight, settled by what the structure lets you do, never by a proof living inside the structure. So I am drawing two lines and putting them next to each other on purpose. The categorical universal property: established. The priority claim, that classical computation is derivative, on substitution-adequacy

and lattice-scoped minimality plus the law-versus-checker fact: also established, and reached here. What is not reached, and what I do not claim, is the cosmic primacy of the A2 category over its rivals. That one is your call, on grounds the argument here does not touch. Derivativeness is not your call. The argument gets all the way to it.

Where priority lives, and where it does not. Here is what does the work. Hold the trust fixed and A2 is the *exact least* local predicate that prices certification. Miss a cert-flip and the floor fails on a one-step trace. Charge a non-cert-flip and you overpay. So the only exact, non-overcharging substitute already *is* A2. That is an equivalence, and it is proved in both directions. There is even a concrete trusted rival in the running, erasure accounting, and it provably loses (`a2_equal_trust_substitution_payoff`, sharp forms `substitution_test_exact_substitute_is_a2` and `substitution_test_rejects_non_a2_exact_substitute`). The structural axis $(\mu, \texttt{cert})$ is the proved-minimal extension over (mem, regs, pc): cert is not a function of the rest, μ is not a function of the rest (`P_full_is_minimal_complete_extension`). Drop either field and certification semantics stop being determinate. And a classical step rule cannot carry A2 as a law. It can only run a fallible checker of it, after the fact, from outside the step. That is what makes classical computation *derivative* and not merely projected: it is the state left over when you forget the law, and you cannot put the law back without rebuilding the substrate. None of that rides on initiality or on the projection/lift asymmetry, and here is the one-line reason to keep your eye on both. Initiality is the freest-object property. $\mathbb{Z}$ is the initial ring, and a $\mathbb{Z}$ with an inert tag bolted on is initial among tagged structures by the identical proof. The projection/lift asymmetry is generic to any field-drop, the tagged $\mathbb{Z}$ included. Neither prices anything, so neither buys priority. And there is a third generic I have to put on the table in the same breath, because it is the one a careful reader actually catches, and I would rather say it than have it said to me. The substitution contest ranks rules that charge a *fixed* event. Erasure accounting loses to A2 at pricing certification, but erasure would lose to almost any honest charger of almost any event, so what the contest settles is which charger, not which event. That certification is the event being priced is written into the setup; it is not a prize the contest awarded. Whether pricing *that* event, rather than head-reversals or anything else a step could be billed for, is itself forced: that I have not shown, and I am not going to claim it. The directional claim is true: classical is the canonical lossy projection, and Thiele strictly extends it. But it stands on the substitution and minimality results above, not on the asymmetry. Bolt A2 into a classical step rule and you have not extended classical computation. You have built the Thiele Machine. Read "initial" as freest. Never as deeper. The word that survives the proofs is "derivative," scoped to a substrate that already prices certification, and it survives on substitution-adequacy and minimality plus the law-versus-checker fact, not on the projection asymmetry, and not on any claim that certification is the one event computation was secretly about all along. That last claim I have not shown, and it is not answered here.

The structural facts vs. the Thiele-specific content. Several load-bearing theorems in this document are instances of standard structural facts applied to the Thiele construction. Naming each one's standard parent and what Thiele adds, so a reader pattern-matching to "this is just X" can see exactly which X and what the substrate-specific delta is:

- **Irrecoverability theorems** (`cert_not_function_of_forget`, `mu_not_function_of_bare_observable`, `cert_addr_not_function_of_forget`). Standard parent: the expressibility limitation of a smaller signature: no derived predicate on the bare classical projection captures the dropped fields. Thiele-specific content: the *choice* of dropped fields (`vm_certified`, `vm_mu`, `csr_cert_addr`) and the explicit reachable-state witnesses (`trace_witness_A`, `forget_witness_uncert`, the bare-observable and cert-addr witness pairs defined). The witnesses are not abstract; they are concrete computational states with named identifiers and verified `forget` equalities. The corollaries (`no_classical_a2_cert_predicate` etc.) are statements about what can be expressed on the bare classical signature, not just about what has been forgotten in the projection.
- **Categorical initiality** (`thiele_is_initial_a2_substrate`). Standard parent: the universal property of the term algebra of an essentially-algebraic theory. Thiele-specific content: the instruction signature (51 opcodes including the four Q_{1+AB} cert-opcodes), the A2 constraint (`cs_cert_costs`), and the basepoint at `init_state`.
- **Substrate-axis undecidability** (`structural_shortcut_undecidable`, `vm_structural_shortcut_undecidable`). Standard parent: Kleene's 1938 diagonal pattern, abstracted from Turing-machine programs to any substrate exposing an internal recursion theorem and an internal representability predicate; the decider it rules out is one answering true exactly when the predicate holds. Thiele-specific content: the typeclass parameterization (`Substrate` and `WithShortcutPredicate` typeclasses) that lifts the pattern above the Turing-machine setting, the choice of extensional predicate (`AdmitsShortcut`, branching on `csr_cert_addr` reachability), and the closed instantiation at `nat_substrate` with zero Hypothesis. Sibling theorem: Rice's 1953 theorem is the same diagonal pattern instantiated at the Turing-machine partial-recursive-functions level. Neither is a corollary of the other; both are instances of Kleene's pattern.
- **μ-uniqueness** (`mu_is_initial_monotone`). Standard parent: the universal property of natural-number recursion. A function fixed by its value at a base case and a step-recurrence is unique up to extensional equality on the inductive structure. Thiele-specific content: the cost schedule that drives the recursion (the per-instruction cost rule with $S(\delta)$ for cert-setters, payload-bit charges for EMIT, $\mathit{flen} \times 8 + S(\delta)$ for LASSERT, etc.) and the reachability invariant that pins the

unique function down on reachable states.

Each of these is a true Coq theorem about the Thiele construction. Two of them carry the priority claim, and neither is generic. The first is minimality: `P_full_is_minimal_complete_extension` proves that $(\mu, \texttt{cert})$ is the proved-minimal extension over (mem, regs, pc): cert is not a function of (mem, regs, pc, μ), and μ is not a function of (mem, regs, pc, cert). That is the state A2 needs, a cert to read and a μ to charge, and dropping either field destroys completeness. The four standard structural facts above (irrecoverability, initiality, undecidability, μ-uniqueness) are generic in the sense that a tagged-$\mathbb{Z}$-shaped field-drop has each one too; on their own they buy a richer extension, not priority. The second is sharper still. Holding trust fixed, A2 is the *exact least* local predicate that prices certification: a substitute that misses a cert-flip fails the floor on a one-step trace, one that charges a non-cert-flip overcharges, and the only exact non-overcharging substitute already is A2, an equivalence proved in both directions (`a2_equal_trust_substitution_payoff`, with sharp forms `substitution_test_exact_substitute_is_a2` and `substitution_test_rejects_non_a2_exact_substitute`). That is the design content of the Thiele construction: A2 lives in the step rule as a law, where flipping certification and charging μ are one and the same step, and a classical step rule can at most run a fallible checker of A2 from outside. Initiality, irrecoverability, and undecidability are generic; minimality, substitution-adequacy, and the law-versus-checked-invariant distinction are not, and that is what makes classical computation derivative rather than a parallel option. The engineering is downstream of *that*: the cost-discipline VM with 51 opcodes, the OCaml extraction with parity-tested correspondence to the Coq semantics, the 47-opcode RTL bisimulation with Qed coverage and zero structural gaps within the RTL-realised subset, and the integer-arithmetic CHSH-LASSERT family of opcodes whose soundness against PSD of the NPA moment matrix is established by an explicit chain of cleared-integer Schur cascades. The latter is the closest the document gets to non-routine algebraic content, and its mathematical bridge to quantum mechanics lives in the externally-cited NPA framework rather than inside the kernel.

25 Conclusion

Two pillars close this document, one at the instance level and one at the substrate level.

At the instance level, the sharpest thing I found is this: an integer-arithmetic check on the CHSH outcome counters forces the NPA moment matrix to be positive semidefinite, and the argument from the one to the other never once reaches for physics. No Hilbert space, no operators, no observables anywhere in the chain. That is the claim, and it is algebra you can follow line by line. Where I checked it is `chsh_lassert_no_trap_implies_quantum_realizable`, and the check is where I confirmed I hadn't smuggled physics in without noticing: `Print Assumptions` on it leans on exactly three standard real-number axioms from Coq's own library: Dedekind-reals

decidability (`sig_forall_dec`, `sig_not_dec`) and functional extensionality, and nothing of mine. The substrate-level pillar below leans on even less.

At the substrate level, the sharpest thing is that a substrate cannot decide its own shortcut-admittance predicate. A halting-shaped wall on the structural axis, built by the same diagonal Kleene used in 1938, run inside the substrate's own machinery instead of borrowed from the time axis. I made it concrete on a substrate built out of three numbers and a fixpoint, every field discharged by hand, no Hypothesis and no Axiom, and the wall holds: no representable decider answers true exactly when a program admits the shortcut (`nat_structural_shortcut_undecidable` and its corollary `nat_self_undecidable`; `Print Assumptions` comes back `Closed under the global context`, nothing assumed at all). That is the unconditional floor, the generic family Rice's theorem also lives in. The certification-axis-specific form, where the predicate is tied to the supra-cert channel, is the VM corollary, conditional on its internal recursion theorem, a routine condition, not a live doubt (Section 19 sets out why).

Those are the two pillars, and neither is the foundation. What makes this document foundational is the universal A2 cost floor: `universal_nfi_any_substrate` holds on *any* substrate that carries a certification predicate and charges A2 on the cert-flip as a law of its step rule (its only premises the two trace endpoints, given a run that starts uncertified and ends certified), and `P_full_is_minimal_complete_extension` pins $(\mu, \texttt{cert})$ as the proved-minimal extension over (mem, regs, pc), the state a step rule needs to read the cert and charge the μ. The foundational weight is the priority claim, and it rides on universality, scoped-minimality, and the law-versus-checker fact. Not on a physics-free Bell bridge, not on a Rice-family diagonal. The CHSH-NPA equivalence and the substrate-axis undecidability are strong companion facts, no more: an algebraic bridge with no physics premises on one side, and a limitative result internal to the substrate's own typeclass machinery on the other. The 51 opcodes, the OCaml runtime, the Bluespec hardware, the parity tests, the simulator harness are scaffolding for one realization of the substrate; the universal cost floor and the lattice-scoped irredundancy of $(\mu, \texttt{cert})$ over (mem, regs, pc) are facts about substrates, and the two theorems above are strong instances of how those facts cash out.

Each of these results carries an explicit falsification condition, and they are gathered in one place (Section 22): a zero-μ trace from uncertified to certified, a column-contractive correlator with a non-PSD NPA matrix, a cost-rule-respecting `CertificationSystem` that certifies at total cost zero, a `nat` function that decides `nat_admits` for its own substrate. Any one of them collapses the matching theorem. You can hunt for any of them with a pencil, and the mechanised ones would stop the checker dead on top of that. I hunted. I came up empty.

I do not want this to sound like I am declaring I am right. I want to push myself, and after investigating everything, maybe the field too. But what do I know.

I built this. I'm not going to point you at the Coq and call that the proof, because it isn't one. Coq doesn't prove things. It checks that a proof term matches a statement, which is a smaller and dumber job, and it will stamp a statement that means nothing if you hand it one. I used it to catch the gaps my own eye slid past, and to pull a running machine and a real chip out of the same source. That is worth a lot. It is not proof. The proof, if it's here, is the argument, and the argument is in English on the pages above: A2 is the only rule that prices certification exactly, a classical step has no field to carry it, so classical computation is the lossy shadow of the thing that does. Don't take the checkmark's word for that. Don't take mine. Follow the argument, step by step, and if one of the steps gives, that's the one I want to hear about.